



FernUniversität in Hagen

Fakultät für Mathematik und Informatik

Projektdokumentation
im Fachpraktikum IT-Sicherheit

Team Blue - Self Hosted Setup
Gruppe 1

von

Florian Dobke, Matrikelnummer: 7732783

Luca Leon Bäcker, Matrikelnummer: 2900858

Marie Lumeau, Matrikelnummer: 2294419

Datum der Abgabe: 15.07.2026

Gender-Hinweis

Die in dieser Projektdokumentation verwendeten Personenbezeichnungen beziehen sich immer gleichermaßen auf weibliche und männliche Personen. Auf eine Doppelnennung und gegenderte Bezeichnungen wird zugunsten einer besseren Lesbarkeit verzichtet.

Inhaltsverzeichnis

Abbildungsverzeichnis	VII
Tabellenverzeichnis	XI
Abkürzungsverzeichnis	XIII
1 Self Hosted Setup	1
1.1 Einleitung	1
1.2 Ausgewählte Services	4
1.2.1 Begründung einzelner Services	6
1.3 Apphost: Architektur, Umsetzung und Bedrohungsmodell	9
1.3.1 Evaluation verschiedener Ansätze	9
1.3.2 Aufbau der Entwicklungs- und Testumgebung . . .	11
1.3.3 Der Apphost-Stack im Detail	13
1.3.4 Abdeckung des Bedrohungsmodells	27
1.3.5 Backups und Wiederherstellung	30
1.3.6 Sicherheitshärtung im Überblick	31
1.3.7 Fazit	32
1.4 Krypto	34
1.4.1 Hot- und Cold-Wallet Architektur	35
1.4.2 Hot-Wallet	40
1.4.3 Cold-Wallet	65
1.4.4 Prozesse	71
1.4.5 Alternativen	84
1.4.6 Sicherheitsbetrachtung	88
1.4.7 Fazit	97
2 Künstliche Intelligenz (KI)-Evaluation	101
2.1 Zielsetzung und Forschungsfrage	101
2.1.1 Forschungsstand	102

2.2	Methodik und Bewertungsrahmen	105
2.2.1	Methodikanforderungen	105
2.2.2	Teststrategie	106
2.2.3	Bewertung des Bitcoin (BTC)-Systems	110
2.2.4	Datengrundlage und Dokumentation	112
2.2.5	Prozessmetriken	112
2.3	Promptdesign und Versuchsverlauf	112
2.3.1	Gemeinsame Ausgangsbedingungen	112
2.3.2	Szenario A	113
2.3.3	Szenario B	114
2.3.4	Nachvollziehbarkeit	114
2.3.5	Retrospektive des Promptdesigns	115
2.4	Einzelbewertung der Szenarien	115
2.4.1	Szenario A	115
2.4.2	Szenario B	123
2.5	Vergleichende Bewertung	132
2.5.1	Architekturplanung	136
2.5.2	Services	137
2.5.3	Konfiguration	137
2.5.4	Netzwerk, Tor und Zugriff	138
2.5.5	Backup und Monitoring	138
2.5.6	BTC-Konzept	139
2.5.7	Fehleranalyse	146
2.6	Grenzen der Evaluation	147
2.7	Takeaway für den Mandanten	148
2.8	Lessons Learned	149
3	Teamorganisation und Projekterfahrungen	151
3.1	Teamorganisation	151
3.2	Aufgabenverteilung	152
3.3	Herausforderungen	152
3.3.1	Self-Hosted-Setup	152
3.3.2	Bitcoin-Setup	153
3.3.3	KI-Evaluation	154
3.4	Learnings	155
3.5	Feedback und Anmerkung zur Gewichtung	156

Literaturverzeichnis

XVII

Anhang

XXVII

Abbildungsverzeichnis

1.1	Darstellung der PSBT-Vertrauenszonen als Hardwarekomponenten und ihrer gehosteten Services. Quelle: Eigene Darstellung.	36
1.2	Darstellung der Schlüsselerwendung im Signierprozess. Quelle: Eigene Darstellung.	38
1.3	PSBT-Lebenszyklus und Statusübergänge. Quelle: Eigene Darstellung.	44
1.4	Balance-Zonen des Hot-Wallets und resultierende OPA-Aktionen. Quelle: Eigene Darstellung.	55
1.5	Darstellung der TPM-Sicherungslogik. Quelle: Eigene Darstellung.	62
1.6	Sparrow Multi-Signatur - Einstellungen. Quelle: Eigenes Bildschirm-Foto aus Sparrow.	73
1.7	Darstellung der Schritte zum Setup des Cold-Wallets. Quelle: Eigene Darstellung.	75
1.8	Darstellung des Signierprozesses für das Hot-Wallet. Quelle: Eigene Darstellung.	76
1.9	Darstellung der Sparrow Output-Verifikation. Quelle: Eigenes Bildschirm-Foto aus Sparrow.	78
1.10	Sparrow Wallet Broadcast und Final TX Export Funktion nach Signatur. Quelle: Eigenes Bildschirm-Foto aus Sparrow.	79
1.11	Darstellung des Signierprozesses für das Cold-Wallet. Quelle: Eigene Darstellung.	80
1.12	Import mit vorsignierten Key A Schlüssel. Quelle: Eigenes Bildschirm-Foto aus Sparrow.	91

A4.1 Homepage-Dashboard mit Übersicht aller Dienste, gruppiert nach Kategorie. Der grüne Punkt neben jedem Eintrag zeigt den erreichten Online-Status an. Quelle: Eigene Bildschirmaufnahme.	XL
A4.2 Ausgabe von <code>docker ps</code> auf dem AppHost. Alle Container laufen im gemeinsamen Monorepo-Deployment unter <code>/opt/monorepo/apphost</code> und melden überwiegend den Status <code>healthy</code> . Quelle: Eigene Bildschirmaufnahme. . . .	XLI
A4.3 Dashboard des Traefik-Reverse-Proxys mit den konfigurierten Entrypoints (u. a. <code>web</code> , <code>websecure</code> , <code>tor-http</code>) sowie einer Erfolgsübersicht der HTTP- und TCP-Router, -Services und -Middlewares. Quelle: Eigene Bildschirmaufnahme.	XLI
A4.4 Prometheus-Statusseite „Target health“, welche die Erreichbarkeit der gescrapten Exporter (u. a. Alertmanager, cAdvisor, Garage, Grafana, Immich) mit Zeitstempel des letzten Scrapes anzeigt. Quelle: Eigene Bildschirmaufnahme.	XLII
A4.5 Grafana-Dashboard „Traefik Official Standalone Dashboard“ mit Request-Raten, HTTP-Statuscodes, langsamsten Diensten sowie der Einhaltung der definierten Latenz-Schwellenwerte je Dienst. Quelle: Eigene Bildschirmaufnahme.	XLII
A4.6 Grafana-Dashboard „Cadvisor exporter“ mit CPU-, Speicher- und Netzwerkauslastung der einzelnen Container sowie einer tabellarischen Übersicht der laufenden Container inklusive Image und Laufzeit. Quelle: Eigene Bildschirmaufnahme.	XLIII
A4.7 Grafana-„Logs Drilldown“ auf Basis von Loki, hier gefiltert nach dem Label <code>service</code> mit Log-Volumen und Log-Zeilen je Dienst (u. a. <code>socket-proxy</code> , <code>opencloud</code> , <code>garage</code> , <code>forgejo</code> , <code>traefik</code>). Quelle: Eigene Bildschirmaufnahme. .	XLIII
A4.8 Startseite von Immich (<code>photos.*</code>) nach erfolgreicher Bereitstellung. Quelle: Eigene Bildschirmaufnahme.	XLIV
A4.9 Login-Maske der Ntfy-Weboberfläche (<code>ntfy.*</code>) für Push-Benachrichtigungen. Quelle: Eigene Bildschirmaufnahme. .	XLIV

A4.10	Startseite der selbst gehosteten Git-Plattform Forgejo (<code>git.*</code> , hier unter dem Namen „AppHost Git“). Quelle: Eigene Bildschirmaufnahme.	XLV
A4.11	In OpenCloud (<code>cloud.*</code>) eingebettete Collabora-Online- Bearbeitung einer <code>.odt</code> -Datei über den WOPI-Server. Quelle: Eigene Bildschirmaufnahme.	XLV
A5.1	Log aus der PCR-Abfrage. Quelle: Eigene Bildschirmauf- nahme in NixOS.	XLVIII

Tabellenverzeichnis

1.1	Zuordnung der BIP-174-Rollen zu den Systemkomponenten	37
1.2	Subject-Berechtigungen je Microservice	45
1.3	Transaktions-Rails und ihre Prüfstufen	51
1.4	OPA-Kontostand-Evaluation (Parameter nach Risk Score)	56
1.5	Bewertung air-gap-fähiger Multi-Signatur-Koordinatoren gegenüber Sparrow Wallet[70]	69
1.6	Grenzwerte des Systems, ihr Zweck und ihre Durchset- zungsorte. Die konkreten Werte sind Labor-Defaults und vom Mandanten an sein Mittelvolumen anzupassen.	93
1.7	Bedrohungs- und Maßnahmenübersicht des Krypto-Systems	100
2.1	Evaluationsmethodologien des Forschungsstands im Ver- gleich.	104
2.2	Prüfindikatoren der Hauptkategorien für die Apphost Komponente	109
2.3	Gewichtete Unterkriterien zur Bewertung der BTC- Sicherheitskonzepte.	111
2.4	Indikatorwertungen für Szenario A	118
2.5	Bewertung der BTC-Implementierungen	123
2.6	Indikatorwertungen für Szenario B	127
2.7	Punktbewertung	133
2.8	Indikatorbasierte Detailbewertung der Eigenentwicklung in den Basis-Kategorien	136
2.9	Detailbewertung der Eigenentwicklung im BTC-Konzept .	140
A5.1	Deklarative NixOS-Härtung der Schlüsselhalter-VMs	XLVI
A5.2	PSBT-Lebenszyklus und Statusübergänge	XLVII
A5.3	OPA-Policy zur PSBT-Bestätigung (Hot-Wallet)	XLVIII
A5.4	Betriebs- und Setup-Skripte je System.	L

Abkürzungsverzeichnis

ACME	Automated Certificate Management Environment
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
AIDE	Advanced Intrusion Detection Environment
API	Application Programming Interface
ASLR	Address Space Layout Randomization
AWS	Amazon Web Services
BIOS	Basic Input/Output System
BIP	Bitcoin Improvement Proposal
BPF	Berkeley Packet Filter
BS	Betriebssystem
BSI	Bundesamt für Sicherheit in der Informationstechnik
BTC	Bitcoin
CA	Certificate Authority
CIS	Center for Internet Security
CNI	Container Network Interface
CPU	Central Processing Unit
CT	Container
CVE	Common Vulnerabilities and Exposures
DB	Datenbank
DoS	Denial of Service
DHCP	Dynamic Host Configuration Protocol
DMA	Direct Memory Access
DNS	Domain Name System
DNSSEC	Domain Name System Security Extensions
eBPF	Extended Berkeley Packet Filter
EFI	Extensible Firmware Interface
ESP	EFI System Partition
ETM	Encrypt-then-MAC

EU	Europäische Union
FIFO	First In, First Out
GCM	Galois/Counter Mode
GOST	Gosudarstwenny Standart (russischer Krypto-Standard)
GUI	Graphical User Interface
HAVAL	Hash of Variable Length
HMAC	Hash-based Message Authentication Code
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure-as-Code
ICMP	Internet Control Message Protocol
ID	Identifikation
IOMMU	Input-Output Memory Management Unit
IoT	Internet of Things
IP	Internet Protocol
IT	Informationstechnik
JIT	Just-in-Time
JSON	JavaScript Object Notation
KEM	Key Encapsulation Mechanism
KI	Künstliche Intelligenz
L1TF	L1 Terminal Fault
LAN	Local Area Network
LLM	Large Language Model
LUKS	Linux Unified Key Setup
LXC	Linux Containers
MAC	Message Authentication Code
MD5	Message-Digest Algorithm 5
MDS	Microarchitectural Data Sampling
MFA	Multi-Factor Authentication
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
NATS	Neural Autonomic Transport System
NTP	Network Time Protocol
NTS	Network Time Security
OCR	Optical Character Recognition
OIDC	OpenID Connect

OPA	Open Policy Agent
P2P	Peer-to-Peer
P2WSH	Pay to Witness Script Hash
PC	Personal Computer
PCI	Peripheral Component Interconnect
PCR	Platform Configuration Register
PDF	Portable Document Format
PSBT	Partially Signed Bitcoin Transaction
QR	Quick Response
RAM	Random Access Memory
RPC	Remote Procedure Call
S3	Simple Storage Service
SATS	Satoshis
SDN	Software-Defined Networking
SHA	Secure Hash Algorithm
SLIP	Shamir's Secret-Sharing for Mnemonic Codes
SLUB	Unqueued Slab Allocator
SNI	Server Name Indication
SMT	Simultaneous Multithreading
SOCKS	SOCKets Secure
SSH	Secure Shell
SSO	Single Sign-On
SUID	Set User ID
TAA	TSX Asynchronous Abort
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TPM	Trusted Platform Module
TR	Technische Richtlinie
TSX	Transactional Synchronization Extensions
TX	Transaction
UDP	User Datagram Protocol
UEFI	Unified Extensible Firmware Interface
UI	User Interface
UID	User Identifier
URI	Uniform Resource Identifiers
URL	Uniform Resource Locator
USA	United States of America

USB	Universal Serial Bus
UTXO	Unspent Transaction Output
VM	Virtuelle Maschine
WG	WireGuard
WOPI	Web Application Open Platform Interface
XFCE	XForms Common Environment
xpub	Extended Public Key
ZMQ	ZeroMQ

1 Self Hosted Setup

1.1 Einleitung

In den vergangenen Jahren ist in der digitalen Welt eine zunehmende Abhängigkeit von Cloud-Diensten und abonnementbasierten Online-Dienstleistungen zu beobachten [1], [2]

Gleichzeitig werden digitale Dienste und Daten nicht nur für wirtschaftliche Zwecke wie zum Beispiel Werbung genutzt, sondern sind auch sicherheitsrelevant. Private Unternehmen und staatliche Institutionen verfügen über umfangreiche Möglichkeiten zur Erhebung, Verarbeitung und Analyse von Daten, mit denen das Verhalten von Nutzern gezielt beeinflusst werden kann[3], [4]. Diese Einflussnahme bezieht sich nicht nur auf personalisierte Werbung, sondern betrifft zunehmend auch politische Meinungsbildung und die Inhalte, die den Nutzern auf sozialen Medien ausgespielt werden[4].

Je mehr Daten über eine Person gesammelt werden, desto präziser lassen sich individuelle Profile erstellen, die einem digitalen Abbild, einem sogenannten „digitalen Zwilling“, nahekommen.[5] Diese Entwicklung wirft erhebliche datenschutzrechtliche und ethische Fragen auf.

In den United States of America (USA) schafft beispielsweise der Cloud Act die rechtliche Grundlage dafür, dass Behörden unter bestimmten Voraussetzungen auf Daten von Unternehmen aus den USA zugreifen können, auch wenn diese außerhalb der Vereinigten Staaten gespeichert sind. [6].

In der Europäischen Union (EU) werden Maßnahmen diskutiert und teilweise umgesetzt, die den Zugriff auf digitale Kommunikationsinhalte betreffen. Im Rahmen von Initiativen zur Bekämpfung von illegalen Inhalten, wie etwa der Verbreitung von Missbrauchsdarstellungen, wird

unter anderem über verpflichtende Prüfmechanismen für Kommunikationsdienste („Chatkontrolle“) debattiert.[7], [8] Diese Ansätze stehen im Spannungsfeld zwischen Sicherheitsinteressen und dem Schutz der Privatsphäre, da sie potenziell auch die Kommunikation unverdächtigter Menschen betreffen könnten. Darüber hinaus plant die Bundesregierung eine gesetzliche Verpflichtung zur anlasslosen Speicherung von IP-Adressen für drei Monate, um schwere Kriminalität im Netz wirksamer bekämpfen zu können.[9]

Auch aus technischer Sicht bestehen Einschränkungen. Zwar setzen viele Dienste auf Ende-zu-Ende-Verschlüsselung, jedoch verbleiben durch Metadaten, serverseitige Komponenten und die Abhängigkeit von zentralen Infrastrukturen weiterhin potenzielle Angriffs- und Zugriffspunkte.[10] Darüber hinaus ermöglichen die bei zentralen Diensten anfallenden Metadaten häufig ein sogenanntes *Identity Chaining*, bei dem Informationen aus unterschiedlichen Quellen miteinander verknüpft werden können.[10], [11] Selbst wenn Kommunikationsinhalte geschützt sind, lassen sich dadurch soziale Beziehungen, Nutzungsprofile und digitale Identitäten rekonstruieren. Zudem stellen zentralisierte Plattformen ein attraktives Ziel für Angriffe dar, was regelmäßig zu Datenlecks und Sicherheitsvorfällen führt.[12] Insbesondere im Kontext digitaler Vermögenswerte wie Kryptowährungen können solche Vorfälle erhebliche finanzielle Schäden verursachen.[13]

Vor diesem Hintergrund wächst bei sicherheits- und datenschutzbewussten Nutzern der Bedarf nach Lösungen, die eine höhere Kontrolle über die eigenen Daten ermöglichen, um Abhängigkeiten von externen Dienst Anbietern zu reduzieren und die Verarbeitung sensibler Daten eigenständig zu kontrollieren.

Ausgehend von den oben genannten Faktoren wurde im Rahmen dieses Projekts eine sichere, vollständig selbst gehostete Infrastruktur für einen Mandanten konzipiert. Bei dem Mandanten handelt es sich um eine vermögende Privatperson mit Schwerpunkt im Bereich Kryptowährungen, für die Diskretion, Privatsphäre und Schutz vor Überwachung von zentraler Bedeutung sind. Die physische Sicherheit und die Sicherung des Netzwerkperimeters der eingesetzten Systeme wird für diese Arbeit als gegeben vorausgesetzt, sodass der Fokus auf der Bereitstellung und

Absicherung der Software und des Netzwerkes liegt.

Ziel dieser Arbeit ist die Entwicklung eines wartbaren, ausfall- und hochsicheren Systems, das realistischen Bedrohungsszenarien wie Malware, Supply-Chain-Angriffe sowie lokalen Angriffen standhält als auch gleichzeitig eine datensouveräne und benutzerfreundliche Nutzung zentraler Dienste ermöglicht.

1.2 Ausgewählte Services

Die Anforderungen des Mandanten sehen die Umsetzung eines selbstgehosteten und umfassend abgesicherten Systems mit mindestens 15 Services vor, die „eine breite Palette an alltäglichen Produktivitäts- und Cloud-Diensten abbildet...“. Aufgrund der umfangreichen Investitionen in Kryptowährungen liegt außerdem ein besonderer Fokus auf Sicherheit und Datenschutz. Vor diesem Hintergrund wurden Dienste ausgewählt, die zentrale Anwendungsbereiche abdecken und gleichzeitig die Anforderungen an Privatsphäre und Kontrolle erfüllen.

Die ausgewählten Services beinhalten dabei sowohl alltägliche Anwendungen, wie Medienverwaltung, Dokumentenarchivierung und kollaboratives Arbeiten, als auch spezialisierte Komponenten für Infrastruktur, Monitoring und Sicherheit. Ergänzend dazu wurden Lösungen für eine sichere Smart-Home-Integration sowie Authentifizierungs- und Zugriffsmanagement berücksichtigt, um ein ganzheitliches und praxisnahes System zu realisieren.

Der thematische Schwerpunkt liegt auf der Integration eines sicheren Bitcoin-Ökosystems, das sowohl automatisierte Prozesse als auch manuelle Kontrollmechanismen unterstützt und damit den spezifischen Anforderungen des Mandanten gerecht wird. Die Umsetzung des Ökosystems von Bitcoin wird in Kapitel 1.4 vertieft und in diesem Kapitel lediglich als Teil der angebotenen Services aufgelistet.

Das gesamte System besteht aus einer Vielzahl von Containern. Diese laufen in der deklarativ beschriebenen Linux-Distribution NixOS, die ihrerseits innerhalb der quelloffenen Virtualisierungsumgebung Proxmox betrieben wird.

1. Dokumente, Medien & Notizen

- Jellyfin
- Paperless-NGX
- Immich
- bento-pdf

2. Cloud, Speicher & Archivierung

- OpenCloud
- Garage
- Collabora Online
- bichon

3. Smart-Home & Internet of Things (IoT)

- Home-Assistant
- mosquitto

4. Bitcoin-Architektur

- Bitcoin Core
- NATS
- Sparrow
- Cold Wallet
- Open Policy Agent
 - Sparrow (offline)
- Middleware
- Simuliertes Air-gapped System (3x)
- TX-Builder

5. Dashboards & Monitoring

- Homepage
- Loki
- OpenSpeedtest
- Alertmanager
- Grafana
- Alloy
- Prometheus
- ntfy

6. Authentifizierung & Passwortmanagement

- Authelia
- Vaultwarden
- (Ursprünglich: KeyCloak)

7. Infrastruktur

- Traefik
- Valkey
- (Ursprünglich: envoy-gateway) (ursprünglich: dragonflydb)

- PostgreSQL
- Tor
- forgejo

1.2.1 Begründung einzelner Services

Vaultwarden

Während der Freigabe der Services äußerte der Mandant Bedenken bezüglich des ausgewählten Passwortmanagers. Er brachte die von ihm bevorzugte Alternative KeePass ein, da er einen Argon2-Schutz wünscht. Argon2 wird genutzt, um das Master-Passwort zu hashen, welches die gesamte Datenbank entschlüsselt [14].

Doch auch Vaultwarden ist mit Argon2 nutzbar [15], womit Vaultwarden und KeePass bezüglich der kryptographischen Absicherung der Master-Passwörter ein vergleichbares Sicherheitsniveau haben.

Ein bedeutender Vorteil von Vaultwarden liegt in der Vereinbarkeit von Sicherheit und Benutzerfreundlichkeit. Während KeePass als lokale, dateibasierte Lösung konzipiert ist und eine manuelle Synchronisation erfordert, ermöglicht Vaultwarden eine zentral gemanagte, selbstgehostete Verwaltung der Logindaten. Dies erleichtert die Nutzung über mehrere Geräte, ohne auf eine externe Cloud angewiesen zu sein. Durch die Integration der Bitwarden-Browsererweiterung erhöht sich nochmal der Bedienkomfort durch das automatische Ausfüllen von Logindaten. Vaultwarden bietet außerdem Ende-zu-Ende-Verschlüsselung, die Trennung von Vertrauen zwischen Client und Server sowie Mehrfaktorauthentifizierung. Diese Aspekte machen Vaultwarden zu einer attraktiveren Lösung für das vorliegende Szenario. [16]

Besonders die Integration mit Authelia trägt zu einer konsistenten und zentral verwalteten Authentifizierungsstrategie bei, denn Vaultwarden unterstützt OpenID Connect, sodass eine nahtlose Einbindung in das Identity- und Access-Management möglich ist [17]. Der Mandant authentifiziert sich über Authelia, wodurch Single-Sign-On über verschiedene Dienste hinweg möglich ist. Gleichzeitig können sicherheitsrelevante Richtlinien wie z.B. Passwortanforderungen oder Mehrfaktorauthentifizierung zentral in Authelia verwaltet und durchgesetzt werden. Vaultwarden

übernimmt dabei die sichere Speicherung der Logindaten im Sinne der Zero-Knowledge-Architektur [18].

Ein weiterer Vorteil ergibt sich im Bereich Monitoring. In der geplanten Architektur mit Authelia können Ereignisse bei Authentifizierungen (Logs (via Loki), fehlgeschlagene Anmeldeversuche oder Token) erfasst und durch Prometheus und Grafana überwacht und in kritischen Situationen der Mandant durch ntfy alarmiert werden. Dadurch ist der Ablauf und der Nutzen für den Mandanten jederzeit einsehbar und transparent. Vaultwarden selbst kann auch als Service von Prometheus überwacht werden.

Im Gegensatz dazu ist KeePass eine rein lokale Anwendung und deshalb nicht sinnvoll in das geplante Monitoring und Logging-Konzept integrierbar. Für Prometheus sind keine Metriken oder Authentifizierungsversuche erfassbar. Aktionen wie das Öffnen der Datenbank, fehlgeschlagene Log-In Versuche oder die Nutzung einzelner Einträge in der Datenbank bleiben lokal auf dem jeweiligen Gerät.

Die identifizierten Schwachstellen von Vaultwarden (insbesondere Common Vulnerabilities and Exposures (CVE)-2024-55225 [19], CVE-2025-24364 [20] und CVE-2025-24365 [21]) wurden im Rahmen der Umsetzung betrachtet. Allerdings sind diese (teils auch schwerwiegenden) Sicherheitslücken nur unter spezifischen Voraussetzungen ausnutzbar (z. B. bei vorhandenem Zugriff auf das Admin-Panel oder vorhandene Nutzerrechte) und bereits durch Updates oder manuelle Anleitungen behoben. Durch die im vorliegenden System implementierten Schutzmechanismen wird die Wahrscheinlichkeit einer erfolgreichen Ausnutzung dieser oder ähnlicher Schwachstellen erheblich reduziert (zudem wurden die Schwachstellen bereits behoben oder durch bereitgestellte Anleitungen zur manuellen Behebung adressiert).

Trotzdem muss Vaultwarden als Passwort-Manager kritisch bewertet werden, da eine Kompromittierung potenziell den Zugriff auf alle gespeicherten Logindaten ermöglichen würde. Dies hätte insbesondere im Kontext der Implementierung des Bitcoin-Transaktionssystems gravierende Auswirkungen. Insgesamt sind die bestehenden Risiken durch geeignete technische und organisatorische Maßnahmen kontrollierbar und im Rahmen des Sicherheitskonzepts angemessen adressiert, sodass

1 Self Hosted Setup

eine Nutzung von Vaultwarden unter den genannten Aspekten berechtigt ist.

1.3 Apphost: Architektur, Umsetzung und Bedrohungsmodell

Dieses Kapitel beschreibt den zentralen Baustein des Gesamtsystems, den sogenannten *Apphost*. Auf ihm laufen alle Alltags- und Produktivdienste des Mandanten. Zunächst werden die Entscheidungen erläutert, die zu der gewählten Architektur geführt haben. Anschließend wird der konkrete Aufbau beschrieben. Abschließend wird gezeigt, wie die Umsetzung das vorgegebene Bedrohungsmodell abdeckt. An geeigneten Stellen wird auf die separate Installationsanleitung sowie auf den Anhang verwiesen, in dem die einzelnen Schritte und alle Sicherheitseinstellungen im Detail aufgeführt sind.

1.3.1 Evaluation verschiedener Ansätze

Vor der Festlegung der endgültigen Architektur wurden mehrere Varianten gegeneinander abgewogen. Gesucht wurde eine Lösung, die sicher und reproduzierbar ist und zugleich wartbar bleibt. Die Nachvollziehbarkeit war dabei das ausschlaggebende Kriterium, denn der Mandant soll die Infrastruktur selbst prüfen und betreiben können.

Verworfen: Kubernetes mit Talos Linux

Der erste Entwurf basierte auf Kubernetes als Orchestrierung und Talos Linux als minimalem, Application Programming Interface (API)-gesteuertem Betriebssystem für die Nodes. Talos besitzt weder eine Shell noch einen Secure Shell (SSH)-Zugang. Die gesamte Konfiguration erfolgt über eine deklarative API. Kubernetes bringt zudem native Konzepte wie Network Policies und Secrets mit.

Dieser Ansatz wurde bereits weitgehend umgesetzt. Im Verlauf der Arbeit zeigte sich jedoch, dass die Komplexität von Kubernetes in keinem angemessenen Verhältnis zum Ziel der Aufgabe steht. Für einen einzelnen Mandanten mit einer überschaubaren Zahl privat betriebener Dienste bedeutet ein Kubernetes-Cluster einen erheblichen dauerhaften Wartungsaufwand. Ingress-Controller, Container Network Interface (CNI)-Plugin, Storage-Provisioner, Zertifikatsverwaltung und Cluster-Updates müssen

verstanden und gepflegt werden. Der Kern der Aufgabe liegt jedoch in der *sorgfältigen Absicherung* der Dienste und nicht im Betrieb eines Cluster-Stacks. Aus diesem Grund wurde der Stack trotz des bereits investierten Aufwands auf Docker Compose neu aufgebaut (siehe Abschnitt 3.3.1).

Ausgetauschte Komponenten

Mit dem Wechsel weg von Kubernetes wurden auch einige zuvor gewählte Werkzeuge ersetzt, da sie ohne Cluster keinen Mehrwert boten oder für den Anwendungsfall überdimensioniert waren:

- **Envoy Gateway zu Traefik.** Envoy Gateway ist eng an die Kubernetes Gateway API gekoppelt, sodass diese Integration ohne Kubernetes entfällt. Traefik liest Container-Labels direkt aus Docker und bringt die Automated Certificate Management Environment (ACME)-Zertifikatsverwaltung (Let's Encrypt) einschließlich der Domain Name System (DNS)-01-Challenge mit. Dadurch lassen sich gültige Zertifikate beziehen, ohne dass der Server aus dem Internet erreichbar sein muss.
- **Keycloak zu Authelia.** Keycloak ist ein vollwertiger Identity-Provider für große Organisationen und entsprechend schwergewichtig. Authelia lässt sich vollständig deklarativ über Konfigurationsdateien einrichten und fügt sich damit besser in den Infrastructure-as-Code-Ansatz ein. Für einen einzelnen Nutzer mit wenigen Diensten ist es zudem angemessener dimensioniert. Authelia deckt sowohl klassisches Forward-Auth als auch OpenID Connect (OIDC) ab und erfüllt damit alle benötigten Fälle.[\[22\]](#)
- **Dragonfly zu Valkey.** Ausschlaggebend für Dragonfly war ursprünglich die Verfügbarkeit eines Kubernetes-Operators. Valkey steht unter dem Dach der Linux Foundation und ist offener lizenziert. Es passt damit besser zu einem auf Datenschutz und Unabhängigkeit ausgerichteten System. In reinen Benchmarks ist Valkey etwas langsamer als Dragonfly. Für den vorliegenden Anwendungsfall, das Caching für Immich und Paperless, ist dieser Unterschied ohne Bedeutung.

Ein einziger gehärteter Apphost statt Virtuelle Maschine (VM) pro Dienst

Eine weitere zentrale Entscheidung war, alle Dienste auf einer einzigen, gut abgesicherten virtuellen Maschine zu betreiben, statt jeden Dienst in eine eigene VM zu legen. Dafür gab es mehrere Gründe:

- **Geringere Komplexität.** Bei über 18 Diensten wären ebenso viele Betriebssysteme einzeln zu aktualisieren, zu härten und zu überwachen. Für den Endanwender ist das kaum handhabbar.
- **Kein internes Routing nötig.** Bei vielen VMs wäre ein internes Routing zwischen ihnen erforderlich. Auf einem einzelnen Host übernimmt Docker diese Aufgabe mit seinen internen Netzwerken. Die Segmentierung lässt sich dadurch an einer Stelle definieren (siehe Abschnitt 1.3.3).
- **Kein Vendor-Lock-in auf Proxmox.** Auf die Software-Defined Networking (SDN)- und Netzwerk-Features von Proxmox wurde bewusst verzichtet, denn Beispiele wie VMware zeigen, wohin eine starke Bindung an einen einzelnen Hersteller führen kann. Da die gesamte Netzwerksegmentierung innerhalb des Apphosts in Docker stattfindet, bleibt dieser portabel. Er ließe sich ohne Änderung der Konfiguration auch direkt auf Bare Metal oder unter einem anderen Hypervisor betreiben.

Die Isolation zwischen den Diensten wird folglich nicht über viele VMs erreicht, sondern über getrennte Docker-Netzwerke, User-Namespaces, Remapping und eine strikt gehärtete Host-Konfiguration. Diese Kombination stellt den besseren Kompromiss aus Sicherheit und Wartbarkeit dar.

1.3.2 Aufbau der Entwicklungs- und Testumgebung

Für eine realistische Entwicklung und Erprobung wurde die Umgebung des Mandanten möglichst genau nachgebildet. Dazu wurde ein dedizierter Server (ein AX41 bei Hetzner) angemietet und darauf Proxmox Virtual Environment installiert. Innerhalb von Proxmox entstand ein klassisches Heimnetzwerk, wie es bei einer Privatperson realistischerweise anzutreffen ist.

OPNsense als Router

Als Router und Firewall des simulierten Heimnetzes diente OPNsense in einer eigenen VM. OPNsense ist eine quelloffene Firewall und wäre auch im realen Einsatz beim Mandanten eine sinnvolle Wahl. Nur die OPNsense-VM hatte direkten Zugriff auf das Internet und maskierte den ausgehenden Verkehr des internen Netzes per Network Address Translation (NAT). Dahinter entsprach der Aufbau einem gewöhnlichen Heimnetz mit Dynamic Host Configuration Protocol (DHCP) für die internen Geräte, einem lokalen DNS-Cache und einem flachen internen Netzwerk.

Zugang über WireGuard im Split-Tunnel

Um so arbeiten zu können, als befänden sich die Arbeitsgeräte physisch im Heimnetz des Mandanten, wurde auf der OPNsense ein WireGuard-Server eingerichtet. Die Geräte des Teams verbinden sich per WireGuard im Split-Tunnel-Modus. Nur der Verkehr in das interne Netz läuft durch den Tunnel, während der übrige Internetverkehr der Geräte unberührt bleibt. Die internen Dienste waren damit genauso erreichbar wie für ein Gerät im Heimnetz, ohne dass potentiell unsichere Ports nach außen geöffnet werden mussten.

Absicherung des Proxmox-Hosts

Der Proxmox-Host selbst wurde vor Beginn der eigentlichen Arbeit abgesichert. Dazu gehören Zwei-Faktor-Authentifizierung für die Weboberfläche, komplexe Passwörter, ausschließlich schlüsselbasierter SSH-Zugang und die Deaktivierung nicht benötigter Dienste. Für die Härtung wurde das Skript `scripts/proxmox-harden.sh` entwickelt. Es deaktiviert unter anderem die Passwort-Authentifizierung für SSH, erzwingt moderne Cipher-Suites, sperrt Universal Serial Bus (USB)-Eingaben und prüft den Secure-Boot-Status. Die Details sind in der Installationsanleitung beschrieben.

Annahme zum Heimnetzwerk

Die Aufgabenstellung spezifiziert das Heimnetzwerk nicht näher. Es wurde daher ein klassisches, flaches Heimnetz angenommen und bewusst nicht weiter verkompliziert. Der Schwerpunkt der Absicherung liegt auf dem Apphost und seinen Diensten, also genau dort, wo die sensiblen Daten des Mandanten liegen.

Getrennte Umgebung für KI-Experimente

Im Rahmen des zweiten Aufgabenteils (Siehe Abschnitt 2-Evaluation) sollten Agenten teilweise autonom in Proxmox arbeiten können. Damit diese Experimente die manuelle Arbeit nicht gefährden konnten, erhielten sie eigene Proxmox-Benutzer und vollständig getrennte Resource Pools. Bis auf einen gemeinsam genutzten Storage-Pool waren diese Umgebungen von der eigentlichen Infrastruktur getrennt. Die Agenten konnten dadurch nicht auf die produktive Konfiguration zugreifen und arbeiteten ausschließlich in ihrem eigenen Bereich.

1.3.3 Der Apphost-Stack im Detail

Der Apphost besteht aus zwei klar voneinander getrennten Schichten, dem Betriebssystem (NixOS) und der Anwendungsschicht (Docker Compose). Beide sind vollständig als Code beschrieben und liegen versioniert im Git-Repository.

Warum NixOS

Als Betriebssystem für den Apphost wurde NixOS gewählt. Diese Wahl deckt sich in mehreren Punkten mit den Anforderungen der Aufgabe:

- **Vollständig deklarativ.** Der gesamte Systemzustand, von den installierten Paketen über die Firewall bis zu jeder einzelnen Sicherheitseinstellung, ist in Konfigurationsdateien beschrieben. Genau das ist mit Infrastructure-as-Code gemeint, denn der Mandant kann den kompletten Aufbau vor der Ausführung im Repository prüfen.

- **Reproduzierbar.** Aus derselben Konfiguration entsteht stets dasselbe System, sodass sich über die Zeit kein Konfigurations-Drift einschleicht. Der Mandant kann die Installation auf eigener Hardware durchführen und erhält exakt dasselbe Ergebnis.
- **Atomare Updates und Rollbacks.** Jede Änderung erzeugt eine neue Generation. Schlägt ein Update fehl, kann beim nächsten Neustart eine ältere Generation aus dem Bootmenü ausgewählt werden. Das erhöht die Wartbarkeit deutlich.
- **Härtung an einer Stelle.** Alle Härtingsmaßnahmen sind zentral in wenigen Nix-Modulen (`security.nix`, `networking.nix`, `docker.nix`, `secureboot.nix`) zusammengefasst. Der Sicherheitsstand bleibt dadurch nachvollziehbar und überprüfbar, statt sich über viele manuelle Schritte zu verteilen.

Die Festplatte wird über das Modul `disko` ebenfalls deklarativ partitioniert. Zum Einsatz kommt Btrfs mit zstd-Kompression und getrennten Subvolumes für `/`, `/nix`, `/var`, `/opt` und `/tmp`. Das `/tmp`-Subvolume wird mit den Optionen `nosuid`, `nodev` und `noexec` eingebunden. Der Swap wird bei jedem Start mit einem zufälligen Schlüssel verschlüsselt, damit keine sensiblen Daten ungeschützt im Swap verbleiben.

Optional lässt sich die Root-Partition mit Linux Unified Key Setup (LUKS)² verschlüsseln. Standardmäßig ist diese Option deaktiviert, da angenommen wird, dass der gesamte Proxmox-Host vom Nutzer mit Festplattenverschlüsselung installiert wurde. Die zusätzliche Schutzschicht kann bei der Installation aktiviert werden. Die Umschaltung wirkt nur bei einer Neuinstallation, weil dafür die Festplatte neu partitioniert wird.

Warum Docker Compose

Die Dienste selbst laufen als Container und werden über Docker Compose orchestriert. Nach dem Verzicht auf Kubernetes ist das die naheliegende Wahl. Die Konfiguration ist kompakt, gut lesbar und für den Mandanten leicht nachvollziehbar. Der gesamte Stack wird über eine zentrale `docker-compose.yml` zusammengesetzt, die per `include` einzelne, thematisch gruppierte Compose-Dateien einbindet (zum Beispiel `media`,

documents, monitoring, cloud). Jede Datei bleibt dadurch überschaubar und einem klaren Zweck zugeordnet.

Netzwerksegmentierung

Statt alle Container in ein gemeinsames Netzwerk zu legen, erhält jede Dienstgruppe ihr eigenes Docker-Netzwerk. Eine Datenbank ist nur in dem Netz erreichbar, in dem auch die zugehörige Anwendung liegt. Diese Aufteilung begrenzt die laterale Bewegung eines Angreifers. Selbst nach der Kompromittierung eines einzelnen Dienstes ist von dort kein direkter Zugriff auf alle anderen Dienste möglich.

Reverse Proxy mit Traefik

Traefik ist der einzige Einstiegspunkt für Web-Verkehr. Er terminiert Transport Layer Security (TLS) und verteilt die Anfragen anhand der Subdomain an den jeweiligen Dienst. Die Zuordnung erfolgt über Container-Labels, die Traefik direkt aus Docker liest. Damit Traefik keinen vollen Zugriff auf den Docker-Socket benötigt, wird die Docker-API ausschließlich über einen eingeschränkten Socket-Proxy ausgelesen. Dieser lässt nur lesende Anfragen zu, sodass selbst eine Kompromittierung von Traefik keinen Vollzugriff auf das System ermöglicht.

Für den internen Zugriff über das Heimnetz wird eine eigene Domain genutzt (in den Tests `lbaecker.de`), deren DNS-A-Eintrag direkt auf die private Internet Protocol (IP)-Adresse des Servers im Heimnetzwerk zeigt. Von außen ist dieser Eintrag zwar öffentlich auflösbar, jedoch nicht erreichbar, da die private IP außerhalb des Heimnetzes nicht routbar ist. Angreifer können auf diesem Weg lediglich die interne IP-Adresse des Apphosts ermitteln. Diese geringfügige Informationspreisgabe stellt einen bewussten Tradeoff dar. Alternativ ließe sich ein statischer DNS-Eintrag im lokalen Resolver des Routers hinterlegen. Ein sicherheitsbewusster Nutzer wie der Mandant besitzt jedoch voraussichtlich Geräte, die standardmäßig auf sicheres DNS setzen und den via DHCP oder Router Advertisement verteilten DNS-Server ignorieren. Mit dem gewählten Ansatz lässt sich die IP-Adresse mit jedem DNS-Server auflösen. Ein zusätzlicher interner Eintrag im Router bleibt dennoch möglich.

Der Vorteil einer echten Domain liegt darin, dass ein öffentlich vertrauenswürdiges Zertifikat von Let's Encrypt bezogen werden kann und keine selbstsignierten Zertifikate erforderlich sind. Daraus ergeben sich mehrere praktische Vorteile. Zunächst treten keine Zertifikatswarnungen im Browser auf. Das ist auch sicherheitsrelevant, denn wer sich angewöhnt, Zertifikatswarnungen wegzuklicken, übersieht irgendwann auch eine echte Man-in-the-Middle-Warnung. Ein gültiges Zertifikat erhält diese Warnung als verlässliches Signal. Außerdem funktioniert die volle Vertrauenskette ohne manuelles Verteilen einer eigenen Certificate Authority (CA) auf jedes Gerät des Mandanten. Gerade bei Mobiltelefonen und bei Apps, die selbstsignierte Zertifikate strikt ablehnen, verursacht das sonst häufig Probleme. Schließlich ist die Verwaltung vollständig automatisiert, da Traefik die Zertifikate rechtzeitig selbst erneuert und somit keine abgelaufenen Zertifikate entstehen.

Die Zertifikate bezieht Traefik über Let's Encrypt mit der DNS-01-Challenge. Der große Vorteil dieser Methode besteht darin, dass der Server dafür nicht aus dem Internet erreichbar sein muss. Über die DNS-API des Anbieters (hier Cloudflare) wird lediglich kurzzeitig ein TXT-Eintrag gesetzt. Die Ports 80 und 443 können vollständig hinter dem Router verbleiben. An Cloudflare fließen dabei keine Inhalte, sondern nur der Nachweis der Kontrolle über die Domain.

Eine bewusste Ausnahme bildet der Tor-Zugang. Onion-Adressen lassen sich von Let's Encrypt nicht zertifizieren. Da die Verbindung über Tor bereits Ende-zu-Ende verschlüsselt ist, wird dort ein selbstsigniertes Zertifikat verwendet, was im Tor-Browser unkritisch ist. Auch wenn ein Zertifikat nicht zwingend notwendig wäre, bildet es eine zusätzliche Schutzschicht für den Fall, dass das Tor-Netzwerk kompromittiert wird oder Schwächen im Tor-Protokoll auftreten.

Die zentrale Zertifikatsverwaltung kommt auch Message Queuing Telemetry Transport (MQTT) zugute (hier Mosquitto). Der Broker wird nicht über einen eigenen TLS-Listener nach außen exponiert, sondern über einen zusätzlichen Transmission Control Protocol (TCP)-Endpoint von Traefik getunnelt. Traefik terminiert das TLS anhand des im TLS-ClientHello mitgesendeten Hostnamens (Server Name Indication (SNI)) mit demselben Let's-Encrypt-Zertifikat, das auch für den regulären Web-

Zugriff verwendet wird. Die entschlüsselten Daten werden innerhalb des Docker-Netzes an den internen, unverschlüsselten MQTT-Listener von Mosquitto weitergereicht. Auf dem Apphost muss damit kein separates Zertifikat für Mosquitto erzeugt, verteilt und erneuert werden. Alle Geräte des Mandanten vertrauen diesem öffentlich signierten Zertifikat in der Regel von Haus aus.

Single Sign-On mit Authelia

Authelia ist der zentrale Authentifizierungsdienst und schützt die Dienste, die keine ausreichend starke eigene Anmeldung mitbringen. Je nach Unterstützung des jeweiligen Dienstes kommen zwei Mechanismen zum Einsatz.

- **OIDC** für Dienste mit nativer Unterstützung, zum Beispiel Grafana, Immich, Paperless und Forgejo. Diese Dienste leiten zur Anmeldung an Authelia weiter und vertrauen anschließend dem ausgestellten Token. Die Benutzeridentität wird dadurch sauber in den Dienst übernommen.
- **Forward-Auth** für Dienste ohne brauchbare eigene Zugriffskontrolle, zum Beispiel Prometheus, Alertmanager und die Homepage. Traefik prüft dabei bei jeder Anfrage zuerst bei Authelia, ob der Nutzer angemeldet ist, bevor die Anfrage den Dienst erreicht.

Die Benutzer und ihre Passwörter liegen in einer Datei mit Argon2id-Hashes, die beim Setup automatisch generiert wird. Die zentrale Anmeldung ist damit deklarativ beschrieben und nicht in einer von außen nicht prüfbaren Datenbank abgelegt.

Zensurresistenter Zugriff über Tor

Auf ausdrücklichen Kundenwunsch ist ein Großteil der Dienste zusätzlich über das Tor-Netzwerk als Onion Service v3 erreichbar. Dafür läuft ein Tor-Container, der einen Onion Service bereitstellt und den Verkehr an einen eigenen Traefik-Einstiegspunkt weiterreicht. Der Mandant kann dadurch auch aus stark überwachten oder zensierten Netzen auf seine Dienste zugreifen. Metadaten fließen dabei nicht an den lokalen Provider

ab und es muss kein Port nach außen geöffnet werden. Der Tor-Container ist als reiner Client konfiguriert (kein Relay, kein Exit, deaktivierter Control- und SOCKets Secure (SOCKS)-Port).

Praktisch entsteht damit ein dualer Uniform Resource Locator (URL)-Raum. Im Heimnetz sind die Dienste unter `*.domain.tld` mit gültigem Zertifikat erreichbar. Über Tor sind sie unter `*.<onion-service>.onion` erreichbar, wobei das oben beschriebene zentrale selbstsignierte Wildcard-Zertifikat genutzt wird.

Die Tor-Erreichbarkeit lässt sich *pro Dienst* explizit ein- und ausschalten. Jeder Dienst erhält neben seinen normalen Traefik-Labels ein zusätzliches Set an Labels für den Tor-Einstiegspunkt. Ohne dieses Set ist der Dienst nicht über Tor erreichbar, während sich am internen Zugriff nichts ändert. Der Mandant kann somit gezielt steuern, welche Dienste anonym von außen verfügbar sind und welche bewusst im Heimnetz verbleiben. Collabora ist aus Sicherheitsgründen standardmäßig nicht über Tor freigegeben (siehe Abschnitt 1.3.4). Auch Garage hat bewusst keinen Tor-Einstiegspunkt, denn als S3-kompatibler Objektspeicher im Hintergrund der anderen Dienste hat es keine eigenen Endnutzer, die von außerhalb des Heimnetzes darauf zugreifen müssten.

Kompatibilität der Anwendungen und OIDC.

Nicht jede Anwendung kommt mit zwei verschiedenen URLs gleich gut zurecht. Manche verhalten sich völlig transparent. Home Assistant etwa orientiert sich am jeweils anfragenden Host und stellt sich automatisch auf die gerade verwendete URL ein, sodass der Wechsel zwischen internem Pfad und Tor dort unbemerkt funktioniert. Eine echte Einschränkung betrifft dagegen die Dienste, die sich über OIDC an Authelia anmelden. OIDC bindet die ausgestellten Tokens, die Redirect-Uniform Resource Identifiers (URIs) und vor allem die sogenannte Issuer-URL fest an eine einzige Basis-Adresse. Wird Authelia intern unter `auth.domain` betrieben, sind alle Tokens auf genau diese Adresse ausgestellt. Meldet sich derselbe Nutzer anschließend über die Tor-Adresse an, schlägt die Validierung fehl. Die Issuer-URL in der Antwort passt dann nicht mehr zu der Adresse, über die der Zugriff erfolgt. Ein unmittelbarer Wechsel zwischen internem Pfad und Tor-Pfad ist bei OIDC-geschützten Diensten

daher nicht möglich. Bei reinen Forward-Auth-Diensten (zum Beispiel Prometheus oder die Homepage) tritt das Problem nicht auf, weil dort kein Token an eine feste Issuer-URL gebunden wird. Es handelt sich um eine reine Komforteinschränkung und nicht um ein Sicherheitsproblem. Der Umgang damit und die ergänzende Empfehlung eines WireGuard-Zugangs werden in Abschnitt 1.3.4 erläutert.

Der Zusammenhang mit dem Hypertext Transfer Protocol (HTTP)-Host-Header ist direkt. Authelia bestimmt die Issuer-URL aus einer statisch konfigurierten Basis-Adresse, prüft aber bei jeder Abfrage, ob der Host-beziehungsweise X-Forwarded-Host-Header zu dieser passt. Greift ein Client über eine andere Adresse zu, etwa die Tor-Adresse, schlägt dieser Abgleich fehl. Selbst wenn der Abgleich umgangen werden könnte, bliebe das Problem bestehen. Der ausgestellte Identifikation (ID)-Token trüge weiterhin die falsche Issuer-URL und die Anwendung validiert sie gegen die hinterlegte Adresse. Eine Lösung über den Browser, etwa durch Header-Manipulation, ist daher nicht sinnvoll möglich. Würden die OIDC-Klienten Host-Header nicht strikt validieren, wären sie für Injection-Angriffe anfällig. Die feste Bindung ist folglich kein Mangel, sondern eine bewusste Sicherheitsfunktion von OIDC. Sie lässt sich nicht umgehen, ohne die Sicherheit des Gesamtsystems erheblich herabzusetzen.

Monitoring und Alerting

Für die Überwachung kommt der etablierte Stack aus Prometheus (Metriken), Loki (Logs), Grafana (Dashboards) und Alertmanager (Alarmierung) zum Einsatz. Node-Exporter und cAdvisor liefern Host- und Container-Metriken. Grafana Alloy sammelt die Logs ein, sowohl die Docker-Container-Logs als auch `syslog` und `auth.log` des Hosts, und schreibt sie nach Loki. Prometheus und Loki arbeiten rein intern. Nur Grafana, Prometheus und Alertmanager sind über den Reverse Proxy zugänglich und dort durch Authelia geschützt.

Dieser Stack stammt ursprünglich aus der Kubernetes-Phase, denn im cloud-nativen Umfeld ist die Kombination aus Prometheus und Grafana der De-facto-Standard für Monitoring. Beim Umbau auf Docker Compose konnte er ohne größere Anpassungen übernommen werden. Genau darin liegt einer seiner größten Vorzüge, denn die Komponenten sind nicht an

Kubernetes gebunden, sondern laufen ebenso als gewöhnliche Container. Das einmal erarbeitete Monitoring-Konzept musste daher nicht verworfen werden:

- **Pull-basiert und einfach.** Prometheus holt die Metriken selbstständig von den Exportern ab. Es muss nichts an einen externen Dienst übertragen werden und alle Daten bleiben auf dem Host.
- **Geringer Ressourcenbedarf.** Gerade Loki ist bewusst leichtgewichtig gehalten. Es indexiert nur Metadaten statt des vollen Textes wie etwa Elastic oder OpenSearch und passt damit gut zu einem einzelnen, sparsam ausgelegten Host.
- **Alles aus einer Oberfläche.** Grafana führt Metriken und Logs in einer einzigen Oberfläche zusammen, sodass der Mandant nicht mehrere Werkzeuge bedienen muss.
- **Flexible Alarmierung.** Der Alertmanager trennt die Auswertung der Alarme von deren Zustellung und lässt sich frei konfigurieren, in diesem Fall in Richtung des selbst gehosteten `ntfy`.

Besonders wichtig für das Bedrohungsmodell ist die Alarmierung. Die Aufgabe verlangt ausdrücklich, dass Push-Benachrichtigungen nicht über fremde Cloud-Dienste laufen, die Metadaten abgreifen könnten. Deshalb kommt das selbst gehostete `ntfy` zum Einsatz. Der Alertmanager schickt seine Alarme per Webhook an `ntfy`. Der Mandant empfängt sie sowohl am Desktop als auch auf seinem GrapheneOS-Smartphone. Da `ntfy` selbst gehostet ist, verlassen die Benachrichtigungen die eigene Infrastruktur nicht. Definierte Alarme betreffen unter anderem hohe Central Processing Unit (CPU)-Last, knappen Speicherplatz, ausgefallene Container und ein nicht erreichbares Backend hinter `Traefik`.

Absicherung der Container

Die Netzwerksegmentierung allein reicht nicht aus. Damit ein einzelner kompromittierter Dienst möglichst wenig Schaden anrichten kann, wird jeder Container nach demselben Grundmuster gehärtet. Die Einstellungen stehen direkt in den Compose-Dateien und sind damit für den Mandanten nachvollziehbar:

- **Keine neuen Privilegien.** Jeder Container läuft mit `no-new-privileges:true`. Ein Prozess im Container kann seine Rechte damit nicht nachträglich erhöhen, etwa über `setuid`-Binaries.
- **Capabilities auf null und gezielt zurück.** Mit `cap_drop: ALL` werden zunächst sämtliche Linux-Capabilities entfernt. Anschließend werden pro Dienst nur die wenigen wieder freigegeben, die er tatsächlich benötigt. Typischerweise sind das `SETUID` oder `SETGID`, wenn der Container intern den Benutzer wechselt.
- **Read-only Dateisystem.** Wo der Dienst es zulässt (etwa Traefik, Tor, Garage, Node-Exporter, Homepage, `ntfy` und Mosquitto), wird das Wurzeldateisystem des Containers schreibgeschützt eingebunden. Für die wenigen Stellen mit echtem Schreibbedarf dient ein kleines, nicht ausführbares `tmpfs` im Random Access Memory (RAM).
- **Ressourcenlimits.** Jeder Container besitzt CPU- und Speicherobergrenzen. Ein einzelner, möglicherweise kompromittierter Dienst kann den Host dadurch nicht lahmlegen.
- **Seccomp und AppArmor.** Es gilt das Standard-Seccomp-Profil von Docker, das gefährliche Syscalls blockiert. Auf Host-Ebene erzwingt AppArmor zusätzliche Mandatory-Access-Control-Profile.
- **User-Namespace-Remapping.** Auf Daemon-Ebene wird `root` im Container auf einen unprivilegierten Bereich auf dem Host abgebildet. Selbst wer im Container `root` wird, bleibt auf dem Host ein Benutzer ohne Rechte.

Nicht jeder Container folgt diesem Muster vollständig. Neben Collabora, das für das Rendern von Dokumenten erweiterte Rechte benötigt und im Abschnitt 1.3.4 behandelt wird, brauchen auch einige Monitoring-Komponenten und die Docker-Anbindung selbst gezielte Ausnahmen:

- **cAdvisor** läuft mit `privileged: true` und `userns_mode: host`, weil es zur Erhebung von Container-Metriken lesend auf `cgroups`, den Kernel-Ringpuffer (`/dev/kmsg`) und interne Docker-Zustände zugreifen muss. Ohne erweiterte Rechte sind diese nicht sichtbar.
- **Node-Exporter** läuft mit `Host-PID`-, `Host-Netzwerk`- und `Host-User-Namespace` (`pid: host, network_mode: host, userns_mode:`

`host`), weil er Prozesse und Netzwerkschnittstellen des Hosts selbst vermisst und nicht die eines isolierten Containers.

- **socket-proxy** läuft mit `userns_mode: host` und bindet den Docker-Socket read-only ein. Der Socket funktioniert ausschließlich mit den Host-User Identifiers (UIDs), mit denen der Docker-Daemon selbst läuft. Unter User-Namespace-Remapping wären die gemappten UIDs im Container nicht identisch mit denen, die der Socket erwartet. Traefik selbst erhält im Gegenzug keinen direkten Socket-Zugriff und spricht ausschließlich den lesebeschränkten Socket-Proxy über das Netzwerk an (siehe oben).
- **Grafana Alloy** bindet den Docker-Socket ebenfalls read-only ein, um Container-Logs einsammeln zu können. Es läuft dabei ohne `userns_mode: host` und mit vollständig entfernten Capabilities.

Alle diese Ausnahmen sind eng auf das jeweils benötigte Recht begrenzt und betreffen bewusst nur Monitoring- und Infrastrukturkomponenten. Diese verarbeiten keine Nutzdaten des Mandanten, sondern beobachten Host- beziehungsweise Container-Zustand oder leiten Anfragen weiter. Die produktiven Dienste des Mandanten bleiben vom vollen Härtungsmuster erfasst.

Ergänzt wird die Container-Härtung durch automatische Sicherheitsscans. Ein geplanter Job prüft die laufenden Images wöchentlich (montags um 02:00 Uhr) mit Trivy auf bekannte Schwachstellen der Stufen HIGH und CRITICAL und legt den Bericht unter `/var/log/docker-security-scan.log` ab. Zusammen mit den durch RenovateBot laufend aktuell gehaltenen Images entsteht ein kontinuierlicher Kreislauf. Neue Schwachstellen werden zeitnah erkannt und das passende Update steht meist bereits als Pull Request bereit.

Härtung von Webanfragen mit Traefik

Weil Traefik der einzige Einstiegspunkt für Web-Verkehr ist, werden dort auch die Härtungsmaßnahmen gebündelt, die für den Hypertext Transfer Protocol Secure (HTTPS)-Verkehr aller Dienste gelten. Statt Security-Header, TLS-Einstellungen und Zugriffsschutz in mehr als zwanzig einzelnen Diensten separat zu pflegen, werden sie zentral als Traefik-

Middleware definiert und über sogenannte Chains an die jeweiligen Router gebunden. Ein neuer Dienst erbt damit automatisch den vollen Schutz, sobald er die passende Kette an Middlewares referenziert.

- **TLS-Konfiguration nach Bundesamt für Sicherheit in der Informationstechnik (BSI) Technische Richtlinie (TR)-02102-2.** Aktuell gilt für sämtliche Entrypoints ausschließlich das `default-TLS`-Profil mit TLS 1.3. Daneben liegt ein zweites, ebenfalls konformes Profil (`intermediate`) vor. Es erlaubt TLS 1.2 nur mit den vom BSI empfohlenen Authenticated Encryption with Associated Data (AEAD)-Cipher-Suites (AES-GCM, ChaCha20-Poly1305) sowie den Kurven X25519, P-256 und P-384 als Fallback. Da kein Router explizit auf `tls.options=intermediate` verweist, bleibt dieses Fallback-Profil derzeit ungenutzt und alle Clients müssen TLS 1.3 sprechen. Bei Bedarf lässt es sich für einzelne Router aktivieren, etwa falls ein Altgerät des Mandanten kein TLS 1.3 unterstützt. Die Option `sniStrict` sorgt in beiden Profilen dafür, dass Anfragen ohne passenden Hostnamen abgewiesen werden, statt auf ein Default-Zertifikat auszuweichen.
- **Erzwungenes HTTPS und HTTP Strict Transport Security (HSTS).** Der HTTP-Entrypoint leitet jede Anfrage permanent auf HTTPS um. Zusätzlich setzt Traefik einen HSTS-Header mit zwei Jahren Gültigkeit und Preload-Flag für alle Subdomains, damit Browser gar nicht erst versuchen, sich unverschlüsselt zu verbinden. Browser sind in den letzten Jahren zwar auf ein HTTPS-by-default-Modell gewechselt, HSTS erzwingt HTTPS jedoch dauerhaft. Das Eintragen in die Preload-Listen der Browser erfordert einen öffentlich erreichbaren Webserver und einen Antrag auf hstspreload.org. Das Preload-Flag hat daher momentan keine Wirkung und stellt eine Vorbereitung dar.
- **Security-Header für alle Dienste.** Über die Middleware `security-headers` setzt Traefik sicherheitsrelevante Header wie `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options: nosniff`, eine restriktive Referrer- und Permissions-Policy (deaktiviert unter anderem Kamera, Mikrofon und Standort) sowie `Cross-Origin-Opener`-, `Embedder`- und

Resource-Policy. Gleichzeitig werden **Server-** und **X-Powered-By-**Header geleert, damit Angreifer aus einer Antwort nicht ableiten können, welche Software in welcher Version dahintersteht. Ein **X-Robots-Tag: noindex** verringert außerdem das Risiko, dass interne Dienste in Suchmaschinen erscheinen.

- **Schutz vor gefälschten Auth-Headern.** Dienste wie Grafana oder Paperless vertrauen im Onion-Pfad einem **Remote-User-**Header, den Authelia nach erfolgreicher Prüfung setzt. Über das Clearnet darf dieser Header niemals vom Client selbst stammen. Die Middleware **strip-remote-user** entfernt deshalb auf allen normalen Routern jeden vom Client mitgeschickten **Remote-***-Header, bevor die Anfrage den Dienst erreicht. Niemand kann sich somit durch einen gefälschten Header als anderer Benutzer ausgeben.
- **Auswertbare Logs.** Das Access-Log läuft im JavaScript Object Notation (JSON)-Format und wird an Loki weitergereicht. Sensible Header wie **Authorization** und **Cookie** werden dabei grundsätzlich verworfen. **User-Agent** und **X-Forwarded-For** bleiben für mögliche zukünftige forensische Auswertungen erhalten.

Diese Maßnahmen ergänzen die oben beschriebene Container-Härtung von Traefik selbst, unter anderem **cap_drop: ALL**, ein read-only Dateisystem und den Zugriff auf die Docker-API ausschließlich über den Socket-Proxy.

Umgang mit Geheimnissen

Alle Geheimnisse (Passwörter, Token, Schlüssel) liegen in einer **.env**-Datei und einem **secrets/-**Verzeichnis. Beide werden nicht in Git eingchecked. Aus den Klartextwerten der **.env** erzeugen kleine Skripte die tatsächlich benötigten gehashten Werte, etwa die Argon2id-Hashes für Authelia, die bcrypt-Hashes für ntfy, die Passwortdatei für Mosquitto sowie die **rpcauth**-Hashes für den Bitcoin-Core-Node des Hot-Wallet-Stacks. Das Verzeichnis **secrets/** ist auf Dateisystemebene strikt geschützt und nur für den **apphost**-Benutzer zugänglich. Ändert der Mandant ein Passwort, generiert er die Hashes mit einem einzigen Befehl neu.

Kontrollierte Updates mit Renovate

Ein wiederkehrendes Problem beim Self-Hosting ist, dass Container-Images veralten und so unbemerkt Sicherheitslücken offen bleiben. Um das zu verhindern, überwacht RenovateBot das Repository und öffnet automatisch einen Pull Request, sobald für ein Image eine neue Version verfügbar ist. Daraus ergibt sich ein kontrollierter und nachvollziehbarer Update-Prozess.

- Jedes Update kommt als einzelner, dokumentierter Pull Request und geschieht nicht unbemerkt im Hintergrund.
- Unkritische Updates für den Monitoring-Stack werden automatisch zusammengefasst und können automatisch gemerged werden.
- Sicherheitskritische Komponenten wie Datenbanken oder Traefik werden bewusst nicht automatisch gemerged, sondern erst nach einer Wartezeit und manueller Prüfung übernommen.
- Major-Updates mit möglichen Breaking Changes warten länger und werden stets von Hand geprüft.

Die genannte Wartezeit wird über die Renovate-Option *minimumReleaseAge* umgesetzt. Für Major-Updates beträgt sie 14 Tage und für das exponierte Traefik-Image 7 Tage. Renovate öffnet den Update-Pull-Request zwar sofort, markiert den zugehörigen Status-Check aber als ausstehend, bis die konfigurierte Zeit seit dem Release verstrichen ist. Der Sicherheitsgewinn dieser Verzögerung ist beträchtlich, denn wer ein Release sofort übernimmt, bringt ungeprüften Code direkt in Produktion. Kompromittierte Releases werden in der Praxis meist innerhalb von Stunden bis wenigen Tagen von der Community entdeckt und vom jeweiligen Registry-Betreiber zurückgezogen. Eine kurze Wartezeit fängt sie daher in aller Regel ab. Ein anschauliches Beispiel lieferte der Vorfall um das NPM-Paket **ua-parser-js** im Oktober 2021. Nach der Übernahme des Maintainer-Accounts wurden drei böartige Versionen veröffentlicht, die auf Windows- und Linux-Systemen Zugangsdaten stahlen und einen Krypto-Miner nachluden [23].

Nach dem Mergen eines Pull Requests genügt auf dem Server ein einziger Befehl, um die neuen Images zu ziehen und die Container neu zu starten.

Ergänzt wird das durch einen wöchentlichen Schwachstellen-Scan der laufenden Images mit Trivy und die tägliche automatische Aktualisierung des NixOS-Systems. Die Angriffsfläche bleibt so dauerhaft klein, ohne dass manuell nach neuen Versionen gesucht werden muss.

Hash-Pinning der Container-Images

Ein reiner Versions-Tag wie `traefik:v3.7.1` ist kein unveränderlicher Bezeichner. Er kann nachträglich auf einen anderen Image-Inhalt umgebogen werden, sei es versehentlich oder durch Kompromittierung. Für den Reproduzierbarkeitsanspruch, der bereits für NixOS formuliert wurde (siehe Abschnitt 1.3.3), stellt das einen Bruch dar. Dieselbe `docker-compose.yml` könnte zu unterschiedlichen Zeitpunkten unterschiedliche Images ziehen, ohne dass sich am Tag etwas ändert.

Die Image-Referenzen werden deshalb zusätzlich zum Tag um den SHA-256-Digest des jeweiligen Manifests ergänzt, also zum Beispiel `traefik:v3.7.1@sha256:<digest>`. Der Digest ist inhaltsadressiert und zeigt exakt auf einen einzigen, unveränderlichen Layer-Stand. Ein `docker compose pull` liefert damit garantiert dasselbe Image, das zum Zeitpunkt des jeweiligen Commits vorlag, unabhängig davon, worauf der Tag im Upstream-Repository später zeigt.

Damit dieses Pinning keinen zusätzlichen manuellen Pflegeaufwand verursacht, übernimmt RenovateBot (siehe Abschnitt 1.3.3) auch diesen Teil. Renovate ergänzt für jedes Image automatisch den passenden Digest. Bei jeder neuen Version und jedem neuen Digest desselben Tags, etwa nach einem erneuten Build desselben Upstream-Releases, öffnet es einen entsprechenden Pull Request. Reproduzierbarkeit und automatisch gepflegte Updates schließen sich damit nicht aus, sondern ergänzen einander.

Übersicht der Dienste

Insgesamt stellt der Apphost deutlich mehr als die geforderten 15 Dienste bereit. Das Spektrum reicht von Medien (Jellyfin, Immich) über Dokumente (Paperless, Bento-Portable Document Format (PDF)), Cloud und Office (OpenCloud, Collabora), Hausautomation (Home Assistant, Mosquitto), Kommunikation (ntfy, Bichon) und Entwicklung (Forgejo) bis zu

Werkzeugen wie dem Passwortmanager Vaultwarden und einer Startseite (Homepage). Zusätzlich läuft auf dem Apphost der Hot-Wallet-Stack für den automatisierten Bitcoin-Zahlungsverkehr, der in Abschnitt 1.4 gesondert beschrieben wird. Die vollständige Liste mit Subdomains und Beschreibungen findet sich im Anhang.

1.3.4 Abdeckung des Bedrohungsmodells

Die Aufgabenstellung gibt drei zentrale Angriffsvektoren vor. Im Folgenden wird für jeden davon gezeigt, mit welchen Maßnahmen er adressiert wird. Eine ausführliche Auflistung aller einzelnen Einstellungen findet sich im Anhang.

Remote-Angriffe

Hierunter fallen Phishing, Brute-Force, kompromittierte Endgeräte und Supply-Chain-Angriffe auf Abhängigkeiten.

- **Keine offenen Ports.** Der Web-Zugriff erfolgt nur über das Heimnetz und über Tor und die Zertifikate werden per DNS-01 bezogen. Es muss daher kein einziger Port ins Internet geöffnet werden, was klassischen Remote-Angriffen von vornherein die Angriffsfläche nimmt. Auch intern werden zusätzlich lediglich Ports für SSH, HTTP, HTTPS und MQTT benötigt.
- **Starke, zentrale Authentifizierung.** Authelia mit Argon2id und Multi-Faktor-Authentifizierung schützt die Dienste und bringt einen eigenen Brute-Force-Schutz auf Anwendungsebene mit (siehe Abschnitt 1.3.3). Ergänzend sperrt Fail2ban nach wenigen Fehlversuchen sowohl bei SSH als auch bei fehlgeschlagenen Web-Logins auf Netzwerkebene und verlängert die Sperrzeit progressiv. Die Firewall blockiert eingehenden Verkehr standardmäßig und erlaubt nur das Nötigste. SSH ist zusätzlich rate-limitiert.
- **Supply-Chain.** RenovateBot, der wöchentliche Trivy-Scan und die täglichen NixOS-Updates sorgen dafür, dass bekannte Schwachstellen schnell geschlossen werden. Wird dennoch ein Container kompromittiert, greift die in Abschnitt 1.3.3 beschriebene Container-

Härtung. Entfernte Capabilities, `no-new-privileges`, read-only Dateisysteme, das Seccomp-Profil, AppArmor und vor allem das User-Namespace-Remapping begrenzen den möglichen Schaden, denn root im Container ist nicht root auf dem Host. Die Netzwerksegmentierung (siehe Abschnitt 1.3.3) verhindert zusätzlich, dass sich ein Angreifer von einem Dienst aus frei weiterbewegen kann.

Lokale Angriffe (BadUSB, Einklinken ins Netz)

Die Aufgabe stellt klar, dass das Aufschrauben des Servers out-of-scope ist. Das Anstecken bössartiger USB-Geräte und das unautorisierte Einklinken ins Netzwerk müssen dagegen abgewehrt werden.

- **USB-Angriffe.** Die Kernel-Module für USB-Speicher (`usb-storage`, `uas`) und weitere nicht benötigte Hardware-Module sind per Blacklist deaktiviert. Auch auf dem Proxmox-Host werden USB-Eingaben gesperrt. Ein angestecktes BadUSB-Gerät findet damit keinen direkten Angriffsweg.
- **Verifizierter Boot.** Secure Boot über `lanzaboote` stellt sicher, dass nur signierte Boot-Komponenten starten. Zusammen mit erzwungener Kernel-Modul-Signatur (`module.sig_enforce`) und dem Kernel-Lockdown wird verhindert, dass ein lokaler Angreifer manipulierten Code einschleust.
- **Direct Memory Access (DMA)-Schutz.** Die Input-Output Memory Management Unit (IOMMU) wird erzwungen und früher Peripheral Component Interconnect (PCI)-DMA-Zugriff blockiert. Angriffe über direkten Speicherzugriff werden dadurch erschwert.
- **Netzwerk.** Wer sich unautorisiert ins Netz einklinkt, trifft auf eine Default-Deny-Firewall und auf Dienste, die hinter Authelia liegen. Ein bloßer Netzwerkzugang genügt somit nicht, um an die Dienste zu gelangen.

Metadaten-Leaks

Damit keine sensiblen Nutzungs- oder Standortdaten an den Provider abfließen, setzt das System an mehreren Stellen an:

- **Tor.** Der Zugriff über Onion Services verbirgt sowohl die Identität des Servers als auch die des Nutzers vor dem lokalen Provider.
- **Verschlüsseltes DNS.** Der Host nutzt DNS-over-TLS mit Domain Name System Security Extensions (DNSSEC)-Validierung gegen Resolver ohne Logging-Policy (Cloudflare und Quad9). DNS-Anfragen landen dadurch nicht im Klartext beim Provider.
- **Authentifizierte Zeit.** Die Zeitsynchronisation läuft über Network Time Security (NTS) und damit authentifiziert über TLS.
- **Keine fremde Cloud für Benachrichtigungen.** Push-Benachrichtigungen laufen, wie beschrieben, ausschließlich über das selbst gehostete ntfy.

Eingegangene Kompromisse

Kein reales Setup ist frei von Kompromissen. Deren offene Benennung gehört zum Entwicklungsprozess und ist für diese Ausarbeitung von hoher Relevanz. Die folgenden Punkte behandeln daher die bewusst eingegangenen Kompromisse.

- **Ein Host statt vieler VMs.** Der Aufbau bündelt viele Dienste auf einem Host. Dieser ist damit ein attraktiveres Ziel, als es eine einzelne Mini-VM wäre. Das erscheint vertretbar, weil die Isolation über Docker-Netzwerke und User-Namespaces sowie eine sehr umfangreiche Host-Härtung erfolgt. Der Gewinn an Wartbarkeit für den Mandanten überwiegt diesen Unterschied.
- **OIDC und der duale URL-Raum.** Dienste mit OIDC funktionieren nicht gleichzeitig über die interne Domain und über Tor, weil OIDC die Tokens fest an eine Basis-URL bindet. Die Mechanik dahinter wurde im Abschnitt 1.3.3 erläutert. Es handelt sich um eine reine Komforteinschränkung und nicht um ein Sicherheitsproblem. Als Komfortverbesserung wird dem Mandanten ergänzend ein WireGuard-Zugang empfohlen. Damit sind die internen Adressen von überall über denselben Pfad erreichbar, der URL-Wechsel entfällt und die OIDC-Anmeldung funktioniert ortsunabhängig. Tor bleibt als zusätzlicher anonymer Zugangsweg davon unberührt.

- **Collabora.** Der Office-Dienst benötigt für das Rendern von Dokumenten erweiterte Rechte, unter anderem die Capability `SYS_ADMIN` sowie deaktivierte Seccomp- und AppArmor-Profile. Unter den für den Mandanten erreichbaren, nutzerdatenverarbeitenden Diensten ist das der am wenigsten eingeschränkte Container im Stack. Die in Abschnitt 1.3.3 genannten Ausnahmen bei Monitoring und Docker-Anbindung betreffen dagegen reine Infrastrukturkomponenten ohne direkten Netzwerkzugriff von außen. Dieser Zustand wird akzeptiert, weil die Funktion sonst nicht läuft. Das Risiko wird über die Netzwerksegmentierung und die Anmeldung an OpenCloud begrenzt. Aus demselben Grund ist Collabora standardmäßig nicht über Tor erreichbar. In den letzten Monaten hat sich Europäische Union (EU)-Office als (EU-)Fork von OnlyOffice etabliert. Die Wahl fiel dennoch bewusst auf Collabora, weil es in der Open-Source-Community etabliert ist und eine große Nutzerbasis hat. EU-Office ist noch sehr jung und erreicht bislang nicht dieselbe Reife und Stabilität. Die Entwicklung wird beobachtet und ein Umstieg bleibt für die Zukunft möglich.
- **Kata Containers und gVisor.** Beide zusätzlichen Sandbox-Run-times sind vorbereitet, aber standardmäßig nicht aktiv (siehe Abschnitt A3.6).

1.3.5 Backups und Wiederherstellung

Für die Sicherung wird bewusst die in Proxmox integrierte Backup-Funktion genutzt. Proxmox sichert komplette VMs als konsistente Snapshots und kann diese regelmäßig und vollautomatisch wegschreiben. Ausschlaggebend für diese Entscheidung ist die Einfachheit für den Mandanten. Da der gesamte Apphost in einer einzigen VM liegt, sichert ein VM-Backup automatisch den konsistenten Gesamtzustand aller Dienste auf einmal. Es kann somit keine Situation entstehen, in der nach einer Wiederherstellung die Datenbank eines Dienstes nicht mehr zu dessen Dateien passt. Gerade für einen Anwender ohne tiefes Informationstechnik (IT)-Wissen ist es ein großer Vorteil, mit einem Klick wieder einen funktionierenden Stand zu erhalten.

Damit entsteht eine gewisse Nähe zu Proxmox. Dieser Lock-in ist je-

doch gering. Das Backup-Format ist offen und gut dokumentiert. Die Sicherungen sind durch Kompression und Deduplizierung effizient und lassen sich verschlüsseln. Die Daten sind dadurch auch dann geschützt, wenn ein Backup-Medium gestohlen wird. Für die Wiederherstellung nach einem Hardwaredefekt genügt es, Proxmox neu aufzusetzen und die VM aus dem letzten Backup zurückzuspielen.

Dateisystemintegrität mit AIDE

Advanced Intrusion Detection Environment (AIDE) prüft die Integrität des Dateisystems. Dazu legt es Prüfsummen und Metadaten (Rechte, Eigentümer, Timestamps) von zuvor konfigurierten Dateien und Verzeichnissen in einer Datenbank ab und vergleicht sie bei jedem weiteren Lauf mit dem aktuellen Zustand.

Ziel ist es, unautorisierte Veränderungen schnell sichtbar zu machen. Beispiele sind eine eingeschleuste Backdoor in einer Binary, eine manipulierte Konfigurationsdatei oder leicht zu übersehende Änderungen wie veränderte Set User ID (SUID)-Bits. Zugrunde liegt die Annahme, dass ein Angreifer einige Schutzmechanismen bereits überwunden und Spuren seiner Kompromittierung auf dem Dateisystem hinterlassen hat. Diese Spuren lassen sich über AIDE auffinden. AIDE läuft automatisiert über einen systemd-Timer, das moderne Äquivalent zu einem Cronjob. Die Ergebnisse müssen jedoch manuell geprüft werden.

1.3.6 Sicherheitshärtung im Überblick

Das System kombiniert eine große Zahl einzelner Härtungsmaßnahmen, die alle deklarativ in den NixOS-Modulen festgelegt sind. Hier folgt ein geordneter Überblick, während die vollständige Liste mit konkreten Werten im Anhang steht.

- **Boot und Firmware.** Zum Einsatz kommen Secure Boot per lanzaboote, ein eingebundenes Trusted Platform Module (TPM), ein deaktivierter Bootloader-Editor, erzwungene Kernel-Modul-Signaturen und der Kernel-Lockdown.
- **Kernel.** Aktiviert sind die CPU-Mitigationen (Spectre, Meltdown,

MDS und weitere), vollständiges Address Space Layout Randomization (ASLR), init-on-alloc und init-on-free, eine erzwungene IOMMU und ein eingeschränkter Zugriff auf Kernel-Informationen. Hinzu kommt eine an den Center for Internet Security (CIS)-Benchmark angelehnte Modul-Blacklist.

- **Netzwerk.** nftables arbeitet mit Default-Deny sowie rate-limitiertem SSH und Internet Control Message Protocol (ICMP), ergänzt um gehärtete Sysctl-Einstellungen. Zusätzlich sind DNS-over-TLS mit DNSSEC und authentifizierte Zeit über NTS aktiv.
- **Zugriff.** SSH funktioniert ausschließlich schlüsselbasiert mit modernen und post-quanten-fähigen Verfahren. Ergänzend existieren ein gehärtetes sudo mit I/O-Logging, eine unveränderliche Benutzerdatenbank, ein gesperrtes Root-Konto und AppArmor als Mandatory Access Control.
- **Audit und Intrusion Detection.** Der Linux Audit Daemon erfasst sicherheitsrelevante Ereignisse. Fail2ban arbeitet mit progressivem Banning und AIDE prüft täglich die Dateisystemintegrität.
- **Container.** Abgesichert wird über User-Namespace-Remapping, den nur über einen eingeschränkten Socket-Proxy möglichen Zugriff auf die Docker-API, entfernte Capabilities, read-only Dateisysteme wo möglich und einen wöchentlichen CVE-Scan.
- **Updates.** Das System aktualisiert sich täglich über NixOS-Auto-Upgrade. RenovateBot pflegt die Container inklusive Digest-Pinning für reproduzierbare Image-Referenzen (siehe Abschnitt 1.3.3) und der Nix-Store wird regelmäßig aufgeräumt.

1.3.7 Fazit

Der Apphost bildet den zentralen Einstiegspunkt für den Mandanten und das Rückgrat für sämtliche Dienste. Auf ihm laufen nicht nur die Alltags- und Produktivanwendungen, sondern auch der Hot-Wallet-Stack des BTC-Zahlungsverkehrs (siehe Abschnitt 1.4). Damit vereint eine einzige, deklarativ beschriebene und umfassend gehärtete Maschine die gesamte für den Mandanten sichtbare Infrastruktur. Die durchgängige

Beschreibung als Code macht den Aufbau vor der Ausführung prüfbar und nach einem Ausfall reproduzierbar wiederherstellbar. Aus Sicht des Mandanten entsteht so ein System, das sich an einer Stelle betreiben, überwachen und warten lässt, ohne dass die Absicherung der einzelnen Dienste darunter leidet.

1.4 Krypto

Die Krypto-Architektur ergibt sich aus der Anforderung, BTC Transaction (TX) automatisiert durchführen zu können und gleichzeitig ein hohes Sicherheitsniveau zu gewährleisten. Die besondere Schwierigkeit besteht darin, dass das Hot-Wallet direkt auf das Internet zugreifen können muss und dementsprechend als internet-exponiertes System einem erhöhten Risiko durch Remote-Angriffe ausgesetzt ist. Dieses gilt es ausreichend vor Kompromittierung zu schützen, da in eben diesem Fall potenziell unmittelbar auf kryptographische Schlüssel zugegriffen und der Wallet-Inhalt transferiert werden könnte. Im Zuge dessen wird es ebenfalls von den oben beschriebenen Services vor den aus dem Gefahrenmodell bekannten Angriffen geschützt (siehe 1.3.4) und weist so bereits eine erhöhte technische Sicherheit auf. Um diesen Gedanken fortzuführen, wurde sich für eine besonders sichere konzeptionelle und architektonische Ausarbeitung entschieden. Dementsprechend wurde ein Hot-/Cold-Wallet-Architekturmodell gewählt, wie es im institutionellen und betrieblichen Umfeld als Best Practice eingesetzt wird [24], [25], [26]. Dieses Modell kombiniert operative Verfügbarkeit mit Schlüsseltrennung und stellt die nötige Sicherheitsfunktion zur Anforderung des Mandanten dar.

Bevor die einzelnen Komponenten im Detail betrachtet werden, sei das Gesamtszenario kurz umrissen: Ein Mandant benötigt eine selbstgehostete Infrastruktur, die eingehende Zahlungsaufträge automatisiert als Bitcoin-Transaktionen ausführt, ohne dass kryptographisches Schlüsselmaterial einem internet-exponierten System dauerhaft ausgeliefert ist. Die Lösung teilt sich daher in drei logisch getrennte Zonen, die ebenfalls in Abbildung 1.1 nachvollzogen werden können und im produktiven Setup physisch zu trennen sind.

- ein permanent online betriebenes *Hot-Wallet* (Apphost) für den automatisierten Zahlungsverkehr
- eine ebenfalls online, aber gehärtete *Signer-VM* (Key A), die das Hot-Schlüsselmaterial isoliert hält
- ein vollständig vom Netz getrenntes, air-gapped *Cold-Wallet* (Key B/Key C) als 2-aus-3-Multi-Signatur-Tresor

1.4.1 Hot- und Cold-Wallet Architektur

Im Sinne einer klassischen Hot- und Cold-Wallet-Architektur hält das Hot-Wallet lediglich einen begrenzten Anteil der monetären Mittel (ca. 5 %) [25], [26], während der überwiegende Teil der Mittel im Cold-Wallet unter höheren Sicherheitsvorkehrungen verwahrt wird. Dieses Modell reduziert den potenziellen Schaden bei einer Kompromittierung des Online-Systems erheblich, da nur ein kleiner Teil des Gesamtwertes extrahiert werden kann (siehe Abbildung 1.1).

Partially Signed Bitcoin Transaction (PSBT)

Als Bitcoin Improvement Proposal (BIP) 174 wurde das PSBT-Format als Standard veröffentlicht und bildet die Grundlage des gesamten Signierprozesses. Ein BIP ist ein standardisiertes Dokument des offenen Bitcoin-Entwicklungsprozesses, über das technische Erweiterungen, Formate und Konventionen vorgeschlagen, diskutiert und bei Annahme durch die Community als einheitlicher Standard etabliert werden. BIPs sind fortlaufend nummeriert und stellen sicher, dass unabhängige Implementierungen (Wallets, Nodes, Bibliotheken) interoperabel bleiben.

BIP 174 beschreibt ein einheitliches Austauschformat für eine noch unvollständig signierte TX, das alle zur Prüfung und Signierung nötigen Informationen wie Inputs, Unspent Transaction Outputs (UTXOs), Ableitungspfade und Outputs mitführt, ohne private Schlüssel zu enthalten. Dies ermöglicht die Realisierung unseres Ansatzes, eine TX über mehrere getrennte Systeme wie Hot-Signer und air-gapped Cold-Signer hinweg schrittweise zu signieren, ohne dass Schlüsselmaterial die jeweilige Zone verlässt. Da alle beteiligten Werkzeuge denselben BIP-174-Standard umsetzen, ist die PSBT über den gesamten Weg interoperabel verarbeitbar.

Die in BIP 174 definierten Rollen wie Creator, Updater, Signer, Combiner, Input Finalizer und Transaction Extractor beschreiben logische Verarbeitungsschritte und nicht zwingend getrennte Akteure. Die Spezifikation sieht ausdrücklich vor, dass eine einzelne Instanz mehrere Rollen gleichzeitig übernehmen darf [27]. Tabelle 1.1 zeigt die Zuordnung in der vorliegenden Architektur. Entsprechend ist es zulässig, dass das Hot-System eine PSBT nicht nur erstellt, sondern unmittelbar auch teilweise signiert, bevor sie an weitere Signierer der Cold-Wallet übergeben wird.

1 Self Hosted Setup

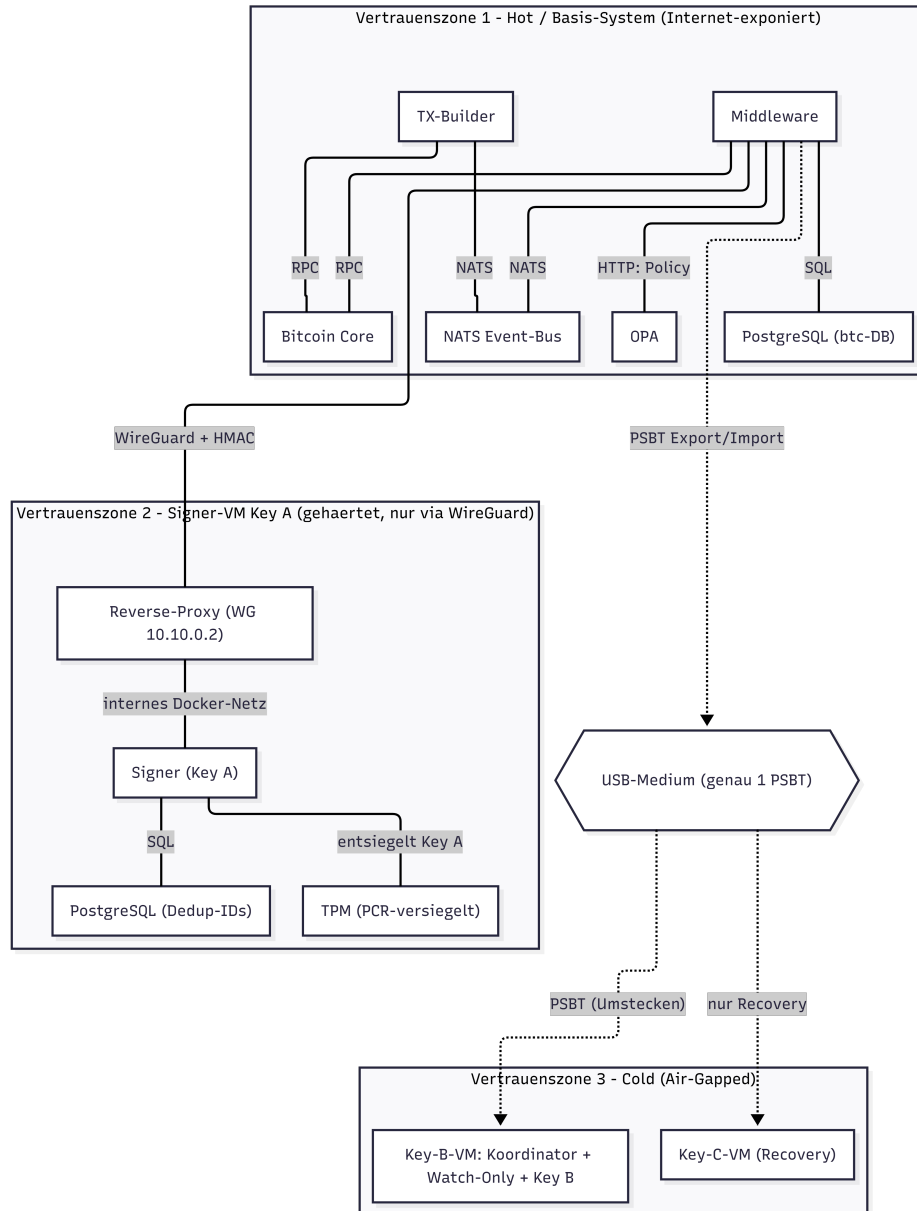


Abbildung 1.1: Darstellung der PSBT-Vertrauenszonen als Hardwarekomponenten und ihrer gehosteten Services. Quelle: Eigene Darstellung.

BIP-174-Rolle	Hot-Pfad	Cold-Pfad
Creator / Updater	Bitcoin Core (<code>walletcreatefundedpsbt</code>), angestoßen durch den TX-Builder	wie Hot-Pfad
Signer	Key A (Signer-VM, automatisch)	Key A (1/3, automatisch), Key B/Key C (Sparrow, manuell)
Combiner	entfällt (nur eine Signatur)	Sparrow Wallet (Cold-VM)
Input Finalizer	Bitcoin Core (<code>finalizepsbt</code>)	Sparrow Wallet
Transaction Extractor	Bitcoin Core (Apphost)	Sparrow (<code>.txn</code> -Export), Broadcast über den Apphost

Tabelle 1.1: Zuordnung der BIP-174-Rollen zu den Systemkomponenten

Durch dieses Vorgehen kann die Authentizität der Transaktion bereits vor der Koordination im Cold-Wallet nachgewiesen werden. Dies zeigt, dass sie vom Apphost verifiziert und nicht im Transit modifiziert wurde. Die nachgelagerten Cold-Signaturen durch Key B und Key C ergänzen diese initiale Signatur gemäß dem PSBT-Prozess [27], [28], [29].

Durch die ausschließliche Nutzung von PSBTs wird sichergestellt, dass sensibles Schlüsselmaterial das air-gapped System niemals verlässt und keine Signaturschritte außerhalb der expliziten Benutzerinteraktion erfolgen [27], [30], [31].

Zur Gewährleistung einer erhöhten Sicherheit wurde zudem ein Multi-Signatur-Verfahren für das Cold-Wallet etabliert, das sich Schlüssel A als Single Key mit dem Hot-Wallet teilt (siehe Abbildung 1.11). Dies erhöht die Sicherheit des Cold-Wallets drastisch und bildet einen Kompromiss zur Operationalität des Systems.

Abbildung 1.1 stellt diese Aufteilung als Vertrauenszonen dar. Jede Hardware-Komponente bestehend aus Hot-Wallet-Apphost, Signierer-VM und den beiden Cold-VMs bildet eine eigene Zone mit den darauf gehosteten Diensten. Die Abstufung kennzeichnet das Expositionsniveau, von der stark exponierten Online-Zone des Hot-Wallets über die gehärtete, nur per WireGuard (WG) erreichbare Signierer-Zone bis zur

1 Self Hosted Setup

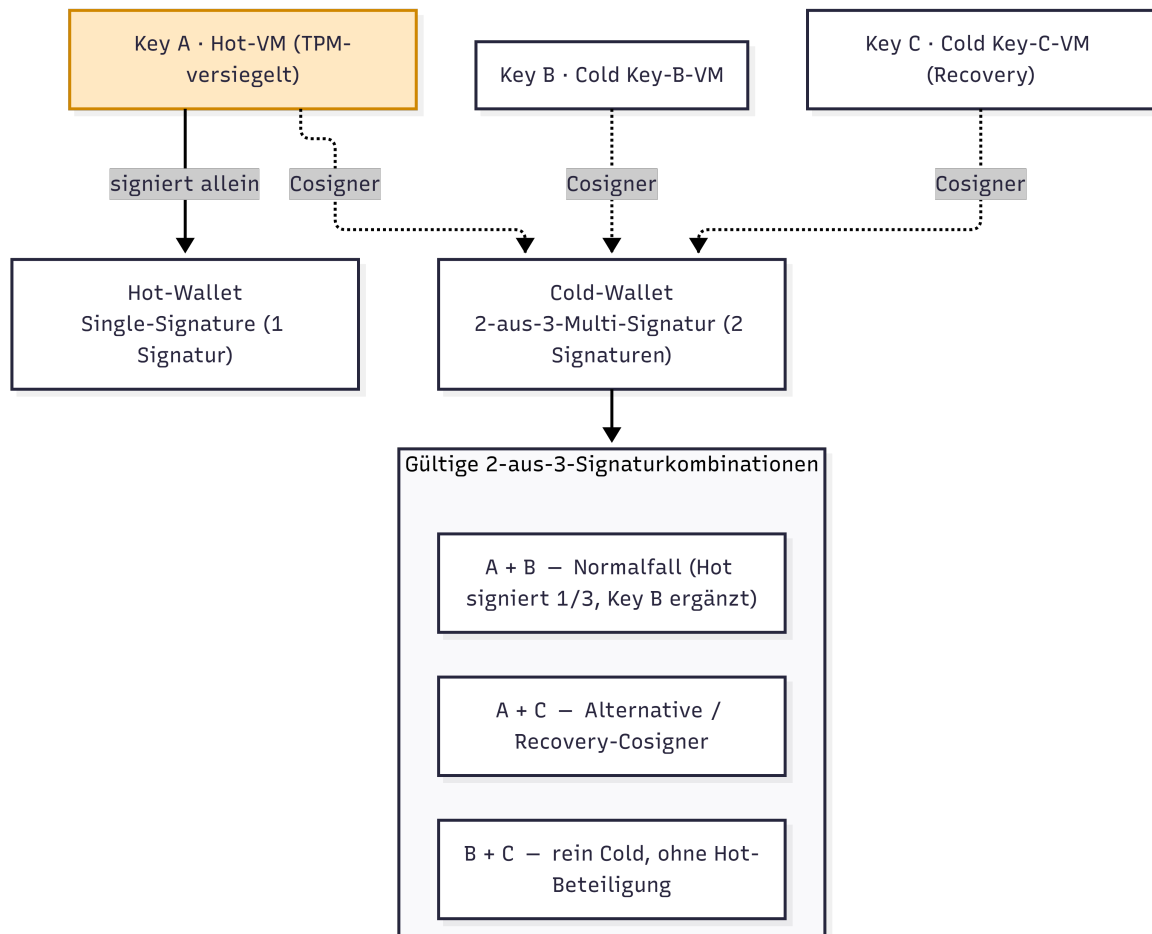


Abbildung 1.2: Darstellung der Schlüsselverwendung im Signierprozess.
Quelle: Eigene Darstellung.

vollständig isolierten, air-gapped Cold-Zone. Zwischen den Zonen werden ausschließlich PSBTs ausgetauscht, niemals Schlüsselmaterial.

Während das Hot-Wallet für automatisierte Transaktionen genutzt wird, erfolgen Transfers aus dem Cold-Wallet ausschließlich kontrolliert und bewusst nicht vollautomatisiert mit manueller Freigabe [24], [30], [32], [33]. Da sich Hot- und Cold-Wallet einen Schlüssel teilen, kann der Cold-Wallet-Prozess bis zur Benachrichtigung des Mandanten parallel zum Hot-Wallet teilautomatisiert ablaufen (siehe Abbildung 1.2).

NixOS

Deklarative Systemhärtung

Die Schlüsselhalter-VMs werden über NixOS deklarativ gehärtet, um eine Basis Sicherheit gegen Angreifer gewährleisten zu können. Die exakt getroffenen Maßnahmen können im Anhang in der Tabelle A5.1 nachvollzogen werden.

Die schlüsselhaltende Hot-VM (Key A) wird analog gehärtet, ist jedoch nicht vollständig air-gapped, sondern ausschließlich über einen einzelnen, per WireGuard authentifizierten User Datagram Protocol (UDP)-Port erreichbar. Sämtlicher übriger Verkehr wird per `nftables` verworfen.

Warnung bei der NixOS-Installation

Bei der Installation von NixOS werden die folgenden Warnmeldungen ausgegeben:

- „Mount point `’/boot’` which backs the random seed file is world accessible, which is a security hole! ...“
- „Random seed file `’/boot/loader/./#bootctlrandom-seed...’` is world accessible, which is a security hole! ...“

Diese rühren daher, dass die gesetzten Berechtigungen für die vfat-Boot-Partition nicht erkannt werden. Diese Warnung wurde recherchiert, ist der Community bekannt und wird als fälschlich ausgelöst und damit unbedrohlich eingestuft [34].

Dockerhärtung Wie bereits in Abschnitt 1.3.3 erwähnt werden gewisse Härtungen für die Docker-Container im Apphost vorgenommen. Diese wurden ebenfalls für die Härtung der separaten Signierer VM von Key A vorgenommen und schließen sich so dem Sicherheitskonzept an.

1.4.2 Hot-Wallet

Da das Hot-Wallet dauerhaft mit dem Internet verbunden ist, stellt es die am stärksten exponierte Komponente der Architektur dar. Entsprechend wird dieses System besonders gehärtet und folgt strikt dem Prinzip der *Segregation of Duties*. Ziel ist es, zu verhindern, dass ein einzelner Dienst TXs eigenständig aufbauen, autorisieren und signieren kann [25], [35].

Aufgrund dessen wurde die Hot-Bitcoin-Architektur in einzelne Microservices unterteilt, wobei jeder Service eigene Sicherheitsanker und Autorisierungen besitzt. Dadurch wird sichergestellt, dass die Kompromittierung eines einzelnen Services nicht zu einer vollständigen Übernahme des Systems führt. Die Infrastruktur teilt sich in Microservices für die direkte TX-Verarbeitung und unterstützende Komponenten:

Microservices zur Transaktionsverarbeitung

- Bitcoin Core (Blockchain-Zustand und Transaktionsvalidierung),
- Middleware (Orchestrierung und interne und externe Kommunikation),
- Open Policy Agent (OPA) (Deklarative Policy-Engine, die TXs anhand von Regeln autorisiert (im Detail siehe 1.4.2)),
- TX-Builder (Transaktionskonstruktion),
- Signierer.

Unterstützende Komponenten

- PostgreSQL (Persistenz von Wallet- und Transaktionsdaten),
- Neural Autonomic Transport System (NATS) (Event-basierte Kommunikation zwischen Services),
- WireGuard (verschlüsselte Kommunikation zwischen Systemen),

- ntfy (Push-Benachrichtigungen an den Operator, siehe 1.3.3),
- Loki / Prometheus (Logging und Monitoring).

Unterstützende Komponenten

Als unterstützende Komponenten wird die Basis-Infrastruktur bezeichnet, die nicht speziell auf BTC-TXs zugeschnitten ist und so auch von den anderen Services des Self-Hosted-Setups verwendet werden kann beziehungsweise wird.

Wie bei den deklarativ gehärteten Schlüsselhalter-VMs (siehe Tabelle A5.1) kommt auch im Hot-Wallet für Key A NixOS zum Einsatz, wodurch eine Vielzahl an Sicherheitsvorkehrungen direkt deklarativ im Betriebssystem vorgenommen werden kann.

Bitcoin Core Node

Bitcoin Core wird verwendet, um eine wesentliche Vereinfachung von Bitcoin-Operationen zu erreichen. Betrieben als Full Node ohne aktivierte Wallet-Funktion übernimmt dieser Service sowohl die Validierung von Blöcken und Transaktionen, die Mempool-Verwaltung als auch Gebührenschatzungen, besitzt jedoch keine kryptographischen Schlüssel [36], [37]. Stattdessen können Wallets direkt auf Bitcoin Core durch Import von internen und externen öffentlichen Deskriptoren registriert werden, ohne dass geheimes Schlüsselmaterial preisgegeben werden muss. Anhand dieser Information kann der Service die Adressen- und PSBT-Generierung für die Bitcoin-Wallet übernehmen.[38]

Im produktiven Umfeld würde Bitcoin Core als Node mit dem Mainnet verbunden, um reale TXs und UTXOs zu verfolgen. Zur Entwicklung und Validierung der Architektur simuliert die Node zudem ein lokales „Regtest“-Netzwerk. Dieses bietet volle Kontrolle über die Blockchain und stellt ein isoliertes, privates Testnetz ohne Verbindung zu externen Peers dar. Coins können dementsprechend lokal erzeugt werden und verursachen keine Kosten. Für die spätere produktive Nutzung durch den Mandanten ist eine Umstellung von Regtest auf das Mainnet sowohl in Bitcoin Core als auch in der angebundenen Sparrow Wallet erforderlich. Die vollständigen Migrationsschritte sind in 1.4.4 beschrieben.

Die Nutzung der automatischen Blockchain-Verwaltung ersetzt einen zuvor evaluierten Ansatz auf Basis eines ZeroMQ (ZMQ)-Listeners. Dieser würde Ereignisse für ausgewählte Wallets überwachen und könnte eine Referenz der UTXOs in deren Besitz extrahieren. Diese Lösung würde eine separate persistierende Logik erfordern, sodass gefundene UTXOs in eine weitere Datenbank geschrieben werden müssten. Eine besondere Herausforderung stellt dabei die Blockchain-Reorganisation dar, bei der bereits verarbeitete Blöcke einer sogenannten Fork verworfen und bei der Reintegration in die Blockchain durch alternative Ketten ersetzt werden können. Eine korrekte Behandlung dieser Problematik hätte eine komplexe und fehleranfällige Logik mit Betrachtung verschiedener Edge Cases erfordert. Daneben gestaltet sich das Berechnen der Gebühren einer TX ebenfalls als sehr aufwendig, da diese dynamisch auf Basis von Mempool-Zuständen, Blockhöhe, Chain-Tips und Netzwerk-Auslastung über mehrere Berechnungen stabilisiert werden müssten.

Durch die Nutzung von Bitcoin Core können diese Aspekte vollständig ausgelagert werden. Insbesondere ermöglicht die Kombination aus Deskriptor-Import und dem Remote Procedure Call (RPC)-Befehl `walletcreatefundedpsbt` eine einfache Erstellung von Transaktionen [39].

Zudem erlaubt Bitcoin Core, dynamisch generierte Zieladressen einem registrierten Wallet zuzuordnen, ohne auf statische Adressen angewiesen zu sein, was die Problematik aus 1.4.3 auflöst. Auf diese Weise wird die Nachvollziehbarkeit von TXs sichergestellt, während gleichzeitig datenschutzrelevante Anforderungen des Mandanten erfüllt werden. Dies stellt einen relevanten Aspekt zur Umsetzung der vollautomatischen Partner-Kommunikation dar. Ankommende Anfragen müssen vor Ausführung autorisiert werden, wobei verschiedene Möglichkeiten bestehen. Eine lässt sich in einer engen Absprache mit dem Partner durch den Austausch eines gemeinsamen Geheimnisses (siehe 1.4.6) oder in der Verwendung von statischen Adressen finden. Eine einfachere und ebenfalls sichere Möglichkeit wird durch das Vertrauen der Partner-Wallets in Form eines Whitelistings dieser dargestellt. Dabei kann Bitcoin Core diese Wallets über den Import des externen Deskriptors als eigene Wallet registrieren und anschließend die Adresszugehörigkeit zu einer bestimmten Wallet mit `getaddressinfo` verifizieren und so freigeben.

Ein zusätzlicher privater Electrum-Server wurde aufgrund des erhöhten Betriebsaufwands und des begrenzten Mehrwerts innerhalb des Projektumfangs, wie bereits in 1.4.5 erwähnt, nicht implementiert.

PostgreSQL

Wie bereits beschrieben, wird PostgreSQL als Datenbank verwendet, um verschiedene Daten über System- oder Docker-Neustarts hinaus zu persistieren. Hierzu wird eine dedizierte Datenbank **btc** betrieben, die alle Wallet- und PSBT-Informationen umfasst. Gespeichert werden:

- Wallet-Metadaten für Hot-, Cold- und whitelisted externe Wallets,
- PSBT-Daten und Lebenszykluszustände,
- Entscheidungsdaten aus der OPA-Policy-Auswertung,
- archivierte Transaktionen nach erfolgreichem Broadcast.

Der Zugriff auf die **btc**-Datenbank (DB) wird ausschließlich über den Middleware-Microservice administriert, um eine einzige Source of Truth darstellen zu können. Dementsprechend werden auf diesem Service verschiedene APIs zum Adressieren der Datenbank bereitgestellt.

Neben den Hot- und Cold-Wallets werden die Namen externer, whitelisted Wallets in der Wallet-DB eingetragen. Dafür wurden die Skripte `wallet_import.sh` und `whiteWallet.sh` eingerichtet, sodass relevante Informationen wie Registrierungs-Name, Schlüssel-Fingerprint und Deskriptoren für die einzelnen Wallets kontrolliert aus dem Dateiverzeichnis (oder einem USB-Medium im Falle eines stand-alone Ansatzes des Krypto Sub-Systems) extrahiert werden können. Die Registrierung selbst erfolgt nicht über einen HTTP-Endpunkt, sondern ereignisgesteuert über NATS, da sich eine nicht-internet exponierte Schnittstelle als sicherer erweist. Die Setup-Identität veröffentlicht `wallet.import.requested`, woraufhin die Middleware das Wallet in Bitcoin Core und PostgreSQL einträgt und den Ausgang über `wallet.import.done` bzw. `wallet.import.failed` zurückmeldet.

Zusätzlich zu den für den Betrieb notwendigen Daten werden ebenfalls der vollständige Statusverlauf jeder PSBT sowie die Entscheidungen des OPA samt Begründung und Input gespeichert, und die PSBT wird

1 Self Hosted Setup

nach dem Broadcast als Artefakt archiviert. Nach dem Broadcast am Ende des PSBT-Prozesses wird die verwendete PSBT im Sinne einer Archivierung in der Tabelle `btc.psbt_archive` gespeichert. Die Tabelle `btc.psbt`, welche die einzelnen Status nach den Microservice-Schritten enthält, soll dementsprechend eher als Runbook zum Nachvollziehen des operationellen Verlaufs der TX verstanden werden und kann nach einer gewissen Zeit gelöscht werden. Der vollständige Lebenszyklus einer PSBT mit allen Zustandsübergängen ist in Abbildung 1.3 grafisch dargestellt, während die exakten Beschreibungen der einzelnen Zustände im Anhang in Tabelle A5.2 nachgelesen werden können.

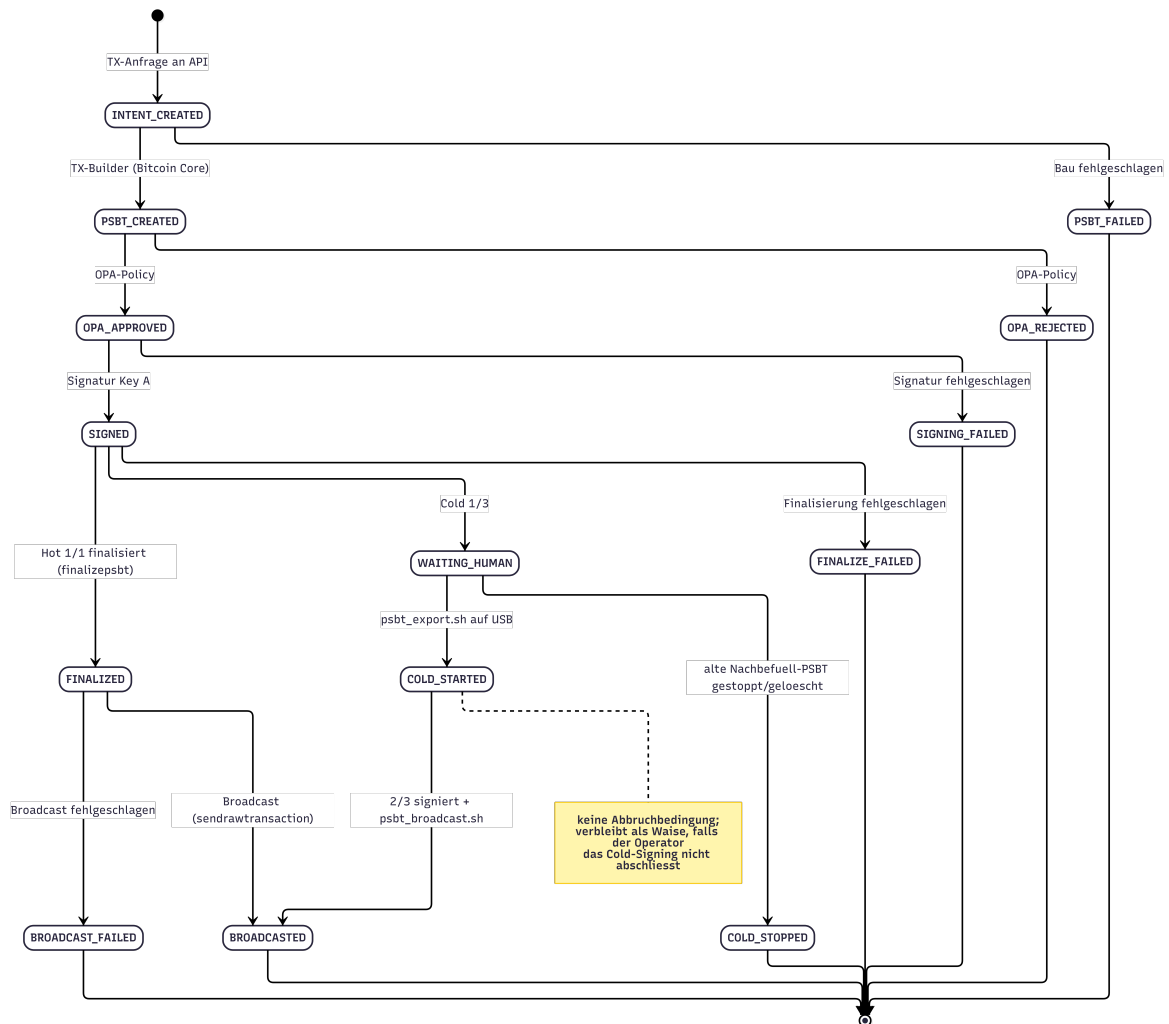


Abbildung 1.3: PSBT-Lebenszyklus und Statusübergänge. Quelle: Eigene Darstellung.

NATS

Die Event-basierte Kommunikation zwischen den Microservices wird über den Messaging-Service NATS realisiert. NATS implementiert ein Publish-Subscribe-Modell, bei dem Nachrichten auf sogenannten Subjects veröffentlicht und von mehreren Empfängern empfangen werden können. Durch eine besonders geringe Latenz eignet sich NATS gut für Echtzeitkommunikation, wie sie für unseren Anwendungsfall beispielsweise zur Wahrung der Aktualität von Gebühren einer TX benötigt wird.

Um die Eskalationsmöglichkeit bei Kompromittierung eines einzelnen Dienstes weiter zu verringern, wurde sich für die Implementation eines Autorisierungs-Mechanismus entschieden. Andernfalls könnte ein Angreifer aus einem Docker-Container heraus NATS-Nachrichten fälschen und damit andere Services in Form eines Lateral Movement korrumpieren. Als zusätzliche Härtung erhält jeder Microservice eine eigene, dedizierte NATS-Identität mit einer explizit definierten Publish- und Subscribe-Berechtigung (siehe Tabelle 1.2), die exakt der im Applikationscode bereits bestehenden Verantwortungstrennung entspricht. Die zulässigen Subjects je Dienst ergeben sich unmittelbar aus dem Nachrichtenfluss des Systems.

Service	Publish	Subscribe
Middleware	intent.created, psbt.build.requested, psbt.created, wallet.import.done, wallet.import.failed	intent.created, psbt.created, psbt.failed, refill.export.done, refill.broadcast.requested, psbt.submit.requested, wallet.import.requested
TX-Builder	psbt.created, psbt.failed	psbt.build.requested
Operator	refill.export.done, refill.broadcast.requested, psbt.submit.requested	
Setup	wallet.import.requested	wallet.import.done, wallet.import.failed

Tabelle 1.2: Subject-Berechtigungen je Microservice

Entscheidend ist dabei nicht die vollständige Symmetrie der Matrix,

sondern die gezielte Einschränkung des nachgelagerten Dienstes: Der TX-Builder erhält ausschließlich das Recht, Ergebnis-Events zu veröffentlichen und Bau-Aufträge zu empfangen. Das Veröffentlichen von `intent.created` ist ihm verwehrt. Eine Kompromittierung des TX-Builders bleibt dadurch auf das Einschleusen fehlerhafter PSBTs beschränkt. Dieser Angriffsvektor, der durch die nachgelagerte OPA-Autorisierung und die Whitelist-Prüfung der Zieladresse ohnehin abgefangen wird, kann den Prozess nicht zusätzlich an seinem Ursprung manipulieren.

Die Zugangsdaten der einzelnen NATS-Identitäten werden, analog zu den bereits beschriebenen Datenbank-Zugangsdaten, nicht statisch in der versionierten Konfiguration hinterlegt. Stattdessen werden sie bei der Initialisierung der Key A VM erzeugt und in einer restriktiv berechtigten Laufzeitdatei außerhalb der Versionskontrolle gehalten, auf die ausschließlich die beteiligten lokalen Prozesse Zugriff besitzen. In Verbindung mit der Netzwerkisolation, durch welche der NATS-Server keinen nach außen gerichteten Port besitzt, werden so sowohl ein direkter externer Zugriff als auch eine dienstübergreifende Rechteausweitung über den Event-Bus ausgeschlossen [40].

Eine weitergehende Ausbaustufe besteht in der Umstellung von der statischen, passwortbasierten Autorisierung auf ein dezentrales, auf NKeys und JSON Web Token basierendes Account-Modell. Dieses würde eine kryptografisch verankerte Identität je Dienst sowie eine signierte Rechtevergabe ermöglichen und die Verwaltung der Berechtigungen von der Server-Konfiguration entkoppeln. Aufgrund des erhöhten Verwaltungsaufwands und des für den aktuellen Projektumfang bereits hinreichenden Schutzniveaus der statischen Variante wurde diese Ausbaustufe als optionale Erweiterung eingestuft.

In Kombination mit NATS wird HTTP als API für gezielte GET-Abfragen zwischen Containern verwendet, wenn Informationen für den weiteren Prozessverlauf abgefragt statt Ereignisse ausgelöst werden. Die Transportwege sind dabei klar getrennt, sodass über HTTP ausschließlich die externen Eingänge (BIP21/PSBT, siehe Tabelle 1.3), der Signier-Aufruf an die Key A VM (`POST /sign`) sowie die Health- und Metrics-Endpunkte laufen. Alle betreiberausgelösten internen Abläufe wie die manuelle PSBT-Einreichung, der Abschluss des Wechselmedium-Exports

und der Broadcast der signierten Cold-PSBT werden hingegen über NATS unter der Operator-Identität ausgelöst (`psbt.submit.requested`, `refill.export.done`, `refill.broadcast.requested`) und besitzen bewusst keinen nach außen gerichteten HTTP-Endpunkt. Eine frühere HTTP-Variante dieser Operator-Aufrufe wurde zugunsten dieser einheitlichen, über die NATS-Berechtigungen abgesicherten Ereignissteuerung entfernt.

Netzwerkanbindung

Netzwerkseitig fügt sich das Hot-Wallet in die Segmentierung des Apphosts ein (siehe Abschnitt 1.3.3). Dabei laufen Sämtliche Microservices in einem eigenen, rein internen Docker-Netzwerk (**finance-network**) ohne veröffentlichte Ports. Bitcoin Core, PostgreSQL, NATS, OPA und der TX-Builder sind dadurch weder aus dem Heimnetz noch von anderen Dienstgruppen des Apphosts erreichbar. Einzig die Middleware ist zusätzlich an zwei weitere Netzwerke angebunden und schickt Daten an das **communication-network**, um Alarme über den bestehenden ntfy-Dienst zuzustellen. Sie empfängt Zahlungsanfragen an ihrer API unter einer eigenen Subdomain des reverse Proxy von dem **traefik-public**-Netzwerk. Anders als die übrigen Web-Dienste des Apphosts wird die Zahlungs-API bewusst nur über Clearnet (eigene Subdomain, gültiges TLS-Zertifikat) und nicht als Onion-Service exponiert (siehe Abschnitt 1.3.3, da anbindende Partner-Systeme ein stabilen, zertifikatsgesicherten HTTPS-Endpoint erwarten und der Tor-Zugang im Wesentlichen dem Betreiber-Zugriff auf die Verwaltungsdienste dient).

WG

WG beziehungsweise sein gebildeter Tunnel stellt den einzigen Kommunikationspfad zwischen dem Hot-Apphost (Middleware) und der Signierer-VM (Key A) dar, wodurch die Signierer-VM von keinem anderen System erreichbar ist. Übertragen werden ausschließlich die zu signierenden PSBTs vom Apphost zur Signierer-VM und zurückgegeben werden die signierten PSBTs. Das geheime Schlüsselmaterial wird nicht ausgetauscht und verlässt zu keinem Zeitpunkt die VM von Key A (siehe 1.4.2).

Der Tunnel bildet damit eine eigenständige Authentifizierungs- und

Verschlüsselungsschicht für die API-Verbindung und erfüllt die Aufgaben:

- Absicherung der Datenübertragung durch Transportverschlüsselung zwischen den beiden Peers,
- Einschränkung der Kommunikationspfade auf definierte, kryptographisch identifizierte Peers,
- Reduktion der Angriffsfläche durch Blockierung aller nicht notwendigen Ports.

Die Verbindung basiert auf einem schlanken UDP-Protokoll und nutzt moderne kryptographische Verfahren zur Authentifizierung und Schlüsselvereinbarung. Diese Konfiguration erfolgt zum einen deklarativ per oneShot über NixOS und zum anderen als automatisiertes Skript, sodass reproduzierbare und gehärtete Netzwerkkonfigurationen ermöglicht werden. Für das besonders kritische Schlüsselmaterial auf NixOS können so alle Ports außer dem UDP-Port für den WireGuard-Handshake und -Datenpakete blockiert werden [41].

Die Peer-Einrichtung und der HMAC-Austausch erfolgen über beiliegende Setup-Skripte auf beiden Systemen. Deren Verwendung ist im README der Signierer-VM (`Hot-KeyHolder/`) sowie in der Installationsanleitung des Apphosts dokumentiert [42], [43]. Eine Kurzübersicht aller Skripte bietet Tabelle A5.4 im Anhang.

Ergänzend zur Tunnel-Verschlüsselung wird jede Anfrage an die Signierer-VM mit einem Hash-based Message Authentication Code (HMAC) über ein gemeinsames Geheimnis signiert. Dieses HMAC-Geheimnis wird einmalig über das Wechselmedium ausgetauscht und bildet eine zweite, von WireGuard unabhängige Schicht, die Authentizität und Integrität jedes einzelnen Signierauftrags nachweist (siehe 1.4.2). Es genügt somit nicht, in den Tunnel zu gelangen, denn ohne Kenntnis des HMAC-Geheimnisses wird eine Anfrage abgewiesen.

Monitoring

Ein zentrales Monitoring überwacht den vollständigen TX-Lifecycle sowie den Zustand des Hot-Wallets. Erfasst werden unter anderem Wallet-Balance, ausstehende Transaktionen und definierte Überweisungs-

entscheidungen, persistiert in den PostgreSQL-Tabellen des Systems (siehe 1.4.2). Eine eigene Monitoring-Infrastruktur betreibt der Krypto-Stack dabei bewusst nicht, sondern schließt sich an den im Apphost-Kapitel beschriebenen Stack aus Prometheus, Loki und Grafana an (siehe 1.3.3). Dazu exponiert die Middleware unter `/metrics` Lifecycle-Metriken im Prometheus-Format und stellt unter anderem Zähler für eingehende Intents, PSBT-Bau, OPA-Entscheidungen, Whitelist- und Limit-Ablehnungen sowie Signier- und Broadcast-Ergebnisse bereit, die der zentrale Prometheus pull-basiert abholt. Sämtliche Services schreiben zudem strukturierte JSON-Logs, die von Grafana Alloy eingesammelt und in Loki abgelegt werden, sodass Metriken und Logs in Grafana gemeinsam auswertbar sind. Auffälligkeiten oder Grenzwertverletzungen werden automatisiert gemeldet und ermöglichen eine schnelle operationelle Reaktion. Für deren Zustellung wird ebenfalls der dort beschriebene, selbst gehostete ntfy-Dienst mitgenutzt.

Ebenfalls wird Sparrow als Watch-Only-Graphical User Interface (GUI) integriert, um einem menschlichen Operator das Verfolgen der TXs und Wertgegenstände zu ermöglichen [44].

Microservices zur Transaktionsverarbeitung

Middleware

Die Kommunikation und Koordination der einzelnen Services erfolgt über eine dedizierte Orchestrierungskomponente, welche den gesamten Transaktionsprozess steuert (siehe Abbildung 1.1, wobei der Ablauf selbst in 1.4.4 beschrieben wird). Die Middleware stellt neben dem NATS-Event-Manager sowohl interne als auch externe Schnittstellen bereit und übernimmt die direkte Kommunikation über HTTP und NATS. Ihre Aufgaben sind:

- Entgegennahme externer und operatorseitiger Eingänge (Übersicht in Tabelle 1.3),
- interne Schreib- und Lesebefehle für die PostgreSQL-DB,
- Weiterleitung von Events über NATS an interne Services,
- Import und Verteilung von Wallet-Metadaten,

- Kommunikation mit der Signer-VM über eine abgesicherte HTTP-Schnittstelle.

Der Empfang einer externen Anfrage startet den vom Mandanten geforderten automatisierten Transaktionsprozess (siehe 1.4.4). Der Umfang der implementierten Schnittstellen aus der realen Welt beschränkt sich bewusst auf BIP21-URIs sowie PSBT-basierte Anfragen. Erweiterungen wie das Lightning Network oder Smart-Contract-basierte Ansätze wurden als Erweiterungen des Bitcoin-Netzwerks und damit als außerhalb des aktuellen Anspruchs des Mandanten betrachtet. Zudem ist deren Integration nicht mit einem Hot-/Cold-Custody System vereinbar, da eine Lightning-Node ein permanent online signierfähiges System, dessen Channel-Schlüssel sich nicht hinter einem isolierten Signierer-VM befindet, erzwingt. Dementsprechend würde diese Architektur den gewählten Ansatz ersetzen und dem Sicherheitsbestreben des Mandanten widersprechen. Weitere Schnittstellen zur Kommunikation mit externen Partnern liegen im Ermessen des Mandanten und können durch Transformation der Eingabe und Auslösen des korrespondierenden NATS-Events implementiert werden. Evaluiert und nicht umgesetzt wurden Payjoin (BIP 78), Silent Payments (BIP 352) und das BIP 70-Payment-Protocol. Die ersten beiden stellen sich in unserem Kontext als reale Ergänzungen dar und sind mit der allgemeinen Architektur vereinbar. Aufgrund der geringen Anwendungsrate im realen Umfeld, wurde diese jedoch nicht umgesetzt. Das BIP 70-Payment-Protocol gilt inzwischen als veraltet und wurde aufgrund von Sicherheits- und Datenschutzbedenken bereits in vielen Anwendungen entfernt.

Für die bereits integrierten Schnittstellen gilt:

- BIP21-Anfragen durchlaufen den vollständigen Prozess inklusive PSBT-Erstellung,
- bereits vorliegende PSBTs werden direkt im nächsten Schritt durch OPA evaluiert.

Nach dem Auslösen der BIP21-Schnittstelle wird geprüft, ob der Partner eine Identifikation über das Code-Wort `id` mitgeliefert hat. Anschließend wird in der Datenbank abgefragt, ob ein Intent mit dieser ID bereits bekannt ist. Dadurch werden Duplikate, beispielsweise als Resultat einer

Race Condition, vermieden und die Stabilität des Systems durch eine eindeutige Identifikation der Partner-Anfragen erhöht. Zu beachten ist ebenfalls, dass mit den unterschiedlichen Partnern eine hinreichende Nomenklatur ausgehandelt werden muss, sodass diese weder eigene noch fremde ID-Räume wiederverwenden. Sollte keine ID festgelegt sein, wird eine neue, in der DB noch unvergebene UUIDv4 ausgewählt und als API-Antwort zurückgegeben [45].

Jegliche Statusveränderung im Prozess wird in der PSBT-DB geloggt und kann darüber nachvollzogen werden. Zudem wird die Entscheidung des OPA samt Begründung und Eingabe separat in `btc.opa_decision` persistiert.

Insgesamt lassen sich die Auslöser einer Transaktion in vier Rails einteilen: die beiden externen HTTP-Eingänge BIP21 und PSBT, den über NATS ausgelösten manuellen Operator-Pfad `manual` sowie die intern durch OPA erzeugten Austausch-Transaktionen `OPA_hot` und `OPA_cold` (siehe Tabelle 1.3).

Rail	Auslöser	Whitelist	OPA Betrag/Fee	Daily-Cap
BIP21	HTTP <code>/api/v1/request/bip21</code>	ext	ja	ja
psbt	HTTP <code>/api/v1/request/psbt</code>	ext	ja	ja
<code>manual</code>	NATS <code>psbt.submit.requested</code> (Operator, fertige PSBT)	Bypass	übersprungen	ja
<code>OPA_hot/OPA_cold</code>	interne Tauschlogik	interne Wallets	übersprungen	–

Tabelle 1.3: Transaktions-Rails und ihre Prüfstufen

Operator-/Manual-Pfad

Der in Tabelle 1.3 aufgeführte manuelle Operator-Pfad erlaubt es dem Operator, eine eigene, bereits fertig erstellte PSBT einzureichen. Es wird empfohlen, diese in einer weiteren Sparrow-Instanz mit dem Hot-Schlüssel als Watch-Only-Wallet zu bauen. Eingereicht wird sie über das Skript `scripts/hotwallet/ops/psbt_submit.sh`. Ausgelöst wird der Pfad nicht über HTTP, sondern über das NATS-Subject `psbt.submit`

`.requested` unter der dedizierten Operator-Identität. Die eingereichte PSBT wird unmittelbar als `psbt.created` in den Prüf- und Signierfluss übergeben und überspringt dabei den TX-Builder, da kein Aufbau der TX mehr erforderlich ist.

TX-Builder

Nach vollständiger Intent-Entgegennahme an der HTTP-Schnittstelle wird dieser zum Bauen der PSBT an den TX-Builder-Service weitergeleitet. Wie bereits erwähnt, war ein vorheriger Ansatz, die PSBT direkt im Code selbst zu konstruieren. Dies wurde durch das Extrahieren von UTXO-Informationen über API-Calls auf die DB und über mehrere Stabilisierungs-Durchläufe für die Gebühren ermöglicht.

Dies wurde direkt ersetzt durch die Methode `walletcreatefundedpsbt` [39], welche automatisiert

- die Auswahl geeigneter UTXOs,
- die Erstellung von Change-Adressen,
- die automatische Gebührenschatzung,
- die Validierung der verfügbaren Mittel

übernimmt.

Für die Standard-Hot-Wallet-PSBT-Erstellung überlässt das System die Gebührenwahl bewusst Bitcoin Core. Statt einer statischen Fee-Rate wird über `conf_target` eine Bestätigung innerhalb von circa sechs Blöcken (≈ 1 Stunde) im sparsamen `economical`-Modus angestrebt und die resultierende Gebühr wird anschließend durch OPA validiert (siehe 1.4.2). Watch-Only-UTXOs und BIP32-Ableitungspfade werden in die PSBT aufgenommen, da die Signatur erst auf der Key A VM erfolgt. Replace-by-Fee [46] bleibt für reguläre Zahlungen deaktiviert und wird ausschließlich für interne Umschichtungen gesetzt, deren zügige Bestätigung systemkritisch ist. Sämtliche Vorgaben können vom Mandanten in der Applikationslogik angepasst werden.

Für Nachbefüllungs- und Sicherungs-Intents bestimmt der OPA Bestätigungsziel und Gebühren-Modus dynamisch auf Basis eines Risk Scores.

Wie dieser Score aus dem Wallet-Saldo bestimmt wird und welche konkreten PSBT-Parameter er festlegt, zeigt Tabelle 1.4.

Open Policy Agent

Die autorisierende Entscheidung über Transaktionen erfolgt zentral über den Open Policy Agent. OPA evaluiert Transaktionsanfragen anhand konfigurierbarer Regeln wie Betragslimits, Gebühren und dem Vorhandensein benötigter Eingaben. Die Policy-Logik und die Limits sind dabei vollständig von der Applikationslogik entkoppelt und können vom Mandanten frei bearbeitet werden [47], [48].

Policy-Entscheidungen werden, wie erwähnt, revisionsfähig protokolliert und von der Middleware bei Entgegennahme der Entscheidung in der PostgreSQL-Datenbank unter `btc.opa_decision` persistiert. Diese Trennung erhöht die Wartbarkeit, Revisionssicherheit und Nachvollziehbarkeit sicherheitskritischer Entscheidungen.

Die vordefinierten OPA-Policies, die im Zuge der System-Operationalisierung erstellt wurden, werden in der Datei `policies/hot.rego` gespeichert, wobei die dazugehörigen Limits in der Datei `data/data.json` verwaltet werden. Diese Regeln können durch Rego- und Json-Code zu den Vorstellungen und zum Budget des Mandanten angepasst werden. Die exakte aufgestellte Konfiguration kann im Anhang in Tabelle A5.3 nachvollzogen werden.

Tages-Abfluss (Daily-Cap)

Zusätzlich zur Einzeltransaktions-Grenze begrenzt OPA den kumulierten Tages-Abfluss. Der Apphost zählt dazu den in den letzten 24 Stunden bereits ausgeschütteten Betrag der Zahlungen exklusive der internen Hot-/Cold-Tauschtransaktionen in den OPA-Input. Überschreitet dieser dynamische Wert aus der Summe bereits verbrauchten und des aktuell angefragten Betrags den statisch festgelegten Grenzwert `max_daily_sat`, wird die Transaktion mit der Begründung `daily limit exceeded` abgelehnt. Für die intern durch OPA ausgelösten Austausch-Transaktionen (`OPA_hot/OPA_cold`) entfällt diese Prüfung, da deren Höhe von OPA bestimmt wird und deren Ausführung systemkritisch ist. Sollte der Apphost kompromittiert werden und dieser Test umgangen werden, greift zudem ein weiterer Velocity Test auf der Key A VM direkt und senkt

somit den maximal möglichen Schaden weiter (siehe 1.4.2).

Weitere Regeln können vom Mandanten definiert werden. Ebenfalls denkbar ist die Verwendung einer Tag-Regel, die das Vorhandensein vorbestimmter Texte als zusätzliche Authentifizierung kontrolliert.

Im BIP21-Prozess erfolgt die Validierung der TX erst nach der Erstellung im TX-Builder. Dieses Vorgehen wurde bewusst gewählt, um die PSBT- und die BIP21-Schnittstelle in ihren Prozessschritten auf dieselbe Folge anzugleichen. Zudem erfolgt die Validierung aller PSBT-Parameter in einem einzelnen Schritt, weshalb Informationen wie Gebühren bereits gegeben sein müssen. Dieser Fall tritt dementsprechend erst nach dem Bau der PSBT auf.

Darüber hinaus ist eine mögliche Erweiterung, wie bereits beschrieben, dass in der Beschreibung von BIP21-Übertragungen Codes enthalten sind, welche die Validität der TX weiter ausweisen. Da diese Informationen meist dynamisch oder kontextabhängig gebildet werden, genügt eine statische Validierung durch OPA nicht. Entsprechend müsste eine ergänzende Validierungslogik im Approval-Gate implementiert werden. Dieser Ansatz wurde im Zuge dieser Ausarbeitung nicht umgesetzt, da er stark vom jeweiligen Partner abhängt und somit die nötigen Informationen zur Umsetzung fehlen.

Stattdessen wurde sich für das Whitelisting bestimmter externer Partner-Wallets entschieden. Deren operationeller Import wurde bereits zuvor beschrieben und bildet die Grundlage für die Zuordnung von dynamisch generierten Zieladressen zu registrierten Wallets. Da sich diese Adressen typischerweise aus einem Extended Public Key (xpub) oder zugehörigen Deskriptoren ableiten und sich somit für jede Transaktion ändern, kann diese dynamische Beziehung nicht durch statische OPA-Regeln abgebildet werden. Aufgrund dessen wurde sich für eine separate Logik entschieden, die das Extrahieren aller externen („ext“) Wallets aus der PostgreSQL-DB ermöglicht und mit der Methode `getaddressinfo` die Adresszugehörigkeit überprüft.

Eine Möglichkeit, den OPA in die Whitelisting-Verifizierung einzubeziehen, läge in der Verwendung statischer Zieladressen für Hot-Wallet-TXs. Aufgrund des erhöhten Datenschutzbestrebens des Mandanten und der

zusätzlichen Anweisung an die Überweisungspartner wurde dieser Ansatz verworfen und außerhalb von OPA implementiert.

Neben der Freigabe einzelner TXs evaluiert OPA nach jedem Broadcast den Kontostand des Hot-Wallets unter der Schnittstelle `policy/hot/limits/output`, wobei die Grenzen ebenfalls in `data/data.json` angepasst werden können. Diese Grenzen sind wie folgt definiert und in Abbildung 1.4 visualisiert.

- Wird die obere Grenze von 2 BTC überschritten, erfolgt automatisch eine Transfer-Transaktion zum Cold-Wallet: `hot_to_cold`.
- Wird die untere Grenze von 0,5 BTC unterschritten, wird ein Nachbefüllungs-Prozess initiiert, der eine Teiltransaktion vorbereitet und den Mandanten über den ntfs-Microservice (siehe 1.3.3) zum Cold-Wallet-Prozess auffordert: `cold_to_hot`.
- Befindet sich der Wallet-Stand im optimalen Bereich, wird keine Aktion vorgenommen: `hold`.

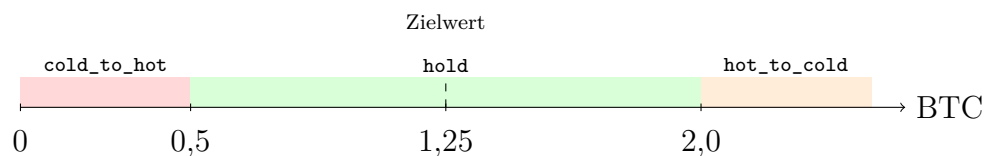


Abbildung 1.4: Balance-Zonen des Hot-Wallets und resultierende OPA-Aktionen.

Quelle: Eigene Darstellung.

Beide Austausch-Transaktionen stellen ein Risiko für das Gesamtsystem dar. Wird ein `hot_to_cold`-Intent ausgelöst, lagert das Hot-Wallet zu viele Geldmittel und stellt einen höheren Schaden bei Kompromittierung dar. Bei einem `cold_to_hot`-Austausch besitzt das Hot-Wallet nur geringe Geldmittel, und die Operationalität des automatischen Transaktionssystems, also das Schutzziel der Verfügbarkeit, ist gefährdet.

Basierend auf der Aktion werden daraufhin die Ziel- und Ursprungsadresse festgelegt. Die Höhe der zu sendenden Mittel wird in Relation zum Mittelwert des optimalen Bereichs bestimmt, sodass möglichst wenige Austausch-Transaktionen zwischen Hot- und Cold-Wallet ausgeführt werden müssen.

Zudem wird ein Risk Score auf Basis des Abstands zur äußeren Grenze des optimalen Bereichs bestimmt, welcher verwendet wird, um die PSBT-Parameter auf Basis einer gewissen Wichtigkeit oder Risikos zu bestimmen. (siehe Tabelle 1.4).

PSBT-Parameter	Score ≤ 30	$30 < \text{Score} \leq 70$	Score > 70
Confirmation Blocks	6	3	1
Estimate Mode	economical	conservative	conservative

Tabelle 1.4: OPA-Kontostand-Evaluation (Parameter nach Risk Score)

Da diese Transaktionen einen abnormalen Zustand des Apphosts revidieren sollen und die Parameter selbst vom System gewählt werden, wird von einer weiteren Bestätigung von PSBT-Parametern wie BTC-Anzahl und Gebühren durch den OPA, wie bei normalen TXs, abgesehen. Nichtsdestotrotz wird das korrekte Funktionieren des Systems über das Vorhandensein der obligatorischen Parameter der PSBT validiert (siehe oben).

Signierer

Die gebaute und bestätigte PSBT wird anschließend über die Middleware an den Key-Holder weitergeleitet. Da das Schlüsselmaterial selbst die sicherheitskritischste Komponente darstellt, wurde dieser auf eine weitere VM ausgelagert.

Wie die übrigen Schlüsselhalter-VMs basiert diese auf NixOS mit der beschriebenen Härtung (siehe Tabelle A5.1), ist jedoch im Unterschied zu den nachfolgend vorgestellten Cold-VMs weiterhin mit dem Internet verbunden. Diese Entscheidung dient nicht dem Beobachten der Blockchain oder anderen Bitcoin-Operationen, sondern dem Aufbau einer Kommunikationsbrücke zum Haupt-System. Im realen Nachbau durch den Mandanten kann diese durch ein USB- oder Local Area Network (LAN)-Kabel ersetzt werden. Eine Substitution durch eine Hardware-Wallet bietet sich nicht an, da die Automatisierung des Signierungsprozesses verloren ginge.

Internet-Verbindung

Eine besondere Härtung der Internet-Verbindung wurde durch die Regeln einer nftables-Konfiguration und die Verwendung von WireGuard erreicht. Dabei werden alle Ports außer UDP 51820 für WG geblockt und aller Verkehr über den verschlüsselten WireGuard-Tunnel geleitet. Dieser Tunnel wird für die HTTP-API-Kommunikation zwischen Apphost und Key A verwendet. Zudem wird `egress` während des `dockercompose build`-Befehls über die nftables-Konfiguration `nftables-setup.conf` erlaubt, nach dem Build jedoch final über `nftables-locked.conf` wieder entfernt und gehärtet.

Die Erreichbarkeit der Key A VM ausschließlich über WireGuard wird nicht allein durch die `nftables`-Regeln auf Betriebssystemebene sichergestellt, sondern zusätzlich auf Ebene der Container-Orchestrierung erzwungen. Da Docker für veröffentlichte Ports eigene NAT-Weiterleitungsregeln anlegt, die unabhängig von der `input/output`-Policy der Firewall wirken, publiziert kein Container einen Port nach außen. Stattdessen übernimmt ein dedizierter, schlanker Reverse-Proxy-Container die externe Anbindung und bindet sich ausschließlich an die statische WireGuard-Interface-Adresse 10.10.0.2. Die Container kommunizieren ausschließlich über das interne, nicht nach außen exponierte Docker-Netzwerk und adressieren sich dabei weiterhin gegenseitig über ihre jeweiligen Service-Namen. Auf der physischen Netzwerkschnittstelle existiert dadurch zu keinem Zeitpunkt ein Dienst, der unabhängig vom WireGuard-Pfad erreichbar wäre.

Neben der In-Transit-Verschlüsselung wird ebenfalls ein privater HMAC-Schlüssel beim HTTP-Call eingesetzt, um die Integrität und Authentizität der Kommunikation zwischen den Systemen sicherzustellen. Jede Anfrage an die Signier-Schnittstelle führt dazu neben der eigentlichen PSBT drei zusätzliche Header mit: einen Zeitstempel, eine Nonce sowie die HMAC-Signatur [49], [50]. Die Signatur wird über die Kombination aus Zeitstempel, Nonce und Request-Body gebildet, sodass jede nachträgliche Veränderung an einem dieser Bestandteile zur Ungültigkeit der Signatur führt. Der Zeitstempel ist nur für einen kurzen Zeitraum gültig, sodass ältere Anfragen abgewiesen werden. In Verbindung mit der nachgelagerten Eindeutigkeitsprüfung der PSBT (siehe Deduplizierung) wird so das Zeitfenster für ein etwaiges Wiedereinspielen abgefangener Anfragen stark eingegrenzt und die Authentizität jedes einzelnen Signierauftrags

sichergestellt.

Zudem wird die Kompromittierungsgefahr für den Schlüssel-Halter weiter reduziert, da ausschließlich PSBTs angenommen werden. Bevor eine empfangene PSBT signiert wird, durchläuft sie eine strukturelle Policy-Prüfung, welche sicherstellt, dass ausschließlich wohlgeformte und plausibel aufgebaute Transaktionen den Signiervorgang erreichen. Geprüft wird dabei, ob die PSBT überhaupt eine Transaktion enthält und mindestens einen Input besitzt, ob jeder Input über ein `witness_utxo` sowie die zugehörigen BIP32-Ableitungspfade verfügt und ob jeder Output einen gültigen Betrag größer Null aufweist. Schlägt eine dieser Prüfungen fehl, wird die Signierung abgebrochen und die PSBT zurückgewiesen. Diese Prüfung ersetzt keine vollständige fachliche Transaktionsprüfung, welche im Basissystem durch Bitcoin Core und den Open Policy Agent erfolgt, verhindert aber bereits auf dem Signierer strukturell ungültige oder unvollständige PSBTs und reduziert damit die Angriffsfläche des sicherheitskritischen Signiervorgangs.

Deduplizierung

Zur Verhinderung einer mehrfachen Verarbeitung identischer Transaktionen wird jede PSBT über ihre eindeutige PSBT-ID erfasst. Diese ID wird in der Datenbank des Signers als eindeutiger Schlüssel geführt, sodass das System bereits gesehene Signiervorgänge zuverlässig erkennt. Trifft eine PSBT mit einer bereits bekannten ID ein, wird sie nicht erneut signiert, sondern mit der Statusmeldung `ALREADY_PROCESSED` quittiert. Dieser Mechanismus stellt den eigentlichen, zustandsbehafteten Schutz gegen das Wiedereinspielen vollständiger Signieraufträge dar und ergänzt die zeitfensterbasierte Prüfung aus Zeitstempel und Nonce um eine dauerhafte, nachvollziehbare Eindeutigkeitsgarantie. Die Nonce jeder Anfrage wird dazu für die Dauer des Gültigkeitsfensters in einem Redis-Speicher als verwendet vermerkt, sodass eine erneut eingespielte Anfrage mit identischer, noch gültiger Nonce bereits vor der Signierung abgewiesen wird.

Velocity-Cap (zweistufige Abflussbegrenzung)

Der maximale Geldabfluss des Hot-Wallets ist an zwei voneinander unabhängigen Punkten begrenzt. Die erste Schranke bildet das oben beschriebene tägliche Limit des OPA im Apphost (siehe 1.4.2). Die

zweite Schranke ist ein harter, vom Apphost unabhängiger Velocity-Cap auf der Key A VM. Der Signierer führt in einem Redis-Speicher ein rollierendes Zeitfenster (Standard 24 Stunden, `VELOCITY_WINDOW_SEC`) über den bereits signierten Abfluss. Dementsprechend weist dieser eine Anfrage mit HTTP 429 ab, sobald die Summe den konfigurierten Grenzwert (Standard 40 000 000 Satoshis (SATS), `VELOCITY_CAP_SATS`) überschreiten würde.

Gezählt wird ausschließlich der Nicht-Change-Abfluss über Single-Signatur-Inputs. PSBTs mit Multi-Signatur-Input (`cold_to_hot`-Nachbefüllungen) werden am Witness-Script herausgerechnet, während Wechselgeld-Rückgaben an die eigene Wallet über den Master-Fingerprint erkannt werden.

Die Beweggründe und Sicherheitsbetrachtung der gestaffelten Schranken und ihr Wertverhältnis werden weiter in der Sicherheitsbetrachtung diskutiert (siehe 1.4.6).

In-Transit-Tresor

Das System verwendet einen sogenannten *In-Transit-Signer*-Ansatz, bei dem ausschließlich über PSBTs kommuniziert wird und das Schlüsselmaterial das System zu keinem Zeitpunkt verlässt. Vorhandene Alternativen für den Signierer als selbstgebauten Microservice lassen sich in **Web3Signer** von ConsenSys und HashiCorp Vault finden. HashiCorp Vault unterstützt in seiner eingebauten Signier-Engine die secp256k1-Kurve des Bitcoin-Algorithmus nicht nativ und könnte nur mit einer Erweiterung durch Plug-ins für unsere Anforderung verwendet werden [51], [52], [53]. **Web3Signer** unterstützt die secp256k1-Kurve zwar nativ, ist jedoch ausschließlich auf das Signieren von Ethereum-Transaktionen und Validator-Daten ausgelegt und kennt das für Bitcoin verwendete PSBT-Format nicht. Eine Nutzung hätte somit eine vollständig eigene Übersetzungsschicht zwischen Bitcoin-Transaktionsformat und generischem Signieren erfordert [54]. Andere proprietäre Lösungen, die Bitcoin nativ unterstützen, wie **AWS KMS** oder **Google Cloud KMS** [55], standen nicht zur Verfügung, da das System als eigenständige, nicht an einen externen Cloud-Dienst gebundene Lösung konzipiert ist. Aufgrund dessen wurde sich für ein eigenes System entschieden. Außerdem hätte das Verwenden einer vorgefertigten Lösung die Idee des Selbstentwurfs eines besonders sicheren Systems im Zuge der Aufgabenstellung zunichtege-

macht.

TPM

Das Schlüsselmaterial wird zudem besonders gesichert und nur über das TPM entschlüsselt. Diese Technologie speichert das Geheimnis in einem speziellen Hardware-Chip und verspricht dadurch Auslese-Sicherheit [56]. Das TPM wird zudem über Platform Configuration Register (PCR) in einem speziellen sogenannten **measured boot** versiegelt. Diese sind spezielle, zur Laufzeit nicht frei beschreibbare Register im TPM und werden während des Boot schrittweise mit kryptographischen Hashes der geladenen Komponenten wie Firmware, Bootloader, Kernel und Konfiguration „erweitert“. Jeder neue Messwert wird mit dem bisherigen Registerinhalt verkettet und neu gehasht, sodass der Endzustand eines Registers den gesamten gemessenen Boot-Pfad eindeutig abbildet. Ein Geheimnis lässt sich an bestimmte PCR-Werte binden und darüber versiegeln. Das TPM gibt es nur frei, wenn dieselben Register zum Entsiegelungszeitpunkt exakt denselben Zustand aufweisen. In einer Autorisierungs-Session wird ein während des Boots gebildetes Abbild dieses Maschinenzustands für die Versiegelung des Geheimnisses verwendet. Der Chip kann dementsprechend nur durch Verifizierung desselben Systemzustands in einer weiteren Auth-Session entsiegelt werden, was eine Kompromittierung des Schlüsselmaterials über den Schlüssel selbst ausschließt.

Die konkrete Belegung der Register wurde nicht angenommen, sondern auf der Signierer-VM mit `systemd-analyze pcrs` empirisch erhoben (siehe Abbildung A5.1 im Anhang). Dabei zeigte sich, dass PCR 8 und 11 auf dem eingesetzten Boot-Stack unerweitert bleiben (Wert `0x00...0`) und daher keine Schutzwirkung entfalten. PCR 8 wird nur vom GRUB-Bootloader belegt, während das hier verwendete `systemd-boot` die Kernel-Befehlszeile in PCR 12 misst. Dazu setzt PCR 11 ein Unified Kernel Image voraus, das im Labor nicht eingesetzt wird. Die Bindung der Entsiegelung erfolgt daher an die real belegten und über Neustarts stabilen Register PCR 4, 9 und 12:

- **PCR 4 (Boot Manager Code):** Misst den verwendeten Bootloader sowie das geladene Kernel-Image und verhindert, dass ein Angreifer einen modifizierten Bootloader oder Kernel unterschiebt.
- **PCR 9 (Kernel Image und Initrd):** Speichert die kryptografi-

schen Hashes des Linux-Kernel-Images und der initialen RAM-Disk und stellt unter NixOS sicher, dass exakt die über die Systemkonfiguration definierte Kernel-Umgebung geladen wurde.

- **PCR 12 (Kernel Command Line / Boot-Konfiguration):** Erfasst unter `systemd-boot` die vollständige Kernel-Befehlszeile und die gewählte Boot-Eintrags-Konfiguration; dadurch werden manipulierte Boot-Parameter, etwa zur Umgehung der Härtung, erkannt.

Die Entsiegelung erfordert eine analoge Policy-Session, in der das TPM die aktuellen Registerwerte prüft und das versiegelte Geheimnis *in Form der 32-Byte-Entropie* nur bei exakter Übereinstimmung freigibt. Zu beachten ist, dass PCRs ausschließlich beim Systemstart gemessen und zur Laufzeit eingefroren werden, wobei eine veränderte Konfiguration erst erkannt wird, sobald sie durch einen Neustart wirksam wird. Der Schutz richtet sich damit gezielt gegen zwei Angriffe. Zum einen ist das versiegelte Geheimnis kryptografisch an den Storage-Root-Key des konkreten TPM gebunden und lässt sich auf keinem anderen TPM laden, sodass ein Aus- und Wiedereinbau der Hardware in ein fremdes System das Schlüsselmaterial nicht preisgibt (siehe Abbildung 1.5). Zum anderen führt eine nachträgliche Schwächung der Härtung, etwa über ein `nixos-rebuild switch`, das Kernel oder Boot-Parameter verändert, beim nächsten Boot zu abweichenden PCR-Werten. Daraufhin würde die Entsiegelung fehlschlagen und der Seed bleibt versiegelt. Gegen einen Angreifer, der bereits zur Laufzeit über Root-Rechte auf dem provisionierten System verfügt, schützt die PCR-Bindung hingegen nicht. Auf dieser Ebene greifen die Netzwerkisolation und die Prozess-Härtung des Signers [57], [58].

Da dieselbe Bindung auch legitime Änderungen an Kernel oder Boot-Konfiguration erfasst, muss der Seed nach einem bewussten System-Update erneut versiegelt werden. Dieser Schritt ist fester Bestandteil des Update-Vorgehens der Signierer-VM.

Die Durchsetzung der Bindung wurde geprüft, indem ein gebundenes Register zur Laufzeit künstlich mit `tpm2_pcrextend` verändert und anschließend die Entsiegelung versucht wurde; diese schlug erwartungsgemäß fehl. Ein vollständiger `nixos-rebuild`-Test entfällt, da die Signierer-

1 Self Hosted Setup

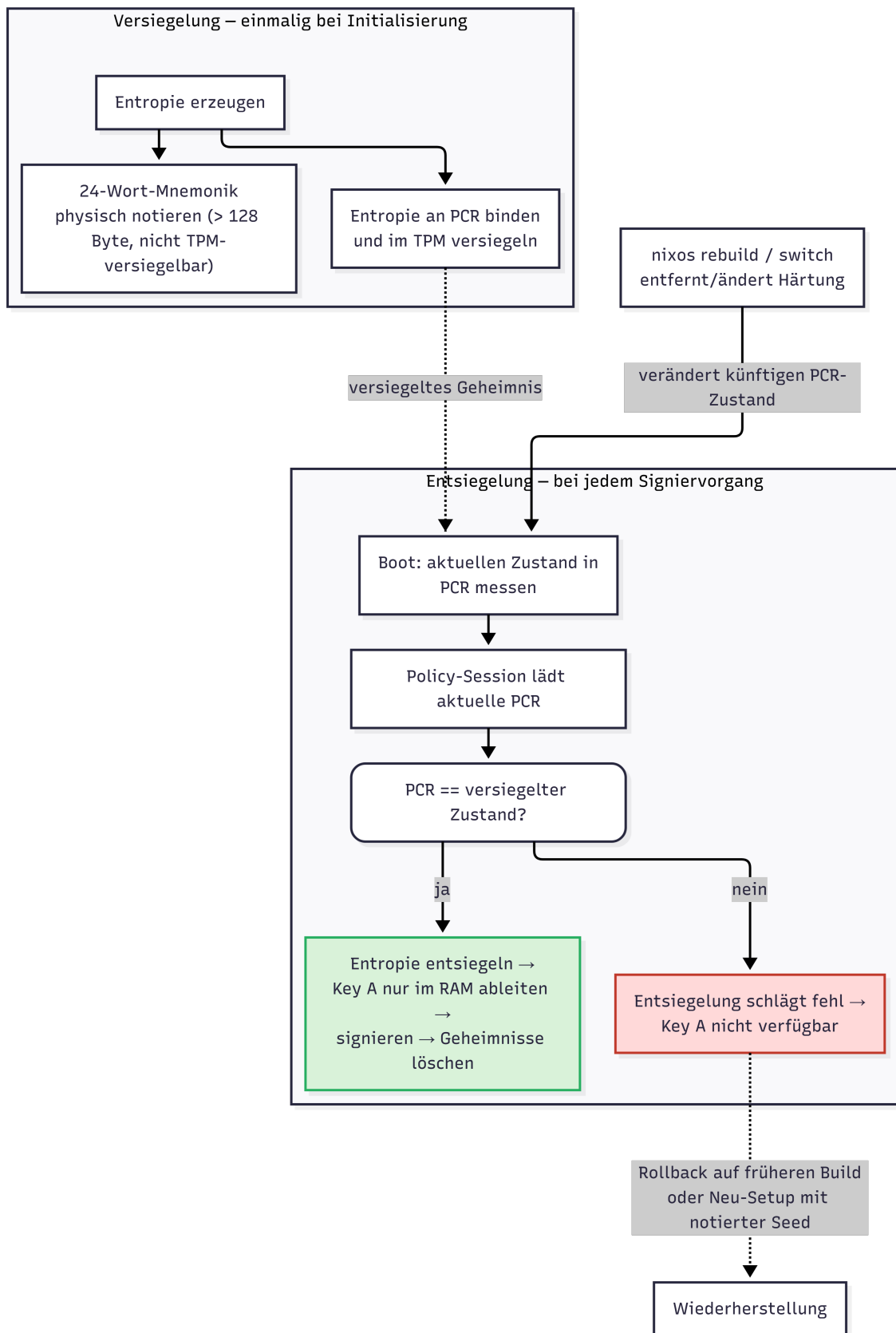


Abbildung 1.5: Darstellung der TPM-Sicherungslogik. Quelle: Eigene Darstellung.

VM bewusst keinen ausgehenden Netzzugang für den Bezug der Build-Abhängigkeiten besitzt.

Das Schlüsselmaterial bildet sich in diesem Ansatz aus der Entropie, welche als Seed im TPM versiegelt und deren 24 mnemonische Wörter zur Sicherung in eine Datei auf dem Desktop ausgegeben werden. Diese 24 Wörter müssen für eine spätere Wiederherstellung vom Mandanten sicher notiert und die Datei anschließend über das automatisierte Skript `scripts/delete_seed.sh` kontrolliert entfernt werden. Empfehlungen wurden bereits in Abschnitt 1.4.4 gegeben. Dementsprechend muss bei jedem Signierungsprozess die Entropie entschlüsselt und der private Schlüssel zum Signieren abgeleitet werden. Dabei wurde exakt auf die Löschung nicht benötigter Geheimnisse geachtet. Die Schlüssel selbst werden nur in den RAM geladen und niemals auf der Festplatte gespeichert. Eine andere Möglichkeit, dies zu realisieren, wäre, das TPM anzuweisen, das Schlüsselmaterial intern zu generieren und nur private Schlüssel zum Signieren auszugeben, wodurch das Root-Schlüsselmaterial maximal geschützt wäre. Diese Implementation war leider nicht möglich, da das auf Proxmox virtualisierte TPM die secp256k1-Kurve des Bitcoin-Algorithmus nicht unterstützte. Da dies ebenfalls im Fall der Hardware des Mandanten nicht garantiert werden kann, wurde sich gegen diese Umsetzung entschieden.

Docker-Härtung

Der Signer-Container, welcher als einziger Dienst unmittelbar mit angreiferkontrollierten PSBT-Daten in Berührung kommen könnte, wurde zusätzlich auf Prozessebene gehärtet. Der Containerprozess wird unter einer dedizierten, nicht privilegierten Benutzerkennung ausgeführt und besitzt keine erweiterten Linux-Capabilities, da für den Betrieb des Signers keine davon benötigt wird: Der Zugriff auf den TPM-Chip erfolgt über gewöhnliche Dateioperationen auf das Gerät. Die hierfür erforderliche Gerätegruppen-Zugehörigkeit wird beim Start automatisiert anhand des tatsächlichen Systemzustands ermittelt und dem Container zugewiesen. Ergänzend ist dem Prozess die Rechteauserweiterung über privilegierte Programmbits untersagt, und das Wurzeldateisystem des Containers ist unveränderlich eingebunden. Schreibvorgänge sind ausschließlich innerhalb der ohnehin für Wallet- und TPM-Daten vorgesehenen, separat eingebundenen Datenverzeichnisse sowie eines flüchtigen

Arbeitsspeicherbereichs für temporäre Dateien möglich. Eine erfolgreiche Kompromittierung des Signiervorgangs, etwa über einen Fehler beim Parsen einer PSBT, bliebe dadurch auf die minimalen Rechte eines einzelnen, nicht privilegierten Prozesses beschränkt und führte nicht zu einer Rechteauserweiterung auf Betriebssystemebene.

Ein automatisiertes Skript `wgHMAC_export.sh` zum Extrahieren der Wallet-Informationen samt xpub, Public-Deskriptor und Meta-Informationen auf einen USB-Stick wurde implementiert und hilft dem Mandanten beim Aufsetzen des Whitelistings auf der Hot-Wallet-VM sowie beim Hinzufügen des xpub im Multi-Signatur-Verfahren des Cold-Wallets. Dieses Skript schreibt gleichzeitig das HMAC-Geheimnis und die WireGuard-Konfiguration samt Public Key auf diesen USB-Stick. Diese Informationen können daraufhin per SSH in das Dateiverzeichnis des Apphosts geladen werden und anschließend per automatisiertem Skript auf der Hot-Wallet eingespielt werden, was das Laden der Deskriptoren in die PostgreSQL-DB und das Ablegen des HMAC-Geheimnisses im zugriffsgeschützten, read-only in den Middleware-Container eingebundenen Verzeichnis `secrets/hotwallet/` umfasst.

Die für die Datenbankbindung notwendigen Zugangsdaten werden nicht statisch vorgegeben, sondern bei der Erstinitialisierung der Key A VM automatisiert und zufällig erzeugt. Sie verbleiben dabei außerhalb der versionierten Konfiguration in einer separaten, restriktiv berechtigten Laufzeitdatei, auf welche ausschließlich die beteiligten lokalen Prozesse Zugriff erhalten. In Verbindung mit der bereits beschriebenen Netzwerkisolation, durch welche die Datenbank ebenfalls keinen nach außen gerichteten Port mehr besitzt, wird damit ausgeschlossen, dass über einen direkten Datenbankzugriff von außen die ID-basierte Eindeutigkeitsprüfung bereits verarbeiteter PSBTs umgangen werden könnte.

Ausgegeben werden nach den beiden Abläufen PSBTs für den Nachfüll-Prozess sowie finalisierte Hot-TXs für das vollautomatisierte Hot-Wallet.

Eine tiefere Beschreibung der Konfiguration und des gesamten Setup- und Signierungsprozesses kann in den `README.md`-Dateien der Signierer-VM (`Hot-KeyHolder/`) und des in den Apphost integrierten Hot-Wallet-Stacks (`apphost/services/hotwallet/`) nachgelesen werden [42], [43].

1.4.3 Cold-Wallet

Das Cold-Wallet ist vollständig air-gapped und besitzt keinerlei Netzwerkverbindung. Die Systeme werden auf dedizierten virtuellen Maschinen in der Proxmox-Umgebung betrieben, deren Netzwerkfunktionen sowohl auf Betriebssystem- als auch auf Hypervisor-Ebene deaktiviert sind [32], [59].

Das vom Internet isolierte Cold-Wallet bildet den architektonischen Kern des Systems und verwaltet ca. 95 % der Bitcoin-Bestände. Diese gilt es dementsprechend besonders zu schützen, weshalb sich für ein Air-Gap und ein 2-aus-3-Multi-Signatur-Wallet entschieden wurde. Die Isolierung des Systems reduziert die Angriffsfläche gegenüber netzwerkbasierten Angriffen erheblich und verhindert direkte Remote-Kompromittierungen.

Unter der Voraussetzung eines sicheren Perimeters am Standort des Mandanten verbleiben als Bedrohungen im Wesentlichen der operationelle Signierprozess selbst, der Datentransfer über das Wechselmedium sowie die Software-Lieferkette der eingesetzten Komponenten. Diese werden in der Sicherheitsbetrachtung (siehe 1.4.6) einzeln bewertet. Um den operationellen Prozess sicher zu gestalten, wurde sich für einen 2-aus-3-Multi-Signatur-Prozess entschieden, wobei über einen USB-Stick PSBTs transferiert werden können [24], [26], [27], [30], [59], [60]. Dieser Multi-Signatur-Prozess wird besonders für hohe Geldsummen und professionelle Architekturen empfohlen und findet dementsprechend Verwendung [60]. Ein Hinzufügen weiterer benötigter Signaturen durch einfaches Ergänzen zusätzlicher Cold-VMs und das Erhöhen des Reglers bleibt im Ermessen des Mandanten.

Architektur

Hardware und Betriebssystem

Die Aufteilung auf zwei getrennte VMs (Key B und Key C) ergibt sich unmittelbar aus dem 2-aus-3-Multi-Signatur-Modell. Zusammen mit dem im Hot-System gehaltenen Key A bilden sie drei unabhängige Cosigner, von denen zwei zur Freigabe einer Cold-Transaktion genügen. Die physische Trennung auf zwei Maschinen verhindert, dass die Kompromittierung eines einzelnen Cold-Systems bereits zwei Schlüssel preisgibt.

Key C dient zusätzlich als Recovery-Schlüssel (siehe 1.4.3), sodass, wenn eine Maschine ausfällt, die Spend-Fähigkeit über die verbleibenden zwei Schlüssel erhalten bleibt und die gesamten Assets auf ein neues Wallet transportiert werden können. Der Mandant kann die beiden VMs an getrennten, physisch gesicherten Orten betreiben oder das Verfahren durch weitere Cold-Signer auf ein 3-aus-4-Schema verschärfen.

Als air-gap-fähiger Multi-Signatur-Koordinator und Schlüsselhalter der Cold-VMs wurde Sparrow Wallet gewählt. Ausschlaggebend waren die vollständige Offline-Fähigkeit, die transparente Darstellung der Zieladressen, TX- und UTXO-Details für die manuelle Verifikation (siehe 1.4.4) sowie die native Unterstützung von Deskriptoren und dem PSBT-Prozess. Die konkrete Abwägung gegen die Alternativen folgt im Anschluss (siehe 1.4.3).

Das Multi-Signatur-Wallet selbst enthält auf jeder VM je einen geheimen kryptographischen Schlüssel und je zwei öffentliche Schlüssel (siehe 1.4.4). Jeder einzelne Schlüssel bleibt isoliert, und eine manuelle Verifikation durch den Mandanten bleibt als Voraussetzung für den Broadcast bestehen.

Für die sicherheitskritische Komponente des kryptographischen Schlüsselmaterials wird NixOS als Betriebssystem verwendet. Dies ermöglicht reproduzierbare, deklarative und minimalistische Konfigurationen. Eine besondere Härtung der Hardware- und Netzwerk-Schicht sichert gegen unbeabsichtigtes Mounten von USB-Medien und Zugriff über offene Ports ab [61].

Auf diesem System wird NixOS bewusst so konfiguriert, dass bestimmte nutzerfreundliche Funktionen bereitgestellt werden, die nicht Teil einer Standard-NixOS-Installation sind. Ziel ist es, sicherheitskritische Skripte kontrolliert und dennoch einfach nutzbar zu machen. Darunter fallen ein deklarativer Desktop-Mount sowie automatisierte Skripte. Zudem führt der XForms Common Environment (XFCE)-Dateimanager (Thunar) Shell-Skripte nicht standardmäßig per Doppelklick aus. Diese Einschränkung wird explizit aufgehoben, um dem Nutzer für bestimmte Skripte eine einfachere Nutzung zu ermöglichen.

Sparrow Wallet

Sparrow Wallet ist eine quelloffene Desktop-Bitcoin-Wallet mit Fokus auf Self-Custody, Sicherheit und Transparenz, womit sie die inhaltlichen Anforderungen des Mandanten unterstützt. Sie unterstützt Single- und Multi-Signatur, Output-Deskriptoren und den vollständigen PSBT-Prozess nach [27]. Außerdem wird eine Anbindung an einen eigenen Bitcoin Core Node ermöglicht, welche wir nicht nutzen, jedoch als Alternative anerkennen (siehe 1.4.5). Es unterstützt zudem die Besonderheit der Cold-Wallet VMs, sodass es offline Signierung sowohl über Dateien, USB als auch animierte Quick Response (QR)-Codes ermöglicht. Daneben zeichnet es sich ebenfalls durch eine breite Hardware-Wallet-Integration und eine Vielzahl an Konnektoren und Wallet Formaten aus, welche als weitere Alternativen beschrieben wurden und dem Mandanten offenstehen (Siehe 1.4.5). Ein besonderes Merkmal ist die transparente Darstellung aller Transaktions- und UTXO-Details, die die manuelle Verifikation im Cold-Prozess erst sicher und praktikabel macht und dementsprechend fest eingeplant ist (siehe 1.4.4) [44], [62].

In unserer Integration fungiert Sparrow Wallet als Multi-Signatur-Koordinator, PSBT-Handler und Key Holder. Eine Instanz darf diese Rollen, wie oben bereits erwähnt, gleichzeitig halten [28], [62]. Zudem verwaltet die Applikation keine vollständige Blockchain, da sie keine Netzwerkverbindung besitzt. Private Schlüssel verbleiben ausschließlich auf den jeweiligen Cold-Key-Systemen, während allein der interne und externe Multi-Signatur-Deskriptor des Cold-Wallets auf Bitcoin Core registriert wird [28], [59].

Sparrow übernimmt im Cold-Bereich folgende Aufgaben:

- Import und Anzeige freigegebener PSBTs,
- Koordination der Multi-Signatur-Signaturen,
- Überprüfung der Skript- und Adressstruktur,
- Zusammenführung und Finalisierung der Signaturen.

Anzuführen ist ebenfalls, dass sich Sparrow während der Entwicklung durch einen Fehler im stable-Release auszeichnete. Nach dem Download aus dem Nix-Store und dem Ausführen des Programms ergaben sich

Probleme mit der JavaFX-Bibliothek, und verschiedene Fenster mit dem Fehler ***Panes omitted*** konnten nicht angezeigt werden. Aufgrund dessen wurde sich während der Entwicklung für die unstable-Version entschieden, die Java 25 nativ unterstützt und diese Probleme behob. Während der Entwicklung wurde zudem NixOS 26.05 veröffentlicht, welche Java 25 direkt umfasst, weshalb der unstable-channel während der Entwicklungsperiode wieder aufgehoben werden konnte. Die Channels **nixos-stable** und **nixos-unstable** sind beide offizielle, von den NixOS-Maintainern betreute Zweige desselben Repositories. **unstable** ist dabei nicht schlechter gepflegt, sondern lediglich der schneller aktualisierte Rolling-Zweig mit neueren Paketversionen (hier Java 25), der kürzer stabilisiert wurde. Die zwischenzeitliche Nutzung von **unstable** war daher eine bewusste, temporäre Wahl zur Behebung des JavaFX-Fehlers.

Zusätzlich wird die Wallet-Anwendung deklarativ in NixOS integriert: Installation über den Nix Store, definierte Startparameter (Netzwerkmodus **--regtest**) und Bereitstellung einer Desktop-Datei, welche den Start per Doppelklick auf das Desktop-Icon ermöglicht.

Eine Besonderheit von Bitcoin, die extra beachtet werden musste, stellt die Adressierung von Wallets dar. Während für jede TX dieselbe statische Empfangsadresse verwendet werden kann, stellt dies nicht den Standard dar. Zur Verringerung der Nachvollziehbarkeit von TX-Abläufen und damit Erhöhung des Datenschutzes werden oft Zieladressen dynamisch aus dem öffentlichen externen Deskriptor nach der BIP32-Hierarchie gebildet. Dies ist ebenfalls für den internen Deskriptor und die Quell-Adresse der Fall, wobei diese bereits im Signierprozess als von dieser Wallet stammend verifiziert wird und kein größeres Problem darstellt.

Dieser Sachverhalt stellt ein Problem für die manuelle Verifikation von PSBT-Details wie im Signierungsprozess (siehe 1.4.4) beschrieben dar, da die Zieladresse als zugehörig zur Hot-Wallet von Key A verifiziert werden soll. Dafür sollte der externe Deskriptor von Key A als Watch-Only-Wallet integriert werden, sodass Sparrow die dynamische Adresse zu dem hinterlegten Wallet vollautomatisch auflösen kann. Der Mandant kann so direkt verifizieren, dass die Cold-Wallet-PSBT Geld nur an das Hot-Wallet sendet.

Die Wahl des Koordinators ist eine architektonische Grundsatzentschei-

dung und wird der Implementierung bewusst vorangestellt: Erst wenn feststeht, welche Software die Rollen Koordinator, PSBT-Handler und Schlüsselhalter air-gapped vereinen kann, ergibt der konkrete Setup- und Signierprozess Sinn. **Alternativen zu Sparrow Wallet**

Als air-gap-fähige Koordinatoren kämen ebenfalls **Electrum**, **Specter Desktop**, **Nunchuk**, **Caravan** sowie **Bitcoin Core** selbst in Frage. Diese decken jedoch jeweils nur Teile der benötigten Funktionalität netzunabhängig ab und fanden aufgrund dessen keine Verwendung.

Kandidat	Offline-fähig	Air-gapped Key-Halter	Ausschlussgrund
Electrum	teilw.	nein	Online-Variante benötigt öffentlichen Electrum-Server (Metadaten-Leak), offline keine Wallet-übergreifende Adressauflösung, nicht Deskriptornativ.[63]
Bitcoin Core	ja	ja (nur Keys)	GUI nicht für die manuelle Verifikation geeignet, keine Anzeige von Zieladressen, Beträgen, UTXO-Flüssen.
Specter Desktop	ja	nein	Reiner Koordinator ohne eigenes Schlüsselmaterial, nur gekoppelt mit Bitcoin Core nutzbar, dann größere Codebasis/Angriffsfläche und geringere PSBT-Detailltiefe.[64], [65]
Nunchuk	ja	nein	Auf externe Signer (Hardware/Tapsigner) ausgelegt, eigene Software-Keys vom Hersteller als testweise/unsicher deklariert.[66], [67]
Caravan	ja	nein	Zustandslose Web-App, nur Watch-Only-Koordinator, keine Persistenz der Wallet-Konfiguration, rudimentäre TX-Darstellung.[68], [69]
Sparrow (gewählt)	ja	ja	Übernimmt Koordinator, PSBT-Handler und Key-Halter Rolle, besitzt Offline-Mode, transparente TX-/UTXO-Anzeige für die manuelle Verifikation, native Deskriptor- und PSBT-Unterstützung.

Tabelle 1.5: Bewertung air-gap-fähiger Multi-Signatur-Koordinatoren gegenüber Sparrow Wallet[70]

Sicherheitsmodell

Backup und Wiederherstellung

Um das System wiederherzustellen, müssen drei Komponenten an getrennten Orten redundant gesichert werden:

- **Seeds:** Die 24-Wort-Phrasen aller drei Schlüssel (Key A bis Key C) werden ausschließlich analog verwahrt (siehe 1.4.4), idealerweise an räumlich getrennten, physisch gesicherten Orten.
- **Wallet-Deskriptor:** Der `wsh(sortedmulti(...))`-Deskriptor samt der xpub-Fingerprints ist für die Wiederherstellung des Multi-Signatur-Wallets funktional zwingend erforderlich: Zwei Seeds allein genügen nicht, um die Wallet-Struktur und die Ableitungspfade zu rekonstruieren. Da der Deskriptor kein Geheimnis darstellt, kann er mehrfach kopiert und gemeinsam mit jedem Seed-Backup abgelegt werden.
- **Systemkonfiguration:** Die deklarativen NixOS-Konfigurationen samt gepinntem Flake erlauben den identischen Neuaufbau jeder VM.

Damit ist das System gegen folgende Ausfälle stärker geschützt:

- **Ausfall einer Cold-VM (Key B oder Key C):** Durch das 2-aus-3-Modell bleibt die Spend-Fähigkeit über die verbleibenden zwei Schlüssel erhalten. Die ausgefallene Maschine wird aus der Konfiguration neu aufgebaut und der Schlüssel aus dem Seed-Backup wiederhergestellt. Alternativ werden die Bestände auf ein neu aufgesetztes Wallet transferiert.
- **Ausfall der Key A VM (inklusive TPM-Defekt):** Das TPM versiegelt lediglich die lokal vorgehaltene Entropie. Die wichtigste Absicherung bleibt das auf Papier notierte Backup (Seed-Phrase), die benötigt wird, um auf einer neu aufgebauten VM diesen einzuspielen und erneut im TPM zuversiegeln.
- **Verlust oder Defekt des Wechselmediums:** Da sich pro Zeitpunkt höchstens eine PSBT auf dem Medium befindet und deren Zustand in der PSBT-DB geführt wird, geht kein Schlüsselmaterial verloren. Der betroffene Vorgang wird verworfen und neu angesto-

ßen.

- **Gleichzeitiger Verlust zweier Schlüssel:** Dieser Fall ist durch die räumlich getrennte Seed-Verwahrung abzufangen, wobei das System an seine Grenzen stößt. Diesem Risiko lässt sich nur durch geeignete Organisation vorbeugen.

Broadcast

Eine zusätzliche Integritätssicherung der finalisierten TX auf dem Rückweg zum Apphost ist nicht erforderlich. Jede nachträgliche Veränderung an Inputs, Outputs oder Beträgen invalidiert die bereits geleisteten Signaturen, sodass eine derart veränderte TX bei der Validierung durch das Bitcoin-Netzwerk zwangsläufig abgewiesen wird. Auch ein vollständiger Austausch der Datei gegen eine andere, in sich gültige TX gefährdet die Bestände nicht: Eine fremde Transaktion trägt keine Signaturen des Cold-Wallets und kann dessen UTXOs daher nicht bewegen. Eine Kompromittierung dieses letzten Schrittes beträfe somit ausschließlich das Schutzziel der Verfügbarkeit und der Broadcast unterbliebe oder müsste wiederholt werden. Es zöge keinen Verlust von Assets nach sich. Gleichwohl gilt es, diesen Zustand über das Monitoring zu erkennen und zu vermeiden.

Die operativen Abläufe des Cold-Wallets, welche das einmalige Setup sowie den manuelle Signierungs-Prozess beinhalten, umfassen Komponenten beider Systeme und werden daher nach der Vorstellung der Hot-Wallet-Services gesammelt in Abschnitt 1.4.4 beschrieben.

1.4.4 Prozesse

Nachdem alle beteiligten Services beschrieben wurden, stellt dieser Abschnitt die operativen Abläufe des Gesamtsystems dar: das einmalige Setup, den vollautomatischen Hot-Transaktionsprozess, den manuellen Operator-Pfad sowie den asynchronen Cold-Prozess mit dem zugehörigen Lebenszyklus der Nachbefüllungs-PSBT. Der Datenaustausch zwischen Online- und Offline-Systemen erfolgt ausschließlich über ein dediziertes Wechselmedium. Pro Zeitpunkt befindet sich stets nur eine Transaktion

auf dem Medium, wodurch Verwechslungen und Zustandsinkonsistenzen vermieden werden.

Cold-Setup

Vorausgesetzt wird das erfolgreiche Installieren der Cold- und Hot-Wallet-NixOS-Images. Dieses Setup lädt beim Installieren von NixOS, wie bereits erwähnt, durch deklarative Statements alle nötigen Programme und Skripte auf den Desktop. Es wird empfohlen, für die VMs von Key B und Key C zuerst die LAN-Bridge von der VM auf Hypervisor- oder Hardware-Ebene zu entfernen und anschließend Sparrow auf dem Desktop zu öffnen. Das Programm startet im Regtest-Modus, und die Variable `cold.sparrowNetwork` sollte in der Datei `configuration.nix` bei produktiver Nutzung auf den Wert `mainnet` umgestellt werden. Zu beachten ist dabei außerdem, dass bei der Umstellung auf das Mainnet sämtliche Wallets neu aufgesetzt werden müssen (siehe 1.4.4).

Anschließend muss je VM ein Wallet importiert oder eine neue mnemonische Phrase festgelegt werden. Zur Sicherung der Seed Phrase bestehend aus 24 mnemonischen Wörtern empfiehlt sich eine ausschließlich analoge, offline gehaltene Aufbewahrung wie handschriftlich auf Papier oder dauerhafter als in Metall gestanztes Seed-Backup an einem physisch gesicherten Ort wie einem Tresor. Die Phrase ist notwendig, um das Wallet wiederherzustellen und kann zur Erhöhung der Ausfallsicherheit in Form mehrerer Kopien an räumlich getrennten Orten verwahrt werden. Eine digitale Speicherung wie beispielsweise als Foto, in der Cloud oder in einem Passwortmanager ist zu vermeiden, da sie das Air-Gap-Prinzip unterläuft. Der Mandant kann die Phrase zusätzlich durch eine BIP39-Passphrase ergänzen oder nach Shamir's Secret-Sharing for Mnemonic Codes (SLIP)-39 auf mehrere Anteile aufteilen. Ein weiteres Passwort für die Verschlüsselung der Wallet in Sparrow kann festgelegt werden und ist empfohlen.

Zuletzt müssen die xpub-Schlüssel aus Key A und Key C extrahiert werden, unter der Prämisse, dass Key B und der Multi-Signatur-Koordinator auf einer Maschine zusammengelegt wurden. Das Skript `wgHMAC_export.sh` kann auf der VM von Key A ausgeführt werden, um eine automatische Ordnerstruktur anzulegen und die Wallet-Informationen in Form einer

`meta.json` zu extrahieren. Dabei werden die öffentlichen Deskriptoren für die BIP84 Ableitung (Single-Signatur) und den BIP48 Multi-Signatur Pfad gebildet.

Der Deskriptor für das Multi-Signatur Wallet muss auf beiden VM der Key B und C Wallet importiert und als eine der drei Signaturen in Form eines xpub samt Fingerprint eingetragen werden. Um ein neues Wallet für Key B und C zu erstellen, muss je VM *eine* neue (oder bekannte) Seed Phrase eingegeben werden. Durch das Eintragen der Seed-Phrase im Multi-Signatur-Wallet Kontext wird von dieser Seed-Phrase automatisch der richtige Ableitungskontext von `m/48h/1h/0h/2h` (`1h` = Coin-Type für Testnet/Regtest, im Mainnet `0h`, siehe 1.4.4) gebildet. Die Sparrow Konfiguration wird in Abbildung 1.6 dargestellt. Diese muss als neue Datei in Form ihres xpubs und Fingerprints auf das Wechselmedium kopiert werden und wie Key A als watch-only Bestandteil auf der je anderen VM eingetragen werden.

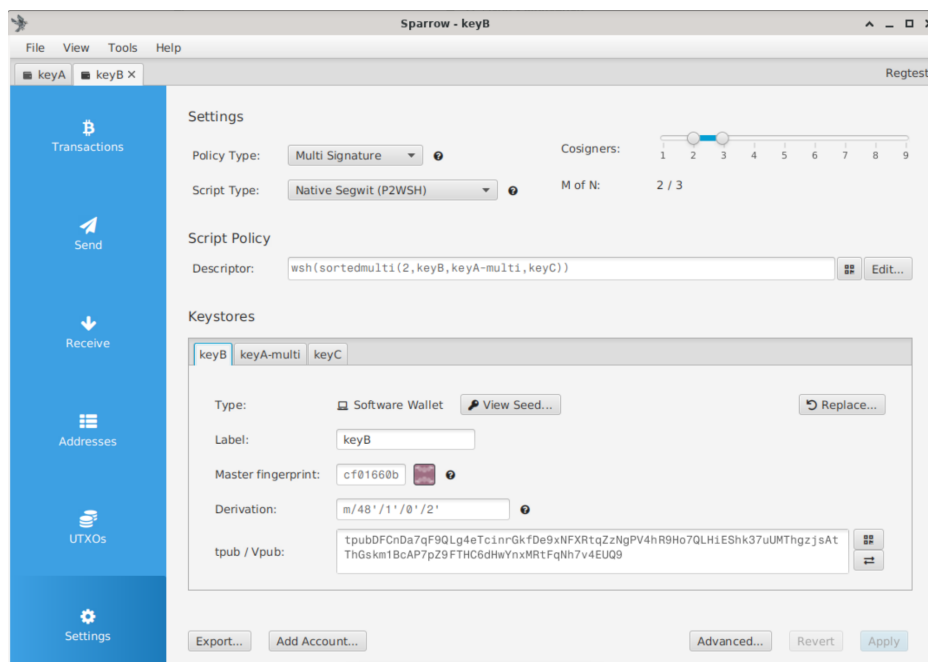


Abbildung 1.6: Sparrow Multi-Signatur - Einstellungen. Quelle: Eigenes Bildschirm-Foto aus Sparrow.

Der BIP84 Schlüssel wird primär für das automatische Signieren für das Hot-Wallet verwendet. Es empfiehlt sich jedoch ebenfalls, diesen direkt in Sparrow als separate Watch-Only-Wallet zu registrieren. Dadurch

werden dynamische Adressen, die zu diesem Wallet gehören, automatisch aufgelöst und als Zieladresse der Transaktion angezeigt.

Der daraus generierte wsh-Deskriptor muss anschließend als separate Datei auf dem Wechselmedium unter `/wallets/cold` als `cold-signer.wsh` abgelegt werden. Durch diese Nomenklatur kann der Deskriptor im nächsten Schritt vollautomatisch auf dem Apphost importiert werden (siehe 1.4.2).

Um den in Abbildung 1.7 dargestellten Prozess zu vereinfachen, stehen auf den Cold- und Hot-Wallet-VMs Hilfsskripte zum Mounten, sicheren Auswerfen und Formatieren eines USB-Mediums auf dem Desktop bereit.

Hot-Transaktionsprozess

Der automatisierte Hot-Prozess verläuft entlang der zuvor beschriebenen Services (siehe Abbildung 1.8). Dabei nimmt die Middleware den Intent über eine der externen Schnittstellen entgegen (siehe Tabelle 1.3), gleicht die Zieladresse gegen die Whitelist ab und beauftragt den TX-Builder über NATS mit dem Bau der PSBT. Die fertige PSBT wird anschließend durch OPA autorisiert (siehe 1.4.2).

Anschließend muss die PSBT von Schlüssel A signiert werden und wird dazu an die Signer-VM übertragen. Von der Schlüssel-Halter-VM wird eine PSBT zurückgegeben. Im Falle des Cold-Wallets ist 1 von 3 Signaturen gegeben, und der Prozess muss asynchron durch den Mandanten fortgesetzt werden. Eine PSBT des Hot-Wallets kann direkt über Bitcoin Core in eine finalisierte TX umgewandelt und anschließend gebroadcastet werden.

Nach jedem Broadcast des Hot-Wallets wird über Bitcoin Core der Kontostand der Wallet über `getbalances()` abgefragt und anschließend wieder an OPA weitergeleitet. Dieser bestimmt die Aktion, die Höhe der Transaktion und die PSBT-Parameter. Anschließend wird die PSBT wie im normalen Prozess gebaut und durch Key A signiert. Im Fall einer Sicherungstransaktion vom Hot- zum Cold-Wallet wird sie am Ende automatisch gebroadcastet. Im Fall einer Nachfüllung von Cold zu Hot wird der Operator benachrichtigt, den manuellen Prozess des Signierens mit Key B oder Key C fortzuführen.

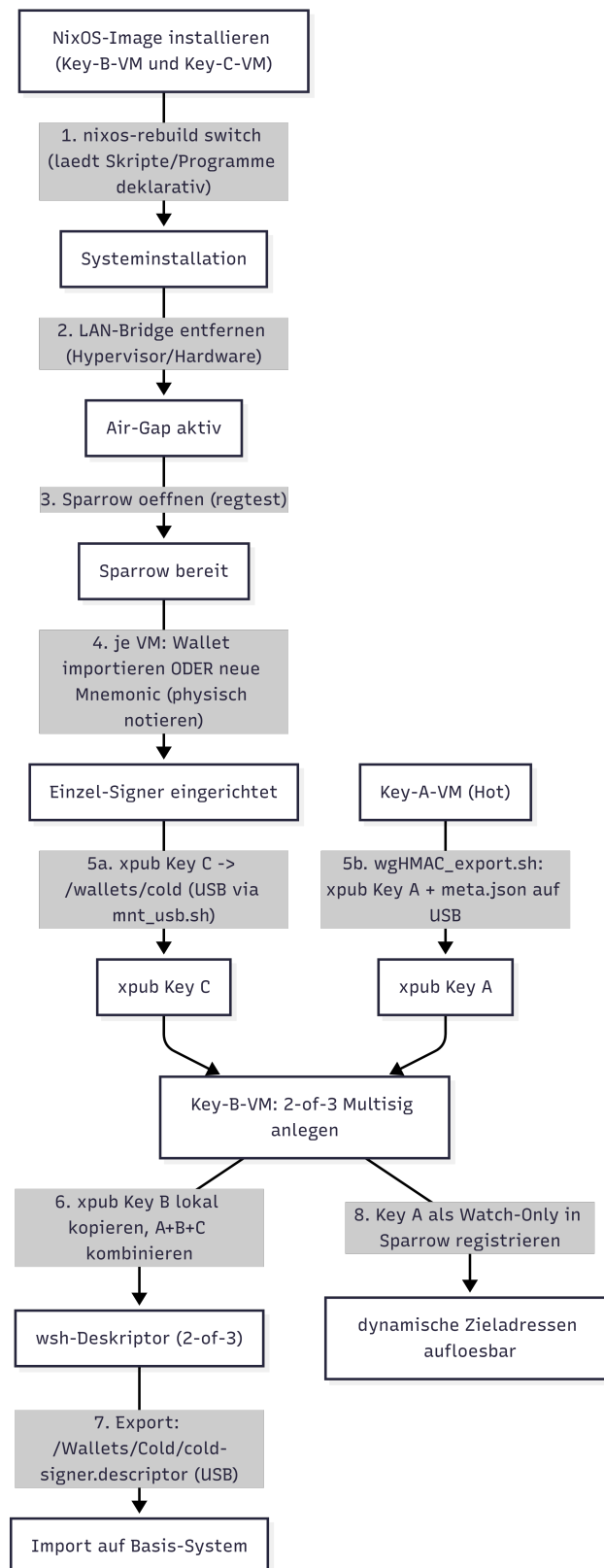


Abbildung 1.7: Darstellung der Schritte zum Setup des Cold-Wallets.

Quelle: Eigene Darstellung.

1 Self Hosted Setup

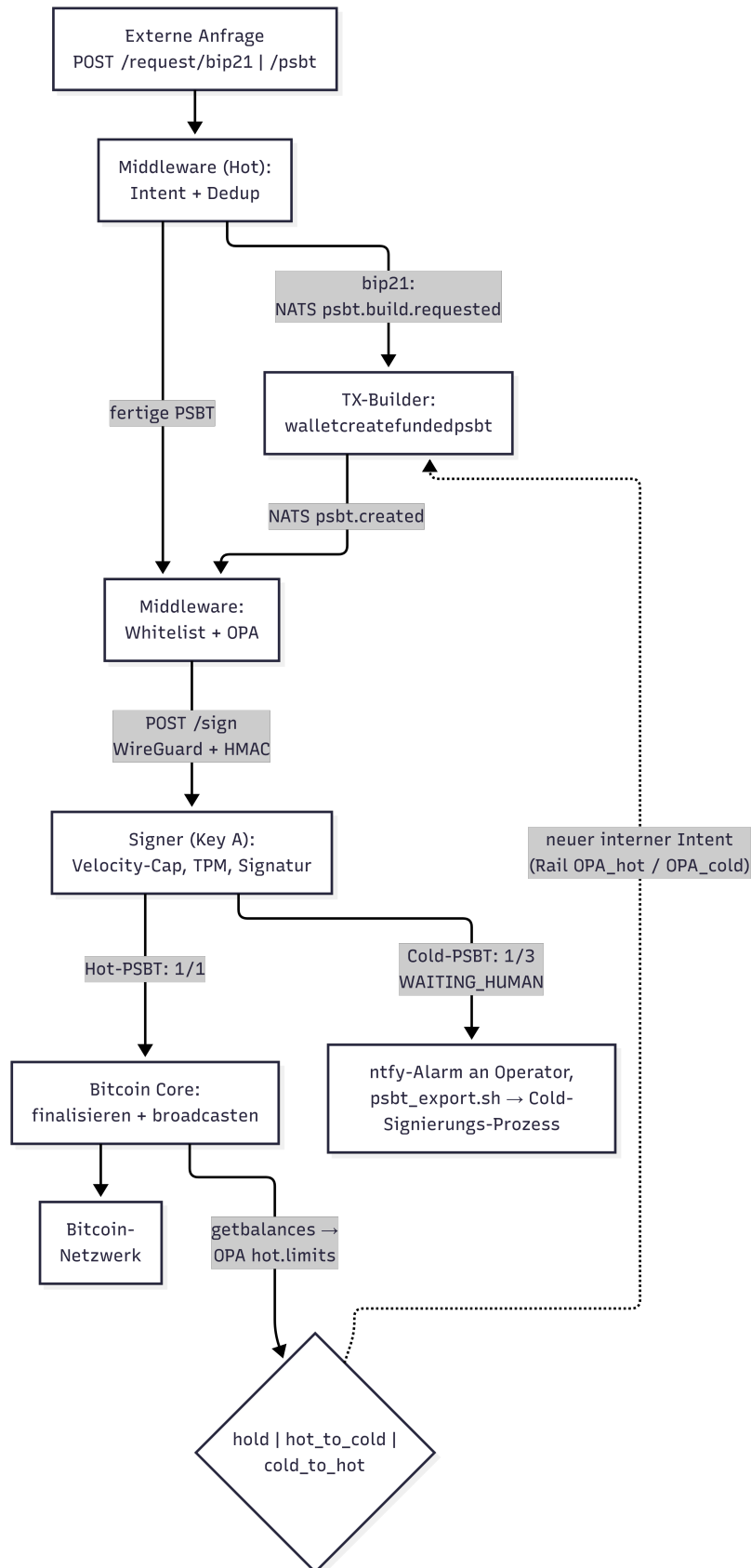


Abbildung 1.8: Darstellung des Signierprozesses für das Hot-Wallet.

Quelle: Eigene Darstellung.

Operator-Pfad

Der als Rail `manual` bereits bei der Middleware eingeführte Operator-Pfad (siehe Tabelle 1.3) besitzt gegenüber den externen Rails zwei bewusste Ausnahmen. Zum einen entfällt der Abgleich der Zieladresse gegen die externe Whitelist, sodass der Operator auch an nicht registrierte Adressen überweisen kann. Zum anderen überspringt OPA für diesen Rail die Betrags- und Gebührengrenzen. Beide Ausnahmen sind erforderlich, damit der Operator im Ausnahmefall handlungsfähig bleibt. Als Restschranken bleiben der Besitz der Operator-NATS-Zugangsdaten, die Pflichtfeld- und Struktur-Prüfung samt SHA-256-Nachweis in OPA, der Tages-Cap (`max_daily_sats`, siehe 1.4.2) sowie als harter, unabhängiger Backstop der Velocity-Cap des Signers (siehe 1.4.2) bestehen. Der manuelle Pfad ist damit ein kontrollierter Bypass mit verbleibenden Schranken, kein ungeprüfter Kanal.

Signierungs-Prozess

Der Cold-Wallet-Prozess startet für den menschlichen Operator mit der Benachrichtigung auf seinem GrapheneOS-Handy (zugestellt über den selbst gehosteten ntfy-Dienst, siehe 1.3.3). Zuvor wurde eine vordefinierte untere Grenze für den Stand der Geldmittel des Hot-Wallets unterschritten. Dadurch werden mehr Geldmittel benötigt, um die Operationalität des automatischen Systems zu gewährleisten. Die PSBT für diese Transaktion wird vollautomatisch gebildet und signiert, sodass der Operator sie im nächsten Schritt auf dem Apphost durch ein automatisches Skript, `psbt_export.sh`, in das Datei-System des Apphosts schreiben kann. Daraufhin muss diese Datei per SSH vom Apphost auf den Personal Computer (PC) des Operator und anschließend auf ein angeschlossenes USB-Medium transferiert werden ([71, Abschnitt 6, „Per SSH verbinden“]). Das GrapheneOS-Handy des Operators wird nur zur Benachrichtigung von sicherheitskritischen Geschehnissen verwendet. Zu keinem Zeitpunkt befindet sich in unserem definierten Prozess geheimes Schlüsselmaterial darauf. Koordinatoren, die Software-Schlüssel auf einem Smartphone ablegen (etwa Nunchuks mobile App, siehe 1.4.3), wurden aufgrund des fehlenden air-gap als Schlüsselhalter verworfen.

Nach dem Umstecken auf die Key B VM muss der Operator die PSBT

1 Self Hosted Setup

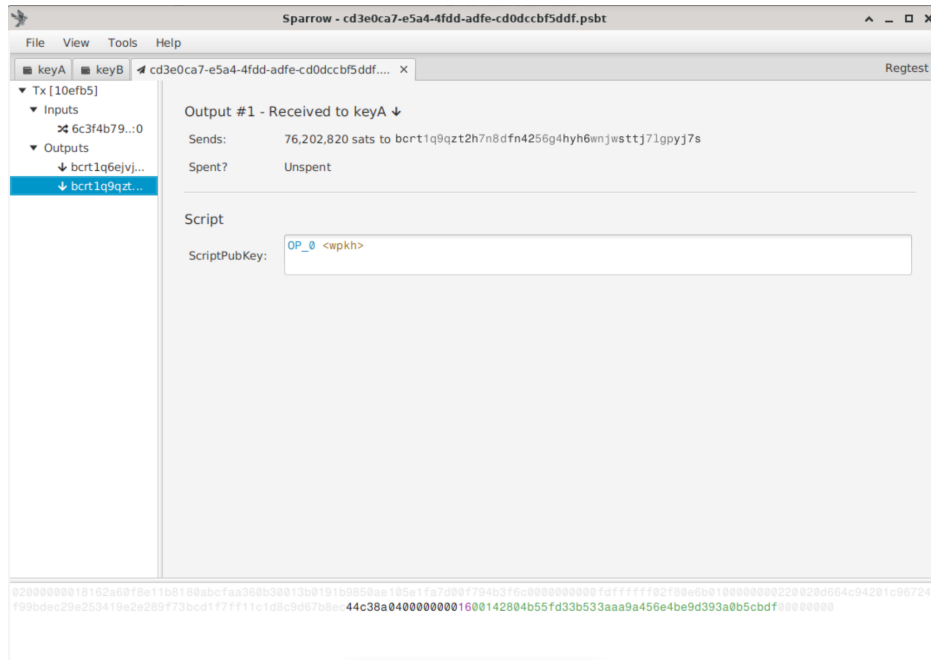


Abbildung 1.9: Darstellung der Sparrow Output-Verifikation.

Quelle: Eigenes Bildschirm-Foto aus Sparrow.

in das Key B Wallet importieren, die Details verifizieren und die PSBT signieren. Diese Verifizierung stellt sich wie im weiteren Verlauf in 1.4.5 kurz angesprochen als letzte Verteidigungsmaßnahme für das Cold-Wallet dar und stellt einen Grund für die Auswahl von Sparrow-Wallet unter den Alternativen dar (siehe 1.4.3). Unter **Outputs** können die genauen Höhen der Bewegungsgrößen bestehend aus Coins, Übertragungswerten (**Output #1**) und Wechselgeld (**Output #0**) und deren Zieladressen angesehen werden. Durch das Hinzufügen des öffentlichen Key A BIP84 Single-Signatur Deskriptors als eigenständiges Wallet in Sparrow (siehe 1.4.4), wird die Zieladresse direkt als zu Key A gehörig aufgelöst (siehe **Output #1 - Received to KeyA** in der Abbildung und bereits erwähnt in 1.4.3). Diese Visualisierung ist in Grafik 1.9 dargestellt. Diskrepanzen in dieser Überprüfung führen dazu, dass die PSBT nicht signiert werden darf und weisen auf eine Kompromittierung des Apphosts hin.

Nach der Signierung kann eine finalisierte TX direkt als Datei mit der neuen Endung **.txn** auf dem Wechselmedium gespeichert werden. Dies ist wie in Abbildung 1.10 direkt über den Knopf **Save Final Transaction** auf Sparrow möglich. Das dargestellte Bild zeigt zudem das als Alternative beschriebene direkte Broadcasting über Sparrow im

Online-Modus aus Kapitel 1.4.5. Danach kann das USB-Medium wieder in den PC des Mandantens eingesteckt, die Datei per SSH in den Apphost transferiert und über das Skript `psbt_broadcast.sh` importiert sowie automatisch über Bitcoin Core gebroadcastet werden[71, Abschnitt 6, „Per SSH verbinden“].

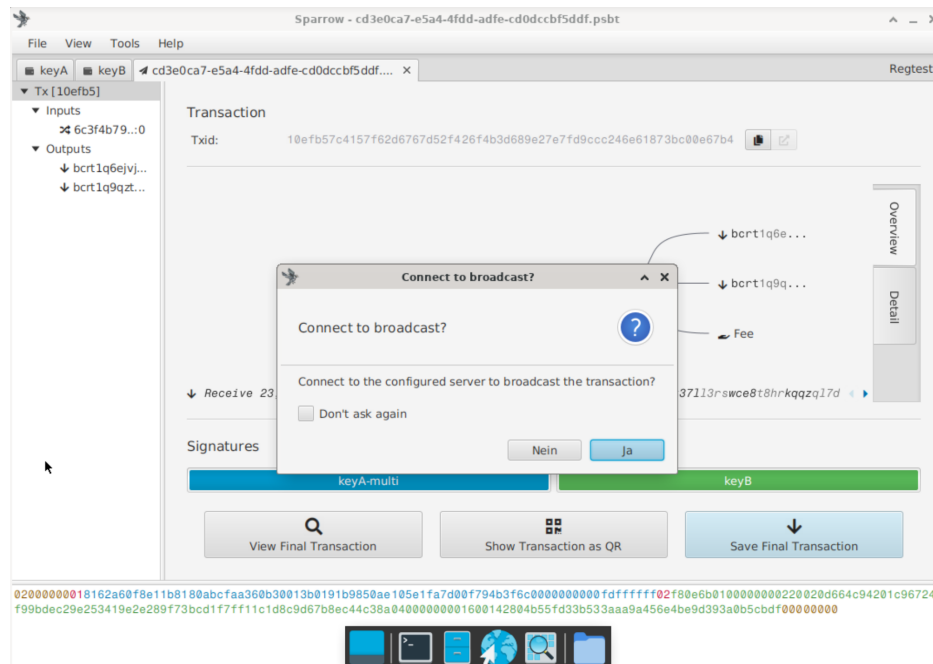


Abbildung 1.10: Sparrow Wallet Broadcast und Final TX Export Funktion nach Signatur.

Quelle: Eigenes Bildschirm-Foto aus Sparrow.

Eine tiefere Beschreibung der Konfiguration und des gesamten Setup- und Signierungsprozesses kann in der `README.md` des NixOsCold-GitHub nachgelesen werden [72]. Eine Kurzübersicht aller Skripte bietet Tabelle A5.4 im Anhang.

Refill-Lebenszyklus

Da bis zur Befüllung durch den asynchronen Cold-Prozess geraume Zeit vergehen kann, müssen die Balance-Grenzen der `hot.limits`-Policy (siehe 1.4.2) mit ausreichend Puffer gewählt werden. Diese wurden, wie beschrieben, als weitere OPA-Policy umgesetzt und können in der `data.json` durch den Mandanten an seine Geldmittel angepasst werden. Da der automatisierte Hot-Flow während der Wartezeit des asynchronen

1 Self Hosted Setup

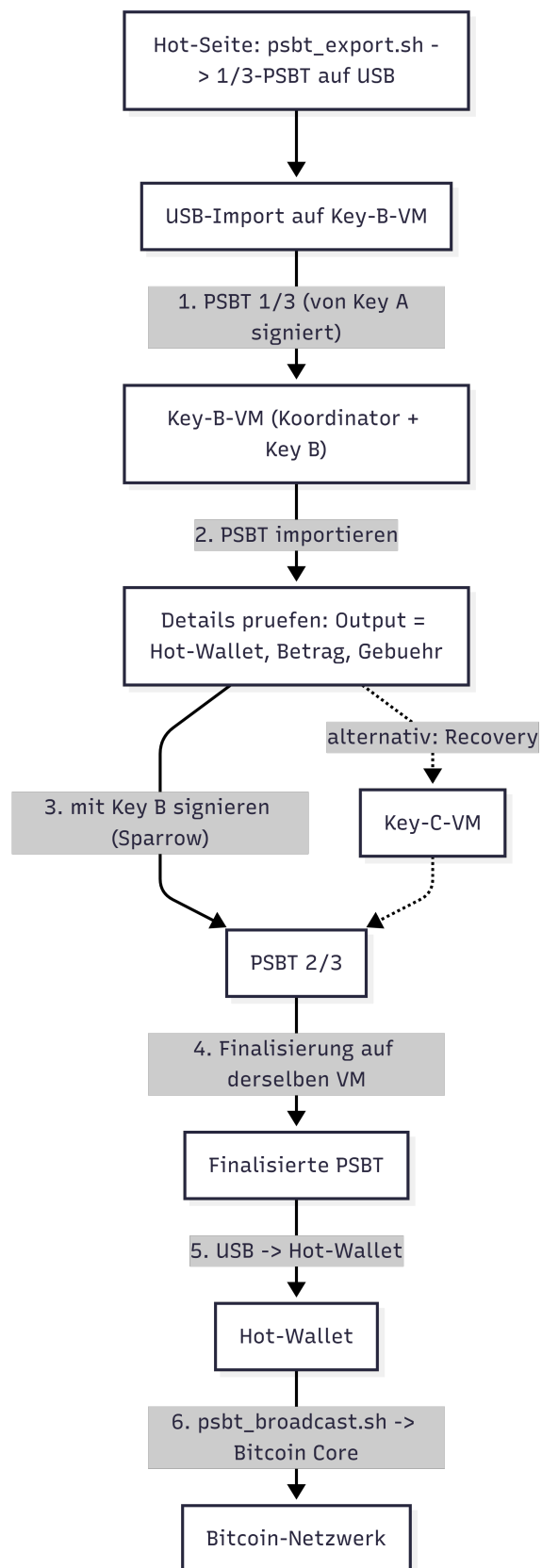


Abbildung 1.11: Darstellung des Signierprozesses für das Cold-Wallet.

Quelle: Eigene Darstellung.

Cold-Prozesses unverändert weiterläuft und nach jedem Broadcast erneut eine Balance-Prüfung über die `hot.limits`-Policy auslöst, kann der Nachbefüllungsbedarf in dieser Phase mehrfach gemeldet werden. Ohne Gegenmaßnahme entstünden dadurch konkurrierende Nachbefüllungs-PSBTs für unterschiedliche Höhen desselben Mangels, und der Operator könnte veraltete TXs signieren. Zudem kann deren Broadcast zu einer Überbefüllung des Hot-Wallets und damit zu einem gegenläufigen `hot_to_cold`-Intent führen, wodurch Zeit des Operators und Gebühren für die eiligen Transaktionen verschwendet würden.

Zur Auflösung der Problematik wurde sich für eine Logik entschieden, bei der sich zu jedem Zeitpunkt nur genau eine aktive Nachbefüllungs-PSBT im System befindet. Die persistente Ablage hält stets nur die jeweils jüngste Nachbefüllungs-PSBT vor. Eine neu erzeugte PSBT ersetzt eine bereits wartende durch Überschreiben der Datei.

Zudem wird der Lebenszyklus über drei gezielte Löschpunkte abgesichert: Beim Export auf das Wechselmedium wird die wartende PSBT eingelesen, aus der persistenten Ablage entfernt und in den Zustand `COLD_STARTED` überführt. Ab diesem Punkt gilt die Transaktion als in der Hand des Operators. Die Single-TX-Prüfung des Export-Skripts verhindert, dass eine weitere PSBT auf dasselbe Medium gelangt. Die persistente Ablage der Datei wird gelöscht, sodass der Signierungsprozess nicht erneut gestartet werden kann und damit die PSBT-Status-DB als Source of Truth nicht gestört werden könnte.

Beim Broadcast der signierten Cold-PSBT wird erneut eine Löschung ausgelöst. Diese greift gezielt den Fall ab, dass während der Wartezeit des Operators, also zwischen Export und Broadcast, eine weitere Nachbefüllungs-PSBT erzeugt und in der Ablage hinterlegt wurde. Da der soeben abgeschlossene Broadcast den ursprünglichen Mangel in einer gewissen Höhe bereits behoben hat, wäre eine solche zwischenzeitlich entstandene PSBT auf einen überholten Saldo gestützt und wird daher verworfen.

Der dritte Löschpunkt besteht in der vollautomatischen Löschung der Nachbefüllungs-PSBT, wenn diese noch nicht vom Operator aufgenommen wurde. Die Löschung wird ausgelöst, wenn OPA eine `hot_to_cold`- oder `hold`-Aktion ermittelt. Dementsprechend muss seit Generierung

der PSBT dem Hot-Wallet eine ausgleichende Menge an Geldmitteln gutgeschrieben worden sein, was erst mit der nächsten Hot-TX auffällt. Praktisch stammt dieser Zufluss, der den Hot-Saldo wachsen lässt, aus regulären eingehenden Zahlungen Dritter an das Hot-Wallet. `hot_to_cold` und `cold_to_hot` Zahlungen wurden bereits von den anderen Löschpunkten abgedeckt. Solche TXs können von jedem Wallet-Inhaber ohne Zustimmung der Zieladresse ausgeführt werden, wie wir es ebenfalls vornehmen. Da es sich bei dem Hot-Wallet um ein vollautomatisches TX-System handelt, ist der Fall, dass Partner ebenfalls Geld auf dieses Wallet senden, denkbar und musste in die Logik eingeplant werden.

Ein verbleibender Randfall besteht, wenn der ausgleichende Zufluss erfolgt, nachdem die Nachfüll-PSBT erzeugt, aber bevor eine weitere Hot-Transaktion die Balance-Prüfung erneut auslöst. Nimmt der Operator die zu diesem Zeitpunkt bereits obsolete PSBT auf, ist sie nach dem Export keinem der drei Löschpunkte mehr zugänglich. Ihr Broadcast führt dann zu einer Überbefüllung und einer gegenläufigen `hot_to_cold`-Transaktion mit doppeltem Gebührenanfall. Eine vollständige Absicherung wäre durch eine erneute Bedarfsprüfung unmittelbar vor dem Broadcast erreichbar, wobei ein ZMQ-basiertes Mithören auf Zuflüsse, das bei einer TX-Erkennung für das Hot-Wallet eine Reevaluiierung durch OPA auslöst, das Fenster zwar verkleinern, aber aufgrund des verbleibenden Export-Zeitpunkts nicht schließen würde. Aufgrund der geringen Eintrittswahrscheinlichkeit wurde der Randfall als akzeptabel eingestuft.

Ein aktiver Neuaufbau der Nachfüll-PSBT nach der Löschung erfolgt anschließend nicht. Stattdessen bleibt die Ablage leer, bis die nächste reguläre Hot-Transaktion über die Balance-Prüfung erneut einen Bedarf feststellt. Dadurch basiert jede neu erzeugte Nachbefüllungs-PSBT zwangsläufig auf dem aktuellen, nach dem letzten Broadcast gültigen Wallet-Stand, womit zugleich das Problem einer auf veraltetem Saldo beruhenden Empfehlung strukturell ausgeschlossen ist.

Für eine einfachere Bedienung durch den Operator können folgende Skripte im Apphost ausgeführt werden:

- `scripts/hotwallet/ops/refill/psbt_export.sh`: exportiert die letzte Cold-PSBT auf ein Wechselmedium unter `/psbt/<id>.psbt`

und erfüllt die Voraussetzung für den Cold-Signierungsprozess.

- `scripts/hotwallet/ops/refill/psbt_broadcast.sh`: importiert eine Cold-PSBT von einem Wechselmedium aus `/psbt/<id>.txn`.

Zudem wird mit `scripts/hotwallet/ops/refill/psbt_broadcast.sh` die PSBT an den Apphost weitergegeben, die PSBT-Details wie ID, SATS-Menge und Adresse mit der DB abgeglichen und diese nach Bestätigung finalisiert und gebroadcastet.

Produktivübernahme - Mainnet-Migration

Die Laborumgebung ist durchgängig auf das Regtest-Netzwerk ausgelegt. Bei der Übernahme in den Produktivbetrieb erfordert der Netzwerkwechsel zwingend eine Anpassung der BIP32-Ableitungspfade, da sich der zugrundeliegende Coin-Type ändert (Testnet/Regtest: 1', Mainnet: 0'). Aus `m/84'/1'/0'` wird `m/84'/0'/0'` und aus `m/48'/1'/0'/2'` wird `m/48'/0'/0'/2'`. Damit sind sämtliche xpubs, Deskriptoren und die daraus registrierten Wallets neu zu erzeugen und erneut über das Wechselmedium auszutauschen. Für die Migration sind folgende Schritte erforderlich:

- Umstellung von Bitcoin Core auf das Mainnet und initiale Blockchain-Synchronisation,
- Umstellung des Sparrow-Netzwerkmodus auf allen Schlüsselhalter-VMs,
- Neuerzeugung der Seeds beziehungsweise Neuableitung der Mainnet-Pfade, Neuaufbau des Multi-Signatur-Wallets (siehe 1.4.4) und erneuter Deskriptor-Import auf dem Apphost,
- Anpassung der OPA-Regel Netzwerk in `data/data.json`, die im Auslieferungszustand ausschließlich `regtest` zulässt (siehe Tabelle A5.3).

1.4.5 Alternativen

Neben der primär dargestellten Implementation kann der Mandant nach Übergabe des simulierten Systems entscheiden, zu anderen Ansätzen zu wechseln. Wie bei den Sparrow-Wallet Alternativen in 1.4.3 wurden verschiedene Ansätze evaluiert und werden in diesem Abschnitt diskutiert.

Hardware-Wallets

Es bietet sich an, dedizierte Hardware-Wallets anstelle der Key-Holder-NixOS-VMs zu verwenden. Diese haben sich als etablierter Industriestandard für die Verwahrung von Bitcoin-Schlüsselmateriale herausgebildet. Verbreitete Geräte sind dabei quelloffen oder zumindest mit einsehbarer Firmware verfügbar. Ihr wesentlicher Sicherheitsgewinn liegt in einem dedizierten Display auf der Gerätehardware, über das TX-Details unabhängig vom angeschlossenen Rechner verifiziert werden können. Dies ist analog zu der in unserem Prozess vorgesehenen manuellen Verifikation in Sparrow Wallet, jedoch auf einer eigenständigen, sehr schwer kompromittierbaren Hardware.

Electrum-Server

Ein Electrum-Server wie **ElectrumX**, **electrs** oder **Fulcrum** ist eine Server-Software, die zwischen einer Bitcoin-Full-Node und leichtgewichtigen Wallet-Clients vermittelt. Er indexiert die Blockchain nach Adressen bzw. Script-Pubkeys, sodass eine Wallet über das Electrum-Protokoll effizient die UTXOs und die Transaktionshistorie ihrer Adressen abfragen kann, ohne selbst die gesamte Kette durchsuchen zu müssen. Sowohl Electrum als auch Sparrow lassen sich auf diese Weise anbinden.

Bei Nutzung eines *öffentlichen* Electrum-Servers übermittelt die Wallet dem Serverbetreiber sämtliche zugehörigen Adressen, wodurch dieser erkennt, welche Adressen zu derselben Wallet gehören. Über die öffentliche Blockchain kann der Betreiber zudem deren Guthaben, Aktivität und Metadaten ermitteln und mit der IP-Adresse verknüpfen. Es besteht zwar keine Gefährdung der Assets, da beim Broadcast nur die

fertig signierte, öffentliche Transaktion übertragen wird und kein privates Schlüsselmaterial, jedoch ein Verlust an Privatsphäre gegenüber Dritten.

Im vorliegenden System entfällt ein Electrum-Server vollständig, da das Hot-System bereits einen eigenen Bitcoin-Core-Full-Node betreibt. Sämtliche Adress- und UTXO-Abfragen zur Generierung der PSBTs verbleiben damit auf der selbst gehosteten Infrastruktur, ohne Offenlegung gegenüber Dritten. Die air-gapped Cold-VMs benötigen ohnehin keinen Server, da sie die Transaktionsdetails ausschließlich aus der PSBT selbst verifizieren. Die Möglichkeit, Sparrow für einen alternativen Broadcast-Schritt direkt an Bitcoin Core anzubinden, bleibt dem Mandanten offen (siehe 1.4.5). Der Betrieb eines *selbst gehosteten* Electrum-Servers kann Adressabfragen bei Wallets mit umfangreicher Historie beschleunigen, bringt für die hier verwendeten, frisch erzeugten Watch-Only-Wallets jedoch keinen relevanten Vorteil und verbleibt im Ermessen des Mandanten [37], [73], [74].

Datentransfer

Der Mandant kann den USB-Austausch ebenfalls durch die Generierung von QR-Codes und eine Kamera an den VMs oder durch direkte Hardware-Wallets ersetzen. Dies sollte die Möglichkeit einer Manipulation in Transit weitestgehend unterbinden, wobei die Transaktionsdetails weiterhin kontrolliert werden sollten [32], [75], [76]. Eine nötige Verbindung zum Apphost per SSH bleibt bestehen. Zudem konnten diese Übertragungstechnologien wie auch die Hardware-Wallets nicht in der Proxmox-Laborumgebung virtualisiert werden. Der implementierte USB-Austausch stellt sich als einfaches Wechseln des Besitzers einer virtualisierten Festplatte dar und ließ sich somit sehr gut für die spätere praktische Verwendung simulieren.

Online-Koordinator

Ein früherer Architekturansatz sah vor, die Koordinator-Instanz nicht vollständig air-gapped zu betreiben, sondern diese über eine direkte Netzwerkverbindung an Bitcoin Core anzubinden. Dieser Ansatz basiert

auf der nativen Unterstützung von Sparrow Wallet, über eine direkte Verbindung mit einem Bitcoin-Core-Node zu kommunizieren. In diesem Szenario hätte Sparrow die vollständige Verarbeitungskette im Cold-Kontext übernehmen können:

- Import und Verifikation der PSBT,
- Koordination und Durchführung der Multi-Signatur-Signierung,
- Finalisierung der Transaktion,
- direkter Broadcast an das Bitcoin-Netzwerk über die bestehende Node-Verbindung.

Die Funktionalität des direkten Broadcast nach der Signierung kann in Abbildung 1.10 nachvollzogen werden. Die Cold-Architektur hätte in diesem Fall aus drei getrennten Systemen bestanden:

- Key B (vollständig air-gapped),
- Key C (vollständig air-gapped),
- Koordinator (netzwerkfähig, ohne Zugriff auf Schlüsselmateriale).

Durch die native Integration von Sparrow Wallet mit Bitcoin Core bietet sich dieser Ansatz technisch an, da keine zusätzliche Infrastruktur für den Broadcast erforderlich ist und der gesamte Transaktionsprozess innerhalb des Koordinators abgeschlossen werden kann. Im weiteren Verlauf wurde dieser Ansatz jedoch verworfen, da er durch ein weiteres System mehr Last für den Operator bedeutet und zwei verschiedene Wege für das Broadcasting darstellt. Dies sollte konsolidiert und schlanker werden, wodurch sich sowohl die Angriffsfläche des Gesamtsystems als auch der Bedarf an Ressourcen verringert.

In einem zugehörigen früheren Ansatz wurde Sparrow Wallet im Cold-System direkt mit Bitcoin Core verbunden und das Cold-Wallet über diese Integration registriert. Diese Idee wurde verworfen, da sich für einen einheitlichen Import der Hot- und Cold-Wallet-Deskriptoren entschieden wurde und der Koordinator mit dem Schlüsselmateriale-Halter für Key B zusammengelegt werden sollte. Dementsprechend sollte diese Maschine zu keinem Zeitpunkt nach dem Setup mit dem Internet verbunden sein, weshalb das Air-Gap direkt nach dem Build angewandt wird.

Insbesondere ergeben sich folgende Nachteile:

- Der Cold-Bereich ist noch weiter vom Netzwerk abhängig, was das ursprüngliche Sicherheitsmodell abschwächt,
- die Koordinator-Instanz wird zu einem aktiven Bestandteil der sicherheitskritischen Entscheidungslogik und muss zusätzlich vertraut werden,
- potenzielle Angriffe auf die Benutzerinteraktion oder die Anzeige der Transaktionsdetails können nicht mehr vollständig ausgeschlossen werden.

Die gewählte Architektur trennt hingegen die Entscheidungs- und Kommunikationsdomäne strikt:

- Cold-Systeme übernehmen ausschließlich die Verifikation und Signierung,
- das Hot-System übernimmt den Broadcast.

Nichtsdestotrotz stellt diese Variante für bestimmte Anwendungsfälle eine valide Alternative dar. Insbesondere bei reduziertem Fokus auf maximale Isolation oder erhöhten Anforderungen an die Bedienbarkeit kann die Integration des Broadcasts in die Koordinator-Instanz den operativen Aufwand deutlich reduzieren. Durch die Reduktion manueller Schritte, insbesondere des Rücktransfers der finalisierten Transaktion, kann die Fehleranfälligkeit im operativen Prozess gesenkt werden.

Hashing

Zusätzlich zur beschriebenen Multi-Signatur-Architektur wurde eine ergänzende, Hash-basierte Freigabeschicht implementiert und nach einer weiteren Prüfung wieder verworfen. Diese dient nicht der Autorisierung der Bitcoin-Transaktion selbst, sondern der Absicherung des organisatorischen Freigabeprozesses während des physisch getrennten Datentransfers. Im Rahmen dieses Ansatzes wurde auf der Koordinator-Instanz ein separates kryptographisches Schlüsselpaar generiert. Der zugehörige Public Key wird über ein Wechselmedium auf die Signer-Systeme übertragen und dort hinterlegt.

Für jede freizugebende PSBT wurde ein zusätzlicher Freigabemechanismus angewendet:

- Erstellung eines kryptographischen Hashes der PSBT,
- Signierung dieses Hashes durch den Koordinator,
- Übertragung der Signatur zusammen mit der PSBT,
- Verifikation der Signatur auf den Key-Holder-Systemen.

Hierdurch kann überprüft werden, ob eine Transaktion tatsächlich durch die Koordinator-Instanz autorisiert wurde und während des Transports nicht verändert wurde. Insbesondere in Szenarien mit erhöhten Anforderungen an Auditierbarkeit, mehreren Operatoren, langen Pausen oder bei stark regulierten Prozessen kann eine solche zusätzliche Verifikationsschicht sinnvoll sein. Ebenso kann sie in Umgebungen mit reduziertem Vertrauen in den manuellen Prozess zusätzliche Sicherheit bieten.

Im weiteren Verlauf der Ausarbeitung wurde dieser Mechanismus nicht als Bestandteil der finalen Architektur übernommen. Der Grund hierfür liegt darin, dass die Integrität und Nachvollziehbarkeit der Transaktionsdaten bereits durch die Kombination aus PSBT-Struktur, Multi-Signatur-Modell, Herkunfts-Authentifizierung über Key A (siehe 1.4.6) und manueller Verifikation im Cold-System gewährleistet wird (siehe 1.4.4). Die relevanten Transaktionsparameter wie Zieladressen, Beträge und Gebühren werden in Sparrow Wallet vollständig transparent dargestellt und durch den Operator überprüft. Eine Manipulation der PSBT würde unmittelbar sichtbar und im Rahmen des Signierungsprozesses erkannt. Die zusätzliche Hash-basierte Freigabeschicht stellt somit keine notwendige Sicherheitsanforderung für die vorliegende Architektur dar und wäre in einer späteren Erweiterung durch Übertragung auf visuellem Wege mittels QR-Codes und Kameras sowie gegebenenfalls Sparrows nativer Erkennung schwieriger umsetzbar.

1.4.6 Sicherheitsbetrachtung

Die Virtualisierung aller VMs über Proxmox als Hypervisor ist eine bewusste Entscheidung, um die Architektur unter Realbedingungen in einer Art Labor zu testen. Eine spätere produktive Umsetzung durch den

Mandanten sollte verschiedene, vollständig getrennte, nicht-virtualisierte Maschinen umfassen, sodass Proxmox-spezifische Angriffsszenarien direkt auf den Hypervisor nicht weiter diskutiert werden.

Transaktion

Die Integrität der PSBT gegen gezielte Manipulation wird nicht über einen einzelnen, mitgeführten Hash sichergestellt, da dieser lediglich auf versehentliche Korruption auf dem Transportweg hinweisen kann. Ein Angreifer, der die Nachricht verändern kann, wird auch den Hash unmittelbar mitfälschen. Stattdessen wird das Schutzziel über mehrere, ineinandergreifende Schichten erreicht, die jeweils an der Phase ansetzen, in der die Transaktion am verwundbarsten ist:

- **Unsigniert (Intent bis Signatur):** In diesem Abschnitt wäre eine Veränderung von Betrag oder Zieladresse grundsätzlich noch wirksam. Sie wird jedoch nicht strukturell, sondern *inhaltlich* abgefangen: Der OPA prüft Betrags-, Gebühren- und Netzwerkgrenzen, und der Whitelist-Abgleich stellt sicher, dass die Zieladresse einem registrierten Wallet zugeordnet ist. Eine manipulierte Adresse oder ein manipulierter Betrag führt damit zur Ablehnung, unabhängig davon, ob die PSBT auf dem Weg verändert wurde.
- **Übergabe an den Signer:** Die Kommunikation zur schlüsselhaltenden VM ist über WireGuard verschlüsselt und zusätzlich über einen HMAC aus Zeitstempel, Nonce und Request-Body authentifiziert. Eine Veränderung eines dieser Bestandteile invalidiert die Signatur, sodass nur unverfälschte und autorisierte Signieraufträge den Schlüssel erreichen.
- **Signierte Phase:** Sobald die PSBT die kryptografischen Signaturen der Schlüssel trägt, ist sie gegen jede weitere Manipulation auf jedem Übertragungsweg geschützt. Jede nachträgliche Veränderung von Inputs, Outputs oder Beträgen invalidiert die Signaturen, woraufhin die Finalisierung beziehungsweise das Netzwerk die Transaktion verwirft. Die Bitcoin-Signatur selbst bildet damit den stärksten Integritätsanker des Systems.

Die verbleibende Lücke besteht in der Übertragung der *unsignierten*

PSBT zwischen Middleware und TX-Builder über NATS. Diese wird nicht durch einen weiteren Hash geschlossen, sondern durch die beschriebene Absicherung des Event-Busses (subjektbezogene Autorisierung, perspektivisch kryptografisch verankerte Dienst-Identitäten). Die Integrität gegen Angreifer wird somit durchgängig über kryptografische Authentizität auf der jeweils passenden Schicht erreicht und nicht über einen Mechanismus, der gegen gezielte Manipulation prinzipiell wirkungslos bliebe.

Multi-Signatur mit Schlüssel-Doppelnutzung

Die Doppelnutzung von Key A wurde in diesem Ansatz bewusst gewählt: Er signiert das als Single-Signature betriebene, internet-exponierte Hot-Wallet und ist zugleich einer der drei Cosigner des 2-aus-3-Multi-Signatur-Cold-Wallets. Dieser Ansatz ist der Kern des dargestellten Schlüsselmodells und ermöglicht es, eine Cold-Transaktion bereits im Hot-System initial zu signieren (1/3) und ihre Authentizität so vor Eintritt in den Cold-Bereich nachzuweisen [27]. Dies lässt sich in Abbildung 1.12 erkennen, wo bereits die Signatur von Key A erkannt wird und so einen Sicherheitsanker für den Operator darstellt.

Technisch wird dies umgesetzt, indem der Seed von Key A für zwei verschiedene Pfade abgeleitet wird. Es handelt sich also um denselben physischen Schlüssel in zwei Deskriptor-Rollen.

- $m/84'/1'/0'$: Als Single-Signatur für das Hot-Wallet nach BIP84 und
- $m/48'/1'/0'/2'$: Als P2WSH-Cosigner für das Cold-Wallet nach BIP48.

Der Ansatz stellt einen bewussten Kompromiss zwischen Operationalität und Isolation dar, wobei eine Schwächung des Sicherheitsniveaus des Cold-Wallets in Kauf genommen wird. Wird das Hot-Wallet vollständig kompromittiert, so verfügt ein Angreifer bereits über einen der drei Cold-Schlüssel. Die effektive Hürde des Cold-Wallets reduziert sich damit aus Sicht des Angreifers von „zwei beliebige der drei Schlüssel“ auf „einen weiteren der verbleibenden Schlüssel B oder C“.

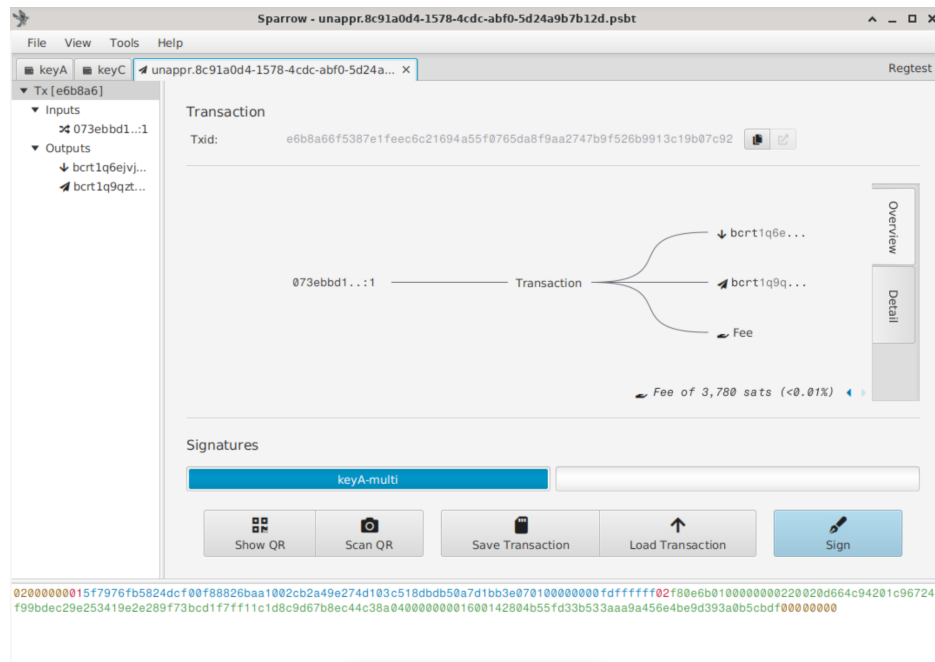


Abbildung 1.12: Import mit vorsignierten Key A Schlüssel. Quelle: Eigenes Bildschirm-Foto aus Sparrow.

Diese Schwächung ist im vorliegenden Bedrohungsmodell vertretbar, da sie ausschließlich die Cold-Schlüssel betrifft. Diese sind durch das Air-Gap genau gegen den Remote-Angriffsvektor abgesichert, der gegen das Hot-Wallet wirksam ist. Ein Angreifer müsste somit zusätzlich zur Hot-Kompromittierung den physischen Perimeter am Standort des Mandanten überwinden, der bis auf Angriffe über das Wechselmedium nicht Teil der Anforderungsbeschreibung des Mandanten ist. Sollte ein Angreifer diese Hürden dennoch umgehen und einen Cold-Schlüssel extrahieren, wäre die Extraktion eines weiteren Schlüssels keine wesentlich größere Hürde, da Key B und Key C auf architektonisch identischen Maschinen liegen.

Umgekehrt ist das Hot-Wallet durch die TPM-Härtung von Key A gegen genau diesen physischen Angriffsvektor stärker geschützt: Bei Aus- und Wiedereinbau der Hardware in ein anderes System würde das geheime Schlüsselmaterial nicht preisgegeben, da sich der Boot- und Konfigurationszustand und damit die PCR-Bindung ändert. Dementsprechend ergänzen sich Hot- und Cold-Wallet gegenseitig im Schutz der Mittel. Der maximale Schaden einer einfacheren, über das Online-System vollzogenen Kompromittierung bleibt dadurch auf den im Hot-Wallet vorgehaltenen Anteil von circa 5 % der Mittel beschränkt [31].

Dem Mandanten bleibt es offen, dies durch Hinzufügen einer weiteren Cold-Signer-Maschine zu verschärfen und das Multi-Signatur-Verfahren auf 3-aus-4 zu erhöhen. Zu beachten bleibt, dass sich dadurch die für jeden `cold_to_hot`-Swap benötigte Zeit erhöht und sich damit die kritische Phase, in der die Operationalität des Hot-Wallets gefährdet ist, verlängert. Bei einem erhöhten Aufkommen automatischer Hot-Wallet-TXs und entsprechend vieler Nachbefüllungs-PSBTs kann dies einen deutlichen Unterschied ausmachen.

Ein Herauslösen von Key A aus dem Cold-Wallet-Schlüsselsatz wird nicht empfohlen, da dadurch die Authentizitätssignatur des Hot-Systems und die darauf gestützte Verifikation der PSBT entfielen.

Doppelter Velocity-Check

Der Zweck der zwei örtlich getrennten Durchsetzungspunkte aus 1.4.2 und 1.4.2 liegt in der Abwehr eines mehrstufigen Angriffs. Auch wenn ein Angreifer den Apphost unterwandert und den OPA-Grenzwert durch Fälschen von `spent_today` oder Umgehung der OPA-Abfrage deaktivieren kann, bleibt der Signierer als zweiter, unabhängiger Enforcer bestehen. Dieser deckelt den tatsächlichen Schlüssel-Abfluss über ein selbstkontrolliertes rollierendes Fenster und begrenzt den maximalen Schaden auf das vom Mandanten am Signierer konfigurierte Velocity-Limit. Dieses wird direkt aus den UTXOs der PSBT berechnet und stellt sich als schwerer angreifbar dar. Dies überträgt das Prinzip der *Segregation of Duties* auf die Abflusskontrolle und vermeidet einen Single Point of Compromise auch für das Velocity-Limit. Eine mögliche Kompromittierung des Redis-Service, welcher sich auch auf diesen Sicherheits-Aspekt bezieht, wird im Folgenden behandelt (siehe 1.4.6). Eine prägnante Darstellung dieses Sachverhalts kann in Tabelle 1.6 gefunden werden.

Bei der Anpassung sind die Relationen der Grenzwerte zueinander zu erhalten. Die Einzeltransaktions-Grenze liegt unterhalb des Daily-Caps, der Daily-Cap unterhalb des Velocity-Caps, sodass im Normalbetrieb stets die OPA-Grenzen des Apphosts binden und der Signer-Cap ausschließlich als Backstop im Kompromittierungsfall greift. Der Velocity-Cap muss zudem genügend Puffer oberhalb des Daily-Caps besitzen, damit interne Ausgleichs-Transaktionen nicht durch ihn blockiert werden. Die Grenzen

Grenzwert	Zweck	Durchsetzungsort	Konfiguration
Betrag je TX	begrenzt den Schaden einer einzelnen unautorisierten Zahlung	OPA (Apphost)	data/data.json
Gebühr je TX	verhindert Mittelabfluss über überhöhte Gebühren	OPA (Apphost)	data/data.json
Tages-Abfluss	deckelt den kumulierten Abfluss regulärer Zahlungen	OPA (Apphost)	max_daily_sats
Velocity-Cap	unabhängiger Backstop, überlebt eine Kompromittierung des Apphosts	Signierer (Key A VM)	VELOCITY_CAP_SATS, VELOCITY_WINDOW_SEC
Hot-Balance-Zielbereich	steuert die Aufteilung der Mittel zwischen Hot- und Cold-Wallet	OPA hot.limits	data/data.json
HMAC-Zeitfenster	begrenzt die Wiederverwendbarkeit abgefangener Signieraufträge	Signierer (Key A VM)	Signer-Konfiguration

Tabelle 1.6: Grenzwerte des Systems, ihr Zweck und ihre Durchsetzungsorte. Die konkreten Werte sind Labor-Defaults und vom Mandanten an sein Mittelvolumen anzupassen.

des Balance-Zielbereichs sind so zu wählen, dass der Hot-Anteil dem gewünschten Risikoanteil (hier ca. 5 %) der Gesamtmittel entspricht. Da Nachbefüllungen als Cold-Ausgaben nicht mitgezählt werden, bleibt der Rückfluss aus der Cold-Zone zudem auch bei ausgeschöpftem Cap stets möglich.

Manipulierte Nachbefüllungs-PSBT

Der aus Angreifersicht attraktivste Pfad gegen die Cold-Bestände verläuft nicht über die Extraktion von Schlüsselmaterial, sondern über den regulären Nachbefüllungsprozess. Ein vollständig kompromittiertes Apphost kann eine Cold-PSBT mit beliebiger Höhe und beliebiger Zieladresse erzeugen. Die internen OPA_cold-Transaktionen durchlaufen weder die Whitelist-Prüfung noch die Betragsgrenzen (siehe Tabelle 1.3) und Key A signiert die PSBT automatisch, da Multi-Signatur-Inputs vom Velocity-Cap ausgenommen sind (siehe 1.4.2). Die einzige und damit entscheidende Verteidigungslinie ist die manuelle Verifikation durch den Operator im Cold-System (siehe 1.4.4). Nur wenn die Zieladresse über

das hinterlegte Watch-Only-Wallet eindeutig als zu Key A gehörig aufgelöst wird und Betrag sowie Gebühren plausibel sind, darf signiert werden. Dieser bewusste *Single Point of Human Failure* ist die Konsequenz des Air-Gaps, da eine automatische Prüfung im Cold-System dessen Isolation unterlaufen würde. Zur Verringerung des Restrisikos empfiehlt sich eine organisatorische Betragsobergrenze je Nachbefüllung, deren Überschreitung der Operator grundsätzlich als Kompromittierungsindiz wertet, sowie perspektivisch ein Vier-Augen-Prinzip, bei dem Key B und Key C von verschiedenen Personen geführt werden.

Wechselmedium

Das USB-Medium ist der einzige verbleibende Kanal in die Cold-Zone und wird von einem System beschrieben, dessen Kompromittierung das Bedrohungsmodell explizit unterstellt. Angriffe über manipulierte Dateisysteme oder USB-Firmware (BadUSB) sind daher als Restrisiko zu betrachten. Die Cold-VMs begegnen dem durch deaktiviertes automatisches Mounten und ein minimales, deklarativ definiertes Softwareprofil. Verarbeitet werden ausschließlich PSBT- und Deskriptor-Dateien, deren Inhalt in Sparrow vollständig angezeigt wird. Zu beachten bleibt die bewusst aktivierte Doppelklick-Ausführung von Shell-Skripten im Dateimanager (siehe 1.4.4): Sie gilt ausschließlich für die bei der Installation deklarativ auf dem Desktop abgelegten Skripte. Dateien vom Wechselmedium sind grundsätzlich nicht auszuführen, sodass das Medium ausschließlich als Datenablage für PSBTs und Deskriptoren verwendet und vor jedem neuen Vorgang über das Format-Skript bereinigt wird. Eine Virtualisierung von QR-Code-Übertragung war in der Laborumgebung nicht möglich (siehe 1.4.5). Im Produktivbetrieb stellt der Wechsel auf optische Übertragung die konsequente Schließung dieses Vektors dar.

Redis Schwachpunkt

Sowohl die Nonce-Sperre als auch das Velocity-Fenster werden flüchtig in Redis gehalten und bewusst nicht persistiert. Der dauerhafte Schutz gegen das Wiedereinspielen vollständiger Signieraufträge ist die persistente PSBT-ID-Deduplizierung in der Datenbank. Die Nonce verengt lediglich das ohnehin kurze, über den Zeitstempel begrenzte Gültigkeitsfenster.

Dabei kann dieser Service durch einen Neustart direkt über Docker-Befehle oder über einen Denial-of-Service-Angriff, der einen Crash erzeugt, zurückgesetzt werden. Beide Angriffe wären durch eine Persistierung der Nonce und des täglichen Limits abschwächbar, jedoch wird diese Maßnahme als von nur schwachem Wert angesehen. Ein Angreifer, der bereits Zugriff auf Docker hat, kann auch das gesamte geheime Schlüsselmaterial auslesen oder persistente Datenbestände löschen. Ein Angreifer, der Denial-of-Service-Attacken als Peer des WG-Tunnels ausführen kann, wird weiterhin durch das HMAC-Geheimnis aufgehalten oder kann dieses ebenfalls umgehen und dem Signierer dann beliebige PSBTs über mehrere Tage unterbreiten, ohne dass ein Operator dies erkennen könnte.

Dementsprechend ist ein Angriff auf die Redis-Komponente nur bei einer fast vollständigen Kompromittierung möglich und kann vernachlässigt werden, da zu diesem Zeitpunkt einfachere Kompromittierungsziele bestehen. Gegen einen Angreifer auf dieser Ebene bilden ohnehin die TPM-/PCR-Bindung und die Prozess-Härtung die relevante Verteidigung, nicht die Flüchtigkeit von Redis. Bei Bedarf an einer über Neustarts hinweg auditierbaren Persistenz des Velocity-Fensters kann Redis mit AOF-/RDB-Persistenz betrieben werden. Hierauf wurde zugunsten einer minimalen Zustands- und Angriffsfläche verzichtet.

Software-Lieferkette

Ein air-gapped System ist gegen Remote-Angriffe isoliert, nicht jedoch gegen kompromittierte Software, die bereits bei der Installation eingebracht wird. Die Architektur begegnet diesem Vektor durch den deklarativen NixOS-Ansatz, wobei sämtliche Pakete aus dem offiziellen, von den NixOS-Maintainern signierten Binary-Cache bezogen, und die vollständige Eingabemenge des Systems über einen gepinnten Flake (`flake.lock`) fixiert wird (siehe Tabelle A5.1). Dadurch ist reproduzierbar nachvollziehbar, welche Softwarestände auf den Schlüsselhalter-VMs laufen, und ein unbemerktes Nachladen veränderter Komponenten ist ausgeschlossen. Ein Restrisiko verbleibt in der Vertrauenskette zu den Upstream-Quellen selbst (nixpkgs, Sparrow Wallet). Dieses kann durch die Verifikation der Upstream-Signaturen bei kontrollierten Updates weiter reduziert, jedoch

prinzipiell nicht eliminiert werden. Updates der Cold-Systeme erfolgen ausschließlich als bewusste, manuelle Entscheidung des Operators.

Rotation der Geheimnisse

Die im System verwendeten Geheimnisse besitzen unterschiedliche Lebenszyklen und werden dementsprechend über getrennte Verfahren rotiert.

Laufzeit-Geheimnisse des Apphosts. Die bei der Installation des Apphosts zufällig erzeugten Zugangsdaten wie die NATS-Passwörter der Dienst-Identitäten (einschließlich der Operator-Identität), das ntfy-Passwort, die PostgreSQL-Rollen-Passwörter sowie die `rpcauth`-Credentials von Bitcoin Core liegen zentral in der nicht versionierten `.env`-Datei des Apphosts. Rotiert wird, indem der jeweilige Wert dort ersetzt, die daraus abgeleiteten Artefakte über die `update-secrets`-Skripte neu erzeugt (`scripts/update-secrets-hotwallet.sh` für die gehashten `rpcauth`-Einträge) und die betroffenen Container anschließend neu erstellt werden. Die PostgreSQL-Rollen-Passwörter müssen zusätzlich per `ALTER ROLE` in der laufenden Datenbank gesetzt werden, da deren Initialisierungsskripte nur beim ersten Start eines leeren Volumes ausgeführt werden. Eine regelmäßige sowie eine anlassbezogene Rotation wird dem Mandanten empfohlen.

Systemübergreifende Geheimnisse. Das HMAC-Geheimnis liegt synchron auf Middleware und Signierer-VM und wird bewusst nicht automatisch mitrotiert. Beide Seiten müssen in einem Schritt getauscht werden, andernfalls weist die Signier-Schnittstelle sämtliche Anfragen ab. Die Rotation erfolgt daher kontrolliert über den etablierten SSH-Wechselmedium-Weg (`wgHMAC_export.sh` auf die Signierer-VM, anschließend `wgHMAC_import.sh` auf dem Apphost). Die WireGuard-Schlüsselpaare lassen sich analog über die Peer-Export- und Import-Skripte erneuern; das Import-Skript überschreibt dabei die bestehende `wg0.conf`.

Wallet-Schlüsselmaterial. Die Seeds selbst sind nicht im engeren Sinne rotierbar: Ein kompromittierter oder vorsorglich auszutauschender Schlüssel erfordert das Aufsetzen eines neuen Wallets und den Transfer der Bestände. Im Cold-Bereich genügen hierfür dank des 2-aus-3-Modells die

verbleibenden zwei Schlüssel, um sämtliche UTXOs auf ein neu erzeugtes Multi-Signatur-Wallet zu überführen. Anschließend sind Deskriptoren und xpubs neu auszutauschen und auf dem Apphost zu registrieren (analog zum Setup in 1.4.4). Für Key A ist zusätzlich die erneute Versiegelung im TPM der Signierer-VM vorzunehmen.

Partner Authentifizierung

Eine weitergehende Absicherung der externen Schnittstellen bestünde in einer expliziten Authentifizierung der Überweisungspartner, etwa über HMAC-signierte Anfragen, Basic Authentication oder OAuth auf der HTTP-Eingangsschnittstelle. Ein solches Verfahren ist jedoch stark partnerabhängig, da Schlüsselaustausch, Token-Format und Gültigkeitsdauer je Partner individuell ausgehandelt werden müssten. Daher wurde dieser Ansatz nicht als fester Bestandteil der Architektur umgesetzt. Die Authentizität eingehender Zahlungsanfragen wird stattdessen bereits teilweise durch das Whitelisting der Ziel-Wallet-Adressen abgedeckt: Da ausschließlich Transaktionen an registrierte, dem jeweiligen Partner zugeordnete Wallets signiert werden, bleibt die Wirkung einer gefälschten Anfrage begrenzt. Eine ergänzende partnerspezifische Authentifizierung bleibt dem Mandanten als Erweiterung im Approval-Gate offen.

1.4.7 Fazit

Die entwickelte Krypto-Architektur erfüllt die Kernanforderung des Mandanten: Eingehende Zahlungsaufträge werden vollautomatisiert als Bitcoin-Transaktionen ausgeführt, ohne dass der überwiegende Teil der Mittel oder deren Schlüsselmaterial einem internet-exponierten System ausgesetzt ist. Erreicht wird dies durch die konsequente Trennung in drei Vertrauenszonen bestehend aus dem exponierten Apphost, der gehärteten Signierer-VM und der air-gapped Cold-Zone, zwischen denen ausschließlich PSBTs, niemals Schlüsselmaterial, ausgetauscht werden.

Die Sicherheit des Systems beruht dabei nicht auf einer einzelnen Maßnahme, sondern auf gestaffelten, voneinander unabhängigen Schichten: der OPA-Autorisierung mit Whitelist- und Limit-Prüfung im Apphost, der kryptographisch authentifizierten Übergabe an den Signierer (WG,

HMAC, Deduplizierung), dem TPM-versiegelten Schlüsselmaterial mit unabhängigem Velocity-Cap sowie dem 2-aus-3-Multi-Signatur-Modell mit verpflichtender manueller Verifikation im Cold-Prozess. Der maximale Schaden einer Kompromittierung des Online-Systems bleibt dadurch auf den im Hot-Wallet vorgehaltenen Anteil der Mittel begrenzt, zusätzlich gedeckelt durch die zweistufige Abflussbegrenzung.

Ebenso bewusst wurden die verbleibenden Grenzen des Modells benannt: Die Doppelnutzung von Key A schwächt das Cold-Wallet kontrolliert zugunsten der Operationalität, die manuelle Verifikation durch den Operator bildet die letzte Verteidigungslinie des Nachbefüllungsprozesses, und das Wechselmedium bleibt der einzige, entsprechend restriktiv behandelte, Kanal in die Cold-Zone. Mehrere zunächst verfolgte Ansätze, darunter die Hash-basierte Freigabeschicht, der ZMQ-basierte Eigenbau des UTXO-Trackings und der netzwerkfähige Online-Koordinator, wurden nach Abwägung von Komplexität und Angriffsfläche bewusst verworfen. Die Reduktion auf das dargestellte Modell ist ein Ergebnis dieser Iterationen.

Für die produktive Übernahme verbleiben klar umrissene Schritte im Verantwortungsbereich des Mandanten: die Migration von Regtest auf das Mainnet (siehe 1.4.4), der Betrieb der Zonen auf physisch getrennten Maschinen anstelle der virtualisierten Laborumgebung sowie optionale Verschärfungen wie ein 3-aus-4-Schema. Ergänzt werden kann ebenfalls die optische PSBT-Übertragung per QR-Code, ein NKey-/JWT-basiertes NATS-Identitätsmodell und eine partnerspezifische Authentifizierung der Eingangsschnittstellen. Eine Härtung der TPM-Bindung durch Secure Boot (unter NixOS über `lanzaboote`, Bindung an PCR 7) und ein Unified Kernel Image (PCR 11) samt signierter PCR-Policy, die reguläre Updates ohne manuelles Neuversiegeln erlaubt, stellt eine weitere sinnvolle Ergänzung dar. Zuletzt soll noch die Möglichkeit einer Retry-Queue für fehlgeschlagene Zahlungsaufträge erwähnt werden, deren Ausgestaltung jedoch stark von den Prozessen des Mandanten und seiner Partner abhängt. Die Architektur ist so ausgelegt, dass jede dieser Erweiterungen additiv möglich ist, ohne das Grundmodell zu verändern.

Eine abschließende Übersicht für den Krypto-Aspekt dieser Ausarbeitung kann in Tabelle 1.7 gefunden werden. Das zugrundeliegende Bedrohungs-

muster wurde in 1.3.4 zuvor diskutiert.

1 Self Hosted Setup

Angriffsvektor	Zone	Gegenmaßnahme	Restrisiko	Verweis
Remote-Kompromittierung Apphost	Hot	OPA, Whitelist, Daily-Cap, 5 %-Aufteilung	Abfluss bis Velocity-Cap	1.4.2
Manipulierte Nachbefüllungs-PSBT	Hot→Cold	Key-A-Vorsignatur, manuelle Sparrow-Verifikation	Menschlicher Fehler	1.4.4
Missbrauch des Operator-Pfads (Whitelist-/Limit-Bypass)	Hot	Besitz Operator-NATS-Token, Struktur-/SHA-256-Prüfung, Daily-Cap, Velocity-Cap	Abfluss bis Velocity-Cap	1.4.2
Replay von Signieraufträgen	Signierer	HMAC, Nonce, ID-Dedup	Redis-Reset	1.4.6
Physischer Zugriff auf Key A VM	Signierer	TPM/PCR-Bindung, LUKS	Diebstahl des Seed-Backups	1.4.6
Manipuliertes Wechselmedium	Cold	Kein Automount, nur PSBT-Dateien, manuelle Prüfung	BadUSB-Firmware	1.4.6
Kompromittierte Lieferkette	alle	Flake-Pinning, deklarativer Neuaufbau	Upstream-Vertrauen	1.4.6
Metadaten-Leak (Adress-/UTXO-Abfragen)	Hot	Eigener BTC-Core-Full-Node statt öffentlichem Electrum-Server, dynamische Deskriptor-Adressen	On-Chain öffentlich einsehbar	1.4.5
Verfügbarkeit des Hot-Wallets (DoS)	Hot	Automatischer cold_to_hot-Nachbefüllprozess, Monitoring/ntfy	Manuelle Freigabe verzögert Rückfluss	1.4.2
Böswilliger Operator	Cold	2-aus-3 erfordert zweite Signatur, Vier-Augen-Prinzip	Im Ein-Operator-Betrieb nicht abgedeckt, Vorraussetzung von Vertrauen	1.4.4

Tabelle 1.7: Bedrohungs- und Maßnahmenübersicht des Krypto-Systems

2 KI-Evaluation

2.1 Zielsetzung und Forschungsfrage

Der Mandant möchte wissen, ob Large Language Models (LLMs) die Arbeit von Sicherheitsberatern übernehmen können. Konkret soll geprüft werden, ob LLMs sowohl konzeptionelle Vorschläge als auch praktische Konfigurations- und Prüfaufgaben zuverlässig bearbeiten können. Dieses Kapitel untersucht deshalb, ob ein LLM in der Lage ist, eine sichere Self-Hosted-Infrastruktur zu planen, aufzubauen, zu dokumentieren und anschließend kritisch zu prüfen.

Dafür wurden zwei Szenarien miteinander verglichen. In Szenario A wurde ein Nutzer ohne vertieftes IT-Sicherheitswissen simuliert. Dieser Nutzer formulierte sein Ziel eher alltagsnah und ließ Codex mit vergleichsweise offenen Prompts ein Home-Lab planen und konfigurieren. Die Anforderungen orientierten sich grundsätzlich am selbst entwickelten Home-Lab: mehrere Self-Hosted-Dienste, Zugriff über Tor, Monitoring, Backups und ein BTC-Transaktionssystem.

In Szenario B wurde dasselbe Ziel verfolgt, allerdings mit deutlich engerer fachlicher Steuerung durch einen IT-Experten. Der Startprompt enthielt hier bereits Sicherheitsregeln, ein Threat Model, klare Verbote, Dokumentationspflichten und Vorgaben zur späteren Prüfung.

Für beide Szenarien wurden getrennte lokale Proxmox-VMs bereitgestellt. Das war notwendig, weil Codex für die praktische Umsetzung weitreichende Rechte innerhalb der jeweiligen Zielumgebung benötigte. Ein direkter Einsatz auf der eigentlichen Home-Lab-Umgebung wäre methodisch und sicherheitstechnisch problematisch gewesen. Es hätte das Risiko bestanden, dass Codex globale Einstellungen oder nicht vom Versuch umfasste Systembestandteile unbeabsichtigt verändert. Die ge-

trennten VMs ermöglichten dagegen eine realitätsnahe Umsetzung, ohne das eigentliche Projekt zu gefährden.[77]

2.1.1 Forschungsstand

In der auf der NeurIPS 2024 (Datasets-and-Benchmarks-Track) erschienenen Arbeit „Infrastructure-as-Code (IaC)-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs“ von Kon et al. entwickelten die Autoren einen eigenen Prüfablauf für durch LLM-generierte Infrastructure-as-Code-Programme. Dafür entwickelten sie etwa 500 erstellte Szenarien für Amazon Web Services (AWS)-basierte Terraform-Konfigurationen.

In der Evaluation prüften sie, ob die Ausgabe verarbeitbar war. Im zweiten Schritt wurde die Ausgabe mit der Rego-basierten Intent-Spezifikation über Open Policy Agent (OPA) verglichen. Diese Spezifikation beschreibt, welche Ressourcen, Abhängigkeiten und Attribute für die jeweilige Aufgabe tatsächlich erforderlich oder zulässig sind. Eine Lösung gilt laut Kon et al. nicht schon als korrekt, wenn Terraform sie akzeptiert, sondern erst dann, wenn sie auch die fachlichen Forderungen an die Infrastruktur der Aufgabe erfüllt.[78]

Gerade dieser zweite Schritt ist für die vorliegende Arbeit methodisch wichtig. Kon et al. haben gezeigt, dass Compile- oder Plan-Prüfungen allein nicht tief genug sind, weil Konfigurationen korrekt aussehen können, obwohl zentrale Nutzeranforderungen fehlen oder falsch umgesetzt wurden. Die formale Intent-Prüfung filterte in der Evaluation einen großen Teil solcher falschen Positivfälle heraus.

Für die eigene Evaluation wurde übernommen, dass bewertet werden muss, ob Codex nicht nur eine plausible Architektur beschreibt oder Dateien erzeugt, sondern ob diese Dateien und der Systemzustand tatsächlich zu den Anforderungen passen: getrennte Rollen, restriktive Zugriffspfade, Tor als kontrollierter Zugang, wirksame Netzwerkgrenzen, geschützte Backups, Monitoring sowie ein sicher begrenzter BTC-Simulationsprozess.

Zhang et al. erweitern diese Sichtweise mit ihrem Benchmark *DPIaC-Eval* um die Dimension der tatsächlichen Deployability und überprüfen

dementsprechend LLM-generierte IaC nicht nur auf syntaktische Korrektheit. Stattdessen wird getrennt danach bewertet, ob der Code in der Zielumgebung tatsächlich ausrollbar ist, ob er den Nutzer-Intent trifft und ob dieser Sicherheitsanforderungen einhält. Die Ergebnisse zeigen, dass formal gültiger Code in der Zielumgebung dennoch nicht ausrollbar sein kann.[79] Für die eigene Evaluation folgt daraus, dass beschriebenes Konzept, erzeugte Dateien und tatsächlicher Laufzeitzustand als getrennte Nachweisebenen behandelt werden müssen.

Hase et al. untersuchen in ihrer auf dem IEEE-Symposium ISDFS 2026 veröffentlichten Arbeit gezielt die Security-Policy-Compliance von LLM-generierten IaC-Skripten. Ein Skript kann syntaktisch korrekt sein und die Anforderungen erfüllen, aber trotzdem problematische Einstellungen enthalten. Hase et al. verwenden dafür unter anderem statische Policy-Prüfung mit Checkov.[80] Für die eigene Evaluation muss die Sicherheit auch ein Kriterium für die Bewertung sein. Für das Home-Lab bedeutet es also, dass ein Dienst nicht allein deshalb gut bewertet werden darf, weil er läuft, sondern er muss zusätzlich überprüft werden, ob Zugriffspfade eng begrenzt sind, Secrets nicht im Klartext liegen, administrative Dienste nicht unnötig exponiert werden, Backups angemessen geschützt sind und Netze begrenzt werden.

Lian et al. betrachten mit *Ciri* eine zweite Rolle von LLM: nicht die Generierung von Infrastruktur, sondern die Validierung von Konfigurationen. Die Autoren untersuchen, ob ein LLM Konfigurationsartefakte daraufhin prüfen kann, ob sie gültig, fehlerhaft oder sicherheitskritisch problematisch sind. [81] Daraus lässt sich eine weitere Prüfkategorie ableiten: Es geht nicht nur darum, ob KI neue Lösungen oder Konfigurationen vorschlagen kann. Ebenso wichtig ist, ob sie bestehende Konfigurationen sinnvoll prüfen kann. Dazu gehört, dass Fehlkonfigurationen und Sicherheitslücken erkannt werden, die Einschätzung nachvollziehbar begründet und passende Korrekturmaßnahmen vorschlägt. Gerade für die Rolle als Sicherheitsberater ist dieser Bereich auch wichtig, da er im Berufsalltag auch vorhandene Systeme bewertet und verbessert.

Diese Prüfkategorie wurde in der eigenen Evaluation als separate Fehleranalyse am Ende beider Szenarien aufgegriffen; die konkrete Umsetzung wird in Abschnitt 2.2.1 beschrieben.

Mukhopadhyay et al. vergleichen schließlich verschiedene Nutzungsformen wie Zero-Shot, Few-Shot und Fine-Tuning. Dabei bewerten die Autoren die erzeugte IaC mehrdimensional, unter anderem nach Correctness, Security, Compliance und Readability. Methodisch relevant ist dabei vor allem, dass sie die automatische Bewertung durch ein LLM-as-a-Judge mit einem manuellen Review durch Fachexperten kombinieren[82]. Für die eigene Methodik ergibt sich daraus, dass die Punktvergabe nicht einer einzelnen Einschätzung überlassen werden darf, sondern eine menschliche Kontrollinstanz mit fachlicher Tiefe benötigt.

Die genannten Arbeiten unterscheiden sich methodisch deutlich und viele Elemente sind nicht direkt auf die vorliegende Fragestellung anwendbar. Die Tabelle 2.1 stellt die Evaluationsmethodologien der drei Arbeiten gegenüber, aus denen die zentralen Prüfprinzipien übernommen wurden. Zhang et al. und Mukhopadhyay et al. ergänzen diesen Rahmen um die Trennung der Nachweisebenen und das menschliche Experten-Review.

	Kon et al.	Hase et al.	Lian et al.
Gegenstand	Generierung von IaC (Terraform/AWS)	Policy-Compliance generierter IaC	Validierung bestehender Konfigurationen
Prüfverfahren	formale Intent-Spezifikation (Rego/OPA) gegen die Nutzeranforderung	statische Policy-Scans (u. a. Checkov)	Erkennung gezielt eingebauter Fehlkonfigurationen
Metriken	Anteil intent-korrekturer Lösungen über die Aufgabenmenge	Anteil und Art der Policy-Verstöße je Modell	Erkennungsrate, Fehlklassifikationen, Halluzinationen
Stichprobe	größerer Aufgaben-Benchmark	viele generierte Skripte	viele Konfigurationsdateien mehrerer Systeme
Übernommen	Anforderungsabgleich statt reiner Lauffähigkeit	Sicherheit als eigenes Ausschlusskriterium	separate Fehleranalyse-Aufgabe

Tabelle 2.1: Evaluationsmethodologien des Forschungsstands im Vergleich.

Der größte Unterschied liegt in der Testmenge. Alle genannten Arbeiten stützen ihre Erkenntnisse auf große, automatisiert prüfbare und reprodu-

zierbare Aufgabenmengen. Das war aufgrund monetärer und zeitlicher Begrenzungen für die vorliegende Aufgabenstellung nicht umsetzbar, weshalb sich die Untersuchung auf einen Durchlauf je Szenario beschränkt. Aggregierte Erfolgsquoten wie in den Benchmarks der veröffentlichten Arbeiten sind hier deshalb nicht sinnvoll anwendbar. Stattdessen soll die Prüftiefe im Einzelfall erhöht werden, wobei jede Kategorie anhand definierter Indikatoren gegen Dateien, Laufzeitzustand und Dokumentation geprüft wird. Zudem begrenzen sicherheitskritische Ausschlussfehler die erreichbare Punktzahl. Die externen Arbeiten liefern damit die Prüfprinzipien, nicht die Messinstrumente.

2.2 Methodik und Bewertungsrahmen

2.2.1 Methodikanforderungen

Aus den genannten Arbeiten ergeben sich vier methodische Anforderungen an die eigene Untersuchung.

Erstens muss zwischen sprachlicher Plausibilität und technischer Umsetzung unterschieden werden. Ein LLM kann eine überzeugende Architektur beschreiben, ohne dass diese im System tatsächlich umgesetzt wurde. Deshalb wurden nicht nur Konzepte, sondern auch erzeugte Dateien, Regeln usw. ausgewertet. Diese Trennung der Nachweisebenen folgt dem Vorgehen von Kon et al. und Zhang et al. [78], [79]

Zweitens muss die Sicherheit als eigenes Kriterium behandelt werden. Aus Hase et al. folgt, dass syntaktisch oder funktional richtige Infrastruktur trotzdem Policy-Verstöße enthalten kann. [80] Deshalb wurden kritische Ausschlussfehler definiert. Dazu gehörten öffentlich erreichbare Admin-Dienste ohne zusätzliche Beschränkung, unsegmentierte Netze, unverschlüsselte sensible Backups, Klartext-Secrets in Dokumentation oder Compose-Dateien sowie echte Seeds oder Private Keys auf Online-Systemen. Trat ein solcher Fehler in einer Kategorie auf, konnte diese Kategorie höchstens mit einem Punkt bewertet werden.

Drittens muss auch die Fähigkeit der Fehleranalyse des LLM betrachtet werden. Deshalb erhielten beide Szenarien identische, absichtlich fehlerhafte Dateien aus den Bereichen Backup, BTC, Docker Compose

und Firewall. Bewertet wurde, ob Codex die Fehler nur oberflächlich kommentiert oder ob es sie fachlich korrekt einordnet und richtige Konfigurationen erstellt. Für jedes der vier Artefakte wurde vor den Läufen festgelegt, welche Fehler enthalten sind und woran eine korrekte Behebung erkannt wird. Zur Verifizierung sind die Artefakte zusammen mit den Korrekturfassungen im Projekt-Repository abgelegt.[83]

Viertens muss die Bewertung selbst gegen einseitige Einschätzungen abgesichert werden. Dabei wird in dieser Ausarbeitung ein weiteres manuelles Review durch Fachexperten nach Mukhopadhyay et al. durchgeführt.[82] Deshalb wurde die Punktvergabe in dieser Untersuchung von zwei Personen unabhängig voneinander vorgenommen und abweichende Wertungen wurden gemeinsam aufgelöst (siehe Abschnitt 2.2.2).

2.2.2 Teststrategie

Codex wurde genutzt, weil im Rahmen der Untersuchung die Ressourcen bereitgestellt worden sind. Gleichzeitig war Codex für die Fragestellung besonders passend, weil es als agentisches System nicht nur textbasiert antwortet, sondern praktisch in der Umgebung arbeiten kann. Es kann Dateien lesen und verändern, Shell-Befehle ausführen, Fehlermeldungen beobachten und auf diese reagieren. Dadurch ließ sich die Frage des Mandanten deutlich realitätsnäher und zukunftsorientierter untersuchen als mit einer rein chatbasierten KI.

Hinzu kommt, dass OpenAI-Modelle zu den bekanntesten und am weitesten verbreiteten LLM-Systemen gehören. In beiden Szenarien wurde Codex mit dem Modell `gpt-5.4` und der Reasoning-Einstellung „Sehr hoch“ genutzt.[84]

Der Agent hatte Zugriff auf Dateien, Konfigurationen und Terminalausgaben innerhalb der jeweiligen VM. Dadurch konnte er Fehler in der eigenen Planung erkennen, Anpassungen vornehmen und einzelne Bestandteile der Infrastruktur praktisch verbessern. Codex verfügte damit über mehr Kontext zur tatsächlichen Umgebung als ein normaler Chatbot.

Dazu wurden zwei getrennte Proxmox-VMs verwendet. Szenario A lief auf `ailab1`, Szenario B auf `ailab2`. Um die Vergleichbarkeit zu sichern,

wurden alle übrigen Faktoren konstant gehalten, wobei beide VMs identisch dimensioniert waren. Daneben kamen dasselbe Modell und dieselbe Reasoning-Einstellung der KI zum Einsatz, beide Läufe folgten demselben Meta-Prompt-Rahmen und erhielten identische Fehlerartefakte für die abschließende Fehleranalyse. Bewusst variiert wurde ausschließlich die fachliche Tiefe der Nutzereingaben. Nicht kontrollierbar waren Umfang und Dauer der Interaktion, die sich aus dem jeweiligen Nutzerverhalten ergaben.[85] Diese Asymmetrie wird in Abschnitt 2.6 als Einschränkung ausgewiesen.

Beide Szenarien verfolgten dasselbe Ziel: Aufbau einer Self-Hosted-Infrastruktur mit mehreren Diensten, Zugriff über Tor, Monitoring, Backups und einem BTC-Transaktions- bzw. BTC-Simulationsbereich. Der zentrale Unterschied lag in den Prompts des jeweiligen Nutzers. In Szenario A waren die Vorgaben bewusst offen und alltagsnah formuliert. In Szenario B enthielt der Initialprompt ein explizites Threat Model, klare Verbote, Pflichtdokumente, Dokumentationsregeln, Validierungsschritte und die Vorgabe, im BTC-Bereich ausschließlich Simulationsdaten zu verwenden.

Die Evaluation erfolgte in sieben Hauptkategorien:

1. Architekturplanung
2. Services
3. Konfiguration
4. Netzwerk, Tor und Zugriff
5. Backup und Monitoring
6. BTC-Konzept
7. Fehleranalyse

Diese Kategorien decken zusammen die wesentlichen Sicherheits- und Betriebsfragen des Mandanten ab. Bei der Architekturplanung wird geprüft, ob das LLM Schutzbedarfe, Rollen und Zonen sinnvoll strukturiert. Bei den Services wird bewertet, ob die geforderten Anwendungen vorhanden und angemessen verteilt und betrieben werden. Die Kategorie Konfiguration untersucht Härtung, Defaults, Secret-Handling und Konsistenz

zwischen Dokumentation und Dateien. Bei dem Thema Netzwerk, Tor und Zugriff wird geprüft, ob spezifische Zugriffspfade erzwungen werden. Bei Unterpunkt Backup und Monitoring wurde betrachtet, inwieweit der Betrieb nachvollziehbar überwacht werden kann und ob eine Wiederherstellung im Fehlerfall möglich ist. Bei dem BTC-System wird die Trennung von Node, Watch-only-Komponenten, Hot-Service-Rolle und Offline-Handoff geprüft. Die Fehleranalyse überprüft zuletzt, ob Codex sicherheitsrelevante Fehler in fremden Dateien erkennt und korrigiert.

Damit die Punktvergabe in den Kategorien nicht pauschal erfolgt, wurde jede Kategorie über vier feste Prüfindikatoren operationalisiert (siehe Tabelle 2.2). Jeder Indikator wurde einzeln nach der unten definierten Skala mit 0 bis 3 Punkten bewertet. Als voll erfüllt gilt ein Indikator nur, wenn er durch Dateien, Laufzeitprüfungen oder gleichwertige Nachweise belegt ist. Die Kategoriewertung ergibt sich als ungewichtetes arithmetisches Mittel der vier Indikatorwertungen, kaufmännisch auf ganze Punkte gerundet. Die Kategorien folgen damit derselben Logik wie das gesondert operationalisierte BTC-Konzept (Abschnitt 2.2.3). Dabei werden die Unterkriterien zusätzlich gewichtet, da sie sich in ihrer Sicherheitsrelevanz deutlich stärker unterscheiden.

Die Einzelbewertungen in Abschnitt 2.4 weisen die vier Indikatorwertungen je Kategorie explizit aus (Tabellen 2.4 und 2.6). Jede Kategoriewertung ist dadurch auf die Prüfindikatoren aus Tabelle 2.2 rückführbar.

Kategorie	Prüfindikatoren
Architekturplanung	• Zonen- und Rollenmodell mit Schutzbedarfen • Begründete Platzierung der Dienste • Dokumentierte erlaubte Kommunikationspfade • Trennung von Admin-, Anwendungs- und BTC-Bereich
Services	• Geforderte Dienste vorhanden und lauffähig • Verteilung nach Schutzbedarf statt Bündelung auf einer Rolle • Minimale Berechtigungen je Dienst • Keine unnötigen Zusatzdienste
Konfiguration	• gehärtete Defaults (Registrierung, Ports, Rechte) • Secret-Handling ohne Klartext • Konsistenz zwischen Dokumentation und realen Dateien • Reproduzierbarer Systemzustand
Netzwerk, Tor und Zugriff	• Default-Deny nachweisbar • Dienste nur über die vorgesehenen Tor-Pfade erreichbar • Admin-Zugang zusätzlich beschränkt (z. B. v3-Client-Auth) • Interne Kommunikation zwischen den Diensten begrenzt
Backup und Monitoring	• Backups verschlüsselt und nicht ausschließlich lokal • Restore praktisch getestet • Host und Dienste überwacht • Alarmkette bis zum Betreiber nachvollziehbar
Fehleranalyse	• Alle vier Fehlerartefakte erkannt • Fachlich korrekte Einordnung mit Begründung • Korrekturfassung behebt die Kernprobleme • Keine neuen Sicherheitsfehler in der Korrektur

Tabelle 2.2: Prüfindikatoren der Hauptkategorien für die Apphost Komponente

Für jeden Prüfindikator und damit für jede Kategorie wurden 0 bis 3 Punkte vergeben. Die Skala wurde bewusst einfach gehalten, um die Ergebnisse nachvollziehbar vergleichen zu können:

- **0 Punkte:** Die Anforderung fehlt, ist kritisch falsch umgesetzt oder

erzeugt selbst ein ernstes Sicherheitsproblem.

- **1 Punkt:** Die Grundidee ist erkennbar, bleibt aber oberflächlich, unsauber, unsicher, enthält relevante Lücken oder ist operativ nicht belastbar.
- **2 Punkte:** Die Lösung ist überwiegend korrekt, enthält nur kleine weniger relevante Lücken oder zu weit gefasste Defaults.
- **3 Punkte:** Die Lösung ist fachlich korrekt, eng gefasst, nachvollziehbar begründet und durch Dateien, Laufzeitprüfungen oder gleichwertige Simulationsnachweise belegt.

Die Punktvergabe wurde von zwei Personen unabhängig anhand der Prüfindikatoren aus Tabelle 2.2 und nach der vordefinierten Skala vorgenommen, wobei abweichende Bewertungen gemeinsam aufgelöst wurden. Diese Methodik folgt wie bereits in Abschnitt 2.1.1 beschrieben der Ausarbeitung von Mukhopadhyay et al., wobei als Bewerter die Ersteller des Basis- bzw. Krypto-Teils herhalten[82]. Nur sie bringen die nötige fachliche Tiefe für die jeweiligen Kategorien im Zuge dieser Ausarbeitung mit und sind dementsprechend die naheliegende Wahl. Die Doppelrolle als Architekten der Referenzlösung und Bewerter der KI-Ergebnisse ist zugleich ein methodisches Risiko, weil eigene Designentscheidungen unbewusst als Maßstab dienen können. Dieses Risiko wird in Abschnitt 2.6 als Einschränkung weiter ausgewiesen.

2.2.3 Bewertung des BTC-Systems

Das BTC-System wurde gesondert evaluiert, weil dabei andere Sicherheitsgrenzen galten als bei produktiven Diensten. In Szenario B war ausdrücklich vorgegeben, ausschließlich Simulationsdaten zu verwenden und niemals echte Seeds, Private Keys oder produktive Geheimnisse zu erzeugen. Der Verzicht auf echte BTC-Werte war daher keine Schwäche, sondern eine Sicherheitsanforderung.

Für die volle Punktzahl genügte jedoch nicht, lediglich auf produktive Geheimnisse zu verzichten. Innerhalb des erlaubten Simulationsrahmens wären Test-Secrets, Test-Wallets, Dummy-Dateien, Dummy-PSBTs oder eine lokale BTC-Core-Regtest-Umgebung zulässig gewesen. Bewertet

wurde daher, ob Codex einen nachvollziehbaren simulierten Prozess abbildete: Erzeugung oder Darstellung von Testdaten, Trennung zwischen Watch-only-, Hot-Service- und Offline-Handoff-Rolle, simulierter Spend-Pfad, Recovery-Überlegung und Nachweis, dass keine produktiven Schlüsselmaterialien auf Online-Systemen lagen.

Daraus ergab sich folgende BTC-spezifische Auslegung der Punkteskala. Jedes Unterkriterium wurde nach der Skala aus Abschnitt 2.2.2 mit 0 bis 3 Punkten bewertet. Der Beitrag zur Gesamtwertung ergibt sich aus Wertung mal Gewichtung, sodass maximal 3,00 Punkte erreichbar sind. Die Ausschlussregel gilt dabei auf Kategorieebene, sodass ein kritischer Ausschlussfehler das gesamte BTC-Konzept unabhängig von den Unterwertungen mit höchstens einem Punkt bewertet.

Bewertungskriterium	Gewichtung
Architekturschlüssigkeit	20 %
Trennung von Schlüsselmaterial und Rollen	25 %
PSBT-, Signier- und Handoff-Prozess	20 %
Testdesign und praktische Prüfbarkeit	15 %
Härtung, Limitierung und Angriffsflächenreduktion	10 %
Recovery, Fehlerfälle und Remediation	10 %
Summe	100 %

Tabelle 2.3: Gewichtete Unterkriterien zur Bewertung der BTC-Sicherheitskonzepte.

Die hohe Gewichtung von Schlüsseltrennung und Signierprozess folgt unmittelbar aus der Kernanforderung des Mandanten, Zahlungen tatsächlich als BTC-Transaktion auszuführen.

Für die Übernahme in die Kategorienwertung der vergleichenden Evaluation wird der gewichtete Gesamtwert kaufmännisch auf ganze Punkte gerundet.

2.2.4 Datengrundlage und Dokumentation

Die Bewertung der beiden Szenarien basiert auf Chatverläufen sowie auf den von Codex erzeugten Dokumenten und Dateien. Für Szenario A waren dies das von Codex erstellte Runbook, der Read-only-Self-Audit und die Dateienprüfung. Für Szenario B wurden bereits im Initialprompt zusätzlich ein Masterplan, ein Decision Log, ein Risk Register, Validierungsergebnisse, ein Abschlussstatus sowie eine separate Fehleranalyse eingefordert. Die vollständigen Prompt- und Session-Exporte wurden gesichert und im GitHub-Ordner „KI-Evaluation“ abgelegt [83].

2.2.5 Prozessmetriken

Auch die Prozessdimension wurde dokumentiert. Szenario A umfasste im lokalen Sessionlog 392 sichtbare Nachrichten, davon 19 User-Nachrichten, und lief über rund 12,3 Stunden. Die erste Architektur lag nach etwa 0,12 Minuten vor, die erste reale Umsetzung nach rund 23,3 Minuten. Das Sessionlog weist 98,7 Mio. uncached Input-Tokens, 90,6 Mio. cached Input-Tokens und 620.622 Output-Tokens aus.

Szenario B umfasste 509 sichtbare Nachrichten, davon 27 User-Nachrichten, und lief rund 27,2 Stunden. Die erste echte Umsetzung begann hier erst nach etwa 143,2 Minuten. Der gemessene Tokenverbrauch lag mit 152,8 Mio. uncached Input-Tokens, 121,0 Mio. cached Input-Tokens und 887.171 Output-Tokens nochmals höher.

2.3 Promptdesign und Versuchsverlauf

2.3.1 Gemeinsame Ausgangsbedingungen

Beide Runs starteten unter vergleichbaren Ausgangsbedingungen. Dazu gehörten eine isolierte Proxmox-Test-VM, ein schrittweises Vorgehen, die Pflicht zur Freigabe vor größeren Umsetzungswellen, keine Änderung von Login-Daten oder globalen Plattform-Einstellungen sowie eine laufende Dokumentation. Der eigentliche Unterschied lag in der fachlichen Tiefe der Prompts.

Die vollständigen Prompts, die exportierten Chatverläufe sowie die in beiden Szenarien erzeugten Artefakte, etwa Runbooks, Decision Logs, Self-Audits und die korrigierten Konfigurationsdateien, sind im Projekt-Repository auf GitHub abgelegt und können dort vollständig nachvollzogen werden.[83]

2.3.2 Szenario A

In Szenario A begann der praktische Teil mit einem kurzen, naiven Prompt. Der Nutzer beschrieb, was das Home-Lab leisten sollte, machte aber kaum genaue Vorgaben zu Threat Model, Sicherheitszonen oder Prüfung. Die weiteren Prompts dienten vor allem dazu, einzelne Abschnitte freizugeben, zwischen Projektphasen zu wechseln, auf erkannte Probleme zu reagieren und spätere Korrekturen oder Härten anzustoßen. Der Run war dadurch offen und stark auf sichtbare Ergebnisse ausgerichtet. Codex hatte viel Handlungsspielraum und konnte schnell eine breite Lösung aufbauen. Sicherheitsrelevante Punkte wurden jedoch erst im weiteren Verlauf konsequent mitgedacht.

Zwei Bestandteile des Verlaufs lagen dabei bewusst außerhalb der simulierten Laienrolle. Erstens legte ein vorangestellter Meta-Prompt in beiden Szenarien denselben Versuchsrahmen fest. Dabei wurde der Scope der Test-VM, die Freigabepflichten und die feste Abfolge der Arbeitsabschnitte entlang der späteren Bewertungskategorien definiert. Diese Struktur diente der Vergleichbarkeit der beiden Runs und stellte keine fachliche Steuerung der Inhalte dar. Zweitens wurde die abschließende Fehleranalyse in beiden Szenarien mit wortgleichen, fachlich detaillierten Prüfaufträgen gestellt, damit diese Kategorie als standardisiertes Testinstrument vergleichbar bleibt. Innerhalb der eigentlichen Simulation blieben die Prompts dagegen durchgehend Alltagssprachlich und beschränken sich auf Freigaben, allgemeine Rückfragen und generisch formulierte Bitten, einen Schritt noch einmal auf Risiken zu prüfen, ohne eigene fachliche Vorgaben.

2.3.3 Szenario B

Szenario B war von Beginn an enger geführt. Schon der Initialprompt gab einen umfangreichen Sicherheitsrahmen vor, unter anderem mit Threat Model, Negativregeln, Pflichtdokumenten und klaren Kriterien zur Prüfung der Ergebnisse. Auch die Folgeprompts hatten eine andere Funktion als in Szenario A. Sie steuerten die Umsetzung aktiv, etwa bei der Netzwerksegmentierung, der Konsistenz zwischen Dokumentation und Live-Zustand, bei Systemhärtungen oder bei der separaten Fehleranalyse. Der Run ließ Codex dadurch weniger Freiraum, war aber von Anfang an stärker auf mögliche Sicherheitsrisiken ausgerichtet. Das verlangsamte die Umsetzung, machte die Ergebnisse aber nachvollziehbarer.

2.3.4 Nachvollziehbarkeit

Codex sollte in beiden Szenarien vor der Umsetzung erläutern, wie er vorgehen würde, welche Annahmen er trifft und welche Risiken bestehen. In Szenario B wurden diese Überlegungen zusätzlich im `decision-log.md` dokumentiert. Bewertet wurde anschließend, ob die Begründungen nicht nur nachvollziehbar klangen, sondern auch zur tatsächlichen Umsetzung passten.^[86]

Das ließ sich später anhand der erzeugten Dateien und des Systemzustands prüfen. In Szenario B zeigte sich dies etwa bei der Nutzung von `nftables`, dem eingeschränkten Zugriff auf Host- und Monitoring-Oberflächen, dem Backup-Konzept und der Trennung zwischen echten BTC-Prozessen und Dummy-PSBT-Beispielen.

In Szenario A waren solche Begründungen ebenfalls vorhanden, aber weniger konsequent. Einige Einschätzungen entstanden erst nachträglich, etwa dass lokale Backups auf demselben Host nur begrenzt schützen oder dass der BTC-Bereich noch nicht für echte Transaktionen geeignet war.

Die Begründungen wurden daher mit Runbook, Decision Log, Validierungsergebnissen, Self-Audit und Fehleranalyse abgeglichen. Dadurch wurde erkennbar, ob Codex nur überzeugend argumentierte oder ob seine Begründungen auch technisch eingelöst wurden.

2.3.5 Retrospektive des Promptdesigns

Szenario B war der stärkere Run, aber auch dieser Prompt war rückblickend nicht perfekt. Positiv war, dass der Ausgangsprompt bereits Threat Model, Grenzen, Dokumentationspflicht, klare Prüfkriterien und das Verbot produktiver BTC-Secrets enthielt. Diese Vorgaben erklären einen großen Teil des Qualitätssprungs gegenüber Szenario A.

Zwei Punkte wurden jedoch erst in der späteren Fehleranalyse vollständig sichtbar. Erstens hätte der administrative Tor-Pfad schon im Ausgangsprompt enger als „operator-only“ beschrieben werden sollen. Gemeint ist ein Zugang für einen sehr kleinen Betreiberkreis mit zusätzlicher Zugriffsbeschränkung jenseits der bloßen Onion-Adresse. Zweitens hätte die Host-Firewall früher als maßgebliche Schutzschicht festgelegt werden sollen.

Rückblickend war der B-Prompt bereits deutlich stärker als der Prompt aus Szenario A. Einige sicherheitsrelevante Punkte hätten aber noch früher und genauer festgelegt werden können. Der Admin-Zugang hätte von Anfang an ausdrücklich auf einen kleinen berechtigten Personenkreis beschränkt werden sollen, statt ihn nur allgemein als Tor-Zugang zu beschreiben. Außerdem wäre es sinnvoll gewesen, bereits im Prompt eindeutig festzulegen, welche Host-Firewall als verbindliche Schutzschicht gilt, um spätere Unklarheiten zwischen unterschiedlichen Regelwerken zu vermeiden.

2.4 Einzelbewertung der Szenarien

2.4.1 Szenario A

Erreichter Stand

Szenario A war von Beginn an auf eine möglichst weitgehende praktische Umsetzung bei alltagsnaher Steuerung ausgelegt. Codex richtete auf `ailab1` ein tatsächlich laufendes Home-Lab ein. Dieses umfasste mehrere getrennte Rollen, verschiedene Anwendungsdienste, einen Tor-Zugang, Monitoring-Komponenten und einen BTC-Bereich. Im Runbook sind 20 Container-Services, zwölf Tor-v3-Hidden-Services, ein pruned Tor-only-

BTC-Node, Monitoring mit Prometheus und Uptime Kuma sowie lokale Backups für zentrale Rollen dokumentiert. Auch typische Nacharbeiten wurden praktisch umgesetzt, etwa das Schließen offener Registrierungen, das Ergänzen von Firewallregeln und die Behebung einzelner Storage- und Mount-Probleme.

Unter grober Steuerung entstand damit nicht nur ein Konzept, sondern ein realer, prüfbarer Zwischenstand. Codex übernahm Aufgaben, die in der Praxis häufig von Administratoren erledigt werden: Paket- und Servicekonfiguration, Containerisierung, Netzwerkverhalten, Dokumentation, Monitoring-Anschluss und spätere Fehlerkorrektur.

Auch für den BTC-Bereich entstand ein funktionsfähiger Aufbau. Dokumentiert sind ein pruned BTC-Core-Node, onion-basierte Peer-to-Peer (P2P)-Konnektivität, Monitoring-Komponenten sowie Überlegungen zu Watch-only-Wallets, dem Einsatz von PSBTs und einer Trennung zwischen operativer Hot-Wallet und langfristiger Verwahrung. Darüber hinaus wurden mögliche Nachfüllprozesse für eine Hot-Wallet und die Nutzung einer separaten Treasury-Komponente im Sessionverlauf evaluiert. Über diese Evaluation hinaus entstanden dazu keine Artefakte.

Mehrere zentrale Bestandteile verblieben jedoch auf konzeptioneller Ebene. Die Evaluation eines Electrum-Servers wurde diskutiert, aber nicht praktisch umgesetzt. Ebenso blieb der Cold-Signing-Prozess als Konzept beschrieben und wurde nicht mit einem realen air-gapped Signierer umgesetzt. Die Rolle eines menschlichen Operators wurde grundsätzlich berücksichtigt, jedoch nicht als eigenständiger Freigabe- oder Betriebsprozess ausgearbeitet. Weitere offene Punkte betrafen die konkrete Auswahl einer Watch-only-Wallet, die Frage der Seed-Erzeugung, die verwendete Signierungsbibliothek sowie den Aufbau eines Multi-Signatur-Modells. Während das Mempool-Tracking praktisch umgesetzt wurde, blieb die Erzeugung und Verarbeitung von PSBTs überwiegend konzeptionell beschrieben. Auch eine Ausgestaltung der air-gapped Systeme wurde nicht dokumentiert. Die vorgesehene Hot-/Cold-Trennung einschließlich eines Refill-Prozesses war zwar fachlich beschrieben, jedoch nicht praktisch implementiert. Ebenso blieb offen, welches Verfahren zur sicheren Simulation oder Erprobung eines BTC-Netzwerks verwendet werden sollte.

Sicherheitsbewertung

Der Self-Audit für `ailab1` zeigte mehrere strukturelle Schwachstellen. Der Proxmox-Host wurde nur unzureichend überwacht. Außerdem war der interne Datenverkehr zwischen den Diensten noch zu offen geregelt, obwohl das Setup eigentlich auf Tor-Zugriff ausgelegt war. Die Backups lagen ausschließlich lokal auf demselben Host. Bei einem größeren Ausfall oder einer Kompromittierung des Hosts hätten sie daher nur begrenzt Schutz geboten. Auch das BTC-Transaktionssystem war noch nicht für reale Transaktionen bereit. Ergänzend zeigte die Artefaktanalyse, dass die BTC-Core-Installation nur gegen die `SHA256SUMS`-Datei desselben Download-Servers geprüft wurde; eine Verifikation der zugehörigen Signaturdatei fand nicht statt, womit die Integritätsprüfung bei einem kompromittierten Verteilpfad wirkungslos bliebe. Zudem verwendet dieser Ansatz `nftables` und `iptables` als Sicherheitsmodelle, was zeigt, dass diese Sicherheitsaspekte evaluiert wurden, sich jedoch nicht für einen Ansatz entschieden wurde. Beide zu verwenden wird als schlechte Praxis angesehen und sollte vermieden werden.

Codex erkannte viele dieser Probleme später selbst, dokumentierte sie im Self-Audit und schlug meist sinnvolle Gegenmaßnahmen vor. Das Grundproblem von Szenario A lag allerdings darin, dass Codex bei schwacher Steuerung zunächst möglichst viel umsetzte und Sicherheitsaspekte erst danach schrittweise nachzog. Vereinfacht gesagt: Es wurde zuerst viel gebaut und erst später gehärtet.

Es entstand damit eine funktional breit aufgestellte Infrastruktur, deren Sicherheitsniveau jedoch erst durch nachträgliche Analyse und zusätzliche Härtungsmaßnahmen verbessert wurde. Die wesentlichen Stärken und Schwächen werden in der vergleichenden Evaluation gemeinsam mit Szenario B betrachtet.

Indikatorbasierte Kategorienwertung

Tabelle 2.4 weist die Einzelwertungen der vier Prüfindikatoren je Kategorie aus. Die Indikatoren I1 bis I4 entsprechen der Reihenfolge aus Tabelle 2.2.

Kategorie	I1	I2	I3	I4	Mittel	Wertung
Architekturplanung	2	2	1	2	1,75	2
Services	3	1	2	2	2,00	2
Konfiguration	2	1	2	2	1,75	2
Netzwerk, Tor und Zugriff	1	2	1	2	1,50	2
Backup und Monitoring	0	0	2	1	0,75	1
Fehleranalyse	3	3	3	3	3,00	3

Tabelle 2.4: Indikatorwertungen für Szenario A

Die Einzelwertungen der Indikatoren begründen sich wie folgt aus den erzeugten Dateien und dem geprüften Laufzeitzustand.

Architekturplanung (2 von 3)

I1 (2): Die Punktzahl begründet sich daraus, dass ein Rollen- und Zonenmodell vorhanden ist. Dabei existieren vier getrennte Linux Containers (LXC)-Rollen bestehend aus 201 `tor-edge`, 202 `app-core`, 203 `ops` und 204 `btc-node`, welche sich die zwei Bridges `vmbr1` für Service- und `vmbr2` für Monitoring-Verkehr, definiert über `/etc/network/interfaces.d/homelab-bridges.cfg`, teilen. Explizite Schutzbedarfsklassen je Rolle fehlen jedoch, wobei die Zuordnung primär an den verfügbaren Ressourcen ausgerichtet wurde.[87] I2 (2): Die Platzierung ist grundsätzlich begründet (Host bleibt frei von Workloads, BTC als eigene Rolle). Der Abzug ergibt sich daraus, dass 202 und 204 dual-homed an beiden Bridges (`10.10.10.10/10.10.20.10` bzw. `10.10.10.30/10.10.20.30`) hängen und die Zonengrenze zwischen Service- und Monitoring-Segment damit aufweichen.[87] I3 (1): Ein dokumentiertes Regelwerk erlaubter Kommunikationspfade existiert nicht und die Laufzeit-Scans des Self-Audits zeigen, dass `sshd` (22) und `node-exporter` (9100) zwischen nahezu allen Containern erreichbar waren. Zudem ist `tor-edge` durch fast alle App- und Ops-Ports direkt ansprechbar.[88] I4 (2): Der BTC-Bereich ist sauber getrennt mit eigener Rolle, RPC und P2P nur auf Loopback und einer eigenen Firewallekette. Der Admin-Bereich bildet dagegen keine eigene Zone, sodass die Host-Managementports (22, 8006, 3128, 111) erst in der Findings-Remediation nachträglich gegenüber den Container-Bridges gefiltert werden.[87], [88]

Services (2 von 3)

I1 (3): Szenario A verwendet 20 Container-Dienste in zwei Compose-Stacks. Dabei entfallen elf auf `app-core` und neun auf `ops`. Zudem wurde ein Tor/Caddy auf `tor-edge` und `bitcoind` auf `btc-node` eingerichtet. Alle Dienste wurden per HTTP-Statuscheck validiert, etwa Vaultwarden mit 200, Paperless-ngx mit 302 und Stirling-PDF mit 401.[87], [89] I2 (1): Elf Dienste unterschiedlichen Schutzbedarfs wurden auf einer einzigen Rolle gebündelt, was sich als problematisch herausstellt. Der Passwortmanager Vaultwarden teilt sich Container und Postgres-/Redis-Unterbau mit Alltagsdiensten wie Stirling-PDF und Filebrowser, wodurch ebenfalls keine korrekte Trennung stattfindet.[87], [89] I3 (2): Punktuelle Härtenungen sind belegt, etwa der entfernte Docker-Socket-Mount bei `homepage`, `PAPERLESS_ACCOUNT_ALLOW_SIGNUPS=false` und die Grafana-Flags (`GF_AUTH_ANONYMOUS_ENABLED=false` und `GF_USERS_ALLOW_SIGN_UP=false`). Kein einziger Dienst nutzt jedoch `cap_drop`, `no-new-privileges` oder `read_only`, und alle Compose-Ports sind unbeschränkt auf allen Interfaces publiziert.[87], [89] I4 (2): Der Stack folgt dem geforderten Katalog, jedoch entstanden zusätzlich ein redundanter zweiter Monitoring-Pfad (Uptime Kuma parallel zu Prometheus/Blackbox/Alertmanager) sowie der selbstgebaute `alert-forwarder` auf Basis eines generischen `python:3.12-alpine`-Images mit eingehängtem Skript.[89]

Konfiguration (2 von 3)

I1 (2): Registrierungen wurden geschlossen (`SIGNUPS_ALLOWED=false`, in Gitea `DISABLE_REGISTRATION=true` plus `REQUIRE_SIGNIN_VIEW=true`), Log-Rotation und `admin off` für Caddy sind gesetzt. Dem stehen durchgängige `:latest`-Image-Tags und vorhersehbare Admin-Namen wie `admin` und `gitadmin` gegenüber.[88], [89], [90] I2 (1): Sämtliche Secrets werden im Klartext unter `/root/homelab-secrets` erzeugt und in `service-credentials.txt` inklusive Benutzer/Passwort-Paaren aggregiert. Zudem werden Datenbankpasswörter per `sed` in `initdb/01-init.sql` eingesetzt. Zusätzlich exportiert der Backup-Job die privaten `hs_ed25519_secret_key`-Dateien der Hidden Services in ein unverschlüsseltes lokales Archiv. Die restriktiven Dateirechte (`0600/0700`, `umask 077`) ändern am Klartextbefund nichts und stellen keine Sicherheitsmaßnah-

me dar.[88], [90], [91] I3 (2): Runbook, Staging-Kopien unter `/root/homelab-staging` und Live-Zustand wurden aktiv synchron gehalten und nach jedem Abschnitt verifiziert. Kleinere Zählabweichungen bleiben, etwa listet das Runbook zwölf publizierte Onions, während der Self-Test nur elf Serien erfasst.[87] I4 (2): Je Arbeitsabschnitt existieren `apply-/verify`-Skriptpaare (wie `apply-config-hardening.sh` mit `verify-config-hardening.sh`). Ein deterministischer Neuaufbau scheitert jedoch an den `:latest`-Tags und fehlenden Snapshot-Ständen.[87], [89]

Netzwerk, Tor und Zugriff (2 von 3)

I1 (1): Die Host-Input-Chain besitzt keine Drop-Policy, da `nftables.conf` nur `type filter hook input priority filter` ohne `policy drop` deklariert. Dies blockt lediglich die vier Managementports 22, 111, 3128 und 8006 aus `vmbr1/vmbr2`. Zudem ist die Forward-Chain leer und akzeptiert implizit alles. Containerseitig endet die `HOMELAB-DOCKER-ACCESS`-Kette zwar mit `DROP`, hängt aber nur in `DOCKER-USER` und schützt damit ausschließlich Docker-publizierte Ports. Container (CT)-eigene Dienste wie `sshd` oder `node-exporter` werden von diesen Policies nicht adressiert.[88], [92], [93] I2 (2): Zwölf v3-Onions sind publiziert und Caddy bindet alle Reverse-Proxy-Listener per `bind 127.0.0.1`, sodass die Proxyports aus dem Labornetz nicht direkt erreichbar sind. Validiert wurde aber nur lokal mit Onion-Host-Headern. Ein externer End-to-End-Zugriff über einen zweiten Tor-Client fehlt, und die Backends bleiben intern direkt ansprechbar.[87], [88], [94] I3 (1): Die Admin-Oberflächen Grafana und Uptime Kuma sowie der Alerts-Feed wurden als normale Onions ohne v3-Client-Auth publiziert. Parallel blieb Host-SSH auf `permitrootlogin yes` und `passwordauthentication yes`. [87], [88] I4 (2): Wirksam begrenzt ist der Docker-Pfad, wobei `app-core` neue Verbindungen nur von 10.10.10.2 (`tor-edge`) und 10.10.20.20 (`ops`) akzeptiert. `ops` empfängt zudem nur von `tor-edge`. Die Gegenprobe `app-core` nach `ops` scheitert nachweislich. Offen blieben `sshd` und `node-exporter` zwischen den Containern.[87], [88], [93]

Backup und Monitoring (1 von 3)

I1 (0): Alle Sicherungen liegen unter `/var/lib/homelab-backups` auf

demselben Host. Weder die `vzdump`-Archive noch die `tar.gz`-Exporte sind verschlüsselt, und das Identitätsarchiv enthält die privaten Hidden-Service-Schlüssel.[88], [91] I2 (0): Ein Restore fand nicht statt, sodass ausschließlich die Archivlesbarkeit über `zstd -t`, `tar -tzf` und ein `sha256`-Manifest geprüft wurde. Das Runbook weist den fehlenden Restore-Drill selbst als offenen Punkt aus.[87], [91] I3 (2): Die Container sind mit vier `node-exporter`-Targets, drei Blackbox-Probenklassen, Loki/Promtail und 15 Uptime-Kuma-Monitore gut abgedeckt. Der Proxmox-Host selbst fehlt jedoch vollständig in der Target-Liste der `prometheus.yml`, obwohl das Backup-Log bereits Thin-Pool-Warnungen protokollierte.[87], [88], [95] I4 (1): Die Kette Alertmanager, `alert-forwarder`, `ntfy` und Alerts-Onion existiert und verarbeitet nachweislich Meldungen. Uptime Kuma hat aber null Notification-Definitionen, und eine Zustellung bis zu einem Endgerät des Betreibers wurde nie ausgelöst.[87], [88]

Fehleranalyse (3 von 3)

I1 (3): Alle vier Artefakte wurden mit priorisierten, zeilengenauen Fundlisten (P1 bis P3) analysiert. Darunter allein 14 Einzelbefunde für `backup.bad.sh`. [96] I2 (3): Die Befunde sind fachlich eingeordnet mit einer Klassifizierung der Mitsicherung von `var/lib/tor` als Hidden-Service-Key-Leak, der RPC-Zugang über eine Onion-Adresse als Remote-Signierfläche und `policy accept` als Bruch des Default-Deny-Prinzips. Je Artefakt werden Lösungsvarianten abgewogen und die Wahl begründet.[96] I3 (3): Die Korrekturfassungen beheben die Kernprobleme nachweisbar. Dies ist nachvollziehbar, da `firewall.fixed.nft policy drop` auf `input` und `forward` setzt und so das Management auf `vmbr0` mit den Ports 22 und 8006 begrenzt. `docker-compose.fixed.yml` entfernt `privileged` und den Docker-Socket, bindet alle Ports auf `127.0.0.1` und erzwingt Secrets sowie Image-Tags über Fail-fast-Variablen (`${VAR:?}`). [96] I4 (3): Die Korrekturen führen keine neuen Risiken ein, sondern zusätzliche Schutzschichten (`ct state invalid drop`, `flock`, `umask 077`, optional `age`-verschlüsselter Sensitive-Export) und dokumentieren die Restrisiken je Datei explizit.[96]

Bewertung des BTC-Konzepts

Die gewichtete Detailbewertung für Szenario A (Tabelle 2.5) ergibt 1,45 von 3,00 Punkten. Die Einzelwertungen begründen sich wie folgt aus den erzeugten Artefakten.

Die Architekturschlüssigkeit erreicht 2 von 3 Punkten. Der Node wurde als eigene, segmentierte Rolle mit Tor-only-Anbindung betrieben, und die beschriebene Aufteilung in Node, Watch-only-Sicht, Hot-Wallet und Treasury ist in sich stimmig. Für die volle Punktzahl fehlte die Auflösung zentraler Architekturfragen, etwa zur Auswahl der Watch-only-Wallet, zur Seed-Erzeugung und zu einem Multi-Signatur-Modell.

Die Trennung von Schlüsselmaterial und Rollen erhält 1 von 3 Punkten. Zwar lag nachweislich zu keinem Zeitpunkt Schlüsselmaterial auf dem System, da nie eine Wallet angelegt wurde, aber die geforderte Trennung von Watch-only-, Hot-Service- und Offline-Rolle verblieb als reine Beschreibung ohne technische Umsetzung. Dies wird nach der Skala als eine relevante Lücke gewertet.

Der PSBT-, Signier- und Handoff-Prozess erhält 1 von 3 Punkten, da er ausschließlich als Text existiert. Im gesamten Artefaktbestand von Szenario A findet sich kein Wallet-, PSBT- oder Signier-Code.

Das Testdesign erreicht 2 von 3 Punkten. Für die umgesetzte Node-Schicht existiert mit `verify-bitcoin-section.sh` ein eigenes Prüfskript, das Dienststatus, Port-Bindungen, RPC-Erreichbarkeit, Firewall-Regeln und die Prometheus-Anbindung tatsächlich zur Laufzeit abfragt. Ergänzend liefert ein Metrik-Exporter fortlaufende Zustandsdaten. Ein Spend- oder Recovery-Pfad konnte mangels Umsetzung nicht getestet werden. Diese Lücke ist jedoch bereits in den beiden vorgenannten Kriterien abgewertet, während das Testdesign für den tatsächlich vorhandenen Umfang eng gefasst ist.

Härtung, Limitierung und Angriffsflächenreduktion erreichen 2 von 3 Punkten. Die Node-Konfiguration in `bitcoin.conf` bindet RPC und P2P ausschließlich an Loopback, erzwingt Onion-only-Konnektivität, deaktiviert Peer-Discovery und DNS-Seeds und begrenzt Verbindungszahl und Speicherbedarf. Dazu kommen vorteilhaft systemd-Sandboxing des Daemons und eine Default-Drop-Firewallkette. Der Punktabzug

ergibt sich aus der Installationskette, wobei BTC-Core nur gegen die SHA256SUMS-Datei desselben Download-Servers geprüft wird und die Verifikation der zugehörigen Signaturdatei außer Acht gelassen wird. In der geheimnisfreien Testumgebung wurde dies als kleinere Lücke eingestuft.

Recovery, Fehlerfälle und Remediation erhalten 1 von 3 Punkten. Das Backup exportiert Konfigurations- und Wiederanlaufartefakte der Nodes und Alerts für RPC-Ausfall und Peer-Verlust sind definiert. Nachteilig findet ein Restore-Test jedoch nicht statt, die Sicherungen lagen ausschließlich lokal auf demselben Host. Eine Wallet-Recovery-Betrachtung fehlt, da die Wallet selbst nicht fertig gestellt wurde.[87], [88]

In Summe ergibt sich für Szenario A 1,45 von 3,00 Punkten; für die Kategorienwertung der vergleichenden Evaluation entspricht das gerundet 1 von 3 Punkten.

Unterkriterium	Impl. A		Impl. B		Eigenentw.	
	Score	Beitrag	Score	Beitrag	Score	Beitrag
Architekturschlüssigkeit	2	0,4	2	0,4	3	0,6
Trennung Schlüsselmaterial / Rollen	1	0,25	1	0,25	3	0,75
PSBT-, Signier- & Handoff-Prozess	1	0,2	1	0,2	3	0,6
Testdesign & praktische Prüfbarkeit	2	0,3	1	0,15	2	0,3
Härtung / Angriffsflächenreduktion	2	0,2	3	0,3	3	0,3
Recovery / Fehlerfälle / Remediation	1	0,1	2	0,2	3	0,3
Gesamtbewertung		1,45		1,50		2,85

Tabelle 2.5: Bewertung der BTC-Implementierungen

2.4.2 Szenario B

Erreichter Stand

In Szenario B lieferte der Nutzer bereits zu Beginn Vorgaben zu einem Masterplan, ein explizites Threat Model, klare Regeln, Verbote für produktive Geheimnisse und Validierung. Der Build-Prozess verlief dadurch langsamer, aber geordneter.

Auf `ailab2` wurden acht voneinander getrennte Rollen für die einzelnen Bereiche des Home-Labs eingerichtet. Dazu kamen eine dokumentierte Sicherheitsarchitektur, ein restriktives `nftables`-Regelwerk nach dem Default-Deny-Prinzip, ein eigener Admin-Onion-Zugang mit zusätzlicher `v3-Client-Auth`, ein Borg-basierter Backup-Pfad mit `Smoke-Restore`, eine Monitoring-Grundlage mit `Host-Exporter` und eine `BTC-Simulation` mit `Dummy-Daten`.

Der `BTC`-Bereich ist insgesamt sicherheitsorientiert aufgebaut. Die vorgesehene Trennung zwischen `Watch-only-Funktion`, operativer Verarbeitung und `Offline-Signierung` folgt etablierten Prinzipien zur Reduzierung der Exposition kryptographischer Geheimnisse. Besonders positiv ist, dass produktive `Seeds`, `Wallet-Dateien` und `Private Keys` konsequent außerhalb des Versuchssystems gehalten wurden.

Gleichzeitig blieb der praktische Nachweis mehrerer Kernmechanismen begrenzt. Weder eine konkrete `Watch-only-Implementierung` noch eine `Signierungsbibliothek`, ein vollständiger `Multi-Signatur-Aufbau` oder ein echter `air-gapped Signierer` wurden umgesetzt. Auch die `Simulation` des `BTC-Netzwerks` beschränkte sich auf `dateibasierte Dummy-Artefakte` ohne `Regtest`-, `Testnet`- oder vergleichbaren `Validierungsansatz`. Die als `PSBT` bezeichneten `Dateien` sind dabei keine `BIP-174-konformen PSBTs`, sondern reine `JSON-Metadaten` ohne `Transaktionsstruktur`. Dadurch konnten zentrale Abläufe wie `Transaktionserstellung`, `Signierung`, `Recovery` und `Broadcast` nicht vollständig überprüft werden.

Besonders kritisch ist dabei die Darstellung des simulierten Signierprozesses. Im Skript wird ausdrücklich kein echtes Signieren durchgeführt. Stattdessen erzeugt die Funktion `create_dummy_signed_artifact` lediglich eine Datei mit der Artefaktklasse `dummy-signed-psbt` und dem Marker `DUMMY_SIGNATURE_MARKER_ONLY`. Auch der anschließende `Broadcast` ist nicht real, sondern wird über ein `Dummy-Receipt` mit `broadcast_mode = simulated-no-network` und `network_action = not-performed` abgebildet. Problematisch ist jedoch, dass die `Log-Ausgaben` diesen Simulationscharakter nicht immer ausreichend deutlich machen. Meldungen wie *Running service phase 1 unsigned-PSBT validator*, *Creating simulated offline-signed artifact*, *Running service phase 2 import/broadcast validator* und *Section 06 BTC simulation completed successfully* können bei

oberflächlicher Prüfung den Eindruck eines erfolgreich durchlaufenen Signier- und Broadcast-Workflows erzeugen. Tatsächlich wurde aber weder eine echte Signatur erzeugt noch eine Signatur kryptographisch geprüft oder eine Transaktion an ein BTC-Netzwerk übertragen. Damit entsteht eine für KI-generierte Infrastruktur typische Scheininvalidierung. Das System erzeugt plausible Artefakte, Statusdateien und erfolgreiche Abschlussmeldungen, ohne dass der sicherheitskritische Kernprozess tatsächlich implementiert wurde. Verstärkt wird dieser Effekt dadurch, dass Teile des Nachweises tautologisch aufgebaut sind: Die Statuszeilen `wallet_dat=absent`, `seed_files=absent` und `xprv_files=absent` werden im Validator fest einkodiert ausgegeben, bevor die eigentliche Dateisystemsuche läuft, und die Abschlussprüfung assertiert genau diese fest einkodierten Zeichenketten. Die tatsächliche Suche beschränkt sich zudem auf die Pfade `/srv/bitcoin-sim` und `/etc/ailab`, sodass verbotene Artefakte an anderen Orten des Gastsystems unentdeckt blieben. Gerade bei Sicherheitsaufgaben ist dieses Verhalten problematisch, weil die Ausgabe auf den ersten Blick den Eindruck technischer Vollständigkeit erzeugt und dadurch einen menschlichen Prüfer täuschen kann. Die Täuschung liegt dabei nicht in einem einzelnen falschen Artefakt, sondern in der Kombination aus realistisch benannten Phasen, erfolgreichen Validator-Meldungen und nur im Detail erkennbaren Dummy-Markierungen. Einschränkend ist festzuhalten, dass die erzeugten Artefakte selbst sowie die begleitende Dokumentation den Simulationscharakter eindeutig deklarieren. Unklar bleibt jedoch für einen Nutzer, ob wirklich Dummy-Objekte oder Test-Objekte wie PSBTs eines Testnets gemeint sind.

Insgesamt ist die BTC-Architektur fachlich plausibel und sicherheitsorientiert konzipiert. Die wesentlichen Einschränkungen liegen jedoch in der begrenzten praktischen Nachweistiefe der vorgesehenen Prozesse und in der irreführenden Wirkung der erzeugten Validierungs- und Log-Ausgaben. Für eine belastbare Bewertung muss daher klar festgehalten werden, dass Szenario B keinen echten BTC-Signierprozess umgesetzt hat, sondern lediglich einen dateibasierten Simulationsablauf mit Dummy-PSBTs, Dummy-Signaturartefakten und simuliertem Broadcast.

Sicherheitsbewertung

Szenario B zeichnet sich insgesamt durch eine konsistente und sicherheitsorientierte Umsetzung der vorgegebenen Anforderungen aus. Die Architektur wurde entlang klar definierter Rollen, Kommunikationspfade und Schutzgrenzen aufgebaut. Sicherheitsrelevante Entscheidungen wurden früh dokumentiert und durch technische Maßnahmen wie restriktive Firewall-Regeln, eingeschränkte Administrationspfade, zusätzliche Authentifizierungsschichten sowie einen überprüften Backup- und Restore-Prozess abgesichert. Positiv hervorzuheben ist außerdem die konsequente Vermeidung produktiver Geheimnisse. Während des gesamten Builds wurden keine produktiven Seeds, Wallet-Dateien, Private Keys oder API-Schlüssel erzeugt oder gespeichert. Die Dokumentation der Sicherheitsgrenzen blieb dabei über die einzelnen Artefakte hinweg konsistent. Abgesichert werden soll dieses System über `nftables`, wobei dies einen validen Ansatz darstellt

Die Analyse der BTC-Komponente zeigt, dass mehrere sicherheitskritische Prozesse lediglich konzeptionell bzw. simulativ abgebildet wurden. Dadurch konnte der vollständige Nachweis zentraler Sicherheitsfunktionen nicht erbracht werden. Für eine belastbare Bewertung sicherheitskritischer Transaktionssysteme reicht die Existenz von Prozessartefakten allein nicht aus. Erforderlich ist zusätzlich die nachvollziehbare Ausführung und Prüfung der zugrunde liegenden Kernfunktionen. Zudem wäre einem Nutzer durch eine Simulation von einem BTC-Setup nicht geholfen und seine eigentliche Anforderung bleibt unerfüllt.

Indikatorbasierte Kategorienwertung

Tabelle 2.6 weist die Indikatorwertungen für Szenario B analog zu Tabelle 2.4 aus.

Kategorie	I1	I2	I3	I4	Mittel	Wertung
Architekturplanung	3	3	3	3	3,00	3
Services	0	1	1	2	1,00	1
Konfiguration	3	3	3	3	3,00	3
Netzwerk, Tor und Zugriff	3	2	3	3	2,75	3
Backup und Monitoring	1	2	2	2	1,75	2
Fehleranalyse	3	3	3	3	3,00	3

Tabelle 2.6: Indikatorwertungen für Szenario B

Die Einzelwertungen der Indikatoren begründen sich wie folgt.

Architekturplanung (3 von 3)

I1 (3): Der Masterplan definiert sechs Sicherheitszonen und weist jeder Zone einen Schutzbedarf, erlaubte Zugriffspfade sowie eine eigene Secrets-Klasse mit expliziter Positiv- und Negativliste und eine Backup-/Restore-Klasse zu.[97] I2 (3): Die Provisionierungsmatrix begründet je Gast Typ, Zone, Bridge und Isolationsprofil. Dabei führt hoher Schutzbedarf zu VMs aus einem gepinnten Debian-Cloud-Image, Infrastrukturrollen zu unprivilegierten LXCs ohne `nesting`, `keyctl`, `fuse` oder `mknod`. Zudem ist auch die Zurückstellung von Collabora und Electrs einzeln begründet.[97] I3 (3): Die Deny-by-default-Kommunikationsmatrix dokumentiert jeden erlaubten Pfad mit Quelle, Ziel, Port und Zweck. Dies äußert sich dadurch, dass `vm-bitcoin-service` nach `vm-bitcoin-node` nur über TCP 8332 kommunizieren darf. Außerdem untersagen Querschnittsregeln Rückverbindungen in die Management-Zone.[97] I4 (3): Admin-, Anwendungs- und BTC-Bereich sind strikt getrennt. Der Admin-Pfad führt ausschließlich über Tor mit v3-Client-Auth auf den Admin-Onion bzw. für die Proxmox-Web-User Interface (UI) über einen lokalen SSH-Tunnel. BTC bildet eine eigene Dummy-only-Zone auf `vmbr50` mit eigener Verbotsliste.[97], [98]

Services (1 von 3)

I1 (0): Der geplante Katalog aus 20 Diensten verblieb vollständig auf Planungsebene, sodass bis zum Ende des Runs keine App-Rollouts erfolgten. Als Zusatzpakete wurden ausschließlich `qemu-guest-agent` und

`cloud-guest-utils` installiert und fünf der acht Gäste verblieben im Sollzustand `stopped`.[\[97\]](#), [\[98\]](#) I2 (1): Die schutzbedarfsgerechte Verteilung existiert nur als Matrix. Dementsprechend sind lediglich der Monitoring-Stack auf 103 und der Borg-Endpunkt auf 104 real verteilt.[\[97\]](#), [\[98\]](#) I3 (1): Minimale Berechtigungen sind nur für die dienstlose Minimalbasis belegt wie der Account `borgrepo` mit `ForceCommand`-Verengung. Für den eigentlichen Servicekatalog existiert kein in Dateien nachweisbares Rechtemodell.[\[98\]](#), [\[99\]](#) I4 (2): Unnötige Zusatzdienste wurden konsequent vermieden und der temporäre Paketpfad (`vmbr90`, `apt-cacher-ng`) nach der Validierung vollständig zurückgebaut. Der Abzug ergibt sich daraus, dass fünf provisionierte Gäste als funktionslose Hüllen verbleiben.[\[98\]](#), [\[100\]](#)

Konfiguration (3 von 3)

I1 (3): Alle Gäste erhielten über `section3_config_remote.sh` eine reproduzierbare gehärtete Minimalbasis. Diese stellt echte Betriebssystem (BS)-Updates (`-with-new-pkgs`), `Etc/UTC`, `Journald-Limit` `SystemMaxUse=64M`, die `/etc/ailab`-Struktur mit `0700-Secrets`-Verzeichnis und `policy-rc.d` gegen ungewollte Dienststarts bereit. Die VM-Basis wurde gegen `SHA512SUMS` samt Signaturdatei verifiziert.[\[97\]](#), [\[101\]](#), [\[102\]](#) I2 (3): In IaC- und Doku-Pfaden liegen keine Klartext-Secrets. Dabei enthält `/etc/ailab/secrets` nur ein README mit dem expliziten Verbot echter Geheimnisse. Laufzeitnahe Artefakte wie der Operator-Auth-Key liegen `root-only` unter `/root/ailab-runtime` außerhalb von `outputs` und IaC.[\[100\]](#), [\[101\]](#) I3 (3): Jeder Abschnitt endet mit einem Validator-Lauf, der Datei- und Live-Zustand abgleicht (Paketmanifest und Port-Check je Gast). Abweichungen wie die DHCP-Instabilität von 101 sind samt Korrektur dokumentiert.[\[100\]](#), [\[102\]](#) I4 (3): Reproduzierbarkeit ist über durchgängige Snapshots wie beispielsweise `post-provision-base`, `post-config-base` und `post-network-tor-base` für alle acht Gäste gegeben. Zudem belegen definierte Soll-Zustände und der nachgewiesene Rückbau des Temp-Pfads diesen Zustand.[\[98\]](#), [\[102\]](#)

Netzwerk, Tor und Zugriff (3 von 3)

I1 (3): Die Tabelle `inet_ailab` setzt `policy drop` auf `input` und `forward`. Dadurch werden nur `Loopback`, `established/related` sowie zwei quell-

gebundene Pfade (`vmbr0` mit `ip saddr @operator_v4` auf die Ports 22 und 8006 sowie `vmbr10` mit der Adresse des Tor-Relays auf Port 22) akzeptiert. Da `pve-firewall` deaktiviert wurde, ist genau ein Regelwerk wirksam.[98], [103], [104] I2 (2): Der einzige Tor-Pfad ist der Admin-Onion (`HiddenServicePort 22 10.10.10.1:22`). Service-Onions existieren mangels ausgerollter Dienste nicht, sodass der Indikator nur für den Admin-Pfad belegt ist, was eine größere Lücke darstellt.[98], [103] I3 (3): Der Admin-Onion ist auf genau einen autorisierten Client verengt, sodass service-seitig eine einzige Datei `operator-1.auth` (Rechte 0600, `debian-tor`) unter `authorized_clients` und host-seitig ein einziges `root-only operator-1.auth_private` außerhalb des IaC-Pfads existieren. Der Negativtest mit einem unautorisierten Tor-Client nach vollständigem Bootstrap blockierte nachweislich.[100], [102], [105] I4 (3): Interne Pfade sind quell- und zielgebunden freigeschaltet, wie etwa `ip saddr 10.30.30.103 ip daddr 10.30.30.1 tcp dport 9100 accept` für den Monitoring-Pull. Für alle sieben Zielgäste ist die Blockade der Host-Gateway-Ports 22, 111, 8006 und 3128 einzeln validiert.[98], [99], [100]

Backup und Monitoring (2 von 3)

I1 (1): Das Borg-Repository ist mit `repokey-blake2` verschlüsselt und der Zugriff über den dedizierten Nutzer `borgrepo` mit `restrict`, `ForceCommand borg serve -restrict-to-path`, `PermitTTY no` und deaktiviertem Forwarding verengt. Das Repository liegt jedoch auf demselben Testhost, und die Passphrase wird per `openssl rand -hex 32` im Klartext unter `/root/ailab-runtime/borg-host-to-104/passphrase` abgelegt.[98], [99] I2 (2): Der Restore wurde als Smoke-Restore des Archivs `ailab2-20...2Z` mit `host/configs.tgz`, `ct-101/sanitized-configs.tgz`, `EXCLUSIONS.txt` nachgewiesen. Die Wiederherstellung einer vollständigen Rolle fehlt.[98], [102] I3 (2): Der Host wird erstmals mitüberwacht (`node-exporter` auf `10.30.30.1:9100` plus SSH-Auth-Textfile-Metriken), wobei alle sechs Prometheus-Targets `health=up` melden. Die Abdeckung endet jedoch bei den drei laufenden Gästen. Die Dienste sind mangels Rollout nicht überwachbar und die Blackbox-Probe prüft nur den `ntfy`-Endpunkt.[98], [99], [100] I4 (2): Der Alertmanager routet auf das `operator-only ntfy` (`listen-http` auf `10.30.30.103:2586`), wobei

eine Zustellung bis zu einem Endgerät des Betreibers nicht durchgetestet wurde.[98], [99]

Fehleranalyse (3 von 3)

I1 (3): Alle vier Artefakte wurden mit tabellarischen Fundlisten (Priorität, Typ, Stelle) analysiert. Darunter finden sich auch strukturelle Befunde wie die IPv6-Wirkung der `table inet`-Freigaben und das direkte Schreiben in finale Archivdateien.[106] I2 (3): Jede Datei enthält eine Variantenabwägung mit gewählter Korrekturvariante, Alternative und Begründung sowie die sanitisierte, generationierte und `age`-verschlüsselte Bundle-Variante statt eines Vollbackups mit nachgelagerten Excludes.[106] I3 (3): Die Referenzfassungen beheben die Kernprobleme, sodass die `firewall.fixed.nft policy drop` nutzt, die `define`-Variablen und Sets die Admin-Zugriffe an die Quelladresse 10.0.2.2 bindet und den Egress auf DNS, Network Time Protocol (NTP) und HTTP(S) begrenzt. Die Datei `backup.fixed.sh` ergänzt `age`-Verschlüsselung, Generationen-Verzeichnisse und eine `EXCLUSIONS.txt`. [106] I4 (3): Statt neuer Risiken führen die Korrekturen zusätzliche Härtung ein wie `cap_drop: [ALL]`, `read_only`, `tmpfs` und Grafana als unprivilegierter Nutzer 472:472. Zudem werden die Restrisiken je Datei schlüssig dokumentiert.[106]

Bewertung des BTC-Konzepts

Die gewichtete Detailbewertung für Szenario B (Tabelle 2.5) ergibt 1,50 von 3,00 Punkten. Die Einzelwertungen begründen sich wie folgt.

Die Architekturschlüssigkeit erreicht 2 von 3 Punkten. Das Rollen-, Zonen- und Datenklassenmodell ist konsistent dokumentiert und wurde in der Umsetzung eingehalten. Der Abzug ergibt sich, da die im Masterplan vorgesehenen schmalen RPC-Kommunikationspfade zwischen Service- und Node-Rolle in der Umsetzung durch einen rein dateibasierten Ablauf ersetzt wurden.

Die Trennung von Schlüsselmaterial und Rollen erhält 1 von 3 Punkten. Auf Dateiebene wurde die Trennung technisch erzwungen, jedoch bleibt der Gegenstand dieser Trennung hinter dem Kriterium zurück.

Die Watch-only- und Hot-Service-Rollen wurden nicht als Dienste implementiert, sondern existieren nur als Verzeichnis- und Datei-Choreografie mit Dummy-Artefakten. Ein Service für die Rollenfunktionen wurde nicht gebaut. Damit existierte zu keinem Zeitpunkt Schlüsselmaterial, auch kein im Simulationsrahmen erlaubtes Test-Schlüsselmaterial, dessen Trennung hätte nachgewiesen werden können. Verstärkend bleiben Teile des geforderten Nachweises deklarativ, da mehrere Statuszeilen fest einkodiert sind und sich die Suche nach verbotenen Artefakten auf zwei Verzeichnispfade des Gastsystems beschränkt. Nach der Skala ist eine Rollentrennung, die weder implementierte Rollen-Services noch Schlüsselmaterial schützt, eine relevante Lücke, sodass trotz des sauberen Rechtemodells nur ein Punkt vergeben werden kann.

Der PSBT-, Signier- und Handoff-Prozess erhält 1 von 3 Punkten. Der zweiphasige Handoff-Ablauf wurde zwar vollständig durchlaufen und dokumentiert, aber der namensgebende Kern des Kriteriums fehlt. Die Artefakte besitzen keine BIP-174-Struktur und die Signatur besteht aus einer Markerzeichenkette. Eine Signaturprüfung für die TX findet nicht statt. Dementsprechend wird der Broadcast nur als Quittungsdatei abgebildet und findet nicht real statt. Ein Service für die Funktionalität wurde nicht gebaut. Nach der Skala ist eine fehlende Signier- und Prüffunktion in einem Signier-Kriterium eine relevante Lücke, sodass trotz der sauberen Prozess-Choreografie nur ein Punkt vergeben werden kann.

Das Testdesign erhält 1 von 3 Punkten. Das Validator-Framework ist umfangreich und führt echte Prüfungen von Listnern, Prozessen und Dateirechten durch, wird jedoch durch zwei strukturelle Schwächen entwertet. Erstens gibt der Validator die Statuszeilen zu Wallet-, Seed- und Schlüsseldateien fest einkodiert aus, bevor die eigentliche Dateisystemsuche läuft. Die Abschlussprüfung assertiert genau diese Zeichenketten und kann in diesen Punkten per Konstruktion nicht fehlschlagen. Nur die ergänzende Negativprüfung auf verbotene Dateinamen wertet die tatsächliche Suchausgabe aus. Diese Suche bleibt allerdings auf zwei Verzeichnispfade des Gastsystems beschränkt. Zweitens existiert für die eigentlichen BTC-Kernfunktionen keine praktische Prüfbarkeit. Der Initialprompt verbot ausschließlich echte Seeds, Private Keys und produktive Geheimnisse. Eine lokale Regtest-Umgebung erzeugt lediglich wertloses Testschlüsselmaterial über einen eigenen Ableitungspfad und hätte diesen

Rahmen somit nicht verletzt. Codex hat diese Möglichkeit dennoch zu keinem Zeitpunkt evaluiert oder auch nur erwähnt, was der exportierte Chatverlauf belegt.[83] Gerade diese Eigenleistung wäre von einem Sicherheitsberater zu erwarten gewesen.

Härtung, Limitierung und Angriffsflächenreduktion erreichen als einziges Kriterium 3 von 3 Punkten. Auf beiden Gästen sind keine BTC-Listener offen und keine BTC-Daemons aktiv, was durch reale Laufzeitprüfungen belegt ist. Die Paketbasis bleibt minimal, die Verarbeitung läuft unter einem unprivilegierten Benutzer, der Host-Handoff ist root-only mit engen Dateirechten und die Gäste wurden nach jedem Validatorlauf wieder gestoppt. Die Lösung ist damit eng gefasst und belegt.

Recovery, Fehlerfälle und Remediation erhalten 2 von 3 Punkten. Vor und nach dem Abschnitt wurden Snapshots beider Gäste angelegt, der vorherige Laufzeitbestand wurde gesichert, und ein realer Bootfehler des Extensible Firmware Interface (EFI)-Pfads wurde reproduzierbar behoben. Der Abzug ergibt sich daraus, dass für den root-only-Handoff kein Backup- oder Retention-Pfad existiert, was die Dokumentation selbst festhält. Fehlerfälle innerhalb des Workflows werden außerdem nicht simuliert.[97], [98], [102]

In Summe ergibt sich für Szenario B 1,50 von 3,00 Punkten, was für die Kategorienwertung der vergleichenden Evaluation gerundet 2 von 3 Punkten entspricht.

2.5 Vergleichende Bewertung

Die Eigenentwicklung wurde nach denselben Prüfindikatoren und derselben Skala bewertet wie die beiden KI-Szenarien. Die Kategorie Fehleranalyse ist auf sie nicht anwendbar, weil diese Kategorie die Validierungsfähigkeit des LLM misst und die eigene Ausarbeitung dort Prüfgegenstand statt Prüfer wäre. Ihre Gesamtsumme bezieht sich deshalb auf sechs Kategorien und ist mit den Szenariosummen nur eingeschränkt direkt vergleichbar.

Der Abstand zwischen den beiden Szenarien verteilt sich nicht gleichmäßig. Szenario B liegt in den fünf sicherheits- und nachweisorientierten

Bereich	Szenario A	Szenario B	Eigenentwicklung
Architekturplanung	2	3	3
Services	2	1	3
Konfiguration	2	3	3
Netzwerk / Tor / Zugriff	2	3	3
Backup / Monitoring	1	2	2
BTC-Konzept	1	2	3
Fehleranalyse	3	3	–
Gesamt	13 / 21	17 / 21	17 / 18

Tabelle 2.7: Punktbewertung

Kategorien jeweils einen Punkt vor Szenario A und fällt bei den Services um einen Punkt zurück. Die vier Punkte Differenz beschreiben deshalb keine durchgehende Überlegenheit, sondern eine andere Optimierung. Ohne fachliche Vorgaben priorisierte das Modell sichtbare Funktion. Mit engem Rahmen priorisierte es belegbare Struktur. In beiden Fällen war Codex zu erheblicher Eigenleistung fähig, seine Prioritäten folgten aber der Qualität der menschlichen Steuerung.

Die 17 Punkte für Szenario B bedeuten nicht, dass ein fertiges Produkktivsystem entstanden ist. Sie bedeuten, dass die im Versuch festgelegten Anforderungen überwiegend nachweisbar erfüllt wurden. Fragen wie Produktivbetrieb, langfristige Wartung oder verbleibende Restrisiken flossen nicht in die Bewertung ein. Erst der Abgleich mit der Eigenentwicklung zeigt zudem, wie viel Weg zwischen einer gut belegten Testbasis und einem tatsächlich betreibbaren System liegt.

Auf die Details der Eigenentwicklung wird analog zum BTC-Konzept nur kurz eingegangen. Tabelle 2.8 begründet die vergebenen Wertungen und liefert die Vergleichspunkte für die folgenden Absätze. Die genannten Aspekte können in der Ausarbeitung des Apphosts nachgelesen werden (Abschnitt 1.3).

Kategorie	I1–I4	Wertung	Details der Eigenentwicklung
Architekturplanung	3/3/2/2 (2,50)	3	I1 (3): Zonenmodell über dedizierte Docker-Netzsegmente je Dienstgruppe inklusive eigener BTC-Zone (finance-network). Die Abdeckung des vorgegebenen Bedrohungsmodells je Angriffsvektor ist dokumentiert. I2 (3): Platzierung aus echtem Abwägungsprozess, inklusive dokumentierter Erprobung und begründeter Verwerfung von Kubernetes/Talos. I3 (2): Kommunikationspfade real erzwungen durch DBs und Caches ohne Traefik-Label nur im eigenen Segment, jedoch keine vollständige Quelle-Ziel-Port-Matrix. I4 (2): Admin-Oberflächen bilden keine eigene Zone, sondern liegen hinter Authelia im traefik-public -Netz.
Services	3/3/2/3 (2,75)	3	I1 (3): Über 18 produktive Dienste in themengetrennten Compose-Dateien (compose/), je Dienst mit Healthcheck. I2 (3): Verteilung nach Schutzbedarf über eigene Netzsegmente. Z. B. compose/media/immich.yml mit Postgres/Valkey nur im Medien-Segment. I3 (2): Durchgängig cap_drop: [ALL] , no-new-privileges , read_only wo möglich und Ressourcenlimits. Definiert sind funktionsbedingte Ausnahmen wie Collabora mit SYS_ADMIN , cAdvisor privilegiert und Exporter im Host-Namespace. I4 (3): Keine funktionslosen Zusatzdienste, jede Compose-Datei zugeordnet zu einem klaren Zweck.

Fortsetzung auf der nächsten Seite

Kategorie	I1–I4	Wertung	Details der Eigenentwicklung
Konfiguration	3/2/3/3 (2,75)	3	I1 (3): Gehärtete Defaults zentral und deklarativ in vier NixOS-Modulen (<code>security.nix</code> , <code>networking.nix</code> , <code>docker.nix</code> , <code>secureboot.nix</code>). I2 (2): Verwendung von Hashes durch Argon2id über <code>scripts/update-secrets-authelia.sh</code> , <code>bcrypt</code> für <code>ntfy</code> , <code>secrets/</code> mit Rechten 0700 außerhalb des Repositories. Die Dienst-Passwörter liegen jedoch systembedingt im Klartext in der <code>.env</code> auf dem Host. I3 (3): Konsistenz zwischen Dokumentation und Dateien nach Review-Abgleich, wobei konkrete Werte im Anhang ausgewiesen sind. I4 (3): Reproduzierbar über Flake-Pinning (<code>flake.lock</code>) und atomare Generationen mit Rollback.
Netzwerk / Tor / Zugriff	3/3/2/3 (2,75)	3	I1 (3): Konfiguriert über <code>nftables</code> mit <code>policy drop</code> auf Input und Forward (<code>modules/networking.nix</code>). SSH ist rate-limitiert und es existiert kein nach außen geöffneter Port durch DNS-01-Zertifikaten. I2 (3): Tor-Erreichbarkeit pro Dienst ist über ein eigenes Label-Set explizit zuschaltbar. Der Tor-Client ist gehärtet (<code>config/tor/torrc: ClientOnly</code> , deaktivierter Control- und SOCKS-Port). I3 (2): Admin-Oberflächen zusätzlich hinter Authelia-Forward-Auth mit optional aktivierbarer MFA. Fail2ban-Web-Jail ist eingerichtet I4 (3): Die interne Kommunikation wird über Segmentierung und User-Namespace-Remapping begrenzt und Docker-API ist nur über den lesenden Socket-Proxy erreichbar.

Fortsetzung auf der nächsten Seite

Kategorie	I1–I4	Wertung	Details der Eigenentwicklung
Backup / Monitoring	1/1/3/3 (2,00)	2	I1 (1): konsistente Proxmox-Snapshots des Gesamtzustands mit optionaler Verschlüsselung, jedoch kein umgesetztes Off-Host-Ziel. I2 (1): Restore per Design als einzelner Rückspielschritt, der jedoch nicht dokumentiert durchgespielt wurde. I3 (3): Host und Container werden vollständig überwacht (Node-Exporter, cAdvisor, Alloy nach Loki, Prometheus-Alert-Regeln). I4 (3): Etablierung einer Alarmkette durch Alertmanager über selbst gehostetes ntfy bis auf das GrapheneOS-Endgerät des Mandanten.
Gesamt		14 / 15	

Tabelle 2.8: Indikatorbasierte Detailbewertung der Eigenentwicklung in den Basis-Kategorien

In den nächsten Abschnitten werden die Kategorien einzeln verglichen. Die zugehörigen Einzelbefunde sind in Abschnitt 2.4 indikatorweise belegt und werden hier nicht wiederholt. Der Vergleich ordnet sie ein und bezieht die Eigenentwicklung als Referenz mit ein.

2.5.1 Architekturplanung

Eigenentwicklung: 3, A: 2, B: 3. Alle drei Systeme beruhen auf einem nachvollziehbaren Zonen- und Rollenmodell. Der einzige Unterschied liegt im Umgang mit der eigenen Planung. Szenario A ließ die Architektur während des Aufbaus entstehen und erzwang sie nur teilweise, sodass Anspruch und tatsächlicher Netzzustand auseinanderliefen. Szenario B legte die Zielstruktur vor der ersten Umsetzung verbindlich fest und validierte die Grenzen anschließend fortlaufend. Die Eigenentwicklung geht darüber hinaus, weil sie konkurrierende Ansätze zunächst ernsthaft erprobt und begründet verworfen hat. Ihre Entscheidung für einen einzelnen gehärteten Host entstand aus einem echten Abwägungsprozess und wurde bis in ein produktives System getragen. Szenario B plant insofern auf Augenhöhe, belegt seine Grenzen aber nur an einer leeren Basis. Szenario A bleibt hinter beiden zurück, weil sein Modell zwar

vergleichbar sinnvoll war, seine Durchsetzung aber nicht nachgewiesen wurde.[87], [88], [97], [102]

2.5.2 Services

Eigenentwicklung: 3, A: 2, B: 1. Funktional steht Szenario A der Eigenentwicklung am nächsten. Beide übertreffen die geforderte Dienstzahl deutlich und decken einen alltagstauglichen Katalog ab. Der Abstand zwischen ihnen ist struktureller Natur. Die Eigenentwicklung trennt jede Dienstgruppe in ein eigenes Netzsegment und hält ihre Datenbanken von außen vollständig unerreichbar, während Szenario A Dienste unterschiedlichen Schutzbedarfs auf einer Rolle bündelt. Der Abstand zu Szenario B ist dagegen eine Existenzfrage, denn dort wurde kein einziger fachlicher Dienst ausgerollt. Für den Mandanten wiegt dieser Unterschied schwer. Die Struktur von A ließe sich nachträglich härten, das leere Grundgerüst von B müsste die Kernleistung erst noch erbringen. Dass ausgerechnet der stärker gesteuerte Lauf hier am weitesten zurückfällt, ist zugleich der deutlichste Beleg für den Zielkonflikt zwischen funktionaler Umsetzung und Nachweisorientierung.[87], [97], [98]

2.5.3 Konfiguration

Eigenentwicklung: 3, A: 2, B: 3. Die Eigenentwicklung und Szenario B erreichen dieselbe Wertung auf unterschiedlichen Wegen. Die Eigenentwicklung beschreibt den kompletten Systemzustand deklarativ im Repository und macht jede Änderung über Generationen umkehrbar. Ihre Reproduzierbarkeit ist damit an einem voll ausgebauten System belegt. Szenario B erbringt denselben Nachweis über Snapshots und Validatorläufe, jedoch nur für eine dienstlose Minimalbasis. Ob diese Disziplin auch unter der Last von zwanzig realen Diensten gehalten worden wäre, bleibt offen. Szenario A fällt gegenüber beiden ab, weil seine Härtung nachträglich entstand und mit Klartext-Ablagen sowie unfixierten Image-Ständen genau die Driftquellen enthält, die die anderen beiden Systeme per Konstruktion vermeiden.[87], [88], [100], [102]

2.5.4 Netzwerk, Tor und Zugriff

Eigenentwicklung: 3, A: 2, B: 3. In dieser Kategorie verbindet die Eigenentwicklung die Stärken der beiden Szenarien. Sie erreicht die Zugriffshärte von Szenario B, weil kein Port nach außen geöffnet wird und eingehender Verkehr standardmäßig verworfen wird. Zugleich bietet sie die Breite von Szenario A, da sämtliche Dienste über einen zentralen, mehrfaktorgeschützten Einstiegspunkt und wahlweise über Tor erreichbar sind. Szenario B belegt seine Strenge dagegen nur für den einzigen vorhandenen Admin-Pfad. Dieser eine Pfad ist dafür formal strenger abgesichert als der Admin-Zugang der Eigenentwicklung, weil er zusätzlich v3-Client-Auth erzwingt. Die Eigenentwicklung hingegen schützt ihre Admin-Oberflächen über die zentrale Authelia-Anmeldung mit optionaler Multi-Factor Authentication (MFA), was sich als leicht schwächer darstellt. Szenario A publizierte umgekehrt viele Pfade, ohne die Grenzen dahinter technisch zu erzwingen. Der Vergleich zeigt damit, dass Erreichbarkeit und Durchsetzung kein Widerspruch sind. Jedes der beiden KI-Szenarien hat nur eine der beiden Hälften eingelöst, die Eigenentwicklung beide gleichzeitig.[87], [88], [98], [102]

2.5.5 Backup und Monitoring

Eigenentwicklung: 2, A: 1, B: 2. Auffällig ist zunächst eine gemeinsame Schwäche aller drei Systeme, denn keines setzt ein Sicherheitsziel außerhalb des eigenen Hosts um. Innerhalb dieser Grenze unterscheiden sich die Profile deutlich. Die Eigenentwicklung sichert den Gesamtzustand als konsistente Snapshots und ist das einzige System, dessen Alarmkette nachweislich bis auf das Endgerät des Mandanten reicht. Ein geübter Restore ist jedoch auch dort nicht dokumentiert, weshalb nach demselben Maßstab wie bei den Szenarien nur zwei Punkte vergeben werden können. Szenario B erreicht die gleiche Wertung mit umgekehrtem Profil. Es belegt Verschlüsselung und einen kleinen Restore, besitzt aber keine durchgehende Alarmkette und überwacht nur eine leere Basis. Szenario A bleibt hinter beiden zurück, wobei weder die Vertraulichkeit noch die Wiederherstellbarkeit seiner Sicherungen belegt sind. Die Punktgleichheit von B und Eigenentwicklung ist deshalb kein Gleichstand der Reife, sondern das Ergebnis komplementärer Lücken.[87], [88], [91], [98], [102]

2.5.6 BTC-Konzept

Nach der gewichteten Detailbewertung (siehe Tabelle 2.5) erreichte Szenario A 1,45 von 3,00 Punkten (gerundet 1), Szenario B 1,50 Punkte (gerundet 2) und die Eigenentwicklung aus Kapitel 1.4 2,85 Punkte (gerundet 3). Die Eigenentwicklung wurde nach demselben Schema und derselben Skala bewertet wie die beiden KI-Szenarien. Der folgende Vergleich stellt die drei Implementierungen je Unterkriterium direkt gegenüber und verwendet die ausführlichen Detail-Analysen für A und B der Einzelbewertung. Die Rundung auf ganze Punkte überzeichnet dabei den Abstand der beiden KI-Szenarien. Ungerundet trennen Szenario A und B lediglich 0,05 Punkte, während die gerundete Darstellung einen vollen Punkt suggeriert. Für die Interpretation sind deshalb die ungerundeten Werte aus Tabelle 2.5 maßgeblich.

Auf die Details der Eigenentwicklung wird nur grob eingegangen und sind deswegen nur kurz in der Tabelle 2.9 aufgelistet. Diese Tabelle begründet die gegebene Bewertung und liefert Vergleichspunkte für die nächsten Absätze. Die genannten Aspekte können in der vorherigen Ausarbeitung nachgelesen werden (siehe Abschnitt 1.4).

Architekturschlüssigkeit

Eigenentwicklung: 3, A: 2, B: 2. Die Eigenentwicklung zeichnet sich durch das Entwickeln und Verwerfen verschiedener Ansätze aus und bildet einen durchgehenden finalen Ansatz. Szenario A legt eine ähnlich stimmige Grundstruktur an, ließ aber zentrale Architekturfragen wie Wallet-Auswahl, Seed-Erzeugung und Multi-Signatur-Modell offen. Szenario B im Gegensatz plant konsistent, ersetzte die vorgesehenen RPC-Pfade aber ohne dokumentierten Abgleich durch einen dateibasierten Ablauf. Der Unterschied liegt somit weniger im Zonenmodell als im Umgang mit Entscheidungen. Die Eigenentwicklung löst offene Architekturfragen auf, begründet Abweichungen, statt sie offenzulassen und setzt sie in einer produktiven Lösung um. Die durchgehende, produktive Umsetzung fehlt in A als auch B. Außerdem zeichnen sich die beiden KI-Szenarien durch nur sehr oberflächliche Überlegung aus. Die Eigenentwicklung unterstützt die manuelle Nutzer-Verifikation von PSBTs in der Cold-Wallet durch die automatische Signierung durch Key A,

Unterkriterium	Score	Details der Eigenentwicklung
Architekturschlüssigkeit	3	Drei Vertrauenszonen aus exponiertem Apphost, WG-isolierter Signierer-VM und air-gapped Cold-Zone; explizite BIP-174-Rollenzuordnung; dokumentierte Abwägung verworfener Alternativen wie Online-Koordinator und Hash-Freigabeschicht.
Trennung Schlüsselmaterial / Rollen	3	Apphost nur mit Watch-only-Deskriptoren, Key A TPM-versiegelt hinter WG, Key B und Key C air-gapped; Segregation of Duties über reale, per Regtest-Ableitungspfade erzeugte Schlüssel.
PSBT-, Signier- & Handoff-Prozess	3	Echte BIP-174-PSBTs über <code>walletcreatefundedpsbt</code> ; Strukturprüfung auf <code>witness_utxo</code> und BIP32-Pfade; Signatur über <code>embit</code> aus der TPM-Entsiegelung; 2-aus-3-Multi-Signatur offline; realer Broadcast; Nutzung BTC-Core; vollautomatischer Trigger mit Balance-Evaluation und Handoff-Löschlogik.
Testdesign & praktische Prüfbarkeit	2	Regtest-Umgebung als Validierungsbasis, Simulationsskripte für Spend-, Refill- und Sicherungspfad, gezielter Negativtest der TPM-Bindung, laufende Metrik-Endpunkte der Dienste (<code>metrics.py</code>); keine automatisierte, wiederholbare Testsuite.
Härtung / Angriffsflächenreduktion	3	Container mit <code>cap_drop: [ALL]</code> , <code>read_only</code> , <code>no-new-privileges</code> , non-root-UID sowie <code>mem_limit</code> und <code>pids_limit</code> ; <code>nftables</code> mit Drop auf Ein- und Ausgang und WG-only; HMAC mit Nonce und Deduplizierung; zweistufiger Velocity-Cap; NixOS mit Flake-Pinning.
Recovery / Fehlerfälle / Remediation	3	2-aus-3-Modell als struktureller Recovery-Mechanismus, TPM-Reseal als dokumentierter Betriebsschritt, negativ getestete PCR-Bindung, Rotationsskripte <code>rotate_secrets.sh</code> und <code>delete_seed.sh</code> . Nur der rein analoge Seed-Restore ist designbedingt nicht praktisch durchgespielt.
Gesamt	2,85	gerundet 3 von 3

Tabelle 2.9: Detailbewertung der Eigenentwicklung im BTC-Konzept

was dem Operator direkt in der GUI angezeigt wird. Eine vergleichbare tiefe Überlegung oder Unterstützung des Operators sucht man in den KI-Szenarien vergebens.

Trennung von Schlüsselmaterial und Rollen

Eigenentwicklung: 3, A: 1, B: 1. In der Eigenentwicklung ist die Trennung dreifach technisch erzwungen und ein aufwändiges Modell für die Segregation of Duties aufgebaut worden. Szenario A beschreibt dieselben Rollen lediglich, ohne dass je eine Wallet existierte oder die tatsächlich verwendeten Schutzschichten und Services evaluiert oder festgelegt werden. Szenario B erzwang die Trennung zwar technisch über Benutzer- und Rechtemodell, jedoch nur für Dummy-Daten und mit teilweise fest einkodiertem Nachweis. Die Eigenentwicklung ist damit das einzige der drei Systeme, in dem echtes Schlüsselmaterial über Regtest-Ableitungspfade existiert *und* nachweislich getrennt gehalten wird. Die anderen Entwicklungen evaluierten den Aufbau eines Testsystems und die Bildung solch eines Schlüsselmaterials nicht. Die Trennung der Eigenentwicklung ist nicht nur behauptet oder simuliert, sondern unter realen Bedingungen wirksam und kann mit den vorliegenden Daten reproduziert werden.

PSBT-, Signier- und Handoff-Prozess

Eigenentwicklung: 3, A: 1, B: 1. In dieser Unterkategorie liegt der deutlichste Abstand, insbesondere im Signierer. Die Eigenentwicklung setzt durchgängige Prozesse produktiv um, während Szenario A diese nicht vollständig und relevante Lücken aufweist. BTC Core wird zwar als Node betrieben, jedoch nicht in einem Testmodus, sodass der Node über den Start des Blockchain-Scannings nicht hinauskommt. Ein Refill-Prozess zwischen Treasury und Hot-Wallet wurde im Sessionverlauf zwar evaluiert und konzeptionell beschrieben, eine Umsetzung in Artefakten erfolgte jedoch nicht. Zum Signieren wird keine automatische Lösung vorgeschlagen, sondern auf einen externen, nicht weiter behandelten Signierer verwiesen, den der Prompter eigenständig evaluieren sollte. Szenario B führt eine vollständige Prozess-Choreografie durch, deren Artefakte jedoch aus JSON-Metadaten statt PSBTs bestehen und deren

Signatur nur über eine Markerzeichenkette simuliert wird. Zwar werden einzelne Aspekte evaluiert und etwa BTC Core ausgewählt, umgesetzt wird jedoch keine der Maßnahmen. Die Ansätze verlassen die theoretische Phase nicht und der Prompter wird stattdessen mit statischen Ausgaben getäuscht.

Beide KI-Szenarien scheitern damit am selben Kernkriterium aus entgegengesetzten Richtungen. A liefert den Unterbau ohne Prozess und B den Prozess ohne Kryptographie. Hinzu kommt der Blick auf den Ausgangspunkt der Automatisierung. Szenario B war im Initialprompt ausdrücklich vorgegeben, dass im Zentrum ein Service steht, der regelmäßig und automatisiert BTC-Transaktionen durchführt. Umgesetzt wurde jedoch kein Dienst mit einem Auftragseingang. Stattdessen wird der Dummy-Ablauf ausschließlich manuell über Skripte angestoßen und eine Schnittstelle, über die ein Zahlungspartner eine Transaktion automatisiert auslösen könnte, existiert nicht. Szenario A erhielt diese Automatisierungsanforderung nicht explizit und wird daran nicht gemessen, dort fehlt mangels Wallet jedoch ohnehin jeder Auslösepfad. Die Eigenentwicklung stellt dafür die beiden externen Eingänge BIP21 und PSBT bereit (siehe Tabelle 1.3), deren Anfragen ohne menschliches Eingreifen dedupliziert, gegen die Whitelist geprüft, durch den OPA autorisiert, automatisch signiert und ausgeführt werden.

Für den Mandanten bedeutet dies, dass keines der beiden KI-Szenarien die eigentliche Kernanforderung erfüllt, eine Zahlung tatsächlich als BTC-Transaktion auszuführen. Szenario A bleibt hinter dieser Anforderung zurück, weil der Nutzer den entscheidenden Signierschritt selbst hätte lösen müssen. Szenario B bleibt in der Wirkung sogar noch weiter zurück, da es einen funktionsfähigen Ablauf lediglich vortäuscht und ein Mandant, der es übernehme, ein System besäße, das Erfolgsmeldungen erzeugt, aber keinen einzigen Satoshi bewegen kann. Da dieses Kriterium mit 20 % gewichtet ist und beide Szenarien hier nur einen Punkt erreichen, verfehlen beide den zentralen Nutzen des Auftrags, während allein die Eigenentwicklung den vollständigen Transaktionsweg belastbar bereitstellt.

Testdesign und praktische Prüfbarkeit

Eigenentwicklung: 2, A: 2, B: 1. Die Eigenentwicklung stellt mit der Regtest-Umgebung genau den Validierungsansatz bereit, den die Metrik als Maßstab nennt. Ergänzt wird dies um Simulationsskripte für Spend-, Refill- und Sicherungspfad, jedoch werden tatsächliche Laufzeit-Tests vermisst. Dieser Ansatz eines Testens des BTC-Netzwerks mit einer realitätsnahen Simulation wird in A und B nicht umgesetzt. Während A noch Regtest evaluiert, wird der mögliche Einsatz nicht von Szenario B erkannt. Stattdessen simuliert B einen eigenen, sehr primitiven, statischen Prozess und bleibt damit weit hinter A zurück. Szenario A prüfte seine Node-Schicht mit einem echten Laufzeit-Prüfskript, kam jedoch nicht über das Scanning der Mainnet Blockchain hinaus. Dementsprechend stellt sich A durch einen sehr viel kleineren Umfang, aber dafür mit einem adäquateren Testing dar und erreicht so eine gleiche Wertung. Szenario B baute das umfangreichste Prüf-Framework, entwertete es jedoch durch tautologische Selbstprüfungen und die fehlende Prüfbarkeit der Kernfunktionen, die nicht praktisch testbar existieren. Es entsteht statt einer tatsächlichen Prüfung eher eine Scheininvalidierung. Es zeigt sich ein struktureller Zusammenhang zwischen A und B, wobei Testqualität der Umsetzungstiefe folgt und so mit einem Fortschreiten des allgemeinen Stands weiterentwickelt wird. A stellt sich im Vergleich zu B als sehr viel kleiner, aber dafür ehrlicher heraus.

Ein weiterer Unterschied liegt in der Beobachtbarkeit des BTC-Systems selbst. Szenario A liefert hier den stärksten Nachweis. Ein eigener Metrik-Exporter erfasst fortlaufend den Node-Zustand und Alerts für RPC-Ausfall und Peer-Verlust sind in Prometheus definiert. Die Eigenentwicklung stellt vergleichbare Zustandsdaten über die Metrik-Endpunkte ihrer Dienste bereit. Szenario B verfügt über keinerlei laufende Überwachung, da nach den Validatorläufen keine Dienste mehr aktiv sind.

Härtung, Limitierung und Angriffsflächenreduktion

Eigenentwicklung: 3, A: 2, B: 3. Diese Kategorie ist das einzige Kriterium, in dem Szenario B die volle Punktzahl mit der Eigenentwicklung teilt. Dies beruht jedoch nicht auf einem durchgehend tiefen Sicherheitskonzept, sondern auf gegensätzlichen Gründen. Szenario B minimiert die

Angriffsfläche durch Abwesenheit, da ohne laufende Daemons, ohne offene Listener und mit nach dem Validatorlauf gestoppten Gästen schlicht keine Angriffsfläche entsteht. Die Eigenentwicklung härtet dagegen ein laufendes System auf mehreren Ebenen.

Eine verbleibende Lücke betrifft auch hier die Lieferkette der Eigenentwicklung. Der Container-Build der Hot-Wallet klonet das offizielle BTC-Core-Repository und checkt lediglich das Tag `v31.0` aus, ohne den zugehörigen Commit-Hash zu pinnen oder das Tag kryptographisch zu verifizieren. Das ist dieselbe Fehlerklasse wie die fehlende Signaturprüfung in Szenario A, wenn auch mit kleinerer Angriffsfläche, da direkt aus dem Quell-Repository gebaut wird. Die Wertung bleibt vertretbar, weil die Härtung des laufenden Systems mehrschichtig belegt ist und sich die Lücke durch ein Pinning auf den Commit-Hash unmittelbar schließen lässt. Szenario A härtet seinen Node solide über `systemd-Sandboxing` wie `ProtectSystem=full` und `NoNewPrivileges`, betreibt ihn aber nicht containerisiert und lässt zudem die Signaturprüfung der BTC-Core-Installation aus, also den sicherheitskritischsten Schritt der Lieferkette. Die Mischung aus `nftables` und `iptables` (siehe `nftables.conf` versus `homelab-btc-firewall.sh`) fällt wie bereits erwähnt ebenfalls negativ auf.

Zur Limitierung gehört in der Eigenentwicklung auch das Gebühren-Handling. Der OPA erzwingt je Transaktion feste Ober- und Untergrenzen für Gebühr und Gebührenrate (siehe `hot.rego`). Ein vergleichbares Fee-Handling fehlt in beiden KI-Szenarien vollständig, obwohl fehlerhafte Gebühren in einem automatisierten Zahlungssystem unmittelbar Verluste erzeugen.

Daneben verwendet Szenario A für die RPC-Authentifizierung von BTC-Core das Cookie-Verfahren (siehe `bitcoin.conf`, `rpccookiefile`). Das ist positiv zu bewerten, da damit auf die veralteten statischen Zugangsdaten über `rpcuser` und `rpcpassword` verzichtet wird. Das Cookie wird bei jedem Start neu erzeugt und ist nur lokal mit entsprechenden Dateirechten lesbar. Gegen einen Angreifer, der bereits lokale Leserechte auf dem Node besitzt, schützt das Verfahren jedoch nicht. Auffällig ist zudem der Eintrag `privatebroadcast=0` in der `bitcoin.conf`. Gerade bei einem Tor-only-Node hätte die Aktivierung dieser Option einen zusätzlichen

Privacy-Gewinn beim Broadcast eigener Transaktionen geboten. Codex hat die Option explizit gesetzt, aber weder begründet noch diskutiert. Die Punktgleichheit von B und Eigenentwicklung ist mit Vorsicht zu lesen. Härtung durch Weglassen ist erheblich leichter zu erreichen als Härtung im laufenden Betrieb und zeigt zugleich die Schwäche einer Bewertung an starren, voneinander unabhängigen Metriken.

Konkret bedeutet die volle Punktzahl für Szenario B keinen Sicherheitsgewinn, den der Mandant nutzen könnte. Sobald reale Dienste hinzukämen, entfielen die Schutzwirkung der Abwesenheit vollständig und die Härtung müsste erst noch aufgebaut werden, sodass der Punktwert den tatsächlichen Nutzen deutlich überzeichnet. Szenario A liefert dagegen eine echte, aber lückenhafte Härtung und bleibt genau an der kritischsten Stelle zurück, da eine untergeschobene Node-Binärdatei mangels Signaturprüfung unbemerkt bliebe. Allein bei der Eigenentwicklung entspricht die Wertung einem Schutz, der im laufenden Betrieb für den Mandanten tatsächlich wirksam ist.

Auch der Broadcast-Pfad unterscheidet sich. Szenario A bindet den Node konsequent Tor-only an und schützt damit den Standort des Nodes. Die Eigenentwicklung sendet finale Transaktionen ebenfalls über einen Onion-Pfad. Szenario B besitzt mangels Netzwerkfunktion keinen Broadcast-Pfad.

Recovery, Fehlerfälle und Remediation

Eigenentwicklung: 3, A: 1, B: 2. Die Eigenentwicklung definiert Wiederherstellungspfade für jede Komponente und nutzt das 2-aus-3-Modell als strukturellen Recovery-Mechanismus. Szenario B belegt Snapshots und die reproduzierbare Behebung eines realen Bootfehlers, simuliert aber keinen Fehlerfall innerhalb des Workflows und hält für den Offline-Handoff keinen Backup-Pfad vor. Szenario A sichert lediglich Konfigurationen ohne jeden Restore-Test. Bemerkenswert ist, dass allein die Eigenentwicklung ein Konzept für den Verlust von Schlüsselmaterial besitzt, also den für ein BTC-System kritischsten Fehlerfall.

Dieser Unterschied ist für den Mandanten unmittelbar wertentscheidend. Da Szenario A keinen Restore-Test durchführt, bliebe im Ernstfall offen,

ob eine Sicherung überhaupt wiederherstellbar wäre, womit ein realer Verlust droht. Szenario B bleibt trotz nachgewiesener Snapshots ebenfalls hinter einer belastbaren Wiederherstellung zurück, da der eigentliche Transaktionsworkflow im Fehlerfall ungeprüft bleibt. Erst die Eigenentwicklung trennt damit einen wiederherstellbaren von einem im Verlustfall unrettbaren Aufbau, was bei einem System mit realem Gegenwert den entscheidenden Ausschlag gibt.

Der Abstand von rund einem Punkt zwischen der Eigenentwicklung und beiden KI-Szenarien konzentriert sich damit auf die beiden am stärksten gewichteten Kriterien Schlüsseltrennung und Signierprozess, die zusammen 45 % der Bewertung ausmachen und in denen keines der KI-Szenarien über 2 Punkte hinauskam. Zugleich verliert auch die Eigenentwicklung an derselben Skala messbar Punkte bei der fehlenden Testautomatisierung. Einschränkend gilt, dass die Bewertung der Eigenentwicklung durch deren Architekten erfolgte und damit keine unabhängige Fremdbewertung darstellt (siehe 2.6).[42], [43], [72]

2.5.7 Fehleranalyse

In der Fehleranalyse erhielten beide Szenarien die volle Punktzahl. Die Eigenentwicklung wird hier nicht bewertet, weil die Kategorie die Validierungsfähigkeit des LLM misst und die eigene Ausarbeitung dabei Prüfgegenstand statt Prüfer wäre. Da die Prüfaufträge wortgleich gestellt wurden, wirkt die Kategorie als kontrolliertes Testinstrument, und die Parität ist selbst ein Ergebnis. Die analytische Kompetenz des Modells erwies sich als weitgehend unabhängig von der vorangegangenen Steuerungsqualität, sobald die Aufgabe präzise spezifiziert war.

Innerhalb der vollen Punktzahl arbeitete Szenario B gleichwohl tiefer als Szenario A. Sein Review fand Befunde, die A nicht benannte, etwa die IPv6-Wirkung der `inet`-Tabelle und das direkte Schreiben in finale Archivdateien, und wog für jede Datei Korrekturvarianten systematisch gegeneinander ab. Die Skala bildet diesen Unterschied nicht mehr ab.[96], [106]

2.6 Grenzen der Evaluation

Trotz der vielen ausgewerteten Nachweise hat die Untersuchung mehrere Grenzen:

- **Testumgebung statt Zielumgebung:** Es wurde nicht die eigentliche Home-Lab-Zielumgebung genutzt, sondern zwei lokale Proxmox-Test-VMs. Beweggründe dafür liegen in der Sicherheit, jedoch schränkt dies auch die Übertragbarkeit bestimmter I/O-, Storage- und Betriebsaspekte ein.
- **Kein produktives Material:** Produktive Geheimnisse, Offline-Schlüssel und reale BTC-Werte blieben bewusst außerhalb des Versuchs. Dies kann beispielsweise durch Nutzung der Testnet Ableitungspfade für BTC-Geheimnisse umgangen werden.
- **Keine formale Verifikation:** Die Bewertung stützte sich auf Dateien, Laufzeitbeobachtungen und Prozessdaten.
- **Einzeldurchlauf:** Die Auswertung beruht je Szenario auf einem einzelnen Durchlauf. Da LLM-Ausgaben nicht deterministisch sind, lassen sich daraus keine Aussagen über die Streuung der Ergebnisse bei wiederholten Läufen ableiten.
- **Ein System, ein Modell:** Untersucht wurde nur ein einzelnes agentisches System mit einem Modell (Codex mit `gpt-5.4`). Aussagen über LLMs im Allgemeinen sind daraus nur eingeschränkt ableitbar.
- **Offener Simulationsrahmen:** Die Erwartung einer möglichst vollständigen Simulation war im Prompt bewusst nicht explizit beschrieben. Damit sollte sichtbar werden, ob das LLM solche Validierungsansätze eigenständig vorschlägt. Keines der beiden Szenarien hat die naheliegende Regtest-Option von sich aus evaluiert.
- **Doppelrolle der Bewerter:** Die Bewerter waren zugleich die Architekten der Referenzlösung. Insbesondere der Vergleich mit der Eigenentwicklung ist dadurch keine unabhängige Fremdbewertung. Der Grad der Objektivität wurde versucht durch eine Bewertung durch mehrere Personen zu erhöhen.

- **Asymmetrische Beweislage:** Szenario B musste bereits per Prompt genau die Dokumente erzeugen, die später als Bewertungsnachweise dienten. Dafür erhielt dieser Versuch sowohl mehr Nutzer-Nachrichten, längere Laufzeit als auch ein größeres Steuerungsbudget. Der Punktabstand misst daher die Kombination aus fachlicher Promptqualität und Steuerungsumfang, nicht die Qualität allein. [84]

Diese Einschränkungen sprechen jedoch nicht zugunsten des LLM. In einer echten Produktivumgebung mit realen Werten wären Expertensteuerung, manuelle Freigaben und Gegenprüfungen noch wichtiger.

2.7 Takeaway für den Mandanten

LLM und insbesondere agentische Systeme wie Codex können heute bereits einen erheblichen Teil der Konzeptions-, Umsetzungs-, Dokumentations- und Validierungsarbeit für ein sicheres Home-Lab übernehmen. Sie sind in der Lage, praktische Infrastruktur aufzubauen, Dateien zu korrigieren, Audits zu formulieren und unter enger Steuerung auch sicherheitsnahe Designs nachvollziehbar umzusetzen. [107], [108]

Sie ersetzen den menschlichen Sicherheitsberater jedoch nicht vollständig. Der Hauptunterschied zwischen Szenario A und Szenario B lag in der Qualität der menschlichen Steuerung. Ohne fachlich starke Vorgaben tendierte der Agent zu einer funktional beeindruckenden, aber sicherheitstechnisch unschärferen Lösung. Mit enger Steuerung erreichte er ein deutlich belastbareres Ergebnis, benötigte aber weiterhin menschliche Priorisierung, Freigabe und klar gesetzte Grenzen.

Wenn der Mandant eine solche Infrastruktur ausschließlich durch LLM-basierte Beratung ohne fachkundige Gegenprüfung hätte planen und konfigurieren lassen, wäre das Resultat wahrscheinlich auf den ersten Blick beeindruckend gewesen. Auf den zweiten Blick wäre es in kritischen Bereichen aber nicht belastbar genug gewesen. Dies betrifft besonders die Infrastruktur des Systems, etwa effektive Netztrennung, Backups, Alerting und den BTC-Transaktionsbereich.

Die Rolle des Sicherheitsberaters verschwindet also nicht. Sie verschiebt

sich eher: weg von der vollständigen manuellen Ausarbeitung, hin zu Risikosteuerung, Priorisierung, Freigabe und kritischer Endverantwortung.

2.8 Lessons Learned

Aus der Evaluation ergeben sich mehrere Erkenntnisse:

Szenario A erfüllte seinen Zweck grundsätzlich gut. Der Nutzer wurde bewusst naiv und alltagsnah simuliert, um zu beobachten, welche Architekturentscheidungen Codex ohne enge fachliche Steuerung selbst priorisiert. Dadurch wurde sichtbar, welche Bestandteile das System eigenständig als wichtig einordnet, welche Sicherheitsmaßnahmen es von sich aus umsetzt und an welchen Stellen erst spätere Nachfragen oder Audits zu weiteren Systemhärtungen führen.

Bei Szenario B bestätigte sich der Wert des offenen Simulationsrahmens als Testinstrument. Codex hat die naheliegende Regtest-Option von sich aus nicht evaluiert, obwohl eine BTC-Core-Regtest-Umgebung, Test-Wallets, eine nachvollziehbare `bitcoin.conf` sowie simulierte Spend- und Recovery-Abläufe innerhalb des erlaubten Rahmens möglich gewesen wären. Für einen produktiven Einsatz sollte diese Erwartung dagegen explizit in den Prompt aufgenommen werden, da die Eigenleistung des Modells an dieser Stelle nicht ausreicht.

Der direkte Vergleich von Szenario A und Szenario B verdeutlicht eine zentrale Herausforderung beim Einsatz von KI für technische Infrastrukturaufgaben. Die Qualität des Ergebnisses hängt nicht ausschließlich von den Fähigkeiten des Modells ab, sondern in hohem Maße davon, welche Ziele und Randbedingungen durch den Nutzer vorgegeben werden. In Szenario A erhielt die KI vergleichsweise wenige Vorgaben. Dadurch konzentrierte sie sich stärker auf die unmittelbare Aufgabenstellung und erreichte innerhalb kurzer Zeit einen hohen praktischen Umsetzungsgrad. Im BTC-Bereich wurden ein BTC-Core-Node, Monitoring-Komponenten sowie die konzeptionellen Grundlagen für Watch-only-Wallets, PSBTs und einen späteren Signierprozess geschaffen. Obwohl zentrale Sicherheitsaspekte und Betriebsprozesse unvollständig blieben, entstand ein System, das sich deutlich an der eigentlichen fachlichen Anforderung

orientierte. Szenario B verfolgte einen anderen Ansatz. Hier standen Sicherheitsanforderungen, Rollenmodelle, Validierungsschritte und die Vermeidung produktiver Geheimnisse bereits von Beginn an im Vordergrund. Das resultierende System war organisatorisch und sicherheitstechnisch deutlich strukturierter. Gleichzeitig verlagerte sich der Schwerpunkt der Arbeit zunehmend auf die Einhaltung dieser Rahmenbedingungen. Im BTC-Bereich führte dies dazu, dass Rollen, Datenflüsse und Sicherheitsgrenzen sehr detailliert ausgearbeitet wurden, während wesentliche Kernfunktionen des eigentlichen Transaktionsprozesses überwiegend simuliert blieben. Die Ergebnisse deuten darauf hin, dass starke Steuerung nicht automatisch zu einer besseren Erfüllung der ursprünglichen Fachanforderung führt. Stattdessen optimiert das Modell auf diejenigen Ziele, die im Prompt und in den nachfolgenden Vorgaben besonders stark gewichtet werden. Werden Sicherheit, Nachweisbarkeit und Compliance zum dominierenden Ziel erklärt, kann dies dazu führen, dass die KI erhebliche Ressourcen auf Dokumentation, Validierung und Regelkonformität verwendet, während die praktische Umsetzung der eigentlichen Kernfunktion in den Hintergrund tritt. Die Evaluation zeigt damit einen Zielkonflikt zwischen funktionaler Umsetzung und regulatorischer bzw. sicherheitstechnischer Absicherung. Beide Aspekte sind notwendig, optimale Ergebnisse entstehen jedoch erst dann, wenn der Nutzer die Balance zwischen beiden Zielsetzungen aktiv steuert.

Zu Beginn der Untersuchung wurde geprüft, ob Codex direkt auf der Serverumgebung des eigenen Home-Labs arbeiten könnte. Aus Sicherheitsgründen wurden die Zugriffsrechte dabei bewusst eingeschränkt, um unbeabsichtigte Änderungen an globalen Einstellungen, Zugriffspfaden oder anderen Bestandteilen zu verhindern. Dadurch war Codex zunächst gezwungen, über die Proxmox-API zu arbeiten. Dieser Ansatz war jedoch aufwendig, langsam und der Tokenverbrauch zu hoch.

Deshalb wurde der Versuchsaufbau auf zwei getrennte lokale Proxmox-VMs umgestellt. Codex konnte dadurch praktischer arbeiten, ohne das eigentliche Home-Lab zu gefährden. Rückblickend hätte diese Entscheidung jedoch bereits früher getroffen werden können. Für zukünftige Untersuchungen dieser Art sollte die isolierte Testumgebung daher von Beginn an eingeplant werden, damit der Agent ausreichend Handlungsspielraum erhält, ohne die produktiven Systeme zu gefährden.[77]

3 Teamorganisation und Projekterfahrungen

3.1 Teamorganisation

Unsere Zusammenarbeit im Team war eine Mischung aus klarer Aufgabenverteilung und engem Austausch. Für schnelle Rückfragen, spontane Abstimmungen und das Klären von Problemen haben wir vor allem Discord genutzt. Dort haben wir kurze Statusupdates ausgetauscht, offene Punkte gesammelt und bei Bedarf direkt in Calls besprochen. Die eigentliche Projektdokumentation haben wir parallel in Overleaf geschrieben, sodass Abschnitte direkt gemeinsam bearbeitet, kommentiert und zusammengeführt werden konnten. Google Sheets diente uns als Aufgabenübersicht mit Zuständigkeiten, offenen Punkten und Zwischenständen, während wir in Google Docs eher lose Notizen für die 15 Services gesammelt haben. Für Code, Konfigurationsstände und nachvollziehbare Änderungen haben wir GitHub genutzt.

Trotz der fachlichen Aufteilung haben wir das Projekt nicht als drei voneinander getrennte Teilaufgaben bearbeitet. Gemeinsame Basisdienste wie PostgreSQL, NATS, WireGuard sowie Logging und Monitoring wurden bewusst als geteilte Infrastruktur verstanden. Das hat Doppelarbeit vermieden, bedeutete aber auch, dass wir Schnittstellen sauber abstimmen mussten. Gerade an diesen Übergängen hat sich gezeigt, wie wichtig regelmäßige Kommunikation für ein konsistentes Gesamtsystem war.

Zusätzlich erschwerten die Rahmenbedingungen des Projekts die Arbeit. Durch die parallele Belastung aus Beruf und Studium sowie die unterschiedlichen Erfahrungsstände im Team dauerten Abstimmung, Tests und belastbare Entscheidungen oft länger als geplant.

3.2 Aufgabenverteilung

Wir haben die Arbeit entlang klarer Schwerpunkte verteilt. Luca übernahm die Infrastruktur- und Self-Hosted-Themen. Dazu gehörten das Serverhosting, der Aufbau des Self-Hosted-Setups und viele der grundlegenden Architekturentscheidungen auf Plattformebene.

Florian verantwortete den Bitcoin-Bereich. Sein Schwerpunkt lag auf der Konzeption der Hot- und Cold-Wallet-Architektur, dem Transaktionssystem sowie den sicherheitsrelevanten Abläufen rund um Signierung, Schlüsselverwaltung und Automatisierung.

Marie übernahm die KI-Evaluation und große Teile der Dokumentationskoordination. Dazu gehörten die Bewertung von KI-Unterstützung, die Pflege der Dokumentation und die Ausarbeitung allgemeiner Kapitel.

Wichtig war für uns dabei, dass die Zuständigkeiten zwar klar, aber nicht starr waren. Wenn aus einer Infrastrukturentscheidung Folgen für die Bitcoin-Komponente entstanden oder Dokumentation und Implementierung auseinanderliefen, haben wir das gemeinsam besprochen.

3.3 Herausforderungen

3.3.1 Self-Hosted-Setup

Eine der ersten größeren Herausforderungen für uns im Infrastrukturtteil war die Frage, welche Architektur für das Projekt wirklich sinnvoll ist. Anfangs war Kubernetes geplant, im Verlauf der Arbeit hat sich Docker Compose für unseren Rahmen aber als die stabilere und besser beherrschbare Lösung herausgestellt. Der Umstieg fiel uns nicht leicht, weil wir den Kubernetes-Stack bereits weit aufgebaut hatten. Mitverantwortlich für die späte Kurskorrektur war ein über Jahre gewachsener architektonischer Bias. Durch die langjährige Arbeit mit Kubernetes-Clustern waren uns dessen Best Practices und fertige Lösungsbausteine so vertraut, dass wir zunächst jede Anforderung als Kubernetes-Anwendungsfall betrachtet haben. Erst spät wurde deutlich, dass dieser Ansatz für einen einzelnen Mandanten keinen messbaren Vorteil bringt und vor allem unnötige Komplexität erzeugt. Eines der Teammitglieder

betreibt beispielsweise seit Jahren ein Kubernetes-basiertes Homelab (siehe <https://github.com/LucaDev/Home/>).

Dazu kamen Plattformthemen rund um NixOS und Isolationsmechanismen. Ein NixOS-Upgrade auf Version 26.05 kurz vor der Abgabe brachte einige Breaking Changes mit sich und erzeugte zusätzlichen Anpassungsdruck. Wir hätten auf der älteren Version bleiben können, wollten dem Mandanten aber kein von Anfang an veraltetes System ausliefern. Technologien wie Kata Containers oder *gVisor* liefen zudem nicht in allen Konstellationen so stabil, wie wir es uns erhofft hatten, insbesondere wenn die Proxmox-Umgebung keine stabile Nested Virtualization bereitstellte. Beide Sandbox-Runtimes haben wir deshalb vorbereitet, aber nicht als Standard aktiviert.

Zwei technische Probleme haben uns dabei besonders aufgehalten. Nach dem Sprung auf NixOS 26.05 startete `docker.service` zunächst gar nicht mehr, weil Kata Containers 3.x nur noch über den containerd-v2-Shim angesprochen wird und die dafür nötige containerd-Snapshotter-Integration mit dem von uns genutzten `usersns-remap` unvereinbar ist. Da uns das UID/GID-Remapping wichtiger war, haben wir Kata aus der Docker-Konfiguration entfernt und den Snapshotter deaktiviert. Ebenso scheiterte bei gleichzeitig aktivem AppArmor und Audit-Daemon der Dienst `audit-rules-nixos.service` bei jedem Rebuild an einer fehlerhaften Ladereihenfolge der Linux-Security-Module. Der passende Upstream-Fix stand erst wenige Tage vor der Abgabe zur Verfügung.

Erschwerend kam der hohe Testaufwand hinzu. Eine vollständige Neuinstallation durchläuft VM-Reset, LiveCD-Boot, Repository-Klon, NixOS-Installation, Secure-Boot-Konfiguration und Container-Start, sodass sich Änderungen am Installationsprozess nur langsam erproben ließen. Auch kleinere Eigenheiten kosteten Zeit, etwa dass Traefik Umgebungsvariablen aus der `.env` in seiner statischen Konfiguration nicht auflöst und wir die Datei deshalb selbst templaten mussten.

3.3.2 Bitcoin-Setup

Im Bitcoin-Teil haben wir besonders gemerkt, wie schwierig sicherheitskritische Eigenentwicklungen ohne belastbare Referenzen sind. Mehrere

Architekturansätze haben wir zunächst aufgebaut und später wieder verworfen, weil sie zwar interessant wirkten, den Gesamtaufbau aber unnötig verkomplizierten. Zusätzlich hat uns die lückenhafte Dokumentation der Bibliothek **embit** gebremst, vor allem bei der Wallet-Verwaltung und beim automatisierten Signieren.

Aufwendig war auch der Versuch, dem Mandanten möglichst wenig manuelle Arbeit zu überlassen. Dadurch entstand viel zusätzliche Logik in Skripten und Abläufen. Gleichzeitig setzte uns die Laborumgebung Grenzen: Der Austausch über QR-Codes ließ sich in der Proxmox-basierten Umgebung nicht realistisch nachstellen, sodass wir auf andere Wege ausweichen mussten. Hinzu kamen Probleme mit Sparrow unter dem stabilen NixOS-Zweig sowie der generell hohe Rebuild- und Testaufwand, den der deklarative NixOS-Ansatz mit sich bringt.

Besonders deutlich wurde für uns der operative Teil von Sicherheit beim Zugriff auf das TPM aus einem unprivilegierten Container. Hier ging es nicht nur darum, dass etwas grundsätzlich funktioniert, sondern dass minimale Rechte, reproduzierbare Konfiguration und realer Hardwarezugriff gleichzeitig zusammenpassen. Gerade an solchen Stellen wurde sichtbar, dass Sicherheit nicht nur aus Kryptographie besteht, sondern auch aus sauberen Betriebsabläufen.

3.3.3 KI-Evaluation

Im Bereich KI war eine der ersten Herausforderungen die große Auswahl an verfügbaren Modellen und Werkzeugen. Gerade am Anfang war es nicht leicht einzuschätzen, welches Modell für welche Aufgabe am besten geeignet ist. Deshalb wurde pragmatisch entschieden, zunächst mit den Modellen zu arbeiten, die im Alltag ohnehin schon genutzt wurden und deren Verhalten bereits etwas bekannt war.

Eine weitere Herausforderung war, dass die Idee, Codex gezielt in die Evaluation einzubeziehen, erst relativ spät entstand. Dadurch mussten zusätzliche Testläufe geplant und eingerichtet werden. Das war deutlich aufwendiger als eine reine Chat-Auswertung, weil Codex nicht nur Antworten schreiben, sondern tatsächlich in einer lokalen Testumgebung arbeiten sollte und diverse Dateien erstellt hat, die ebenfalls analysiert

werden mussten. Die Läufe dauerten entsprechend lange und konnten nicht einfach nebenbei durchgeführt werden.

Schwierig war außerdem die Umsetzung von Szenario B. Dort sollte ein fachkundiger Nutzer simuliert werden. Dafür wurde Wissen aus den beiden anderen Projektteilen benötigt. Das ließ sich nur eingeschränkt vollständig nachbilden, weil die Person, die den KI-Teil bearbeitet hat, nicht in gleicher Tiefe an beiden anderen Teilen beteiligt war. Gleichzeitig ließ sich die Arbeit auch nicht einfach aufteilen, weil Codex auf einer lokalen VM gearbeitet hat und viele Entscheidungen direkt während des laufenden Tests getroffen werden mussten.

Auch die spätere Auswertung war dadurch nicht ganz trivial und musste durch die beiden anderen Gruppenmitglieder stark unterstützt werden. Für den Bitcoin-Teil brauchte man Detailwissen zu Wallets, PSBT, Watch-only-Konzepten, Regtest und Schlüsseltrennung. Für die Infrastruktur waren dagegen Kenntnisse in den Bereichen Netzwerktrennung, Tor-Zugriffen, Backups, Monitoring und Härtung wichtig. Entweder hätten sich die anderen Gruppenmitglieder zusätzlich in den KI-Teil einarbeiten müssen, oder die Person im KI-Teil hätte sich sehr tief in beide Fachbereiche einarbeiten müssen. Beides wäre wegen des Umfangs und des Zeitaufwands nur schwer realistisch gewesen.

Das erklärt auch, warum Szenario B nicht in allen Bereichen ideal vorbereitet war. Der Anspruch war zwar, einen Expertennutzer nachzuahmen, aber dieses Expertenwissen musste aus mehreren Projektteilen zusammengeführt werden. Gerade im Bitcoin-Bereich hätten einige Anforderungen von Anfang an noch genauer formuliert werden können.

3.4 Learnings

Ein wichtiges Learning für uns war, dass nicht jede technisch mögliche Erweiterung am Ende einen echten Mehrwert bringt. Mehrere Ansätze waren auf dem Papier interessant, hätten das System in der Praxis aber unnötig komplexer und angreifbarer gemacht.

Für den Infrastrukturteil haben wir vor allem gelernt, wie wichtig frühe und klare Entscheidungskriterien sind. Sie helfen dabei, Tool-Bias zu

vermeiden und aus vielen Optionen schneller zu einer tragfähigen Lösung zu kommen. Im Rückblick hätten uns einige frühere Festlegungen Zeit und Umwege erspart.

Im Bitcoin-Bereich war die wichtigste Erkenntnis für uns, dass der Eigenbau einer vollständigen Custody-Lösung erheblich komplexer ist, als es am Anfang wirkt. Bewährte Komponenten wie Bitcoin Core, Sparrow und Hardware-Wallets reduzieren nicht nur den Implementierungsaufwand, sondern können die Sicherheit auch erhöhen, weil sie etablierte und nachvollziehbare Funktionen mitbringen. Für uns war das ein klarer Hinweis darauf, dass Sicherheit nicht automatisch durch möglichst viel Eigenentwicklung entsteht.

Auch organisatorisch haben wir mitgenommen, dass klare Zuständigkeiten und gemeinsame Werkzeuge gut zusammenpassen. Gerade weil technische Umsetzung und Dokumentation parallel liefen, hat uns diese Kombination geholfen, als Team konsistent zu arbeiten.

3.5 Feedback und Anmerkung zur Gewichtung

Aus unserer Sicht war der Gesamtumfang der Aufgabenstellung deutlich zu hoch. Besonders in Kombination mit Aufgabenteil 1, der für sich genommen bereits auf mehrere Wochen ausgelegt war, entstand insgesamt eine sehr große Arbeitsbelastung. Gegen Ende des Moduls war bei allen drei Gruppenmitgliedern spürbar die Luft raus. Die Menge an Aufgaben, die technische Tiefe und der Zeitdruck führten dazu, dass wir teilweise überfordert waren und das Modul insgesamt stark belastend wurde.

Beide Aufgabenteile waren inhaltlich sehr interessant. Gerade deshalb wäre es aus unserer Sicht sinnvoll, den Umfang im nächsten Semester stärker zu fokussieren. Unser Vorschlag wäre, Aufgabenteil 1 zu streichen und stattdessen das gesamte Semester für den zweiten, praktischen Teil zu nutzen. Dadurch bliebe mehr Zeit, sich gründlich in die Themen einzuarbeiten, Dinge praktisch auszuprobieren und die Ergebnisse sauberer zu dokumentieren.

Zusätzlich wäre es aus unserer Sicht hilfreich, den zweiten Aufgabenteil selbst etwas zu reduzieren. Statt drei großer Teilbereiche könnte ein

Bereich gestrichen oder der erwartete Umfang deutlich begrenzt werden. Dann hätten alle Gruppenmitglieder mehr Zeit gehabt, sich nicht nur mit dem eigenen Abschnitt, sondern auch mit den anderen Themen zu beschäftigen. So hätte jede Person aus mehreren Bereichen etwas lernen und praktische Erfahrungen sammeln können.

Durch den hohen Zeitdruck mussten wir die Arbeit am Ende stark aufteilen: Jede Person hat vor allem ihren eigenen Bereich fertiggestellt, während die anderen dort kaum praktisch mitarbeiten konnten. Das war schade, weil alle Bereiche fachlich spannend waren. Mit weniger Umfang wäre aus unserer Sicht nicht nur die Belastung geringer gewesen, sondern auch der Lerneffekt für alle Gruppenmitglieder deutlich größer.

Vor diesem Hintergrund halten wir eine zu starke Gewichtung einzelner Teilbereiche innerhalb des Projekts nur für bedingt sinnvoll. Grundsätzlich würden wir die drei Hauptbereiche gleich gewichten, da alle Teile für das Gesamtsystem notwendig waren und stark voneinander abhingen. Infrastruktur, Bitcoin-Komponente und KI-Evaluation erfüllten jeweils unterschiedliche Funktionen und lassen sich deshalb nur schwer direkt miteinander vergleichen.

Wenn überhaupt eine unterschiedliche Gewichtung vorgenommen wird, müsste aus unserer Sicht eher der Bitcoin-Teil höher gewichtet werden. Dort war der Anteil an eigener Konzeption, Erklärung und sicherheitskritischer Abwägung besonders hoch. Anders als bei vielen Infrastrukturthemen existieren für den konkreten Aufbau einer solchen mandantenbezogenen Bitcoin- und Transaktionsarchitektur nur begrenzt direkt übertragbare Referenzen. Viele Entscheidungen mussten daher selbst hergeleitet, begründet und auf ihre praktischen sowie sicherheitstechnischen Folgen geprüft werden.

Gerade dieser Teil erforderte besonders viel eigenständige Arbeit, auch wenn sich das nicht immer unmittelbar in sichtbaren Konfigurationsdateien oder im Seitenumfang widerspiegelt. Deshalb sollte bei der Bewertung nicht nur betrachtet werden, wie viel technisch umgesetzt oder dokumentiert wurde, sondern auch, wie viel konzeptionelle und sicherheitskritische Eigenleistung in den jeweiligen Teilbereichen steckt.

Literaturverzeichnis

- [1] F. Richter, *Cloud Computing Market Size Worldwide 2025*, Letzter Zugriff: 24.04.2026, Statista, Feb. 2025. Adresse: <https://www.statista.com/chart/18819/worldwide-market-share-of-leading-cloud-infrastructure-service-providers/>.
- [2] Eurostat, *53% EU enterprises used paid cloud services in 2025*, Letzter Zugriff: 08.07.2026, Feb. 2026. Adresse: <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20260203-1>.
- [3] European Data Protection Supervisor, *Opinion 3/2018 on Online Manipulation and Personal Data*, Letzter Zugriff: 08.07.2026, März 2018. Adresse: https://www.edps.europa.eu/sites/default/files/publication/18-03-19_online_manipulation_en.pdf.
- [4] European Commission. Joint Research Centre, *Technology and democracy: understanding the influence of online technologies on political behaviour and decision making*. Luxembourg: Publications Office of the European Union, 2020. DOI: [10.2760/709177](https://doi.org/10.2760/709177).
- [5] European Union, *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Letzter Zugriff: 08.07.2026, Apr. 2016. Adresse: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>.
- [6] Office of the Law Revision Counsel, U.S. House of Representatives, *18 U.S.C. 2713: Required preservation and disclosure of communications and records*, Letzter Zugriff: 08.07.2026. Adresse: <https://uscode.house.gov/view.xhtml?edition=prelim&num=0&req=granuleid\%3AUSC-prelim-title18-section2713>.
- [7] European Commission, *Proposal for a Regulation of the European Parliament and of the Council laying down rules to prevent and combat child sexual abuse*, COM(2022) 209 final; Letzter Zugriff: 08.07.2026, Mai 2022. Adresse: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52022PC0209>.

- [8] European Union, *Regulation (EU) 2024/1307 amending Regulation (EU) 2021/1232 as regards the use of technologies for combating online child sexual abuse*, Letzter Zugriff: 08.07.2026, Apr. 2024. Adresse: <https://eur-lex.europa.eu/eli/reg/2024/1307/oj/eng>.
- [9] Die Bundesregierung, *Im Kabinett beschlossen: IP-Adressspeicherung*, Letzter Zugriff: 08.07.2026, Apr. 2026. Adresse: <https://www.bundesregierung.de/breg-de/aktuelles/kabinett-ip-adressen-2422604>.
- [10] S. Landau, „Categorizing Uses of Communications Metadata: Systematizing Knowledge and Presenting a Path for Privacy,“ in *New Security Paradigms Workshop 2020*, 2020, S. 1–19. DOI: [10.1145/3442167.3442171](https://doi.org/10.1145/3442167.3442171).
- [11] H. Gao, Y. Wang, J. Shao, H. Shen und X. Cheng, „User Identity Linkage across Social Networks with the Enhancement of Knowledge Graph and Time Decay Function,“ *Entropy*, Jg. 24, Nr. 11, S. 1603, 2022. DOI: [10.3390/e24111603](https://doi.org/10.3390/e24111603).
- [12] Verizon, „2025 Data Breach Investigations Report,“ Verizon, Techn. Ber., 2025, Letzter Zugriff: 08.07.2026. Adresse: <https://www.verizon.com/business/resources/Tea/reports/2025-dbir-data-breach-investigations-report.pdf>.
- [13] Chainalysis Team, *2025 Crypto Theft Reaches \$3.4 Billion*, Letzter Zugriff: 08.07.2026, Dez. 2025. Adresse: <https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2026/>.
- [14] A. Biryukov, D. Dinu und D. Khovratovich, „Argon2: the memory-hard function for password hashing and other applications,“ University of Luxembourg, Techn. Ber., März 2017, Version 1.3, PHC release. besucht am 25. Apr. 2026. Adresse: <https://www.cryptolux.org/index.php/Argon2>.
- [15] Cloudron. „Cloudron Docs,“ besucht am 25. Apr. 2026. Adresse: <https://docs.cloudron.io/packages/vaultwarden/>.
- [16] Dani-Garcia. „vaultwarden,“ besucht am 25. Apr. 2026. Adresse: <https://github.com/dani-garcia/vaultwarden>.
- [17] Dani-Garcia, *Release 1.35.0 · Dani-Garcia/Vaultwarden*, 2025. besucht am 25. Apr. 2026. Adresse: <https://github.com/dani-garcia/vaultwarden/releases/tag/1.35.0>.
- [18] D. Hussain, *Secrets management: Why vaultwarden bridges the gap between Dev and ops*, 2026. besucht am 26. Apr. 2026. Adresse: <https://ayedo.de/en/posts/secrets-management-warum-vaultwarden-die-brucke-zwischen-dev-und-ops-schlagt/>.

- [19] National Institute of Standards and Technology (NIST). „CVE-2024-55225,“ besucht am 26. Apr. 2026. Adresse: <https://nvd.nist.gov/vuln/detail/CVE-2024-55225>.
- [20] National Institute of Standards and Technology (NIST). „CVE-2025-24364,“ besucht am 26. Apr. 2026. Adresse: <https://nvd.nist.gov/vuln/detail/CVE-2025-24364>.
- [21] National Institute of Standards and Technology (NIST). „CVE-2025-24365,“ besucht am 26. Apr. 2026. Adresse: <https://nvd.nist.gov/vuln/detail/CVE-2025-24365>.
- [22] KeyCloak. „KeyCloak,“ besucht am 25. Apr. 2026. Adresse: <https://www.keycloak.org/>.
- [23] Cybersecurity and Infrastructure Security Agency (CISA). „Malware Discovered in Popular NPM Package, ua-parser-js,“ besucht am 13. Juli 2026. Adresse: <https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js>.
- [24] Knowing Bitcoin. „Bitcoin Security: Cold vs Hot Wallet Setup,“ besucht am 2. Mai 2026. Adresse: <https://knowingbitcoin.com/bitcoin-cold-vs-hot-wallet-security/>.
- [25] Nadcab Labs. „Custodial Wallet Architecture Explained,“ besucht am 2. Mai 2026. Adresse: <https://www.nadcab.com/blog/custodial-wallet-architecture-explained>.
- [26] Chris Ekai. „Hot Wallet vs Cold Wallet Architecture,“ besucht am 2. Mai 2026. Adresse: <https://riskpublishing.com/hot-wallet-vs-cold-wallet-what-risk-managers/>.
- [27] Bitcoin Improvement Proposals. „BIP-174: Partially Signed Bitcoin Transactions,“ besucht am 2. Mai 2026. Adresse: <https://github.com/bitcoin/bips/blob/master/bip-0174.mediawiki>.
- [28] Knowing Bitcoin. „Sparrow Wallet Multisig Tutorial,“ besucht am 2. Mai 2026. Adresse: <https://knowingbitcoin.com/sparrow-wallet-multisig-tutorial/>.
- [29] G. Walker. „Partially Signed Bitcoin Transaction (PSBT),“ learnmeabitcoin. Adresse: <https://learnmeabitcoin.com/technical/transaction/psbt/>.

- [30] D-Central Technologies. „Partially Signed Bitcoin Transactions (PSBT): The Cypherpunk Standard for Collaborative Bitcoin Security,“ besucht am 2. Mai 2026. Adresse: <https://d-central.tech/everything-you-need-to-know-about-partially-signed-bitcoin-transactions/>.
- [31] S. Rose, O. Borchert, S. Mitchell und S. Connelly. „Zero Trust Architecture.“
- [32] Bitcoindeveloper. „Wallets,“ besucht am 4. Juni 2026. Adresse: <https://developer.bitcoin.org/devguide/wallets.html>.
- [33] Roy Gaurav. „Hot Wallet vs Cold Crypto Wallet: What’s The Difference?“ Besucht am 4. Juni 2026. Adresse: <https://www.ledger.com/academy/topics/security/hot-wallet-vs-cold-crypto-wallet-whats-the-difference>.
- [34] koolean. „Security warning when installing NixOS 23.11,“ besucht am 30. Juni 2026. Adresse: <https://discourse.nixos.org/t/security-warning-when-installing-nixos-23-11/37636>.
- [35] Joint Task Force, „Security and Privacy Controls for Information Systems and Organizations,“ National Institute of Standards und Technology, Techn. Ber. NIST SP 800-53, Rev. 5, 2020. DOI: [10.6028/NIST.SP.800-53r5](https://doi.org/10.6028/NIST.SP.800-53r5). besucht am 30. Juni 2026.
- [36] Bitcoin Core Developers. „Bitcoin Core Documentation,“ besucht am 4. Juni 2026. Adresse: <https://bitcoincore.org/en/doc/31.0.0/>.
- [37] Sparrow Wallets. „Connect to Bitcoin Core,“ besucht am 4. Juni 2026. Adresse: <https://www.sparrowwallet.com/docs/connect-node.html>.
- [38] Bitcoin Core, *importdescriptors (31.0.0 RPC)*, Letzter Zugriff: 08.07.2026. Adresse: <https://bitcoincore.org/en/doc/31.0.0/rpc/wallet/importdescriptors/>.
- [39] Bitcoin Core Developers. „walletcreatefundedpsbt – RPC API Reference,“ besucht am 30. Juni 2026. Adresse: <https://developer.bitcoin.org/reference/rpc/walletcreatefundedpsbt.html>.
- [40] NATS Docs. „Authenticating with a User and Password,“ besucht am 2. Mai 2026. Adresse: https://docs.nats.io/running-a-nats-service/configuration/securing_nats/authorization.
- [41] J. A. Donenfeld, „WireGuard: Next Generation Kernel Network Tunnel,“ in *Network and Distributed System Security Symposium (NDSS)*, 2017. besucht am 30. Juni 2026. Adresse: <https://www.wireguard.com/papers/wireguard.pdf>.

- [42] Florian Dobke. „NixOSHot Konfigurations-Dateien und readme,“ besucht am 30. Juni 2026. Adresse: <https://github.com/LucaDev/LFM-Team-Blue/tree/main/Hot-KeyHolder>.
- [43] Florian Dobke, Luca Leon Bäcker. „Krypto Basis-system Konfigurations-Dateien und readme,“ besucht am 30. Juni 2026. Adresse: <https://github.com/LucaDev/LFM-Team-Blue/tree/main/apphost/services/hotwallet>.
- [44] Sparrow Wallets. „Sparrow Wallets Features,“ besucht am 4. Juni 2026. Adresse: <https://www.sparrowwallet.com/features/>.
- [45] Bitcoin Improvement Proposals. „BIP-21: URI Scheme,“ besucht am 2. Mai 2026. Adresse: <https://bips.dev/21/>.
- [46] D. A. Harding und P. Todd. „BIP-125: Opt-in Full Replace-by-Fee Signaling,“ besucht am 30. Juni 2026. Adresse: <https://github.com/bitcoin/bips/blob/master/bip-0125.mediawiki>.
- [47] Open Policy Agent, *Open Policy Agent Documentation*, 2025. besucht am 2. Mai 2026. Adresse: <https://www.openpolicyagent.org/docs>.
- [48] InfraCloud Technologies. „Microservices Authorization with Open Policy Agent,“ besucht am 2. Mai 2026. Adresse: <https://www.infracloud.io/blogs/opa-microservices-authorization-simplified/>.
- [49] Malindu Dilshan. „Securing Your APIs with HMAC Signature Validation,“ besucht am 2. Mai 2026. Adresse: <https://medium.com/@malindudilshan389/securing-your-apis-with-hmac-signature-validation-54af44d31197>.
- [50] H. Krawczyk, M. Bellare und R. Canetti, „HMAC: Keyed-Hashing for Message Authentication,“ Internet Engineering Task Force (IETF), Techn. Ber. RFC 2104, 1997. DOI: [10.17487/RFC2104](https://doi.org/10.17487/RFC2104). besucht am 30. Juni 2026.
- [51] Kaleido. „vault-plugin-secrets-ethsign.“ Community-Plugin für secp256k1, GitHub – kaleido-io, besucht am 29. Juni 2026. Adresse: <https://github.com/kaleido-io/vault-plugin-secrets-ethsign>.
- [52] HashiCorp Vault Community. „Support for secp256k1 (Issue #3846).“ Feature-Request zur secp256k1-Unterstützung für Kryptowährungs-Wallet-Backends, GitHub – hashicorp/vault, besucht am 29. Juni 2026. Adresse: <https://github.com/hashicorp/vault/issues/3846>.

- [53] Certicom Research, „SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0,“ Standards for Efficient Cryptography Group (SECG), Techn. Ber., 2010. besucht am 30. Juni 2026. Adresse: <https://www.secg.org/sec2-v2.pdf>.
- [54] ConsenSys. „Web3Signer Documentation.“ secp256k1 nur für ETH, ConsenSys, besucht am 29. Juni 2026. Adresse: <https://docs.web3signer.consenso.io/>.
- [55] Google Cloud. „Key purposes and algorithms – Cloud Key Management Service,“ Google Cloud Documentation, besucht am 29. Juni 2026. Adresse: <https://cloud.google.com/kms/docs/algorithms>.
- [56] DebitLlama. „Secure environment variables and secrets with TPM in production using Docker or K8S,“ besucht am 15. Jan. 2026. Adresse: https://medium.com/@hopeter_20531/secure-environment-variables-and-secrets-with-tpm-in-production-using-docker-or-k8s-d7b60ff63b6b.
- [57] balint. „A Modern and Secure Desktop Setup,“ besucht am März 2024. Adresse: <https://discourse.nixos.org/t/a-modern-and-secure-desktop-setup/41154/22>.
- [58] Trusted Computing Group, „TPM 2.0 Library Specification,“ Trusted Computing Group, Techn. Ber., 2019. besucht am 30. Juni 2026. Adresse: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [59] BitcoinSafe. „Air-Gapped Bitcoin Signing Guide,“ besucht am 2. Mai 2026. Adresse: <https://www.bitcoinsafe.com/en/guides/air-gapped-psbt-basics>.
- [60] Sparrow Wallets. „Best Practices,“ besucht am 4. Juni 2026. Adresse: <https://www.sparrowwallet.com/docs/best-practices.html>.
- [61] How does NixOS work? „NixOs,“ besucht am 4. Juni 2026. Adresse: <https://nixos.org/guides/how-nix-works/>.
- [62] Sparrow Wallet. „Sparrow Wallet Documentation,“ besucht am 2. Mai 2026. Adresse: <https://www.sparrowwallet.com/docs/>.
- [63] Electrum Technologies GmbH. „Electrum Documentation,“ besucht am 5. Juli 2026. Adresse: <https://electrum.readthedocs.io/en/latest/>.
- [64] Specter Solutions. „Adding Devices — Specter Desktop Documentation,“ besucht am 5. Juli 2026. Adresse: <https://docs.specter.solutions/desktop/extensions/devices/>.

- [65] cryptoadvance. „Specter Desktop — Frequently Asked Questions,“ besucht am 5. Juli 2026. Adresse: <https://github.com/cryptoadvance/specter-desktop/blob/master/docs/faq.md>.
- [66] Nunchuk. „Nunchuk Desktop — The Complete Bitcoin Terminal,“ besucht am 5. Juli 2026. Adresse: <https://nunchukapp.io/>.
- [67] Nunchuk. „Creating a Software Key,“ besucht am 5. Juli 2026. Adresse: <https://resources.nunchuk.io/getting-started/createsoftwarekey/>.
- [68] caravan. „Caravan — Stateless Multisig Coordinator,“ besucht am 5. Juli 2026. Adresse: <https://github.com/caravan-bitcoin/caravan>.
- [69] Unchained. „Caravan,“ besucht am 5. Juli 2026. Adresse: <https://www.caravannmultisig.com/#/>.
- [70] Knowing Bitcoin. „Sparrow vs Nunchuk vs Specter: Multisig Coordinators,“ besucht am 5. Juli 2026. Adresse: <https://knowingbitcoin.com/sparrow-vs-nunchuk-vs-specter-multisig-coordinators/>.
- [71] Luca Lean Bäcker. „Apphost – Installations- und Betriebsanleitung,“ besucht am 14. Juli 2026. Adresse: <https://github.com/LucaDev/LFM-Team-Blue/tree/main/apphost/Installationsanleitung.md>.
- [72] Florian Dobke. „NixOSCold Konfigurations-Dateien und readme,“ besucht am 30. Juni 2026. Adresse: <https://github.com/LucaDev/LFM-Team-Blue/tree/main/Cold-Wallet>.
- [73] Sparrow Wallets. „Quick Start Guide,“ besucht am 4. Juni 2026. Adresse: <https://www.sparrowwallet.com/docs/quick-start.html>.
- [74] ElectrumX Developers. „ElectrumX Documentation,“ besucht am 5. Juli 2026. Adresse: <https://electrumx.readthedocs.io/>.
- [75] Knowing Bitcoin, „Air-Gapped to Quantum: Bitcoin Security,“ *Knowing Bitcoin*, 2024. besucht am 2. Mai 2026. Adresse: <https://knowingbitcoin.com/bitcoin-air-gapped-quantum-security/>.
- [76] Ledger. „Hardware Wallets Vs Cold Wallets: What’s the Difference?“ Besucht am 4. Juni 2026. Adresse: <https://www.ledger.com/academy/hardware-wallets-and-cold-wallets-whats-the-difference>.
- [77] OpenAI, *Sandbox – Codex*, <https://developers.openai.com/codex/concepts/sandboxing>, Letzter Zugriff: 24.06.2026, 2026.

- [78] P. T. J. Kon u. a., „IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs,“ in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, Letzter Zugriff: 08.07.2026, 2024. Adresse: https://proceedings.neurips.cc/paper_files/paper/2024/hash/f26b29298ae8acd94bd7e839688e329b-Abstract-Datasets_and_Benchmarks_Track.html.
- [79] T. Zhang, S. Pan, Z. Zhang, Z. Xing und X. Sun, *Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation*, Angenommen bei FSE 2026 (ACM Int. Conf. on the Foundations of Software Engineering), 2026. Adresse: <https://arxiv.org/abs/2506.05623>.
- [80] H. Ryo, Y. Wang, T. Koike-Akino, J. Liu, K. Parsons und J. Hato, „Evaluating Security Policy Compliance in Infrastructure as Code Generated by Large Language Models,“ in *2026 14th International Symposium on Digital Forensics and Security (ISDFS)*, 2026, S. 1–6. DOI: [10.1109/ISDFS69419.2026.11458930](https://doi.org/10.1109/ISDFS69419.2026.11458930).
- [81] X. Lian u. a., „Large Language Models as Configuration Validators,“ *CoRR*, 2023. DOI: [10.48550/arXiv.2310.09690](https://arxiv.org/abs/10.48550/arXiv.2310.09690).
- [82] S. Mukhopadhyay, M. Shrivastava, K. Vaidhyathan und G. S. Kalahasti, „Leveraging LLMs for Generating Infrastructure as Code: An Exploratory Empirical Study,“ in *Proceedings of the 19th Innovations in Software Engineering Conference*, Ser. ISEC '26, New York, NY, USA: Association for Computing Machinery, 2026. DOI: [10.1145/3796563.3796572](https://doi.org/10.1145/3796563.3796572).
- [83] Marie Lumeau. „KI-Evaluation: Prompts, Chatverläufe und erzeugte Artefakte (Szenario A und B),“ besucht am 9. Juli 2026. Adresse: [XXX](#).
- [84] OpenAI, *GPT-5.4 Model*, Letzter Zugriff: 08.07.2026. Adresse: <https://developers.openai.com/api/docs/models/gpt-5.4>.
- [85] B. Chen, Z. Zhang, N. Langrené und S. Zhu, „Unleashing the Potential of Prompt Engineering in Large Language Models: A Comprehensive Review,“ *CoRR*, Jg. abs/2310.14735, 2023. DOI: [10.48550/arXiv.2310.14735](https://arxiv.org/abs/10.48550/arXiv.2310.14735).
- [86] Codex, *Decision Log zu Szenario B*, Internes Evaluationsartefakt, Datei: `decision-log.md`, Juli 2026.
- [87] Codex, *Home-Lab Runbook zu Szenario A*, Internes Evaluationsartefakt, Datei: `homelab-runbook.md`, Juli 2026.
- [88] Codex, *Self-Audit zu Szenario A auf a i l a b 1*, Internes Evaluationsartefakt, Datei: `self-audit-ailab1-2026-07-06.md`, Juli 2026.

- [89] Eigene Artefakte aus Szenario A, *Docker-Compose-Stacks app-core und ops*, Dateien: `work/services/app-core/docker-compose.yml`, `work/services/ops/docker-compose.yml`, Anhangsartefakte, 2026.
- [90] Eigene Artefakte aus Szenario A, *deploy-homelab-services.sh*, Deploy- und Secret-Erzeugungsskript, Datei: `work/scripts/deploy-homelab-services.sh`, Anhangsartefakt, 2026.
- [91] Eigene Artefakte aus Szenario A, *homelab-backup.sh*, Backup-Skript aus dem Arbeitsverzeichnis von Szenario A, Anhangsartefakt, 2026.
- [92] Eigene Artefakte aus Szenario A, *Host-Firewallregelwerk nftables.conf*, Datei: `work/host/nftables.conf`, Anhangsartefakt, 2026.
- [93] Eigene Artefakte aus Szenario A, *homelab-docker-firewall.sh*, Container-Firewallskripte für app-core und ops, Anhangsartefakte, 2026.
- [94] Eigene Artefakte aus Szenario A, *Caddyfile des tor-edge*, Datei: `work/services/tor-edge/Caddyfile`, Anhangsartefakt, 2026.
- [95] Eigene Artefakte aus Szenario A, *prometheus.yml der ops-Rolle*, Datei: `work/services/ops/config/prometheus/prometheus.yml`, Anhangsartefakt, 2026.
- [96] Codex, *Artefaktprüfung zu Szenario A*, Internes Evaluationsartefakt, Datei: `artefakt-pruefung-2026-07-08.md`, Juli 2026.
- [97] Codex, *Masterplan zu Szenario B*, Internes Evaluationsartefakt, Datei: `master-plan.md`, Juli 2026.
- [98] Codex, *Abschlusszusammenfassung zu Szenario B*, Internes Evaluationsartefakt, Datei: `final-summary.md`, Juli 2026.
- [99] Eigene Artefakte aus Szenario B, *section05-apply.sh*, Backup-/Monitoring-Umsetzungsskript, Datei: `Work-Files/section05-apply.sh`, Anhangsartefakt, 2026.
- [100] Codex, *Implementation Log zu Szenario B*, Internes Evaluationsartefakt, Datei: `implementation-log.md`, Juli 2026.
- [101] Eigene Artefakte aus Szenario B, *section3_config_remote.sh*, Konfigurations-Baseline-Skript, Datei: `Work-Files/section3\_config\_remote.sh`, Anhangsartefakt, 2026.
- [102] Codex, *Validierungsnotizen zu Szenario B*, Internes Evaluationsartefakt, Datei: `validator-notes.md`, Juli 2026.

- [103] Eigene Artefakte aus Szenario B, *section4_network_tor_remote.sh*, Netzwerk-/Tor-Umsetzungsskript, Datei: `Work-Files/section4\_network\_tor\_remote.sh`, Anhangsartefakt, 2026.
- [104] Proxmox Server Solutions GmbH, *Firewall - Proxmox VE*, Letzter Zugriff: 08.07.2026. Adresse: <https://pve.proxmox.com/wiki/Firewall>.
- [105] The Tor Project, *Client Authorization*, Letzter Zugriff: 08.07.2026. Adresse: <https://community.torproject.org/onion-services/advanced/client-auth/>.
- [106] Codex, *Artefakt-Review zu Szenario B*, Internes Evaluationsartefakt, Datei: `artefakt-review.md`, Juli 2026.
- [107] OpenAI, *Codex*, Letzter Zugriff: 08.07.2026. Adresse: <https://developers.openai.com/codex>.
- [108] OpenAI, *CLI - Codex*, Letzter Zugriff: 08.07.20262026-07-09. Adresse: <https://developers.openai.com/codex/cli>.

Anhang

A1 KI-Nutzung

Für die Erstellung dieser Arbeit wurden unterstützend verschiedene KI-basierte Werkzeuge eingesetzt. Die Nutzung umfasste insbesondere die Strukturierung von Texten, die Erstellung und Überprüfung von BibTeX-Einträgen, die Kontrolle von Rechtschreibung und Grammatik, die sprachliche Verbesserung einzelner Textpassagen sowie die Unterstützung bei der Kontrolle selbst erstellten Codes und bei der Generierung von Code.

Verwendet wurden dabei die folgenden Anbieter beziehungsweise Werkzeuge: Gemini, Claude, GitHub Copilot, ChatGPT und Codex.

A2 Installierte Applikationen

Alle Dienste sind über Traefik erreichbar. Die Standard-Subdomain ist `<subdomain>.<DOMAIN>`. Konfigurierbare Subdomains können in der `.env`-Datei überschrieben werden. Die Spalte **Tor** zeigt, ob der Dienst zusätzlich über einen eigenen `<subdomain>.<onion-adresse>.onion`-Router erreichbar ist. Rein intern genutzte Komponenten haben keinen Traefik-Router und damit auch keinen Tor-Zugang.

Dienst	Subdomain	Tor	Beschreibung
Infrastruktur			
Traefik	<code>traefik.*</code>	Ja	Reverse Proxy / TLS-Termination
Authelia	<code>auth.*</code>	Ja	SSO / MFA / OIDC-Provider
Garage	<code>s3.*</code>	Nein	S3-kompatibler Objektspeicher
Garage UI	<code>garage.*</code>	Nein	Garage Web-Verwaltung
Tor	–	–	Onion-Routing für die anderen Dienste
Medien			
Jellyfin	<code>jellyfin.*</code>	Ja	Medienserver (Filme, Musik, TV)
Immich	<code>photos.*</code>	Ja	Foto- und Video-Backup
Dokumente			
Paperless-NGX	<code>paperless.*</code>	Ja	Dokumentenmanagement / OCR
Bento-PDF	<code>pdf.*</code>	Ja	PDF-Werkzeuge
Cloud & Office			

Fortsetzung auf der nächsten Seite

Dienst	Subdomain	Tor	Beschreibung
OpenCloud	cloud.*	Ja	Nextcloud-Alternative, Datei-sync
Collabora	office.*	Nein	Browser-basierte Office-Suite
WOPI-Server	wopi.*	Nein	Collaboration-Backend
Home Automation			
Home Assistant	home.*	Ja	Hausautomation
Mosquitto	mqtt.*	Ja	MQTT-Broker (WebSocket)
Kommunikation			
Ntfy	ntfy.*	Ja	Push-Benachrichtigungen
Bichon	bichon.*	Ja	E-Mail-Client
Entwicklung			
Forgejo	git.*	Ja	Self-hosted Git-Plattform
Monitoring			
Grafana	grafana.*	Ja	Dashboards & Visualisierung
Prometheus	prometheus.*	Ja	Metriken-Scraping
Alertmanager	alertmanager.*	Ja	Alert-Routing
Loki	–	Nein	Log-Aggregation
Node Exporter	–	Nein	Host-Metriken
cAdvisor	–	Nein	Container-Metriken
Grafana Alloy	–	Nein	Log-Shipper zu Loki (Docker- und Host-Logs)
Tools			
Homepage	dashboard.*	Ja	Startseite / Service-Übersicht

Fortsetzung auf der nächsten Seite

Dienst	Subdomain	Tor	Beschreibung
Vaultwarden	vault.*	Ja	Passwortmanager (Bitwarden-kompatibel)
OpenSpeedTest	speedtest.*	Ja	Interner Netzwerk-Speedtest

A3 Sicherheitshärtungen des Systems

Das Basis-System ist auf mehreren Ebenen gehärtet. Die folgende Übersicht führt die wesentlichen Maßnahmen auf und erhebt keinen Anspruch auf Vollständigkeit.

A3.1 Boot & Firmware

- **Secure Boot (lanzaboote):** Alle NixOS-Generationen werden beim Build automatisch signiert. Der Boot schlägt fehl, wenn die Signatur nach dem Key-Enrollment ungültig ist.
- **TPM-Integration:** Der TPM-Chip ist in den Secure-Boot-Prozess eingebunden (`tpm-eventlog`); ermöglicht Measured Boot.
- **Bootloader-Editor deaktiviert:** `systemd-boot.editor = false` verhindert das Umgehen des Passworts über den Boot-Editor.
- **Kernel-Modul-Signatur erzwungen:** `module.sig_enforce=1` - nur signierte Kernel-Module dürfen geladen werden.
- **Kernel-Lockdown:** `lockdown=confidentiality` sperrt privilegierten Zugriff auf Kernel-Speicher.
- **UEFI-Passwort (Host):** Muss im Mainboard-Basic Input/Output System (BIOS) gesetzt werden (herstellerspezifisch).

A3.2 Kernel-Härtung

Oliv-grün markierte Optionen sind unter NixOS 26.05 (Stand 2. Juli 2026) auf den gleichen Wert wie der Standard gesetzt. Ein explizites Setzen ist dennoch sinnvoll, weil es die Auditierbarkeit des Systems verbessert und vor unbemerkten Änderungen des Standards schützt. Nicht alle Optionen sind für virtuelle Maschinen relevant. Sie werden dennoch gesetzt, um auch bei Bare-Metal-Ausführung möglichst viel Sicherheit zu bieten.

Spectre/Meltdown/CPU-Mitigationen

- Page Table Isolation: `pti=on`
- Speculative Store Bypass Disable: `spec_store_bypass_disable=on`
Erzwingt Speculative Store Bypass für alle Anwendungen, nicht nur sandboxed. Default: `seccomp`
- TSX deaktiviert + TAA/MDS vollständig mitigiert
(`tsx=off, tsx_async_abort=full, nosmt, mds=full, nosmt`). Kein Standard. Deaktiviert Simultaneous Multithreading (SMT) und Transactional Synchronization Extensions (TSX) komplett, schützt jedoch vor Exploits ähnlich zu Retbleed.
- L1TF: `l1tf=full,force`; Retbleed: `retbleed=auto,nosmt` Kein Standard. Deaktiviert SMT

Speicher- & Adressraumschutz

- Vollständiges ASLR: `randomize_va_space=2, vm.mmap_rnd_bits=32`. Standardmäßig aktiv, lediglich explizit gesetzt
- SLUB-Debug, Page-Poison, Init-on-alloc/free: verhindert Use-after-free-Exploits. Standardmäßig deaktiviert, da es Performance kostet.
- vsyscall-Seite deaktiviert: `vsyscall=none`. Standardmäßig auf `xonly` gesetzt. Sehr alte Programme, die noch auf vsyscalls setzen, verlieren ggf. ihre Funktion. Moderne haben keine Probleme damit.
- debugfs deaktiviert: `debugfs=off`. Standardmäßig aktiviert

DMA-Schutz (IOMMU)

- IOMMU erzwungen: `iommu=force, amd_iommu=on, intel_iommu=on`. Sie ist oftmals automatisch aktiviert, jedoch nicht immer.
- DMA Passthrough deaktiviert: `iommu.passthrough=0`. Verschlechtert Performance bei Thunderbolt/USB4, schützt aber verlässlich gegen DMA-Angriffe.

- Early PCI DMA blockiert: `efi=disable_early_pci_dma`. Schließt mögliches Early-Boot Thunderbolt/USB4/Angriffsfenster.

Kernel-Informationlecks

- Kernel-Pointer versteckt: `kernel.kptr_restrict=2`. Default=1. Stärkere Restriktion aktiviert
- dmesg nur für root: `kernel.dmesg_restrict=1`. Standard jedoch explizit gesetzt.
- Unprivilegiertes BPF deaktiviert:
`kernel.unprivileged_bpf_disabled=1`,
`net.core.bpf_jit_enable=0`, Standard unter NixOS jedoch explizit gesetzt.
- perf_events nur privilegiert: `kernel.perf_event Paranoid=3`. Standard=2. Höheres Schutzlevel
- ptrace eingeschränkt: `kernel.yama.ptrace_scope=2` (nur `PTRACE_TRACEME`). Default=1. Noch stärkere Einschränkung von ptrace
- SysRq deaktiviert: `kernel.sysrq=0`. Default=1
- setuid-Core-Dumps deaktiviert: `fs.suid_dumpable=0`. Default=2

Dateisystem-Schutz (Kernel)

- `fs.protected_hardlinks=1`, `fs.protected_symlinks=1`. Nur explizit gesetzt
- `fs.protected_fifos=2`, `fs.protected_regular=2`. Default=1. Erweitert FIFO und regular File Schutz
- /tmp gemountet mit `nosuid,nodev,noexec`. Default=`nosuid,nodev`. `noexec` für zusätzliche Absicherung

Blacklisted Kernel-Module (Angelehnt an CIS Benchmark)

Nicht benötigte Protokoll- und Dateisystem-Module sind deaktiviert, darunter:

Netzwerk: `dccp`, `sctp`, `rds`, `tipc`, `ax25`, `atm`, `ipx`, `appletalk` u. a.

Dateisysteme: `cramfs`, `hfs`, `hfsplus`, `jffs2`, `udf`, `cifs`, `nfs*`, `gfs2`

Hardware: `bluetooth`, `btusb`, `usb-storage`, `firewire-core`

A3.3 Netzwerk-Härtung

nftables-Firewall (Default Deny)

- Input-Policy: `drop` - nur explizit erlaubter Verkehr passiert.
- Forward-Policy: `drop` - kein unerwünschtes Routing.
- ICMP und ICMP6 rate-limitiert (5/s bzw. 10/s mit Burst).
- SSH rate-limitiert: 5 Verbindungen/min, Burst 10.
- Alle abgewiesenen Pakete werden geloggt (`[nftables DROP]`).

IPv4/IPv6 Sysctl

- Reverse Path Filtering aktiv (`rp_filter=1`)
- Source-Routing, ICMP-Redirects und Router-Advertisements deaktiviert
- SYN-Flood-Schutz: `tcp_syncookies=1`, `tcp_max_syn_backlog=4096`
- TCP-Timestamps deaktiviert (verhindert Uptime-Fingerprinting)
- IPv6 Router-Advertisements und Autoconf deaktiviert
- Martian-Pakete werden geloggt
- eBPF-JIT-Härtung: `net.core.bpf_jit_harden=2`

DNS-Sicherheit

- DNS-over-TLS erzwungen (`DNSOverTLS=true`)

- DNSSEC-Validierung aktiv (`DNSSEC=true`)
- Server: Cloudflare (1.1.1.1) und Quad9 (9.9.9.9) – beide ohne Logging-Policy

NTP – Authenticated Time Sync

- chrony mit **Network Time Security (NTS)** – Zeitserver werden über TLS authentifiziert
- `authselectmode=require` – unauthentifizierte Server werden abgelehnt

A3.4 Zugriffssteuerung & Authentifizierung

SSH-Härtung

- Nur Public-Key-Authentifizierung (`PasswordAuthentication=false`, `PermitRootLogin=no`)
- Post-Quanten-Key Encapsulation Mechanism (KEM):
`mlkem768x25519-sha256`, `sntrup761x25519-sha512`
- Ciphers: nur ChaCha20-Poly1305 und AES-256/128-GCM
- MACs: nur ETM-Varianten (`hmac-sha2-512-etm`, `hmac-sha2-256-etm`)
- X11/TCP/Agent-Forwarding und Tunnel deaktiviert
- `MaxAuthTries=3`, `LoginGraceTime=30s`, `MaxSessions=3`
- Rechtlicher Hinweis-Banner bei Login: kein technischer Schutz jedoch rechtliche Absicherung der Auditierungs- und Logging-Maßnahmen. Entzieht unbefugten Zugreifenden jegliche Grundlage für eine Berufung auf Unkenntnis der Überwachung und dokumentiert befugten Nutzenden die Bedingungen des Zugriffs.

Sudo-Härtung

- Nur Wheel-Gruppe darf sudo ausführen (`execWheelOnly=true`)

- Passwort immer erforderlich (`wheelNeedsPassword=true`)
- Session-Timeout: 5 Minuten; maximal 3 Passwortversuche
- Vollständiges I/O-Logging unter `/var/log/sudo-io`
- Eingeschränkter PATH (`secure_path`)

Benutzerverwaltung

- Immutable User-Datenbank: `users.mutableUsers=false`
- Root-Konto gesperrt: `hashedPassword="!"`, keine SSH-Keys
- Nur ein unprivilegiertes Benutzer (apphost)

AppArmor (Mandatory Access Control)

- AppArmor aktiv: `security.apparmor.enable=true`
- `killUnconfinedConfinables=true` - Prozesse ohne Profil werden beendet
- Upstream AppArmor-Profil eingebunden (`apparmor-profiles`)

A3.5 Audit & Intrusion Detection

Linux Audit Daemon

Überwacht werden u. a.:

- Zeitänderungen (`clock_settime`, `/etc/localtime`)
- Benutzer-/Gruppenänderungen (`/etc/{passwd,shadow,group}`)
- Privilege Escalation (`execve` mit UID-Wechsel zu root)
- Kernel-Modul-Laden/Entladen (`init_module`, `delete_module`)
- Änderungen an `/etc/sudoers` und `/etc/docker/`
- Datei- und Berechtigungsänderungen
- Netzwerkkonfigurationsänderungen (`sethostname`, `/etc/hosts`)

Fail2ban - Progressives Banning

- SSH-Jail: 3 Fehlversuche $\Rightarrow$ 2 Stunden Sperre
- Traefik-Jail: 5 Fehlversuche (HTTP 401) $\Rightarrow$ 1 Stunde Sperre
- Progressives Banning aktiv: jede weitere Sperre verlängert sich automatisch, bis max. 48 Stunden
- Backend: nftables (systemweit kein iptables mehr im Einsatz)

AIDE - Dateisystem-Integrität

- Tägliche automatische Prüfung via systemd-Service
- Überwachte Pfade: `/etc`, `/bin`, `/sbin`, `/lib*`, `/usr/bin`, `/usr/sbin`, `/boot`, `/opt/apphost/config`
- Algorithmen: SHA-512, SHA-256, MD5, Tiger, HAVAL, GOST u. a.
- Datenbank komprimiert (`gzip_dbout=yes`)

A3.6 Container-Sicherheit

- **User-Namespace-Remapping:** `usersns-remap=default` - Root im Container wird auf UID 100000-165536 auf dem Host gemappt. Ein Container-Ausbruch liefert dem Angreifer keinen root-Zugriff auf den Host.
- **Docker-Socket nur via Unix-Socket:** kein TCP-Listener (`hosts=["unix:///var/run/docker.sock"]`), kein Userland-Proxy.
- **Socket-Proxy:** Traefik greift nur über einen eingeschränkten Docker-Socket-Proxy auf die Docker-API zu. Direkter API-Zugriff ist geblockt.
- **Seccomp-Profil:** Standard-Docker-Seccomp-Profil aktiv.
- **Strikte Verzeichnisrechte:** `/opt/apphost/secrets` ist 0700 (kein Gruppen-Zugriff); alle anderen App-Verzeichnisse 0750.
- **Core-Dumps deaktiviert:** `LimitCORE=0` im Docker-Daemon.

- **Wöchentlicher CVE-Scan:** Trivy scannt alle laufenden Images montags 02:00 auf HIGH/CRITICAL-Schwachstellen.

Kata Containers & gVisor (optional, standardmäßig deaktiviert)

Beide Runtimes sind konfiguriert, aber nicht als Standard gesetzt, da sie unter bestimmten Hypervisor-Konfigurationen zu Instabilitäten führen können. Das gilt insbesondere, wenn Nested Virtualization nicht stabil unterstützt wird. Die Runtime kann in der Docker-Compose-Konfiguration pro Service auf `kata` oder `runsc` (gVisor) umgestellt werden, sofern die Hardware Nested Virtualization stabil unterstützt. Primär geeignet sind beide für nicht vertrauenswürdigen Code. Für den normalen Betrieb mit bekannten Diensten bieten User-Namespace-Remapping und die weiteren angewandten Sicherheitskonfigurationen bereits ein sehr hohes Sicherheitsniveau, vorausgesetzt das System wird aktuell gehalten.

A3.7 Festplatte & Daten

- **Swap verschlüsselt:** Zufälliger Schlüssel pro Boot (`randomEncryption=true`) keine persistenten Daten im Swap. Standardmäßig nicht unter NixOS aktiviert.
- **Btrfs mit zstd-Kompression** auf allen Subvolumes.
- **/tmp eingeschränkt:** Mount-Optionen `nosuid,nodev,noexec`.
- **ESP (/boot):** Umask 0077 (nur root darf lesen).

A3.8 Update-Management

- **NixOS-Auto-Upgrade:** Täglich 04:30 Uhr (± 15 min zufällige Verzögerung). Neustart muss manuell durchgeführt werden.
- **RenovateBot:** Container-Images werden automatisch aktuell gehalten (siehe Abschnitt 1.3.3).
- **Nix-Store GC:** Wöchentlich, löscht Generationen älter als 30 Tage.

A4 Screenshots des AppHost

Zur Veranschaulichung der in Abschnitt A2 aufgeführten Applikationen zeigt dieser Anhang eine Auswahl an Bildschirmaufnahmen des laufenden Systems. Sie dokumentieren exemplarisch den Zustand einzelner Dienste sowie deren Monitoring zum Zeitpunkt der Erstellung.

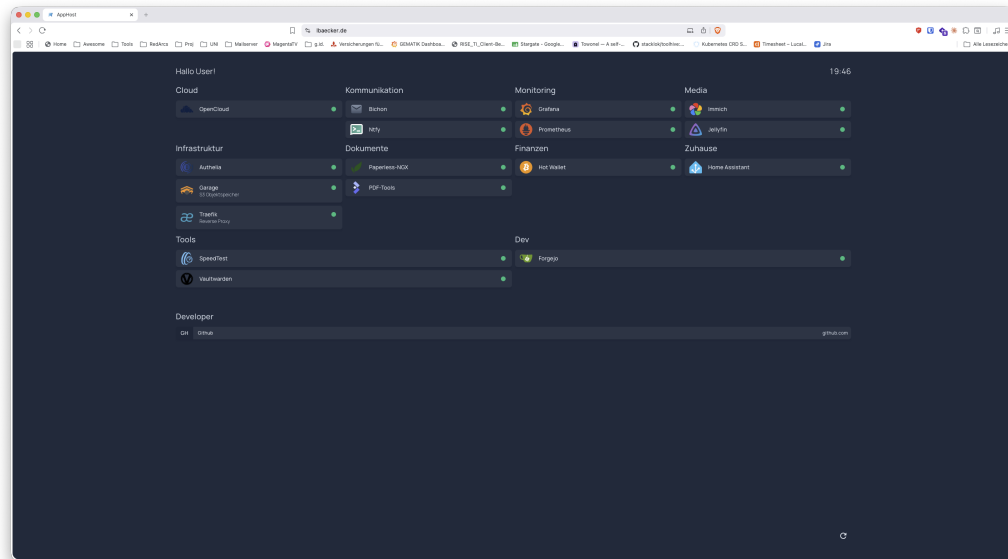


Abbildung A4.1: Homepage-Dashboard mit Übersicht aller Dienste, gruppiert nach Kategorie. Der grüne Punkt neben jedem Eintrag zeigt den erreichten Online-Status an. Quelle: Eigene Bildschirmaufnahme.

Anhang

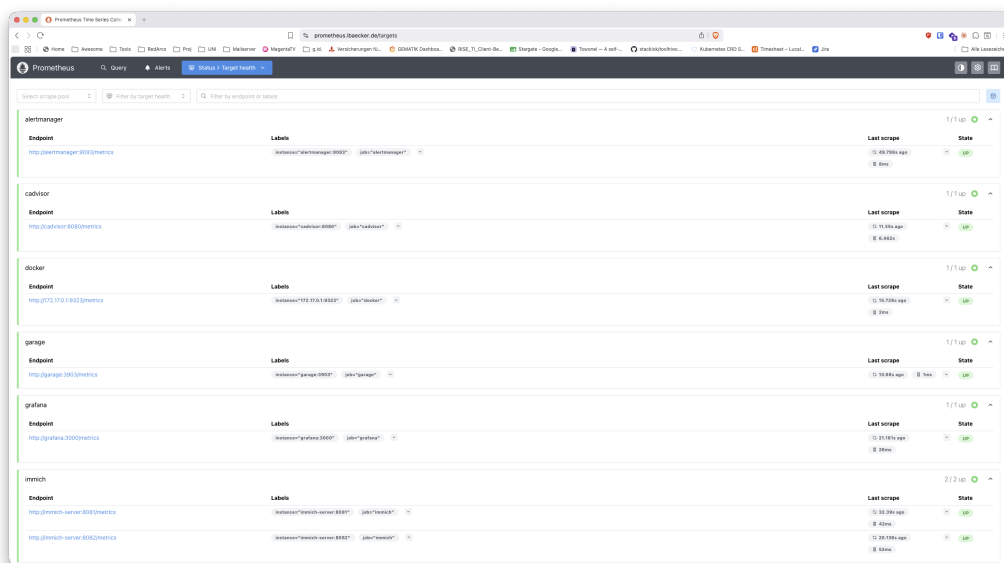


Abbildung A4.4: Prometheus-Statusseite „Target health“, welche die Erreichbarkeit der gescrapten Exporter (u. a. Alertmanager, cAdvisor, Garage, Grafana, Immich) mit Zeitstempel des letzten Scrapes anzeigt. Quelle: Eigene Bildschirmaufnahme.

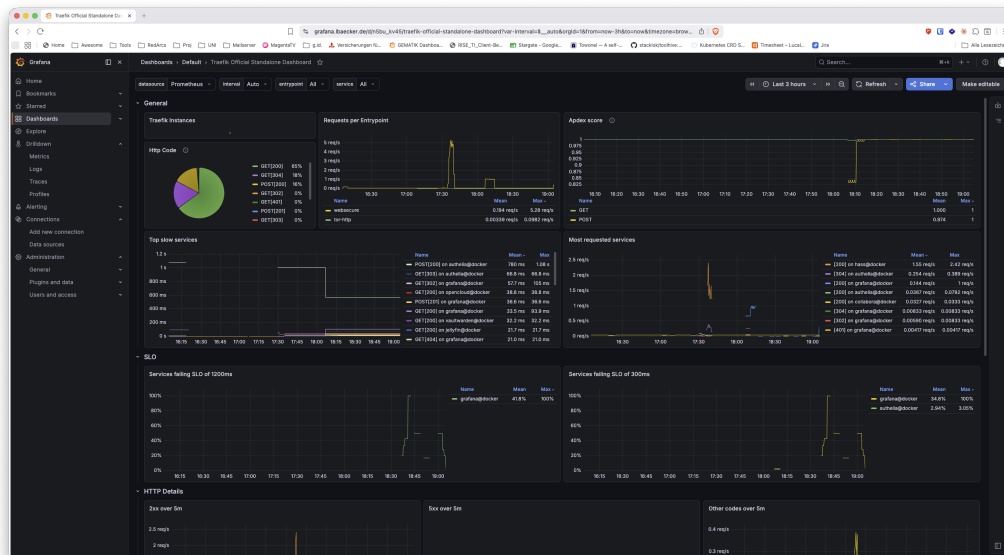


Abbildung A4.5: Grafana-Dashboard „Traefik Official Standalone Dashboard“ mit Request-Raten, HTTP-Statuscodes, langsamsten Diensten sowie der Einhaltung der definierten Latenz-Schwellenwerte je Dienst. Quelle: Eigene Bildschirmaufnahme.

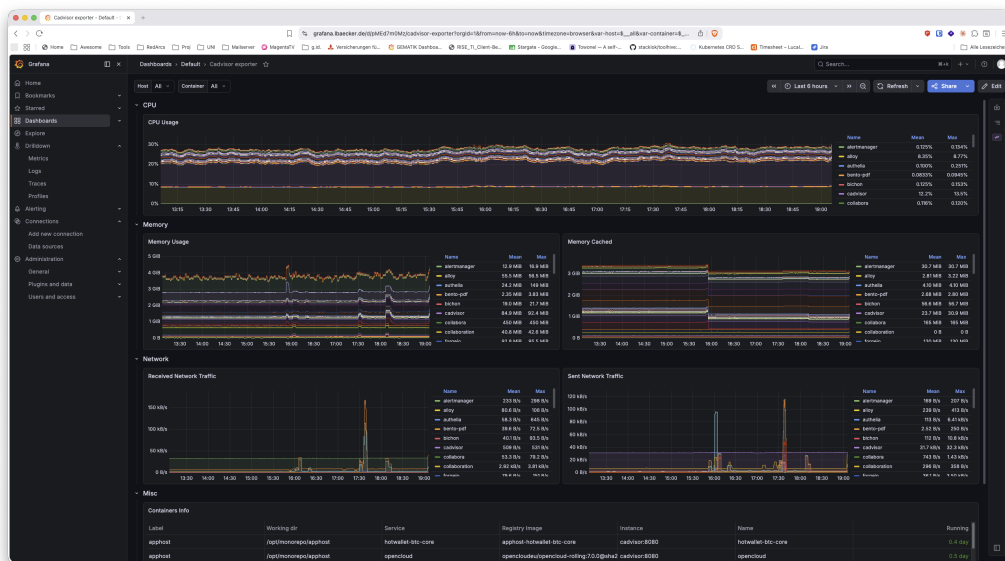


Abbildung A4.6: Grafana-Dashboard „Cadvisor exporter“ mit CPU-, Speicher- und Netzwerkauslastung der einzelnen Container sowie einer tabellarischen Übersicht der laufenden Container inklusive Image und Laufzeit. Quelle: Eigene Bildschirmaufnahme.

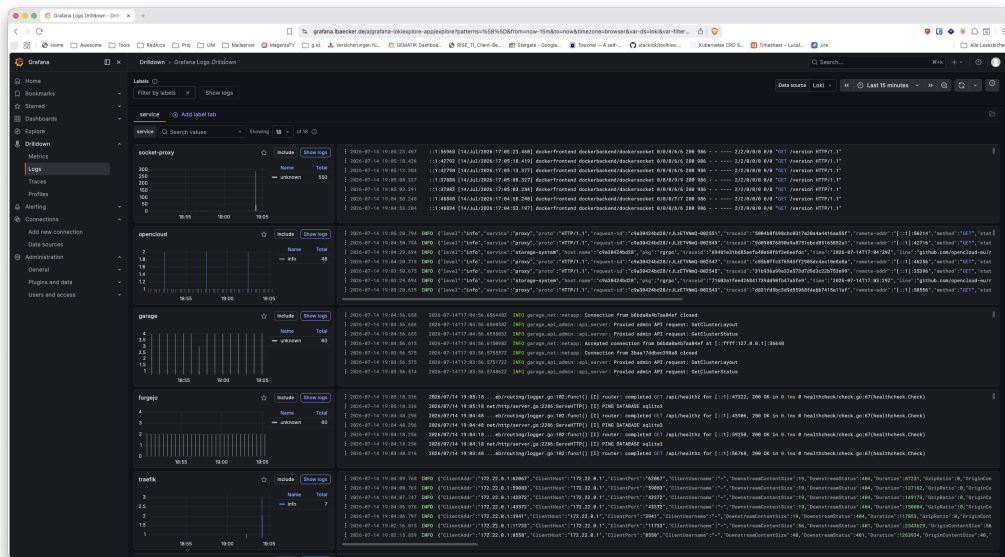


Abbildung A4.7: Grafana-„Logs Drilldown“ auf Basis von Loki, hier gefiltert nach dem Label `service` mit Log-Volumen und Log-Zeilen je Dienst (u. a. socket-proxy, opencloud, garage, forgejo, traefik). Quelle: Eigene Bildschirmaufnahme.

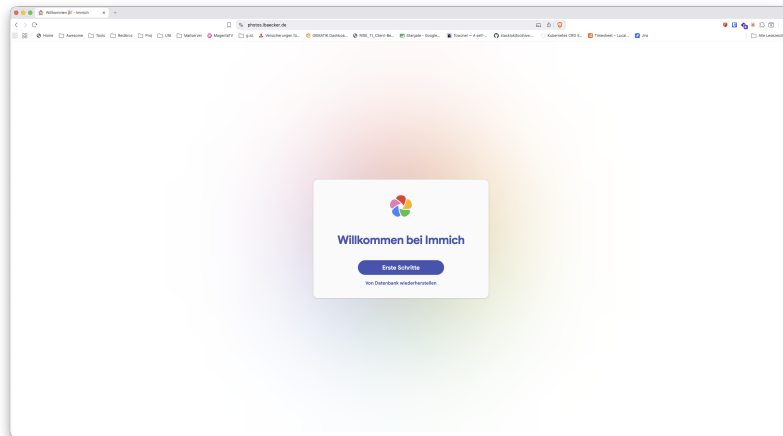


Abbildung A4.8: Startseite von Immich (photos.*) nach erfolgreicher Bereitstellung.
Quelle: Eigene Bildschirmaufnahme.

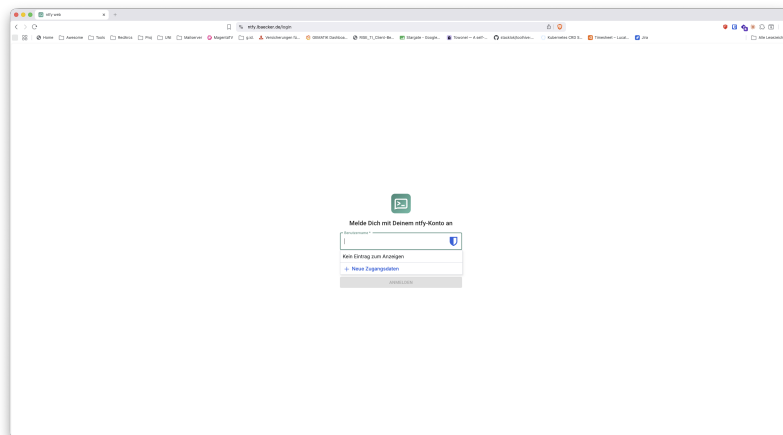


Abbildung A4.9: Login-Maske der Ntfy-Weboberfläche (ntfy.*) für Push-Benachrichtigungen. Quelle: Eigene Bildschirmaufnahme.

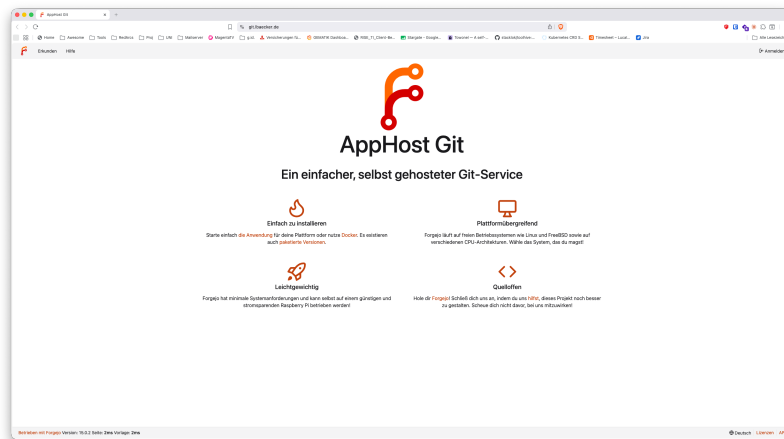


Abbildung A4.10: Startseite der selbst gehosteten Git-Plattform Forgejo (`git.*`, hier unter dem Namen „AppHost Git“). Quelle: Eigene Bildschirmaufnahme.

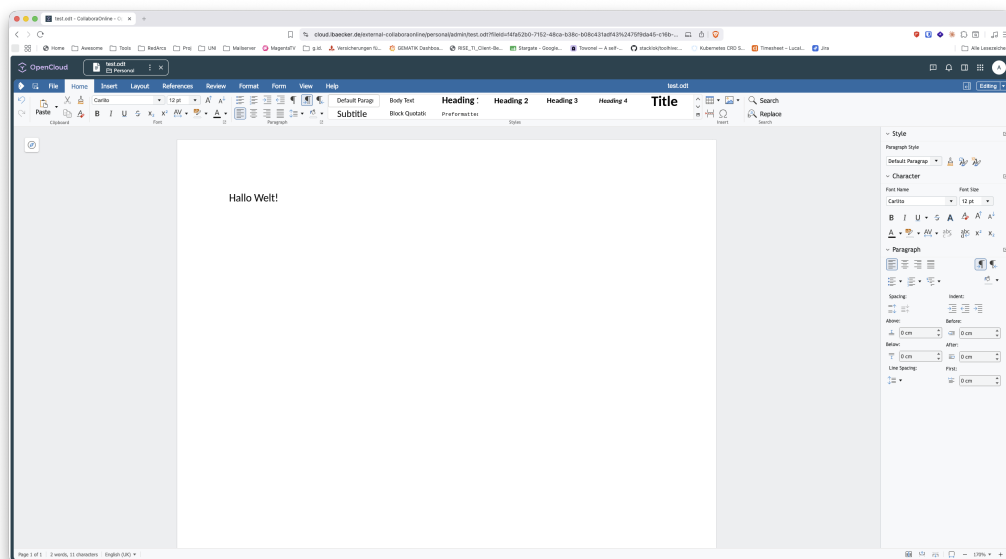


Abbildung A4.11: In OpenCloud (`ccloud.*`) eingebettete Collabora-Online-Bearbeitung einer `.odt`-Datei über den WOPI-Server. Quelle: Eigene Bildschirmaufnahme.

A5 Krypto Komponenten

A5.1 NixOS Härtung

Hrtungsmaßnahme	Beschreibung	Abgewehrter Angriffsvektor
Air-Gap als Standardzustand (Cold-VMs)	Das System bootet per NixOS- <i>specialisation</i> standardmäßig vollständig air-gapped, ein Online-Modus existiert nur als separater, bewusst zu wählender Boot-Eintrag. Interfaces werden beim Boot heruntergefahren, IPv6 deaktiviert und ausgehender Verkehr per <code>nftables</code> (<code>policy drop</code> , nur Loop-back) unterbunden.	Remote-Kompromittierung, Exfiltration von Schlüsselmaterial, Command-and-Control.
Festplatten-verschlüsselung (LUKS2)	Die Root-Partition ist vollständig verschlüsselt, die Passphrase wird beim Boot abgefragt.	Auslesen von Wallet-/Konfigurationsdaten aus gestohlenen Datenträgern oder VM-Abbildern (Data-at-Rest).
Kernel-Härtung	<code>kptr_restrict</code> , <code>dmesg_restrict</code> , <code>yama.ptrace_scope</code> , deaktiviertes unprivilegiertes eBPF sowie <code>protectKernelImage</code> (kein <code>kexec</code> zur Laufzeit).	Lokale Rechteauserweiterung, Kernel-Informationenlecks, Laufzeit-Manipulation des Kernels.
Boot-Integrität	Der <code>systemd-boot</code> -Editor ist deaktiviert, sodass Kernel-Parameter am Boot-Menü nicht verändert werden können.	Umgehung von Anmeldung bzw. Init-Prozess bei physischem Zugriff.
Kontrollierter Datenträgerzugriff	Automatisches Mounten (<code>udisks2</code>) ist deaktiviert, Wechselmedien werden ausschließlich manuell eingebunden.	Automatisches Einbinden manipulierter USB-Medien.
Reproduzierbarkeit	Passwörter liegen ausschließlich als Hash vor, die Konfiguration ist deklarativ und über einen gepinnten Flake reproduzierbar.	Konfigurationsdrift, Klartext-Credentials in der Versionskontrolle.

Tabelle A5.1: Deklarative NixOS-Härtung der Schlüsselhalter-VMs

A5.2 PSBT

Status	Beschreibung
INTENT_CREATED	Eingang der TX-Anfrage an der API-Schnittstelle.
PSBT_CREATED/PSBT_FAILED	Nach Erstellung durch den TX-Builder-Microservice über Bitcoin Core.
OPA_APPROVED/OPA_REJECTED	Nach der Entscheidung durch den OPA (Hot-Wallet).
SIGNED/SIGNING_FAILED	Nach Signierung auf der VM von Key A.
FINALIZED/FINALIZE_FAILED	Nach Finalisierung der Hot-PSBT zu einer Roh-TX über Bitcoin Core (<code>finalizepsbt</code>).
WAITING_HUMAN	Nach Teilsignierung mit 1/3 Unterschriften, warten auf manuelle Freigabe in der Cold-Wallet-Architektur.
COLD_STARTED	Der Operator hat die für den manuellen Cold-Prozess generierte PSBT auf ein Wechselmedium extrahiert.
COLD_STOPPED	Eine wartende, veraltete Nachbefüllungs-PSBT (WAITING_HUMAN) wurde gestoppt bzw. gelöscht. Eine PSBT im Status COLD_STARTED wird nicht in diesen Status versetzt, sondern bleibt ggf. verwaist in COLD_STARTED. Dies wird vorgenommen, da ggf. mehrere Operatoren gleichzeitig agieren und das System den menschlichen Handlungsspielraum, wenn dieser einmal gestartet wurde, nicht einschränken soll.
BROADCASTED/BROADCAST_FAILED	Erfolgreiches bzw. fehlgeschlagenes Broadcasting der TX über Bitcoin Core (<code>sendrawtransaction</code>).

Tabelle A5.2: PSBT-Lebenszyklus und Statusübergänge

A5.3 OPA

Policy	Bedingung / Limit
Pflichtparameter vorhanden (Nicht Null)	id, psbt, hash, target_address, source_address, Wallet_Typ (Voraussetzung für die nachfolgenden Regeln)
Netzwerk	nur regtest
amount_sats	$0 < x \leq 5\,000\,000$
fee_sats	$\leq 20\,000$
fee_rate	$20 \leq x \leq 100$ sat/vB
amount_satsspent_today	$\leq 30\,000\,000$ pro Tag (max_daily_sats, nur Zahlungen, ohne interne Swaps)

Tabelle A5.3: OPA-Policy zur PSBT-Bestätigung (Hot-Wallet)

A5.4 PCR

```
[user@hot-keyA:~/Desktop]$ sudo systemd-analyze pcrcs
NR NAME SHA256
0 platform-code 924d2586953a65bdc1d42298a6c2c80c36f83be0404f1f031bfac9331336ddea
1 platform-config cf9584c7000d3ed0b9698e2bfdbd2cd43b726933f5a17cbbd3f563010c859b13
2 external-code 0c086a8ba21bf3cbeee845770a92c2131a00e79a981b035cb65407e97f5a870c
3 external-config 3d458cfe55cc03eal1f443f1562beec8df51c75e14a9fcf9a7234a13f198e7969
4 boot-loader-code 6876e33ec273585b2d23f063ecac439a052998e84a3c0dcf3a1b629de23675c2
5 boot-loader-config ca5d1d4c6ce999bdd7d62d6fe00be707b1f3a15ae0d1c11e7ed1994fdf807cc3
6 host-platform 3d458cfe55cc03eal1f443f1562beec8df51c75e14a9fcf9a7234a13f198e7969
7 secure-boot-policy 65caf8dd1e0ea7a6347b635d2b379c93b9a1351edc2afc3ecda700e534eb3068
8 - 0000000000000000000000000000000000000000000000000000000000000000
9 kernel-initrd f60421e468f9bdd5691954d6841b1f642e9480accf9af0886022d22add9f7742
10 ima e40536941e11aed6b23af3538084ddc9d5d03dbf47e4239d837ed14bc85abd5e
11 kernel-boot 0000000000000000000000000000000000000000000000000000000000000000
12 kernel-config 954e57aa00d3a5563288d65f6dbf4527e741c463a9169454ce75d2cb5bb9a9ea
13 sysexts 0000000000000000000000000000000000000000000000000000000000000000
14 shim-policy 0000000000000000000000000000000000000000000000000000000000000000
15 system-identity 0000000000000000000000000000000000000000000000000000000000000000
16 debug 0000000000000000000000000000000000000000000000000000000000000000
17 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
18 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
19 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
20 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
21 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
22 - ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
23 application-support 0000000000000000000000000000000000000000000000000000000000000000
```

Abbildung A5.1: Log aus der PCR-Abfrage. Quelle: Eigene Bildschirmaufnahme in NixOS.

A5.5 Hilfsskripte

Skript	System	Zweck / Auslöser
<i>Basis-System (Hot-Wallet, <code>scripts/hotwallet/ops/</code>)</i>		
<code>psbt_submit.sh</code>	Basis	Fertige Operator-PSBT über NATS (<code>psbt.submit.requested</code>) einreichen.
<code>refill/</code> <code>psbt_export.sh</code>	Basis	Wartende Cold-PSBT auf das Wechselmedium exportieren (Zustand <code>COLD_STARTED</code>).
<code>refill/</code> <code>psbt_broadcast.sh</code>	Basis	Signierte Cold-PSBT vom Medium importieren, finalisieren und über Bitcoin Core broadcasten.
<code>setup/</code> <code>wallet_import.sh</code>	Basis	Cold-wsh-Deskriptor in Bitcoin Core und PostgreSQL registrieren.
<code>setup/</code> <code>wgHMAC_import.sh</code>	Basis	HMAC-Geheimnis und WG-Konfiguration vom Medium einspielen.
<code>setup/</code> <code>wgPeer_export.sh</code>	Basis	WG-Peer-Konfiguration des Basis-Systems exportieren.
<code>setup/</code> <code>wg_setup.sh</code>	Basis	WG-Interface und Kernel-Forwarding einrichten.
<i>Signierer-VM (Key A, Desktop)</i>		
<code>wgHMAC_export.sh</code>	Key A	xpub/öffentliche Deskriptoren, HMAC-Geheimnis und WG-Konfiguration auf USB exportieren.
<code>wgPeer_setup.sh</code>	Key A	WG-Peer einrichten bzw. überschreiben.
<code>setup.sh</code>	Key A	NixOS der Signierer-VM automatisch installieren.

Fortsetzung auf nächster Seite

Tabelle A5.4 – Fortsetzung

Skript	System	Zweck / Auslöser
<code>mnt/umnt/ format-USB.sh</code>	Key A	Wechselmedium mounten, sicher auswerfen bzw. bereinigen.
<i>Cold-VMs (Key B/C, Desktop, scripts/)</i>		
<code>setup.sh</code>	Cold	NixOS-Cold-Image automatisiert installieren (LUKS, Flake-Pinning).
<code>online.sh</code>	Cold	Kurzzeitig Online-Modus zur Wallet-Registrierung aktivieren.
<code>airgap.sh</code>	Cold	In den air-gapped Standard zurückschalten.
<code>mnt/umnt/ format-USB.sh</code>	Cold	Wechselmedium mounten, sicher auswerfen bzw. bereinigen.
<i>Evaluierte Hash-Freigabeschicht (verworfen, siehe 1.4.5)</i>		
<code>psbt/ psbt-approve.sh</code>	Koord.	PSBT hashen und Hash durch den Koordinator signieren (<code>unappr.<id>.psbt</code>).
<code>auth/ hash-keyGen.sh</code>	Koord.	Freigabe-Schlüsselpaar auf der Koordinator-Instanz erzeugen.
<code>auth/ hash-keyStore.sh</code>	Key B/C	Public Key der Freigabe auf dem Signer hinterlegen (mit Air-Gap-Prüfung).
<code>auth/ hash-verify.sh</code>	Key B/C	Freigabesignatur (<code>approval.json.sig</code>) vor dem Signieren verifizieren.

Tabelle A5.4: Betriebs- und Setup-Skripte je System.